

Primitive Recognition Calculus Lean Formalization Bundle

Generated 2026-05-27. Scope: full Lean source content under IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/. Intended as a NotebookLM PDF companion to Delta_Manifesto_20260526.pdf and Delta_Formal_Unification_From_Distinction.pdf.

Total files: 33. **Total Lean lines:** 11228.

File Manifest

No.	Path	Lines
1	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Strength.lean	54
2	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Basic.lean	134
3	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/SameDiff.lean	105
4	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Quotient.lean	71
5	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Orbit.lean	106
6	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitArithmetic.lean	194
7	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/IntegerRational.lean	1289
8	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/IntegerOrder.lean	67
9	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitDivisibility.lean	283
10	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitEuclidean.lean	473
11	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RationalField.lean	284
12	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCost.lean	242
13	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceIncrementTriangle.lean	261
14	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceVerifierTriangle.lean	104
15	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceTriangle.lean	94
16	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCauchy.lean	236
17	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealNullSetoid.lean	135
18	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealMulBoundedContinuity.lean	236
19	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealBoundednessModulus.lean	142
20	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealProductContinuity.lean	304
21	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealOrderCongruence.lean	206
22	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteness.lean	592
23	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteOrderedField.lean	370
24	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteOrderedFieldPromoted.lean	88
25	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompletion.lean	97
26	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/TraceClosure.lean	107
27	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/TraceLogic.lean	237
28	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/FormalSystem.lean	123

No.	Path	Lines
29	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Inevitability.lean	90
30	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RecognizerBridge.lean	117
31	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCNativeCostUniqueness.lean	3798
32	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Kernel.lean	253
33	IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/UniversalFoundation.lean	336

1. Strength.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Strength.lean · Lines: 54

```

0001 /-
0002   PrimitiveRecognitionCalculus/Strength.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Spec anchor:
0008     K1. Strength Ledger
0009
0010   This module records the proof-strength tags used by the Primitive
0011   Recognition Calculus kernel. The tags are not mathematical assumptions.
0012   They are audit labels attached to later definitions and theorems.
0013 -/
0014
0015 import Mathlib
0016
0017 namespace IndisputableMonolith
0018 namespace Foundation
0019 namespace PrimitiveRecognitionCalculus
0020
0021 /-- K1. The strength tag attached to a PRC claim. -/
0022 inductive StrengthTag where
0023   /-- Forced by distinction and finite repetition alone. -/
0024   | deltaOnly
0025   /-- Uses the completed orbit or completed stable trace families. -/
0026   | traceClosure
0027   /-- Uses selection of witnesses from stable families. -/
0028   | choice
0029   /-- Uses controlled subtrace-class or power-class formation. -/
0030   | powerComprehension
0031   /-- Uses excluded middle or full classical reasoning as an extension. -/
0032   | classicalExtension
0033   deriving DecidableEq, Repr
0034
0035 /-- A small audit record tying a claim label to its strength tag. -/
0036 structure StrengthClaim where
0037   label : String
0038   tag : StrengthTag
0039   statement : String
0040   deriving Repr
0041
0042 /-- K1 audit sanity: the 6-only tag is not the trace-closure tag. -/
0043 theorem deltaOnly_ne_traceClosure :
0044   StrengthTag.deltaOnly ≠ StrengthTag.traceClosure := by
0045   decide
0046
0047 /-- K1 audit sanity: choice is not a 6-only claim. -/
0048 theorem choice_ne_deltaOnly :
0049   StrengthTag.choice ≠ StrengthTag.deltaOnly := by
0050   decide
0051
0052 end PrimitiveRecognitionCalculus
0053 end Foundation
0054 end IndisputableMonolith

```

2. Basic.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Basic.lean · Lines: 134

```

0001 /-
0002   PrimitiveRecognitionCalculus/Basic.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Spec anchors:
0008     K2.1-K2.5, R1-R4
0009
0010   The  $\delta$ -only syntactic kernel: distinction act, side, endpoint, finite
0011   trace, trace append, and trace extension. Lean equality is used only by
0012   the verifier to prove facts about this syntax.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Strength
0017
0018 namespace IndisputableMonolith
0019 namespace Foundation
0020 namespace PrimitiveRecognitionCalculus
0021
0022 /-- K2.1. The primitive distinction act. At the object level this is  $\delta$ . -/
0023 inductive DistinctionAct where
0024   | delta
0025   deriving DecidableEq, Repr
0026
0027 /-- K2.2. The two sides forced by a distinction. -/
0028 inductive Side where
0029   | left
0030   | right
0031   deriving DecidableEq, Repr
0032
0033 /-- K2.3. An endpoint is a side of the primitive distinction. -/
0034 structure Endpoint where
0035   side : Side
0036   deriving DecidableEq, Repr
0037
0038 /-- The left endpoint of  $\delta$ . -/
0039 def Endpoint.left : Endpoint :=
0040   (Side.left)
0041
0042 /-- The right endpoint of  $\delta$ . -/
0043 def Endpoint.right : Endpoint :=
0044   (Side.right)
0045
0046 /-- K2.4. A finite trace is empty or extended by one distinction act. -/
0047 inductive Trace where
0048   | empty
0049   | extend : Trace → DistinctionAct → Trace
0050   deriving DecidableEq, Repr
0051
0052 namespace Trace
0053
0054 /-- R3. One-step extension by  $\delta$ . -/
0055 def step (T : Trace) : Trace :=
0056   Trace.extend T DistinctionAct.delta
0057
0058 /-- Append two traces. This is the syntactic composition operation. -/
0059 def append : Trace → Trace → Trace
0060   | T, Trace.empty => T
0061   | T, Trace.extend U a => Trace.extend (append T U) a
0062
0063 @[simp] theorem append_empty (T : Trace) :
0064   append T Trace.empty = T := by
0065   rfl
0066
0067 @[simp] theorem append_extend (T U : Trace) (a : DistinctionAct) :
0068   append T (Trace.extend U a) = Trace.extend (append T U) a := by
0069   rfl
0070
0071 @[simp] theorem empty_append (T : Trace) :
0072   append Trace.empty T = T := by
0073   induction T with
0074   | empty => rfl
0075   | extend T a ih =>
0076     simp [append, ih]
0077
0078 /-- R4. Trace append is associative. -/
0079 theorem append_assoc (T U V : Trace) :
0080   append (append T U) V = append T (append U V) := by
0081   induction V with
0082   | empty => rfl
0083   | extend V a ih =>
0084     simp [append, ih]
0085
0086 /-- K2.5. 'Extends T U' means 'U' is 'T' followed by some suffix trace. -/
0087 def Extends (T U : Trace) : Prop :=
0088    $\exists V : \text{Trace}, \text{append } T V = U$ 
0089
0090 /-- R4. Trace extension is reflexive. -/

```

```

0091 theorem extends_refl (T : Trace) :
0092   Extends T := by
0093   exact (Trace.empty, rfl)
0094
0095 /-- R4. Trace extension is transitive. -/
0096 theorem extends_trans {T U V : Trace}
0097   (hTU : Extends T U) (hUV : Extends U V) :
0098   Extends T V := by
0099   rcases hTU with (A, hA)
0100   rcases hUV with (B, hB)
0101   refine (append A B, ?_)
0102   rw [← append_assoc, hA, hB]
0103
0104 /-- The trace with exactly `n` repeated  $\delta$ -extensions. -/
0105 def orbitTrace : Nat → Trace
0106 | 0 => Trace.empty
0107 | Nat.succ n => step (orbitTrace n)
0108
0109 /-- Length of a finite trace. -/
0110 def length : Trace → Nat
0111 | Trace.empty => 0
0112 | Trace.extend T _ => Nat.succ (length T)
0113
0114 @[simp] theorem length_empty :
0115   length Trace.empty = 0 := by
0116   rfl
0117
0118 @[simp] theorem length_extend (T : Trace) (a : DistinctionAct) :
0119   length (Trace.extend T a) = Nat.succ (length T) := by
0120   rfl
0121
0122 /-- K2.12 preview. The length of the nth orbit trace is n. -/
0123 theorem length_orbitTrace (n : Nat) :
0124   length (orbitTrace n) = n := by
0125   induction n with
0126   | zero => rfl
0127   | succ n ih =>
0128     simp [orbitTrace, step, ih]
0129
0130 end Trace
0131
0132 end PrimitiveRecognitionCalculus
0133 end Foundation
0134 end IndisputableMonolith

```

3. SameDiff.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/SameDiff.lean · Lines: 105

```

0001 /-
0002   PrimitiveRecognitionCalculus/SameDiff.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Spec anchors:
0008     K2.6-K2.10, R5-R7, K4.2-K4.3
0009
0010   This module does not define object equality as Lean equality. It defines
0011   an admissible trace-judgment surface: SameT, DiffT, consistency, and the
0012   substitution rule for contexts that respect SameT.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Basic
0017
0018 namespace IndisputableMonolith
0019 namespace Foundation
0020 namespace PrimitiveRecognitionCalculus
0021
0022 /-- K2.6-K2.8. An admissible trace judgment surface. -/
0023 structure TraceJudgment where
0024   /-- K2.7. Object-level equality at a trace. -/
0025   same : Trace → Endpoint → Endpoint → Prop
0026   /-- K2.6. Object-level witnessed difference at a trace. -/
0027   diff : Trace → Endpoint → Endpoint → Prop
0028   /-- R5. SameT must be reflexive at each trace. -/
0029   same_refl_proof : ∀ T : Trace, Reflexive (same T)
0030   /-- R5. SameT must be symmetric at each trace. -/
0031   same_symm_proof : ∀ T : Trace, Symmetric (same T)
0032   /-- R5. SameT must be transitive at each trace. -/
0033   same_trans_proof : ∀ T : Trace, Transitive (same T)
0034   /-- R6. SameT and DiffT cannot both hold for the same ordered pair. -/
0035   same_diff_exclusive :
0036     ∀ {T : Trace} {a b : Endpoint}, same T a b → diff T a b → False
0037
0038 namespace TraceJudgment
0039
0040 /-- Reflexivity of SameT, extracted from the admissibility field. -/
0041 theorem same_refl (J : TraceJudgment) (T : Trace) (a : Endpoint) :
0042   J.same T a a :=
0043   J.same_refl_proof T a
0044
0045 /-- Symmetry of SameT, extracted from the admissibility field. -/
0046 theorem same_symm (J : TraceJudgment) (T : Trace) {a b : Endpoint}
0047   (h : J.same T a b) :
0048   J.same T b a :=
0049   J.same_symm_proof T h
0050
0051 /-- Transitivity of SameT, extracted from the admissibility field. -/
0052 theorem same_trans (J : TraceJudgment) (T : Trace) {a b c : Endpoint}
0053   (hab : J.same T a b) (hbc : J.same T b c) :
0054   J.same T a c :=
0055   J.same_trans_proof T hab hbc
0056
0057 end TraceJudgment
0058
0059 /-- K2.8. A trace is consistent for a judgment surface if it never asserts
0060   SameT and DiffT for the same endpoints. -/
0061 def Consistent (J : TraceJudgment) (T : Trace) : Prop :=
0062   ∀ a b : Endpoint, ~ (J.same T a b ∧ J.diff T a b)
0063
0064 /-- R6. The exclusivity field gives consistency at every trace. -/
0065 theorem consistent_of_exclusive (J : TraceJudgment) (T : Trace) :
0066   Consistent J T := by
0067   intro a b h
0068   exact J.same_diff_exclusive h.1 h.2
0069
0070 /-- A predicate respects SameT at a trace. -/
0071 def RespectsSame (J : TraceJudgment) (T : Trace)
0072   (P : Endpoint → Prop) : Prop :=
0073   ∀ {a b : Endpoint}, J.same T a b → P a → P b
0074
0075 /-- K2.10 and R7. Substitution for contexts that respect SameT. -/
0076 theorem substitute
0077   (J : TraceJudgment) (T : Trace) (P : Endpoint → Prop)
0078   (hP : RespectsSame J T P) {a b : Endpoint}
0079   (hsame : J.same T a b) (ha : P a) :
0080   P b :=
0081   hP hsame ha
0082
0083 /-- A verifier-level model of the SameT/DiffT interface.
0084
0085   This is not PRC's object-level primitive. It is a sanity model showing the
0086   interface is inhabited in Lean's verifier language. -/
0087 def verifierEqualityJudgment : TraceJudgment where
0088   same := fun _ a b => a = b
0089   diff := fun _ a b => a ≠ b
0090   same_refl_proof := by

```

```
0091   intro T a
0092   rfl
0093   same_symm_proof := by
0094     intro T a b h
0095     exact h.symm
0096   same_trans_proof := by
0097     intro T a b c hab hbc
0098     exact hab.trans hbc
0099   same_diff_exclusive := by
0100     intro T a b hsame hdiff
0101     exact hdiff hsame
0102
0103 end PrimitiveRecognitionCalculus
0104 end Foundation
0105 end IndisputableMonolith
```

4. Quotient.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Quotient.lean · Lines: 71

```

0001 /-
0002   PrimitiveRecognitionCalculus/Quotient.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Spec anchors:
0008     K2.11, K4.4
0009
0010   Quotienting is permitted only after SameT is known to be an equivalence.
0011   This module builds endpoint trace-classes and the corresponding recursion
0012   principle for SameT-respecting maps.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.SameDiff
0017
0018 namespace IndisputableMonolith
0019 namespace Foundation
0020 namespace PrimitiveRecognitionCalculus
0021
0022 /-- K2.11. The setoid induced by SameT at a fixed trace. -/
0023 def sameSetoid (J : TraceJudgment) (T : Trace) : Setoid Endpoint where
0024   r := J.same T
0025   iseqv := {
0026     refl := J.same_refl_proof T
0027     symm := by
0028       intro a b h
0029       exact J.same_symm_proof T h
0030     trans := by
0031       intro a b c hab hbc
0032       exact J.same_trans_proof T hab hbc
0033   }
0034
0035 /-- K2.11. Endpoint trace-classes under SameT. -/
0036 def EndpointClass (J : TraceJudgment) (T : Trace) : Type :=
0037   Quot (sameSetoid J T)
0038
0039 /-- The class of an endpoint. -/
0040 def endpointClassOf (J : TraceJudgment) (T : Trace)
0041   (a : Endpoint) : EndpointClass J T :=
0042   Quot.mk (sameSetoid J T) a
0043
0044 /-- K4.4. SameT endpoints determine the same quotient class. -/
0045 theorem endpointClass_eq_of_same
0046   (J : TraceJudgment) (T : Trace) {a b : Endpoint}
0047   (h : J.same T a b) :
0048   endpointClassOf J T a = endpointClassOf J T b :=
0049   Quot.sound h
0050
0051 /-- K4.4. Quotient recursion for SameT-respecting maps. -/
0052 def endpointClassLift
0053   (J : TraceJudgment) (T : Trace) {α : Sort _}
0054   (f : Endpoint → α)
0055   (hf : ∀ {a b : Endpoint}, J.same T a b → f a = f b) :
0056   EndpointClass J T → α :=
0057   Quot.lift f (by
0058     intro a b h
0059     exact hf h)
0060
0061 @[simp] theorem endpointClassLift_mk
0062   (J : TraceJudgment) (T : Trace) {α : Sort _}
0063   (f : Endpoint → α)
0064   (hf : ∀ {a b : Endpoint}, J.same T a b → f a = f b)
0065   (a : Endpoint) :
0066   endpointClassLift J T f hf (endpointClassOf J T a) = f a := by
0067   rfl
0068
0069 end PrimitiveRecognitionCalculus
0070 end Foundation
0071 end IndisputableMonolith

```


5. Orbit.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Orbit.lean · Lines: 106

```

0001 /-
0002   PrimitiveRecognitionCalculus/Orbit.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Spec anchors:
0008     K2.12, R8, K4.5
0009
0010   Base-neutral arithmetic begins as the finite orbit of repeated 6. Lean's
0011   Nat is used here only as a verifier representation, and the equivalence is
0012   proved explicitly.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Basic
0017
0018 namespace IndisputableMonolith
0019 namespace Foundation
0020 namespace PrimitiveRecognitionCalculus
0021
0022 /-- K2.12. The base-neutral finite orbit of repeated distinction. -/
0023 inductive DistinctionNat where
0024   | zero
0025   | succ : DistinctionNat → DistinctionNat
0026   deriving DecidableEq, Repr
0027
0028 namespace DistinctionNat
0029
0030 /-- R8. Zero is not a successor. -/
0031 theorem zero_ne_succ (n : DistinctionNat) :
0032   zero ≠ succ n := by
0033   intro h
0034   cases h
0035
0036 /-- R8. Successor is injective. -/
0037 theorem succ_injective :
0038   Function.Injective succ := by
0039   intro a b h
0040   cases h
0041   rfl
0042
0043 /-- R8. Induction over the 6-orbit. -/
0044 theorem induction {P : DistinctionNat → Prop}
0045   (hzero : P zero)
0046   (hsucc : ∀ n : DistinctionNat, P n → P (succ n)) :
0047   ∀ n : DistinctionNat, P n := by
0048   intro n
0049   induction n with
0050   | zero => exact hzero
0051   | succ n ih => exact hsucc n ih
0052
0053 /-- Verifier representation of the orbit as Lean Nat. -/
0054 def toNat : DistinctionNat → Nat
0055   | zero => 0
0056   | succ n => Nat.succ (toNat n)
0057
0058 /-- Build an orbit position from a verifier Nat. -/
0059 def ofNat : Nat → DistinctionNat
0060   | 0 => zero
0061   | Nat.succ n => succ (ofNat n)
0062
0063 @[simp] theorem toNat_zero :
0064   toNat zero = 0 := by
0065   rfl
0066
0067 @[simp] theorem toNat_succ (n : DistinctionNat) :
0068   toNat (succ n) = Nat.succ (toNat n) := by
0069   rfl
0070
0071 @[simp] theorem ofNat_zero :
0072   ofNat 0 = zero := by
0073   rfl
0074
0075 @[simp] theorem ofNat_succ (n : Nat) :
0076   ofNat (Nat.succ n) = succ (ofNat n) := by
0077   rfl
0078
0079 /-- K4.5. Transport from Lean Nat to the 6-orbit and back is identity. -/
0080 theorem toNat_ofNat (n : Nat) :
0081   toNat (ofNat n) = n := by
0082   induction n with
0083   | zero => rfl
0084   | succ n ih =>
0085     simp [ofNat, ih]
0086
0087 /-- K4.5. Transport from the 6-orbit to Lean Nat and back is identity. -/
0088 theorem ofNat_toNat (n : DistinctionNat) :
0089   ofNat (toNat n) = n := by
0090   induction n with

```

```
0091 | zero => rfl
0092 | succ n ih =>
0093   simp [toNat, ih]
0094
0095 /-- K4.5. The 6-orbit is equivalent to Lean Nat as a verifier display. -/
0096 def equivNat : DistinctionNat = Nat where
0097   toFun := toNat
0098   invFun := ofNat
0099   left_inv := ofNat_toNat
0100   right_inv := toNat_ofNat
0101
0102 end DistinctionNat
0103
0104 end PrimitiveRecognitionCalculus
0105 end Foundation
0106 end IndisputableMonolith
```

6. OrbitArithmetic.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitArithmetic.lean · Lines: 194

```

0001 /-
0002   PrimitiveRecognitionCalculus/OrbitArithmetic.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Spec anchors:
0008     K4.5 (orbit arithmetic), K4.6 (balanced length), K4.7 (cross-multiplication)
0009
0010   Addition and multiplication on the 6-orbit, with the transport theorems
0011   to Lean Nat needed for the balanced and cross-multiplication
0012   characterizations of 'PRCInt' and 'PRCRat'.
0013
0014   Strength: 6-only. The construction uses only the inductive structure of
0015   the orbit and the verifier-level transport already established in
0016   'Orbit.lean'. No project-local axioms.
0017 -/
0018
0019 import MathLib
0020 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Orbit
0021
0022 namespace IndisputableMonolith
0023 namespace Foundation
0024 namespace PrimitiveRecognitionCalculus
0025 namespace DistinctionNat
0026
0027 /-! ## Addition on the 6-orbit -/
0028
0029 /-- K4.5. Addition of orbit positions: concatenation of repetition. -/
0030 def add : DistinctionNat → DistinctionNat → DistinctionNat
0031   | a, zero => a
0032   | a, succ b => succ (add a b)
0033
0034 instance : Add DistinctionNat := (add)
0035
0036 theorem add_def (a b : DistinctionNat) :
0037   a + b = add a b := rfl
0038
0039 theorem add_zero_eq (a : DistinctionNat) :
0040   a + zero = a := rfl
0041
0042 theorem add_succ_eq (a b : DistinctionNat) :
0043   a + succ b = succ (a + b) := rfl
0044
0045 theorem zero_add_eq (a : DistinctionNat) :
0046   zero + a = a := by
0047   induction a with
0048   | zero => rfl
0049   | succ n ih =>
0050     show succ (zero + n) = succ n
0051     rw [ih]
0052
0053 theorem succ_add_eq (a b : DistinctionNat) :
0054   succ a + b = succ (a + b) := by
0055   induction b with
0056   | zero => rfl
0057   | succ n ih =>
0058     show succ (succ a + n) = succ (succ (a + n))
0059     rw [ih]
0060
0061 theorem add_comm (a b : DistinctionNat) :
0062   a + b = b + a := by
0063   induction a with
0064   | zero =>
0065     rw [zero_add_eq, add_zero_eq]
0066   | succ n ih =>
0067     rw [succ_add_eq, add_succ_eq, ih]
0068
0069 theorem add_assoc (a b c : DistinctionNat) :
0070   (a + b) + c = a + (b + c) := by
0071   induction c with
0072   | zero => rfl
0073   | succ n ih =>
0074     show (a + b) + succ n = a + (b + succ n)
0075     rw [add_succ_eq, add_succ_eq, add_succ_eq, ih]
0076
0077 /-! ## Transport to Lean Nat -/
0078
0079 /-- K4.5. The verifier display of orbit addition matches Lean Nat addition. -/
0080 theorem toNat_add (a b : DistinctionNat) :
0081   (a + b).toNat = a.toNat + b.toNat := by
0082   induction b with
0083   | zero =>
0084     rw [add_zero_eq, toNat_zero, Nat.add_zero]
0085   | succ n ih =>
0086     show (succ (a + n)).toNat = a.toNat + (succ n).toNat
0087     rw [toNat_succ, toNat_succ, ih]
0088     omega
0089
0090 /-- The verifier display is injective: equal Nat displays come from equal

```

```

0091 orbit positions. -/
0092 theorem toNat_inj {a b : DistinctionNat} (h : a.toNat = b.toNat) :
0093   a = b := by
0094   have := congrArg DistinctionNat.ofNat h
0095   rwa [ofNat_toNat, ofNat_toNat] at this
0096
0097 /-! ## Cancellation -/
0098
0099 /-- K4.5. Left cancellation for orbit addition. -/
0100 theorem add_left_cancel {a b c : DistinctionNat}
0101   (h : a + b = a + c) : b = c := by
0102   apply toNat_inj
0103   have h' : (a + b).toNat = (a + c).toNat := by rw [h]
0104   rw [toNat_add, toNat_add] at h'
0105   exact Nat.add_left_cancel h'
0106
0107 /-- K4.5. Right cancellation for orbit addition. -/
0108 theorem add_right_cancel {a b c : DistinctionNat}
0109   (h : a + c = b + c) : a = b := by
0110   apply add_left_cancel (a := c)
0111   rw [add_comm c a, add_comm c b]
0112   exact h
0113
0114 /-! ## Multiplication on the 6-orbit -/
0115
0116 /-- K4.7. Multiplication of orbit positions: nested repetition. -/
0117 def mul : DistinctionNat → DistinctionNat → DistinctionNat
0118   | _, zero => zero
0119   | a, succ b => mul a b + a
0120
0121 instance : Mul DistinctionNat := {mul}
0122
0123 theorem mul_def (a b : DistinctionNat) :
0124   a * b = mul a b := rfl
0125
0126 theorem mul_zero_eq (a : DistinctionNat) :
0127   a * zero = zero := rfl
0128
0129 theorem mul_succ_eq (a b : DistinctionNat) :
0130   a * succ b = a * b + a := rfl
0131
0132 theorem zero_mul_eq (a : DistinctionNat) :
0133   zero * a = zero := by
0134   induction a with
0135   | zero => rfl
0136   | succ n ih =>
0137     show zero * n + zero = zero
0138     rw [add_zero_eq, ih]
0139
0140 theorem succ_mul_eq (a b : DistinctionNat) :
0141   succ a * b = a * b + b := by
0142   induction b with
0143   | zero =>
0144     rw [mul_zero_eq, mul_zero_eq, add_zero_eq]
0145   | succ n ih =>
0146     show succ a * n + succ a = (a * n + a) + succ n
0147     rw [ih, add_succ_eq, add_succ_eq]
0148     congr 1
0149     rw [add_assoc, add_assoc, add_comm a n]
0150
0151 theorem mul_comm (a b : DistinctionNat) :
0152   a * b = b * a := by
0153   induction a with
0154   | zero =>
0155     rw [zero_mul_eq, mul_zero_eq]
0156   | succ n ih =>
0157     rw [succ_mul_eq, mul_succ_eq, ih]
0158
0159 /-- K4.7. The verifier display of orbit multiplication matches Lean Nat. -/
0160 theorem toNat_mul (a b : DistinctionNat) :
0161   (a * b).toNat = a.toNat * b.toNat := by
0162   induction b with
0163   | zero =>
0164     show (a * zero).toNat = a.toNat * zero.toNat
0165     rw [mul_zero_eq, toNat_zero]
0166     omega
0167   | succ n ih =>
0168     show (a * n + a).toNat = a.toNat * (succ n).toNat
0169     rw [toNat_add, toNat_succ, ih, Nat.mul_succ]
0170
0171 /-- K4.7. Product of nonzero orbit positions is nonzero. -/
0172 theorem mul_ne_zero {a b : DistinctionNat}
0173   (ha : a ≠ zero) (hb : b ≠ zero) :
0174   a * b ≠ zero := by
0175   intro h
0176   have hn : (a * b).toNat = zero.toNat := by rw [h]
0177   rw [toNat_mul, toNat_zero] at hn
0178   rcases Nat.mul_eq_zero.mp hn with hzero | hzero
0179   · have : a = zero := by
0180     apply toNat_inj
0181     rw [toNat_zero]
0182     exact hzero
0183     exact ha this
0184   · have : b = zero := by
0185     apply toNat_inj
0186     rw [toNat_zero]
0187     exact hzero
0188     exact hb this
0189
0190 end DistinctionNat
0191

```

```
0192 end PrimitiveRecognitionCalculus
0193 end Foundation
0194 end IndisputableMonolith
```

7. IntegerRational.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/IntegerRational.lean · Lines: 1289

```

0001 /-
0002   PrimitiveRecognitionCalculus/IntegerRational.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Spec anchors:
0008     K4.6, K4.7, K4.8, K4.9, K4.10, A5
0009
0010   Quotient-native PRC integers and rationals. The equivalence relations are
0011   internal:
0012
0013     - PRCInt: signed orbits identified by balanced orbit length,
0014       a.pos + b.neg = b.pos + a.neg.
0015     - PRCRat: ratio orbits identified by cross-multiplication of the
0016       signed numerator by the denominator,
0017       a.num · b.den ~ b.num · a.den (balanced equality of SignedOrbits).
0018
0019   The maps to verifier `Z` and `Q` are conservative displays, not
0020   definitional.
0021 -/
0022
0023 import MathLib
0024 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Orbit
0025 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitArithmetic
0026
0027 namespace IndisputableMonolith
0028 namespace Foundation
0029 namespace PrimitiveRecognitionCalculus
0030
0031 namespace DistinctionNat
0032
0033 /-! ## Internal comparison support for signed-orbit order -/
0034
0035 /-- Truncated subtraction on finite 6-orbit positions. -/
0036 def truncatedSub : DistinctionNat → DistinctionNat → DistinctionNat
0037 | a, zero => a
0038 | zero, succ _ => zero
0039 | succ a, succ b => truncatedSub a b
0040
0041 /-- Boolean `≤` on finite 6-orbit positions, by structural recursion only. -/
0042 def leq : DistinctionNat → DistinctionNat → Bool
0043 | zero, _ => true
0044 | succ _, zero => false
0045 | succ a, succ b => leq a b
0046
0047 /-- Absolute difference of two finite 6-orbit positions. -/
0048 def absDiff (a b : DistinctionNat) : DistinctionNat :=
0049   truncatedSub a b + truncatedSub b a
0050
0051 /-- Verifier display of internal truncated subtraction. -/
0052 theorem toNat_truncatedSub (a b : DistinctionNat) :
0053   (truncatedSub a b).toNat = a.toNat - b.toNat := by
0054   induction a generalizing b with
0055   | zero =>
0056     cases b with
0057     | zero => rfl
0058     | succ b => simp [truncatedSub]
0059   | succ a ih =>
0060     cases b with
0061     | zero => rfl
0062     | succ b =>
0063       simp [truncatedSub, ih]
0064
0065 /-- Internal Boolean order agrees with the verifier `Nat` order. -/
0066 theorem leq_eq_true_iff (a b : DistinctionNat) :
0067   leq a b = true ↔ a.toNat ≤ b.toNat := by
0068   induction a generalizing b with
0069   | zero =>
0070     cases b with
0071     | zero => simp [leq]
0072     | succ b => simp [leq]
0073   | succ a ih =>
0074     cases b with
0075     | zero => simp [leq]
0076     | succ b =>
0077       simp [leq, ih]
0078
0079 /-- Internal Boolean order is false exactly when verifier order is reversed. -/
0080 theorem leq_eq_false_iff (a b : DistinctionNat) :
0081   leq a b = false ↔ b.toNat < a.toNat := by
0082   rw [← Bool.not_eq_true, leq_eq_true_iff]
0083   omega
0084
0085 /-- Verifier display of internal absolute difference. -/
0086 theorem toNat_absDiff (a b : DistinctionNat) :
0087   (absDiff a b).toNat =
0088     Int.natAbs ((a.toNat : ℤ) - (b.toNat : ℤ)) := by
0089   unfold absDiff
0090   rw [toNat_add, toNat_truncatedSub, toNat_truncatedSub]

```

```

0091 by_cases h : b.toNat ≤ a.toNat
0092 · have hzero : b.toNat - a.toNat = 0 := Nat.sub_eq_zero_of_le h
0093   rw [hzero, Nat.add_zero]
0094   have hnonneg : 0 ≤ (a.toNat : Z) - (b.toNat : Z) := by
0095     omega
0096   have hcast : ((Int.natAbs ((a.toNat : Z) - (b.toNat : Z))) : Z) =
0097     (a.toNat : Z) - (b.toNat : Z) := by
0098     rw [Int.natAbs_of_nonneg hnonneg]
0099   apply Nat.cast_injective (R := Z)
0100   rw [hcast]
0101   omega
0102 · have hle : a.toNat ≤ b.toNat := by omega
0103   have hzero : a.toNat - b.toNat = 0 := Nat.sub_eq_zero_of_le hle
0104   rw [hzero, Nat.zero_add]
0105   have hnonpos : (a.toNat : Z) - (b.toNat : Z) ≤ 0 := by
0106     omega
0107   have hcast : ((Int.natAbs ((a.toNat : Z) - (b.toNat : Z))) : Z) =
0108     -((a.toNat : Z) - (b.toNat : Z)) := by
0109     have hnég_nonneg : 0 ≤ -((a.toNat : Z) - (b.toNat : Z)) := by
0110       omega
0111     have hnég_abs : ((Int.natAbs (-((a.toNat : Z) - (b.toNat : Z)))) : Z) =
0112       -((a.toNat : Z) - (b.toNat : Z)) := by
0113       rw [Int.natAbs_of_nonneg hnég_nonneg]
0114       rwa [Int.natAbs_neg] at hnég_abs
0115     apply Nat.cast_injective (R := Z)
0116     rw [hcast]
0117     omega
0118
0119 end DistinctionNat
0120
0121 /-! ## Signed orbits and the balanced-length equivalence -/
0122
0123 /-- K4.6. A signed orbit difference. Intended meaning: `pos - neg`. -/
0124 structure SignedOrbit where
0125   pos : DistinctionNat
0126   neg : DistinctionNat
0127   deriving DecidableEq, Repr
0128
0129 namespace SignedOrbit
0130
0131 /-- A verifier display of a signed orbit as an integer. -/
0132 def toInt (z : SignedOrbit) : Z :=
0133   (z.pos.toNat : Z) - (z.neg.toNat : Z)
0134
0135 @[simp] theorem toInt_mk (a b : DistinctionNat) :
0136   toInt (a, b) = (a.toNat : Z) - (b.toNat : Z) := by
0137   rfl
0138
0139 /-- The zero signed orbit. -/
0140 def zero : SignedOrbit :=
0141   (DistinctionNat.zero, DistinctionNat.zero)
0142
0143 @[simp] theorem zero_toInt :
0144   zero.toInt = 0 := by
0145   rfl
0146
0147 /-- The unit signed orbit. -/
0148 def one : SignedOrbit :=
0149   (DistinctionNat.succ DistinctionNat.zero, DistinctionNat.zero)
0150
0151 @[simp] theorem one_toInt :
0152   one.toInt = 1 := by
0153   rfl
0154
0155 /-- A nonnegative signed orbit built from a 0-orbit position. -/
0156 def ofOrbit (n : DistinctionNat) : SignedOrbit :=
0157   (n, DistinctionNat.zero)
0158
0159 @[simp] theorem ofOrbit_toInt (n : DistinctionNat) :
0160   (ofOrbit n).toInt = n.toNat := by
0161   show (n.toNat : Z) - 0 = n.toNat
0162   ring
0163
0164 /-- Pointwise addition of signed orbits. -/
0165 def add (a b : SignedOrbit) : SignedOrbit where
0166   pos := a.pos + b.pos
0167   neg := a.neg + b.neg
0168
0169 @[simp] theorem add_pos (a b : SignedOrbit) :
0170   (add a b).pos = a.pos + b.pos := rfl
0171
0172 @[simp] theorem add_neg (a b : SignedOrbit) :
0173   (add a b).neg = a.neg + b.neg := rfl
0174
0175 theorem add_toInt (a b : SignedOrbit) :
0176   (add a b).toInt = a.toInt + b.toInt := by
0177   show ((a.pos + b.pos).toNat : Z) - ((a.neg + b.neg).toNat : Z) =
0178     ((a.pos.toNat : Z) - (a.neg.toNat : Z)) +
0179     ((b.pos.toNat : Z) - (b.neg.toNat : Z))
0180   rw [DistinctionNat.toNat_add, DistinctionNat.toNat_add]
0181   push_cast
0182   ring
0183
0184 /-- Pointwise negation of signed orbits: swap pos and neg. Named
0185   `negate` rather than `neg` to avoid collision with the structure field. -/
0186 def negate (a : SignedOrbit) : SignedOrbit where
0187   pos := a.neg
0188   neg := a.pos
0189
0190 @[simp] theorem negate_pos (a : SignedOrbit) :
0191   (negate a).pos = a.neg := rfl

```

```

0192 @[simp] theorem negate_neg (a : SignedOrbit) :
0193   (negate a).neg = a.pos := rfl
0194
0195 theorem negate_toInt (a : SignedOrbit) :
0196   (negate a).toInt = -a.toInt := by
0197     show (a.neg.toNat : Z) - (a.pos.toNat : Z) =
0198       -((a.pos.toNat : Z) - (a.neg.toNat : Z))
0199   ring
0200
0201 /-- Signed orbit multiplication:
0202   `(p₁ - n₁) * (p₂ - n₂) = (p₁p₂ + n₁n₂) - (p₁n₂ + n₁p₂)` -/
0203 def mul (a b : SignedOrbit) : SignedOrbit where
0204   pos := a.pos * b.pos + a.neg * b.neg
0205   neg := a.pos * b.neg + a.neg * b.pos
0206
0207 @[simp] theorem mul_pos (a b : SignedOrbit) :
0208   (mul a b).pos = a.pos * b.pos + a.neg * b.neg := rfl
0209
0210 @[simp] theorem mul_neg (a b : SignedOrbit) :
0211   (mul a b).neg = a.pos * b.neg + a.neg * b.pos := rfl
0212
0213 theorem mul_toInt (a b : SignedOrbit) :
0214   (mul a b).toInt = a.toInt * b.toInt := by
0215     show ((a.pos * b.pos + a.neg * b.neg).toNat : Z) -
0216       ((a.pos * b.neg + a.neg * b.pos).toNat : Z) =
0217       ((a.pos.toNat : Z) - (a.neg.toNat : Z)) *
0218       ((b.pos.toNat : Z) - (b.neg.toNat : Z))
0219   rw [DistinctionNat.toNat_add, DistinctionNat.toNat_add,
0220       DistinctionNat.toNat_mul, DistinctionNat.toNat_mul,
0221       DistinctionNat.toNat_mul, DistinctionNat.toNat_mul]
0222   push_cast
0223   ring
0224
0225 /-- Subtraction on signed orbits via the negation. -/
0226 def sub (a b : SignedOrbit) : SignedOrbit :=
0227   add a (negate b)
0228
0229 theorem sub_toInt (a b : SignedOrbit) :
0230   (sub a b).toInt = a.toInt - b.toInt := by
0231     show (add a (negate b)).toInt = a.toInt - b.toInt
0232   rw [add_toInt, negate_toInt]
0233   ring
0234
0235 /-- Scale a signed orbit by a (positive-only) orbit position. -/
0236 def scaleByNat (z : SignedOrbit) (d : DistinctionNat) : SignedOrbit where
0237   pos := z.pos * d
0238   neg := z.neg * d
0239
0240 @[simp] theorem scaleByNat_pos (z : SignedOrbit) (d : DistinctionNat) :
0241   (z.scaleByNat d).pos = z.pos * d := rfl
0242
0243 @[simp] theorem scaleByNat_neg (z : SignedOrbit) (d : DistinctionNat) :
0244   (z.scaleByNat d).neg = z.neg * d := rfl
0245
0246 theorem scaleByNat_toInt (z : SignedOrbit) (d : DistinctionNat) :
0247   (z.scaleByNat d).toInt = z.toInt * (d.toNat : Z) := by
0248     show ((z.pos * d).toNat : Z) - ((z.neg * d).toNat : Z) =
0249       ((z.pos.toNat : Z) - (z.neg.toNat : Z)) * (d.toNat : Z)
0250   rw [DistinctionNat.toNat_mul, DistinctionNat.toNat_mul]
0251   push_cast
0252   ring
0253
0254 /-! ### K4.9. Balanced-length equivalence -/
0255
0256 /-- K4.9. Two signed orbits are equivalent when their orbit lengths balance:
0257   `a.pos + b.neg = b.pos + a.neg`. This is the internal PRC integer relation,
0258   defined entirely on 0-orbit positions. -/
0259 def balanced (a b : SignedOrbit) : Prop :=
0260   a.pos + b.neg = b.pos + a.neg
0261
0262 instance instDecidableBalanced (a b : SignedOrbit) :
0263   Decidable (balanced a b) := by
0264     unfold balanced
0265     unfold balanced
0266     infer_instance
0267
0268 /-- K4.9. Characterization of balanced length by Nat-level addition. -/
0269 theorem balanced_iff_toNat_eq (a b : SignedOrbit) :
0270   balanced a b ↔
0271     a.pos.toNat + b.neg.toNat = b.pos.toNat + a.neg.toNat := by
0272     unfold balanced
0273     constructor
0274     · intro h
0275       have := congrArg DistinctionNat.toNat h
0276       rwa [DistinctionNat.toNat_add, DistinctionNat.toNat_add] at this
0277     · intro h
0278       apply DistinctionNat.toNat_inj
0279       rw [DistinctionNat.toNat_add, DistinctionNat.toNat_add]
0280       exact h
0281
0282 /-- K4.9. The balanced characterization agrees with the verifier integer
0283   display. This is the bridge from the internal PRC relation to the
0284   conservative 'Z' view. -/
0285 theorem balanced_iff_toInt_eq (a b : SignedOrbit) :
0286   balanced a b ↔ a.toInt = b.toInt := by
0287     rw [balanced_iff_toNat_eq]
0288     unfold SignedOrbit.toInt
0289     constructor
0290     · intro h
0291       have : ((a.pos.toNat + b.neg.toNat : ℕ) : Z) =
0292         ((b.pos.toNat + a.neg.toNat : ℕ) : Z) := by exact_mod_cast h

```



```

0293   push_cast at this
0294   linarith
0295   · intro h
0296   have : (a.pos.toNat : Z) + (b.neg.toNat : Z) =
0297     (b.pos.toNat : Z) + (a.neg.toNat : Z) := by linarith
0298   exact_mod_cast this
0299
0300 theorem balanced_refl (a : SignedOrbit) : balanced a a := by
0301   unfold balanced
0302   rw [DistinctionNat.add_comm]
0303
0304 theorem balanced_symm {a b : SignedOrbit} (h : balanced a b) :
0305   balanced b a := by
0306   unfold balanced at *
0307   exact h.symm
0308
0309 theorem balanced_trans {a b c : SignedOrbit}
0310   (hab : balanced a b) (hbc : balanced b c) : balanced a c := by
0311   rw [balanced_iff_toNat_eq] at hab hbc
0312   omega
0313
0314 theorem balanced_equivalence : Equivalence balanced := {
0315   refl := balanced_refl
0316   symm := balanced_symm
0317   trans := balanced_trans
0318 }
0319
0320 /-! ### K4.13. Signed-orbit order and absolute value -/
0321
0322 /-- Internal nonnegativity: a signed orbit balances with a positive orbit. -/
0323 def nonneg (z : SignedOrbit) : Prop :=
0324   ∃ k : DistinctionNat, SignedOrbit.balanced z (SignedOrbit.ofOrbit k)
0325
0326 /-- Computable nonnegative flag from structural comparison of the two sides. -/
0327 def nonnegFlag (z : SignedOrbit) : Bool :=
0328   DistinctionNat.leq z.neg z.pos
0329
0330 /-- Strict negativity as failure of the structural nonnegative flag. -/
0331 def negativeFlag (z : SignedOrbit) : Bool :=
0332   !z.nonnegFlag
0333
0334 /-- Internal signed-orbit order: `a ≤ b` when `b - a` is nonnegative. -/
0335 def le (a b : SignedOrbit) : Prop :=
0336   nonneg (SignedOrbit.sub b a)
0337
0338 /-- Internal strict order: nonnegative difference with nonzero difference. -/
0339 def lt (a b : SignedOrbit) : Prop :=
0340   le a b ∧ ¬ SignedOrbit.balanced a b
0341
0342 /-- Absolute value of a signed orbit as an orbit position. -/
0343 def abs (z : SignedOrbit) : DistinctionNat :=
0344   DistinctionNat.absDiff z.pos z.neg
0345
0346 theorem nonnegFlag_eq_true_iff (z : SignedOrbit) :
0347   z.nonnegFlag = true ↔ 0 ≤ z.toInt := by
0348   unfold nonnegFlag SignedOrbit.toInt
0349   rw [DistinctionNat.leq_eq_true_iff]
0350   omega
0351
0352 theorem nonnegFlag_eq_false_iff (z : SignedOrbit) :
0353   z.nonnegFlag = false ↔ z.toInt < 0 := by
0354   rw [! Bool.not_eq_true, nonnegFlag_eq_true_iff]
0355   omega
0356
0357 /-- Internal nonnegativity agrees with the verifier integer display. -/
0358 theorem nonneg_iff_toInt_nonneg (z : SignedOrbit) :
0359   nonneg z ↔ 0 ≤ z.toInt := by
0360   constructor
0361   · intro h
0362     rcases h with (k, hk)
0363     have hdisplay := (SignedOrbit.balanced_iff_toInt_eq z (SignedOrbit.ofOrbit k)).mp hk
0364     rw [SignedOrbit.ofOrbit_toInt] at hdisplay
0365     omega
0366   · intro hz
0367     refine (DistinctionNat.ofNat z.toInt.toNat, ?)
0368     rw [SignedOrbit.balanced_iff_toInt_eq, SignedOrbit.ofOrbit_toInt,
0369       DistinctionNat.toNat_ofNat]
0370     omega
0371
0372 /-- The structural nonnegative flag is equivalent to internal nonnegativity. -/
0373 theorem nonnegFlag_eq_true_iff_nonneg (z : SignedOrbit) :
0374   z.nonnegFlag = true ↔ nonneg z := by
0375   rw [nonnegFlag_eq_true_iff, nonneg_iff_toInt_nonneg]
0376
0377 theorem negativeFlag_eq_true_iff_toInt_neg (z : SignedOrbit) :
0378   z.negativeFlag = true ↔ z.toInt < 0 := by
0379   unfold negativeFlag
0380   by_cases h : z.nonnegFlag = true
0381   · rw [h]
0382     simp
0383     rw [nonnegFlag_eq_true_iff] at h
0384     omega
0385   · have hf : z.nonnegFlag = false := by
0386     cases hflag : z.nonnegFlag with
0387     | false => rfl
0388     | true =>
0389       ex falso
0390       exact h hflag
0391   rw [hf]
0392   simp
0393   exact (nonnegFlag_eq_false_iff z).mp hf

```

```

0394
0395 /-- Verifier display of internal absolute value. -/
0396 theorem abs_toNat (z : SignedOrbit) :
0397   z.abs.toNat = Int.natAbs z.toInt := by
0398   unfold abs SignedOrbit.toInt
0399   exact DistinctionNat.toNat_absDiff z.pos z.neg
0400
0401 theorem abs_eq_zero_iff_toInt_eq_zero (z : SignedOrbit) :
0402   z.abs = DistinctionNat.zero ↔ z.toInt = 0 := by
0403   constructor
0404   · intro h
0405     have hnatz : z.abs.toNat = 0 := by
0406       rw [h, DistinctionNat.toNat_zero]
0407     rw [abs_toNat] at hnatz
0408     exact Int.natAbs_eq_zero.mp hnatz
0409   · intro h
0410     apply DistinctionNat.toNat_inj
0411     rw [abs_toNat, h, Int.natAbs_zero, DistinctionNat.toNat_zero]
0412
0413 theorem abs_ne_zero_of_toInt_ne_zero {z : SignedOrbit}
0414   (h : z.toInt ≠ 0) :
0415   z.abs ≠ DistinctionNat.zero := by
0416   intro hz
0417   exact h ((abs_eq_zero_iff_toInt_eq_zero z).mp hz)
0418
0419 theorem abs_ne_zero_of_not_balanced_zero {z : SignedOrbit}
0420   (h : ¬ SignedOrbit.balanced z SignedOrbit.zero) :
0421   z.abs ≠ DistinctionNat.zero := by
0422   apply abs_ne_zero_of_toInt_ne_zero
0423   intro hz
0424   exact h ((SignedOrbit.balanced_iff_toInt_eq z SignedOrbit.zero).mpr (by
0425     rw [hz, SignedOrbit.zero_toInt]))
0426
0427 theorem le_iff_toInt_le (a b : SignedOrbit) :
0428   le a b ↔ a.toInt ≤ b.toInt := by
0429   unfold le
0430   rw [nonneg_iff_toInt_nonneg, SignedOrbit.sub_toInt]
0431   omega
0432
0433 theorem lt_iff_toInt_lt (a b : SignedOrbit) :
0434   lt a b ↔ a.toInt < b.toInt := by
0435   unfold lt
0436   rw [le_iff_toInt_le, SignedOrbit.balanced_iff_toInt_eq]
0437   omega
0438
0439 end SignedOrbit
0440
0441 /-- K4.9. Signed-orbit equivalence is the balanced-length relation. -/
0442 def signedOrbitEquiv (a b : SignedOrbit) : Prop :=
0443   SignedOrbit.balanced a b
0444
0445 theorem signedOrbitEquiv_equivalence :
0446   Equivalence signedOrbitEquiv :=
0447   SignedOrbit.balanced_equivalence
0448
0449 theorem signedOrbitEquiv_iff_toInt_eq (a b : SignedOrbit) :
0450   signedOrbitEquiv a b ↔ a.toInt = b.toInt :=
0451   SignedOrbit.balanced_iff_toInt_eq a b
0452
0453 /-- K4.8. Setoid for quotient-native PRC integers. -/
0454 def signedOrbitSetoid : Setoid SignedOrbit where
0455   r := signedOrbitEquiv
0456   iseqv := signedOrbitEquiv_equivalence
0457
0458 /-- K4.8. PRC integers as signed-orbit quotient classes. The quotient is
0459 taken by the internal balanced-length relation; the verifier display into
0460 `Z` is a downstream theorem. -/
0461 def PRCInt : Type :=
0462   Quot signedOrbitSetoid
0463
0464 namespace PRCInt
0465
0466 /-- K4.8. Constructor from a signed orbit display. -/
0467 def mk (z : SignedOrbit) : PRCInt :=
0468   Quot.mk signedOrbitSetoid z
0469
0470 /-- K4.8/A5. Conservative verifier display of a PRC integer as `Z`. -/
0471 def toInt : PRCInt → ℤ :=
0472   Quot.lift SignedOrbit.toInt (by
0473     intro a b h
0474     exact (signedOrbitEquiv_iff_toInt_eq a b).mp h)
0475
0476 @[simp] theorem toInt_mk (z : SignedOrbit) :
0477   toInt (mk z) = z.toInt := by
0478   rfl
0479
0480 /-- K4.8. The zero PRC integer. -/
0481 def zero : PRCInt :=
0482   mk SignedOrbit.zero
0483
0484 @[simp] theorem zero_toInt :
0485   zero.toInt = 0 := by
0486   rfl
0487
0488 /-- K4.8. The unit PRC integer. -/
0489 def one : PRCInt :=
0490   mk SignedOrbit.one
0491
0492 @[simp] theorem one_toInt :
0493   one.toInt = 1 := by
0494   rfl

```

```

0495
0496 /-- K4.8. Equal balanced signed orbits determine equal PRC integers. -/
0497 theorem mk_eq_mk_of_balanced {a b : SignedOrbit}
0498   (h : SignedOrbit.balanced a b) :
0499     mk a = mk b :=
0500     Quot.sound h
0501
0502 /-! ### PRCInt operations -/
0503
0504 /-- Addition respects balanced equivalence. -/
0505 private theorem add_respects_balanced {a₁ a₂ b₁ b₂ : SignedOrbit}
0506   (ha : SignedOrbit.balanced a₁ a₂) (hb : SignedOrbit.balanced b₁ b₂) :
0507     SignedOrbit.balanced (SignedOrbit.add a₁ b₁) (SignedOrbit.add a₂ b₂) := by
0508     rw [SignedOrbit.balanced_iff_toInt_eq] at *
0509     rw [SignedOrbit.add_toInt, SignedOrbit.add_toInt, ha, hb]
0510
0511 /-- K4.8. Addition on PRC integers, lifted from signed-orbit addition. -/
0512 def add : PRCInt → PRCInt → PRCInt :=
0513   Quot.lift
0514     (fun a b => mk (SignedOrbit.add a b))
0515     (by
0516       intro a b₁ b₂ h
0517       apply Quot.sound
0518       exact add_respects_balanced (SignedOrbit.balanced_refl a) h)
0519     (by
0520       intro a₁ a₂ b h
0521       apply Quot.sound
0522       exact add_respects_balanced h (SignedOrbit.balanced_refl b))
0523
0524 @[simp] theorem add_mk (a b : SignedOrbit) :
0525   add (mk a) (mk b) = mk (SignedOrbit.add a b) := by
0526     rfl
0527
0528 @[simp] theorem toInt_add (a b : PRCInt) :
0529   (add a b).toInt = a.toInt + b.toInt := by
0530     refine Quot.induction_on a (fun a => ?_)
0531     refine Quot.induction_on b (fun b => ?_)
0532     show (SignedOrbit.add a b).toInt = a.toInt + b.toInt
0533     exact SignedOrbit.add_toInt a b
0534
0535 /-- Negation respects balanced equivalence. -/
0536 private theorem negate_respects_balanced {a₁ a₂ : SignedOrbit}
0537   (h : SignedOrbit.balanced a₁ a₂) :
0538     SignedOrbit.balanced (SignedOrbit.negate a₁) (SignedOrbit.negate a₂) := by
0539     rw [SignedOrbit.balanced_iff_toInt_eq] at *
0540     rw [SignedOrbit.negate_toInt, SignedOrbit.negate_toInt, h]
0541
0542 /-- K4.8. Negation on PRC integers, lifted from signed-orbit swap. -/
0543 def negate : PRCInt → PRCInt :=
0544   Quot.lift
0545     (fun a => mk (SignedOrbit.negate a))
0546     (by
0547       intro a b h
0548       apply Quot.sound
0549       exact negate_respects_balanced h)
0550
0551 @[simp] theorem negate_mk (a : SignedOrbit) :
0552   negate (mk a) = mk (SignedOrbit.negate a) := by
0553     rfl
0554
0555 @[simp] theorem toInt_negate (a : PRCInt) :
0556   (negate a).toInt = -a.toInt := by
0557     refine Quot.induction_on a (fun a => ?_)
0558     show (SignedOrbit.negate a).toInt = -a.toInt
0559     exact SignedOrbit.negate_toInt a
0560
0561 /-- K4.8. The toInt display is injective: distinct PRC integers have
0562 distinct verifier displays. -/
0563 theorem toInt_injective : Function.Injective toInt := by
0564   intro a b h
0565   induction a using Quot.ind with
0566   | _ a =>
0567     induction b using Quot.ind with
0568     | _ b =>
0569       apply Quot.sound
0570       exact (signedOrbitEquiv_iff_toInt_eq a b).mpr h
0571
0572 /-- Multiplication respects balanced equivalence. -/
0573 private theorem mul_respects_balanced {a₁ a₂ b₁ b₂ : SignedOrbit}
0574   (ha : SignedOrbit.balanced a₁ a₂) (hb : SignedOrbit.balanced b₁ b₂) :
0575     SignedOrbit.balanced (SignedOrbit.mul a₁ b₁) (SignedOrbit.mul a₂ b₂) := by
0576     rw [SignedOrbit.balanced_iff_toInt_eq] at *
0577     rw [SignedOrbit.mul_toInt, SignedOrbit.mul_toInt, ha, hb]
0578
0579 /-- K4.8. Multiplication on PRC integers, lifted from signed-orbit
0580 multiplication. -/
0581 def mul : PRCInt → PRCInt → PRCInt :=
0582   Quot.lift
0583     (fun a b => mk (SignedOrbit.mul a b))
0584     (by
0585       intro a b₁ b₂ h
0586       apply Quot.sound
0587       exact mul_respects_balanced (SignedOrbit.balanced_refl a) h)
0588     (by
0589       intro a₁ a₂ b h
0590       apply Quot.sound
0591       exact mul_respects_balanced h (SignedOrbit.balanced_refl b))
0592
0593 @[simp] theorem mul_mk (a b : SignedOrbit) :
0594   mul (mk a) (mk b) = mk (SignedOrbit.mul a b) := by
0595     rfl

```

```

0596
0597 @[simp] theorem toInt_mul (a b : PRCInt) :
0598   (mul a b).toInt = a.toInt * b.toInt := by
0599   refine Quot.induction_on a (fun a => ?_)
0600   refine Quot.induction_on b (fun b => ?_)
0601   show (SignedOrbit.mul a b).toInt = a.toInt * b.toInt
0602   exact SignedOrbit.mul_toInt a b
0603
0604 /-- K4.8. Subtraction on PRC integers as add ◦ negate. -/
0605 def sub (a b : PRCInt) : PRCInt :=
0606   add a (negate b)
0607
0608 @[simp] theorem toInt_sub (a b : PRCInt) :
0609   (sub a b).toInt = a.toInt - b.toInt := by
0610   show (add a (negate b)).toInt = a.toInt - b.toInt
0611   rw [toInt_add, toInt_negate]
0612   ring
0613
0614 /-! ### Ring axioms on PRC integers -/
0615
0616 theorem add_comm (a b : PRCInt) : add a b = add b a := by
0617   apply toInt_injective
0618   simp [Int.add_comm]
0619
0620 theorem add_assoc (a b c : PRCInt) :
0621   add (add a b) c = add a (add b c) := by
0622   apply toInt_injective
0623   simp [Int.add_assoc]
0624
0625 theorem zero_add (a : PRCInt) : add zero a = a := by
0626   apply toInt_injective
0627   simp
0628
0629 theorem add_zero (a : PRCInt) : add a zero = a := by
0630   apply toInt_injective
0631   simp
0632
0633 theorem add_negate (a : PRCInt) : add a (negate a) = zero := by
0634   apply toInt_injective
0635   simp
0636
0637 theorem negate_add (a : PRCInt) : add (negate a) a = zero := by
0638   apply toInt_injective
0639   simp
0640
0641 theorem mul_comm (a b : PRCInt) : mul a b = mul b a := by
0642   apply toInt_injective
0643   simp [Int.mul_comm]
0644
0645 theorem mul_assoc (a b c : PRCInt) :
0646   mul (mul a b) c = mul a (mul b c) := by
0647   apply toInt_injective
0648   simp [Int.mul_assoc]
0649
0650 theorem one_mul (a : PRCInt) : mul one a = a := by
0651   apply toInt_injective
0652   simp
0653
0654 theorem mul_one (a : PRCInt) : mul a one = a := by
0655   apply toInt_injective
0656   simp
0657
0658 theorem zero_mul (a : PRCInt) : mul zero a = zero := by
0659   apply toInt_injective
0660   simp
0661
0662 theorem mul_zero (a : PRCInt) : mul a zero = zero := by
0663   apply toInt_injective
0664   simp
0665
0666 theorem left_distrib (a b c : PRCInt) :
0667   mul a (add b c) = add (mul a b) (mul a c) := by
0668   apply toInt_injective
0669   simp [Int.mul_add]
0670
0671 theorem right_distrib (a b c : PRCInt) :
0672   mul (add a b) c = add (mul a c) (mul b c) := by
0673   apply toInt_injective
0674   simp [Int.add_mul]
0675
0676 /-! ### PRCInt is isomorphic to  $\mathbb{Z}$  -/
0677
0678 /-- Construct a PRC integer from a verifier Int by routing the positive
0679 and negative parts through the 5-orbit. -/
0680 def ofInt (n :  $\mathbb{Z}$ ) : PRCInt :=
0681   mk (DistinctionNat.ofNat n.toNat, DistinctionNat.ofNat (-n).toNat)
0682
0683 @[simp] theorem toInt_ofInt (n :  $\mathbb{Z}$ ) :
0684   (ofInt n).toInt = n := by
0685   show ((DistinctionNat.ofNat n.toNat).toNat :  $\mathbb{Z}$ ) -
0686     ((DistinctionNat.ofNat (-n).toNat).toNat :  $\mathbb{Z}$ ) = n
0687   rw [DistinctionNat.toNat_ofNat, DistinctionNat.toNat_ofNat]
0688   omega
0689
0690 @[simp] theorem ofInt_toInt (a : PRCInt) :
0691   ofInt a.toInt = a := by
0692   apply toInt_injective
0693   rw [toInt_ofInt]
0694
0695 /-- K4.8. The PRC integer surface is literally isomorphic to verifier ' $\mathbb{Z}$ '.
0696 The verifier ' $\mathbb{Z}$ ' is therefore not assumed; it is a downstream display the

```

```

0697 PRC quotient happens to reproduce. -/
0698 def equivInt : PRCInt = ℤ where
0699   toInt := toInt
0700   invFun := ofInt
0701   left_inv := ofInt_toInt
0702   right_inv := toInt_ofInt
0703
0704 @[simp] theorem ofInt_add (m n : ℤ) :
0705   ofInt (m + n) = add (ofInt m) (ofInt n) := by
0706   apply toInt_injective
0707   simp
0708
0709 @[simp] theorem ofInt_mul (m n : ℤ) :
0710   ofInt (m * n) = mul (ofInt m) (ofInt n) := by
0711   apply toInt_injective
0712   simp
0713
0714 @[simp] theorem ofInt_neg (n : ℤ) :
0715   ofInt (-n) = negate (ofInt n) := by
0716   apply toInt_injective
0717   simp
0718
0719 @[simp] theorem ofInt_zero : ofInt 0 = zero := by
0720   apply toInt_injective
0721   simp
0722
0723 @[simp] theorem ofInt_one : ofInt 1 = one := by
0724   apply toInt_injective
0725   simp
0726
0727 end PRCInt
0728
0729 /-! ## Operation instances on PRCInt -/
0730
0731 namespace PRCInt
0732
0733 instance instZero : Zero PRCInt := (zero)
0734 instance instOne : One PRCInt := (one)
0735 instance instAdd : Add PRCInt := (add)
0736 instance instMul : Mul PRCInt := (mul)
0737 instance instNeg : Neg PRCInt := (negate)
0738 instance instSub : Sub PRCInt := (sub)
0739
0740 @[simp] theorem add_eq (a b : PRCInt) : a + b = add a b := rfl
0741 @[simp] theorem mul_eq (a b : PRCInt) : a * b = mul a b := rfl
0742 @[simp] theorem neg_eq (a : PRCInt) : -a = negate a := rfl
0743 @[simp] theorem sub_eq (a b : PRCInt) : a - b = sub a b := rfl
0744 @[simp] theorem zero_eq : (0 : PRCInt) = zero := rfl
0745 @[simp] theorem one_eq : (1 : PRCInt) = one := rfl
0746
0747 /- K4.8. The toInt display is a ring homomorphism (additive). -/
0748 theorem toInt_add' (a b : PRCInt) :
0749   (a + b).toInt = a.toInt + b.toInt := by
0750   simp
0751
0752 /- K4.8. The toInt display is a ring homomorphism (multiplicative). -/
0753 theorem toInt_mul' (a b : PRCInt) :
0754   (a * b).toInt = a.toInt * b.toInt := by
0755   simp
0756
0757 /- K4.8. The toInt display preserves negation. -/
0758 theorem toInt_neg' (a : PRCInt) :
0759   (-a).toInt = -a.toInt := by
0760   simp
0761
0762 /- K4.8. The toInt display preserves zero. -/
0763 theorem toInt_zero' : (0 : PRCInt).toInt = 0 := by
0764   simp
0765
0766 /- K4.8. The toInt display preserves one. -/
0767 theorem toInt_one' : (1 : PRCInt).toInt = 1 := by
0768   simp
0769
0770 end PRCInt
0771
0772 /-! ## Ratio orbits and cross-multiplication -/
0773
0774 /- K4.7. A rational orbit display: integer numerator over nonzero orbit
0775 denominator. -/
0776 structure RatioOrbit where
0777   num : SignedOrbit
0778   den : DistinctionNat
0779   den_ne_zero : den ≠ DistinctionNat.zero
0780
0781 namespace RatioOrbit
0782
0783 /- The denominator's verifier Nat is nonzero. -/
0784 theorem den_toNat_ne_zero (q : RatioOrbit) :
0785   q.den.toNat ≠ 0 := by
0786   intro h
0787   have hden : DistinctionNat.ofNat q.den.toNat = DistinctionNat.ofNat 0 := by
0788     rw [h]
0789   rw [DistinctionNat.ofNat_toNat, DistinctionNat.ofNat_zero] at hden
0790   exact q.den_ne_zero hden
0791
0792 /- A verifier display of a ratio orbit as a rational number.
0793 Spec tag A5: this is a transport wrapper. The internal characterization
0794 is cross-multiplication. -/
0795 def toRat (q : RatioOrbit) : ℚ :=
0796   (q.num.toInt : ℚ) / (q.den.toNat : ℚ)
0797

```

```

0798 /-- The denominator used by `toRat` is nonzero in the verifier rationals. -/
0799 theorem den_cast_ne_zero (q : RatioOrbit) :
0800   (q.den.toNat : ℚ) ≠ 0 := by
0801     exact_mod_cast q.den.toNat_ne_zero
0802
0803 /-! ### Ratio-orbit arithmetic -/
0804
0805 /-- K4.11. Zero ratio orbit. -/
0806 def zero : RatioOrbit where
0807   num := SignedOrbit.zero
0808   den := DistinctionNat.succ DistinctionNat.zero
0809   den_ne_zero := by
0810     intro h
0811     exact DistinctionNat.zero_ne_succ DistinctionNat.zero h.symm
0812
0813 @[simp] theorem zero_toRat :
0814   zero.toRat = 0 := by
0815     unfold zero toRat
0816     simp
0817
0818 /-- K4.11. Unit ratio orbit. -/
0819 def one : RatioOrbit where
0820   num := SignedOrbit.one
0821   den := DistinctionNat.succ DistinctionNat.zero
0822   den_ne_zero := by
0823     intro h
0824     exact DistinctionNat.zero_ne_succ DistinctionNat.zero h.symm
0825
0826 @[simp] theorem one_toRat :
0827   one.toRat = 1 := by
0828     unfold one toRat
0829     simp
0830
0831 /-- K4.11. Addition of ratio orbits:
0832   `a/b + c/d = (ad + cb)/(bd)`. -/
0833 def add (a b : RatioOrbit) : RatioOrbit where
0834   num := SignedOrbit.add (a.num.scaleByNat b.den) (b.num.scaleByNat a.den)
0835   den := a.den * b.den
0836   den_ne_zero := DistinctionNat.mul_ne_zero a.den_ne_zero b.den_ne_zero
0837
0838 theorem add_toRat (a b : RatioOrbit) :
0839   (add a b).toRat = a.toRat + b.toRat := by
0840     unfold add_toRat
0841     rw [SignedOrbit.add_toInt, SignedOrbit.scaleByNat_toInt,
0842         SignedOrbit.scaleByNat_toInt, DistinctionNat.toNat_mul]
0843     have hA : (a.den.toNat : ℚ) ≠ 0 := a.den_cast_ne_zero
0844     have hB : (b.den.toNat : ℚ) ≠ 0 := b.den_cast_ne_zero
0845     field_simp [hA, hB]
0846     push_cast
0847     ring_nf
0848
0849 /-- K4.11. Negation of ratio orbits. -/
0850 def negate (a : RatioOrbit) : RatioOrbit where
0851   num := SignedOrbit.negate a.num
0852   den := a.den
0853   den_ne_zero := a.den_ne_zero
0854
0855 theorem negate_toRat (a : RatioOrbit) :
0856   (negate a).toRat = -a.toRat := by
0857     unfold negate toRat
0858     rw [SignedOrbit.negate_toInt]
0859     have hA : (a.den.toNat : ℚ) ≠ 0 := a.den_cast_ne_zero
0860     field_simp [hA]
0861     push_cast
0862     ring_nf
0863
0864 /-- K4.11. Subtraction of ratio orbits. -/
0865 def sub (a b : RatioOrbit) : RatioOrbit :=
0866   add a (negate b)
0867
0868 theorem sub_toRat (a b : RatioOrbit) :
0869   (sub a b).toRat = a.toRat - b.toRat := by
0870     unfold sub
0871     rw [add_toRat, negate_toRat]
0872     ring
0873
0874 /-- K4.11. Multiplication of ratio orbits:
0875   `a/b * c/d = (ac)/(bd)`. -/
0876 def mul (a b : RatioOrbit) : RatioOrbit where
0877   num := SignedOrbit.mul a.num b.num
0878   den := a.den * b.den
0879   den_ne_zero := DistinctionNat.mul_ne_zero a.den_ne_zero b.den_ne_zero
0880
0881 theorem mul_toRat (a b : RatioOrbit) :
0882   (mul a b).toRat = a.toRat * b.toRat := by
0883     unfold mul toRat
0884     rw [SignedOrbit.mul_toInt, DistinctionNat.toNat_mul]
0885     have hA : (a.den.toNat : ℚ) ≠ 0 := a.den_cast_ne_zero
0886     have hB : (b.den.toNat : ℚ) ≠ 0 := b.den_cast_ne_zero
0887     field_simp [hA, hB]
0888     push_cast
0889     ring_nf
0890
0891 /-- K4.12. Reciprocal of a nonzero ratio orbit.
0892
0893 The numerator sign is selected by the structural signed-orbit comparison
0894 `nonnegFlag`, and the denominator is the internal signed-orbit absolute value.
0895 The verifier integer display is used only in the transport theorem below. -/
0896 def recipNonzero (a : RatioOrbit)
0897   (h : ¬ SignedOrbit.balanced a.num SignedOrbit.zero) : RatioOrbit where
0898   num :=

```

```

0899   if a.num.nonnegFlag then
0900     SignedOrbit.ofOrbit a.den
0901   else
0902     SignedOrbit.negate (SignedOrbit.ofOrbit a.den)
0903   den := a.num.abs
0904   den_ne_zero := SignedOrbit.abs_ne_zero_of_not_balanced_zero h
0905
0906 theorem recipNonzero_toRat (a : RatioOrbit)
0907   (h : ~ SignedOrbit.balanced a.num SignedOrbit.zero) :
0908   (recipNonzero a h).toRat = (a.toRat)-1 := by
0909   unfold recipNonzero toRat
0910   have hDen : (a.den.toNat : Q) ≠ 0 := a.den_cast_ne_zero
0911   have hNumInt : a.num.toInt ≠ 0 := by
0912     intro hz
0913     exact h ((SignedOrbit.balanced_iff_toInt_eq a.num SignedOrbit.zero).mpr (by
0914       rw [hz, SignedOrbit.zero_toInt]))
0915   have hNum : (a.num.toInt : Q) ≠ 0 := by exact_mod_cast hNumInt
0916   by_cases hflag : a.num.nonnegFlag = true
0917   · have hnonneg : 0 ≤ a.num.toInt :=
0918     (SignedOrbit.nonnegFlag_eq_true_iff a.num).mp hflag
0919     have hpos : 0 < a.num.toInt := by omega
0920     simp [hflag, SignedOrbit.ofOrbit_toInt, SignedOrbit.abs_toNat]
0921     have habs : |(a.num.toInt : Q)| = (a.num.toInt : Q) := by
0922       exact abs_of_pos (by exact_mod_cast hpos)
0923     rw [habs]
0924   · have hflagFalse : a.num.nonnegFlag = false := by
0925     cases hflag' : a.num.nonnegFlag with
0926     | false => rfl
0927     | true =>
0928       ex falso
0929     exact hflag hflag'
0930   have hneg : a.num.toInt < 0 :=
0931     (SignedOrbit.nonnegFlag_eq_false_iff a.num).mp hflagFalse
0932   simp [hflagFalse, SignedOrbit.ofOrbit_toInt, SignedOrbit.negate_toInt,
0933     SignedOrbit.abs_toNat]
0934   have habs : |(a.num.toInt : Q)| = -(a.num.toInt : Q) := by
0935     exact abs_of_neg (by exact_mod_cast hneg)
0936   rw [habs]
0937   field_simp [hDen, hNum]
0938
0939 /-- K4.12. Total reciprocal of ratio orbits, sending zero to zero as in `Q`. -/
0940 def recip (a : RatioOrbit) : RatioOrbit :=
0941   if h : SignedOrbit.balanced a.num SignedOrbit.zero then
0942     zero
0943   else
0944     recipNonzero a h
0945
0946 theorem recip_toRat (a : RatioOrbit) :
0947   (recip a).toRat = (a.toRat)-1 := by
0948   unfold recip
0949   by_cases h : SignedOrbit.balanced a.num SignedOrbit.zero
0950   · have hzero := (SignedOrbit.balanced_iff_toInt_eq a.num SignedOrbit.zero).mp h
0951     simp [h, hzero, toRat, zero]
0952   · simp [h, recipNonzero_toRat]
0953
0954 /-! ### K4.10. Cross-multiplication equivalence -/
0955
0956 /-- K4.10. Two ratio orbits are equivalent under cross-multiplication when
0957   `a.num · b.den` balances `b.num · a.den` as signed orbits. This is the
0958   internal PRC rational relation, defined entirely on 0-orbit positions. -/
0959 def crossEq (a b : RatioOrbit) : Prop :=
0960   SignedOrbit.balanced (a.num.scaleByNat b.den) (b.num.scaleByNat a.den)
0961
0962 /-- K4.10. Cross-multiplication agrees with rational equality of the
0963   verifier displays. -/
0964 theorem crossEq_iff_toRat_eq (a b : RatioOrbit) :
0965   crossEq a b ↔ a.toRat = b.toRat := by
0966   unfold crossEq toRat
0967   rw [SignedOrbit.balanced_iff_toInt_eq]
0968   rw [SignedOrbit.scaleByNat_toInt, SignedOrbit.scaleByNat_toInt]
0969   have hA : (a.den.toNat : Q) ≠ 0 := a.den_cast_ne_zero
0970   have hB : (b.den.toNat : Q) ≠ 0 := b.den_cast_ne_zero
0971   constructor
0972   · intro h
0973     field_simp
0974     have hQ : (a.num.toInt : Q) * (b.den.toNat : Q) =
0975       (b.num.toInt : Q) * (a.den.toNat : Q) := by exact_mod_cast h
0976     linarith
0977   · intro h
0978     have : (a.num.toInt : Q) * (b.den.toNat : Q) =
0979       (b.num.toInt : Q) * (a.den.toNat : Q) := by
0980       field_simp at h
0981       linarith
0982     have hZ : (a.num.toInt * b.den.toNat : Z) =
0983       (b.num.toInt * a.den.toNat : Z) := by exact_mod_cast this
0984     exact hZ
0985
0986 theorem crossEq_refl (a : RatioOrbit) : crossEq a a := by
0987   unfold crossEq
0988   exact SignedOrbit.balanced_refl _
0989
0990 theorem crossEq_symm {a b : RatioOrbit} (h : crossEq a b) : crossEq b a := by
0991   unfold crossEq at *
0992   exact SignedOrbit.balanced_symm h
0993
0994 theorem crossEq_trans {a b c : RatioOrbit}
0995   (hab : crossEq a b) (hbc : crossEq b c) : crossEq a c := by
0996   rw [crossEq_iff_toRat_eq] at hab hbc
0997   rw [hab, hbc]
0998
0999 theorem crossEq_equivalence : Equivalence crossEq := {

```

```

1000 refl := crossEq_refl
1001 symm := crossEq_symm
1002 trans := crossEq_trans
1003 }
1004
1005 end RatioOrbit
1006
1007 /-- K4.8. Ratio-orbit equivalence is cross-multiplication. -/
1008 def ratioOrbitEquiv (a b : RatioOrbit) : Prop :=
1009   RatioOrbit.crossEq a b
1010
1011 theorem ratioOrbitEquiv_equivalence :
1012   Equivalence ratioOrbitEquiv :=
1013     RatioOrbit.crossEq_equivalence
1014
1015 theorem ratioOrbitEquiv_iff_toRat_eq (a b : RatioOrbit) :
1016   ratioOrbitEquiv a b ↔ a.toRat = b.toRat :=
1017     RatioOrbit.crossEq_iff_toRat_eq a b
1018
1019 /-- K4.8. Setoid for quotient-native PRC rationals. -/
1020 def ratioOrbitSetoid : Setoid RatioOrbit where
1021   r := ratioOrbitEquiv
1022   iseqv := ratioOrbitEquiv_equivalence
1023
1024 /-- K4.8. PRC rationals as nonzero-denominator ratio-orbit quotient classes,
1025 identified by cross-multiplication of orbit-level numerator and denominator. -/
1026 def PRCRat : Type :=
1027   Quot ratioOrbitSetoid
1028
1029 namespace PRCRat
1030
1031 /-- K4.8. Constructor from a ratio orbit display. -/
1032 def mk (q : RatioOrbit) : PRCRat :=
1033   Quot.mk ratioOrbitSetoid q
1034
1035 /-- K4.8/A5. Conservative verifier display of a PRC rational as `Q`. -/
1036 def toRat : PRCRat → Q :=
1037   Quot.lift RatioOrbit.toRat (by
1038     intro a b h
1039     exact (ratioOrbitEquiv_iff_toRat_eq a b).mp h)
1040
1041 @[simp] theorem toRat_mk (q : RatioOrbit) :
1042   toRat (mk q) = q.toRat := by
1043   rfl
1044
1045 /-- K4.8. Equal cross-multiplied ratio orbits determine equal PRC rationals. -/
1046 theorem mk_eq_mk_of_crossEq {a b : RatioOrbit}
1047   (h : RatioOrbit.crossEq a b) :
1048   mk a = mk b :=
1049     Quot.sound h
1050
1051 /-- K4.8. The toRat display is injective on the quotient. -/
1052 theorem toRat_injective : Function.Injective toRat := by
1053   intro a b h
1054   induction a using Quot.ind with
1055   | a =>
1056     induction b using Quot.ind with
1057     | b =>
1058       apply Quot.sound
1059       exact (ratioOrbitEquiv_iff_toRat_eq a b).mpr h
1060
1061 /-! ### PRCRat operations -/
1062
1063 /-- K4.11. Zero PRC rational. -/
1064 def zero : PRCRat :=
1065   mk RatioOrbit.zero
1066
1067 @[simp] theorem zero_toRat :
1068   zero.toRat = 0 := by
1069   exact RatioOrbit.zero_toRat
1070
1071 /-- K4.11. Unit PRC rational. -/
1072 def one : PRCRat :=
1073   mk RatioOrbit.one
1074
1075 @[simp] theorem one_toRat :
1076   one.toRat = 1 := by
1077   exact RatioOrbit.one_toRat
1078
1079 private theorem add_respects_cross {a₁ a₂ b₁ b₂ : RatioOrbit}
1080   (ha : RatioOrbit.crossEq a₁ a₂) (hb : RatioOrbit.crossEq b₁ b₂) :
1081   RatioOrbit.crossEq (RatioOrbit.add a₁ b₁) (RatioOrbit.add a₂ b₂) := by
1082   rw [RatioOrbit.crossEq_iff_toRat_eq] at *
1083   rw [RatioOrbit.add_toRat, RatioOrbit.add_toRat, ha, hb]
1084
1085 /-- K4.11. Addition on PRC rationals, lifted from ratio-orbit addition. -/
1086 def add : PRCRat → PRCRat → PRCRat :=
1087   Quot.lift₂
1088     (fun a b => mk (RatioOrbit.add a b))
1089     (by
1090       intro a b₁ b₂ h
1091       apply Quot.sound
1092       exact add_respects_cross (RatioOrbit.crossEq_refl a) h)
1093     (by
1094       intro a₁ a₂ b h
1095       apply Quot.sound
1096       exact add_respects_cross h (RatioOrbit.crossEq_refl b))
1097
1098 @[simp] theorem add_mk (a b : RatioOrbit) :
1099   add (mk a) (mk b) = mk (RatioOrbit.add a b) := by
1100   rfl

```



```

1101
1102 @[simp] theorem toRat_add (a b : PRCRat) :
1103   (add a b).toRat = a.toRat + b.toRat := by
1104     refine Quot.induction_on a (fun a => ?_)
1105     refine Quot.induction_on b (fun b => ?_)
1106     show (RatioOrbit.add a b).toRat = a.toRat + b.toRat
1107     exact RatioOrbit.add_toRat a b
1108
1109 private theorem negate_respects_cross {a b : RatioOrbit}
1110   (h : RatioOrbit.crossEq a b) :
1111     RatioOrbit.crossEq (RatioOrbit.negate a) (RatioOrbit.negate b) := by
1112     rw [RatioOrbit.crossEq_iff_toRat_eq] at *
1113     rw [RatioOrbit.negate_toRat, RatioOrbit.negate_toRat, h]
1114
1115 /-- K4.11. Negation on PRC rationals. -/
1116 def negate : PRCRat → PRCRat :=
1117   Quot.lift
1118     (fun a => mk (RatioOrbit.negate a))
1119     (by
1120       intro a b h
1121       apply Quot.sound
1122       exact negate_respects_cross h)
1123
1124 @[simp] theorem negate_mk (a : RatioOrbit) :
1125   negate (mk a) = mk (RatioOrbit.negate a) := by
1126     rfl
1127
1128 @[simp] theorem toRat_negate (a : PRCRat) :
1129   (negate a).toRat = -a.toRat := by
1130     refine Quot.induction_on a (fun a => ?_)
1131     show (RatioOrbit.negate a).toRat = -a.toRat
1132     exact RatioOrbit.negate_toRat a
1133
1134 /-- K4.11. Subtraction on PRC rationals. -/
1135 def sub (a b : PRCRat) : PRCRat :=
1136   add a (negate b)
1137
1138 @[simp] theorem toRat_sub (a b : PRCRat) :
1139   (sub a b).toRat = a.toRat - b.toRat := by
1140     show (add a (negate b)).toRat = a.toRat - b.toRat
1141     rw [toRat_add, toRat_negate]
1142     ring
1143
1144 private theorem mul_respects_cross {a₁ a₂ b₁ b₂ : RatioOrbit}
1145   (ha : RatioOrbit.crossEq a₁ a₂) (hb : RatioOrbit.crossEq b₁ b₂) :
1146     RatioOrbit.crossEq (RatioOrbit.mul a₁ b₁) (RatioOrbit.mul a₂ b₂) := by
1147     rw [RatioOrbit.crossEq_iff_toRat_eq] at *
1148     rw [RatioOrbit.mul_toRat, RatioOrbit.mul_toRat, ha, hb]
1149
1150 /-- K4.11. Multiplication on PRC rationals, lifted from ratio-orbit multiplication. -/
1151 def mul : PRCRat → PRCRat → PRCRat :=
1152   Quot.lift₂
1153     (fun a b => mk (RatioOrbit.mul a b))
1154     (by
1155       intro a b₁ b₂ h
1156       apply Quot.sound
1157       exact mul_respects_cross (RatioOrbit.crossEq_refl a) h)
1158     (by
1159       intro a₁ a₂ b h
1160       apply Quot.sound
1161       exact mul_respects_cross h (RatioOrbit.crossEq_refl b))
1162
1163 @[simp] theorem mul_mk (a b : RatioOrbit) :
1164   mul (mk a) (mk b) = mk (RatioOrbit.mul a b) := by
1165     rfl
1166
1167 @[simp] theorem toRat_mul (a b : PRCRat) :
1168   (mul a b).toRat = a.toRat * b.toRat := by
1169     refine Quot.induction_on a (fun a => ?_)
1170     refine Quot.induction_on b (fun b => ?_)
1171     show (RatioOrbit.mul a b).toRat = a.toRat * b.toRat
1172     exact RatioOrbit.mul_toRat a b
1173
1174 private theorem recip_respects_cross {a b : RatioOrbit}
1175   (h : RatioOrbit.crossEq a b) :
1176     RatioOrbit.crossEq (RatioOrbit.recip a) (RatioOrbit.recip b) := by
1177     rw [RatioOrbit.crossEq_iff_toRat_eq] at *
1178     rw [RatioOrbit.recip_toRat, RatioOrbit.recip_toRat, h]
1179
1180 /-- K4.12. Total reciprocal on PRC rationals, lifted from ratio-orbit
1181 reciprocal and sending zero to zero. -/
1182 def recip : PRCRat → PRCRat :=
1183   Quot.lift
1184     (fun a => mk (RatioOrbit.recip a))
1185     (by
1186       intro a b h
1187       apply Quot.sound
1188       exact recip_respects_cross h)
1189
1190 @[simp] theorem recip_mk (a : RatioOrbit) :
1191   recip (mk a) = mk (RatioOrbit.recip a) := by
1192     rfl
1193
1194 @[simp] theorem toRat_recip (a : PRCRat) :
1195   (recip a).toRat = (a.toRat)⁻¹ := by
1196     refine Quot.induction_on a (fun a => ?_)
1197     show (RatioOrbit.recip a).toRat = (a.toRat)⁻¹
1198     exact RatioOrbit.recip_toRat a
1199
1200 /-! ### PRCRat field-style laws, proved via the injective display -/
1201

```

```

1202 theorem add_comm (a b : PRCRat) : add a b = add b a := by
1203   apply toRat_injective
1204   simp [Rat.add_comm]
1205
1206 theorem add_assoc (a b c : PRCRat) :
1207   add (add a b) c = add a (add b c) := by
1208   apply toRat_injective
1209   simp [Rat.add_assoc]
1210
1211 theorem zero_add (a : PRCRat) : add zero a = a := by
1212   apply toRat_injective
1213   simp
1214
1215 theorem add_zero (a : PRCRat) : add a zero = a := by
1216   apply toRat_injective
1217   simp
1218
1219 theorem add_negate (a : PRCRat) : add a (negate a) = zero := by
1220   apply toRat_injective
1221   simp
1222
1223 theorem mul_comm (a b : PRCRat) : mul a b = mul b a := by
1224   apply toRat_injective
1225   simp [Rat.mul_comm]
1226
1227 theorem mul_assoc (a b c : PRCRat) :
1228   mul (mul a b) c = mul a (mul b c) := by
1229   apply toRat_injective
1230   simp [Rat.mul_assoc]
1231
1232 theorem one_mul (a : PRCRat) : mul one a = a := by
1233   apply toRat_injective
1234   simp
1235
1236 theorem mul_one (a : PRCRat) : mul a one = a := by
1237   apply toRat_injective
1238   simp
1239
1240 theorem left_distrib (a b c : PRCRat) :
1241   mul a (add b c) = add (mul a b) (mul a c) := by
1242   apply toRat_injective
1243   simp [Rat.mul_add]
1244
1245 theorem mul_recip_cancel {a : PRCRat} (h : a.toRat ≠ 0) :
1246   mul a (recip a) = one := by
1247   apply toRat_injective
1248   rw [toRat_mul, toRat_recip, one_toRat]
1249   field_simp [h]
1250
1251 /-! ### Operation instances on PRCRat -/
1252
1253 instance instZero : Zero PRCRat := (zero)
1254 instance instOne : One PRCRat := (one)
1255 instance instAdd : Add PRCRat := (add)
1256 instance instMul : Mul PRCRat := (mul)
1257 instance instNeg : Neg PRCRat := (negate)
1258 instance instSub : Sub PRCRat := (sub)
1259 instance instInv : Inv PRCRat := (recip)
1260
1261 @[simp] theorem add_eq (a b : PRCRat) : a + b = add a b := rfl
1262 @[simp] theorem mul_eq (a b : PRCRat) : a * b = mul a b := rfl
1263 @[simp] theorem neg_eq (a : PRCRat) : -a = negate a := rfl
1264 @[simp] theorem sub_eq (a b : PRCRat) : a - b = sub a b := rfl
1265 @[simp] theorem inv_eq (a : PRCRat) : a⁻¹ = recip a := rfl
1266 @[simp] theorem zero_eq : (0 : PRCRat) = zero := rfl
1267 @[simp] theorem one_eq : (1 : PRCRat) = one := rfl
1268
1269 theorem toRat_add' (a b : PRCRat) :
1270   (a + b).toRat = a.toRat + b.toRat := by
1271   simp
1272
1273 theorem toRat_mul' (a b : PRCRat) :
1274   (a * b).toRat = a.toRat * b.toRat := by
1275   simp
1276
1277 theorem toRat_neg' (a : PRCRat) :
1278   (-a).toRat = -a.toRat := by
1279   simp
1280
1281 theorem toRat_inv' (a : PRCRat) :
1282   (a⁻¹).toRat = (a.toRat)⁻¹ := by
1283   simp
1284
1285 end PRCRat
1286
1287 end PrimitiveRecognitionCalculus
1288 end Foundation
1289 end IndisputableMonolith

```

8. IntegerOrder.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/IntegerOrder.lean · Lines: 67

```

0001 /-
0002   PrimitiveRecognitionCalculus/IntegerOrder.lean
0003
0004   Round-trip sources:
0005   6/PRC_Kernel_Spec_20260526.html
0006   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0007
0008   Spec anchors:
0009   Build Order step 1: internal signed-orbit order and absolute value.
0010
0011   This module is the named certificate surface for integer order. The core
0012   declarations live in `IntegerRational.lean` because `RatioOrbit.recipNonzero`
0013   needs `SignedOrbit.abs` and `SignedOrbit.nonnegFlag` without creating an
0014   import cycle.
0015 -/
0016
0017 import MathLib
0018 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerRational
0019
0020 namespace IndisputableMonolith
0021 namespace Foundation
0022 namespace PrimitiveRecognitionCalculus
0023
0024 /-- Step-1 certificate for internal signed-orbit order and absolute value. -/
0025 structure IntegerOrderCertificate : Prop where
0026   truncated_sub_display :
0027     ∀ a b : DistinctionNat,
0028       (DistinctionNat.truncatedSub a b).toNat = a.toNat - b.toNat
0029   leq_display :
0030     ∀ a b : DistinctionNat,
0031       DistinctionNat.leq a b = true ↔ a.toNat ≤ b.toNat
0032   absdiff_display :
0033     ∀ a b : DistinctionNat,
0034       (DistinctionNat.absDiff a b).toNat =
0035         Int.natAbs ((a.toNat : ℤ) - (b.toNat : ℤ))
0036   signed_nonneg_display :
0037     ∀ z : SignedOrbit, SignedOrbit.nonneg z ↔ 0 ≤ z.toInt
0038   signed_nonneg_flag_display :
0039     ∀ z : SignedOrbit, z.nonnegFlag = true ↔ 0 ≤ z.toInt
0040   signed_abs_display :
0041     ∀ z : SignedOrbit, z.abs.toNat = Int.natAbs z.toInt
0042   signed_le_display :
0043     ∀ a b : SignedOrbit, SignedOrbit.le a b ↔ a.toInt ≤ b.toInt
0044   signed_lt_display :
0045     ∀ a b : SignedOrbit, SignedOrbit.lt a b ↔ a.toInt < b.toInt
0046   abs_nonzero_internal :
0047     ∀ z : SignedOrbit,
0048       (¬ SignedOrbit.balanced z SignedOrbit.zero) →
0049         z.abs ≠ DistinctionNat.zero
0050
0051 /-- The internal signed-orbit order surface is closed. -/
0052 theorem integer_order_certificate : IntegerOrderCertificate where
0053   truncated_sub_display := DistinctionNat.toNat_truncatedSub
0054   leq_display := DistinctionNat.leq_eq_true_iff
0055   absdiff_display := DistinctionNat.toNat_absDiff
0056   signed_nonneg_display := SignedOrbit.nonneg_iff_toInt_nonneg
0057   signed_nonneg_flag_display := SignedOrbit.nonnegFlag_eq_true_iff
0058   signed_abs_display := SignedOrbit.abs_toNat
0059   signed_le_display := SignedOrbit.le_iff_toInt_le
0060   signed_lt_display := SignedOrbit.lt_iff_toInt_lt
0061   abs_nonzero_internal := by
0062     intro z h
0063     exact SignedOrbit.abs_ne_zero_of_not_balanced_zero h
0064
0065 end PrimitiveRecognitionCalculus
0066 end Foundation
0067 end IndisputableMonolith

```

9. OrbitDivisibility.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitDivisibility.lean · Lines: 283

```

0001 /-
0002   PrimitiveRecognitionCalculus/OrbitDivisibility.lean
0003
0004   Round-trip sources:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006   6/PRC_Structural_Brainstorm_20260527.html
0007
0008   Spec anchors:
0009   Build Order step 2: divisibility, units, factorization, and primality on
0010   orbit positions.
0011
0012   Strength: 6-only for definitions. Nat divisibility and Nat arithmetic appear
0013   only in verifier transport theorems.
0014 -/
0015
0016 import Mathlib
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitArithmetic
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022 namespace DistinctionNat
0023
0024 /-! ## Native divisibility on finite 6-orbit positions -/
0025
0026 /-- The multiplicative unit orbit position. -/
0027 def one : DistinctionNat :=
0028   succ zero
0029
0030 @[simp] theorem one_toNat :
0031   one.toNat = 1 := by
0032     rfl
0033
0034 theorem one_ne_zero :
0035   one ≠ zero := by
0036     intro h
0037     exact zero_ne_succ zero h.symm
0038
0039 theorem mul_one_eq (a : DistinctionNat) :
0040   a * one = a := by
0041     unfold one
0042     rw [mul_succ_eq, mul_zero_eq, zero_add_eq]
0043
0044 theorem one_mul_eq (a : DistinctionNat) :
0045   one * a = a := by
0046     rw [mul_comm, mul_one_eq]
0047
0048 theorem mul_assoc (a b c : DistinctionNat) :
0049   (a * b) * c = a * (b * c) := by
0050     apply toNat_inj
0051     rw [toNat_mul, toNat_mul, toNat_mul, toNat_mul]
0052     exact Nat.mul_assoc a.toNat b.toNat c.toNat
0053
0054 /-- Native orbit divisibility: `a` divides `b` when multiplying `a` by another
0055 orbit position yields `b`. -/
0056 def divides (a b : DistinctionNat) : Prop :=
0057   ∃ k : DistinctionNat, a * k = b
0058
0059 /-- Native unit predicate. In the finite 6-orbit, the only multiplicative unit
0060 is the one-step orbit. -/
0061 def unit (a : DistinctionNat) : Prop :=
0062   a = one
0063
0064 /-- Native nontrivial factorization. Both factors must be nonzero and non-unit. -/
0065 def nontrivialFactorization (n : DistinctionNat) : Prop :=
0066   ∃ a b : DistinctionNat,
0067     a ≠ zero ∧ b ≠ zero ∧ ¬ unit a ∧ ¬ unit b ∧ a * b = n
0068
0069 /-- Native prime orbit position: nonzero, non-unit, and with no nontrivial
0070 factorization. -/
0071 def primeOrbit (p : DistinctionNat) : Prop :=
0072   p ≠ zero ∧ ¬ unit p ∧ ¬ nontrivialFactorization p
0073
0074 theorem divides_refl (a : DistinctionNat) :
0075   divides a a := by
0076     exact (one, mul_one_eq a)
0077
0078 theorem divides_zero (a : DistinctionNat) :
0079   divides a zero := by
0080     exact (zero, mul_zero_eq a)
0081
0082 theorem one_divides (a : DistinctionNat) :
0083   divides one a := by
0084     exact (a, one_mul_eq a)
0085
0086 theorem divides_trans (a b c : DistinctionNat)
0087   (hab : divides a b) (hbc : divides b c) :
0088     divides a c := by
0089       rcases hab with (m, hm)
0090       rcases hbc with (n, hn)

```

```

0091   refine (m * n, ?_)
0092   rw [← mul_assoc, hm, hn]
0093
0094 /-- Divisibility is native, but it displays as Nat divisibility. -/
0095 theorem divides_iff_toNat_dvd (a b : DistinctionNat) :
0096   divides a b ↔ a.toNat | b.toNat := by
0097   constructor
0098   · intro h
0099     rcases h with (k, hk)
0100     refine (k.toNat, ?_)
0101     have hnNat := congrArg DistinctionNat.toNat hk
0102     rw [toNat_mul] at hnNat
0103     exact hnNat.symm
0104   · intro h
0105     rcases h with (k, hk)
0106     refine (ofNat k, ?_)
0107     apply toNat_inj
0108     rw [toNat_mul, toNat_ofNat]
0109     exact hk.symm
0110
0111 theorem unit_iff_toNat_eq_one (a : DistinctionNat) :
0112   unit a ↔ a.toNat = 1 := by
0113   constructor
0114   · intro h
0115     unfold unit at h
0116     rw [h, one_toNat]
0117   · intro h
0118     unfold unit
0119     apply toNat_inj
0120     rw [h, one_toNat]
0121
0122 private theorem ofNat_ne_zero_of_ne_zero {n : Nat} (h : n ≠ 0) :
0123   ofNat n ≠ zero := by
0124   intro hz
0125   have hnNat := congrArg DistinctionNat.toNat hz
0126   rw [toNat_ofNat, toNat_zero] at hnNat
0127   exact h hnNat
0128
0129 private theorem not_unit_ofNat_of_ne_one {n : Nat} (h : n ≠ 1) :
0130   ¬ unit (ofNat n) := by
0131   intro hu
0132   rw [unit_iff_toNat_eq_one] at hu
0133   rw [toNat_ofNat] at hu
0134   exact h hu
0135
0136 /-- Native nontrivial factorization displays as ordinary Nat nontrivial
0137 factorization. -/
0138 theorem nontrivialFactorization_iff_toNat (n : DistinctionNat) :
0139   nontrivialFactorization n ↔
0140     ∃ a b : Nat,
0141       a ≠ 0 ∧ b ≠ 0 ∧ a ≠ 1 ∧ b ≠ 1 ∧ a * b = n.toNat := by
0142   constructor
0143   · intro h
0144     rcases h with (a, b, ha0, hb0, ha1, hb1, hmul)
0145     refine (a.toNat, b.toNat, ?_, ?_, ?_, ?_, ?_)
0146   · intro hz
0147     have : a = zero := by
0148       apply toNat_inj
0149       rw [hz, toNat_zero]
0150     exact ha0 this
0151   · intro hz
0152     have : b = zero := by
0153       apply toNat_inj
0154       rw [hz, toNat_zero]
0155     exact hb0 this
0156   · intro h1
0157     apply ha1
0158     rw [unit_iff_toNat_eq_one]
0159     exact h1
0160   · intro h1
0161     apply hb1
0162     rw [unit_iff_toNat_eq_one]
0163     exact h1
0164   · have hnNat := congrArg DistinctionNat.toNat hmul
0165     rw [toNat_mul] at hnNat
0166     exact hnNat
0167   · intro h
0168     rcases h with (a, b, ha0, hb0, ha1, hb1, hmul)
0169     refine (ofNat a, ofNat b, ?_, ?_, ?_, ?_, ?_)
0170   · exact ofNat_ne_zero_of_ne_zero ha0
0171   · exact ofNat_ne_zero_of_ne_zero hb0
0172   · exact not_unit_ofNat_of_ne_one ha1
0173   · exact not_unit_ofNat_of_ne_one hb1
0174   · apply toNat_inj
0175     rw [toNat_mul, toNat_ofNat, toNat_ofNat, hmul]
0176
0177 /-- Native prime-orbit predicate displays as the Nat no-nontrivial-factor
0178 predicate, without defining primality by importing Nat prime theory. -/
0179 theorem primeOrbit_iff_toNat_no_nontrivial_factor (p : DistinctionNat) :
0180   primeOrbit p ↔
0181     p.toNat ≠ 0 ∧ p.toNat ≠ 1 ∧
0182     ¬ ∃ a b : Nat,
0183       a ≠ 0 ∧ b ≠ 0 ∧ a ≠ 1 ∧ b ≠ 1 ∧ a * b = p.toNat := by
0184   unfold primeOrbit
0185   rw [unit_iff_toNat_eq_one, nontrivialFactorization_iff_toNat]
0186   constructor
0187   · intro h
0188     rcases h with (hp0, hp1, hfac)
0189     refine (?, hp1, hfac)
0190   intro hz
0191   have : p = zero := by

```

```

0192   apply toNat_inj
0193   rw [hz, toNat_zero]
0194   exact hp0 this
0195   · intro h
0196     rcases h with (hp0, hp1, hfac)
0197     refine (?, hp1, hfac)
0198   intro hz
0199   exact hp0 (by rw [hz, toNat_zero])
0200
0201 /-- If an orbit is prime, every native factorization has a unit factor. -/
0202 theorem unit_or_unit_of_mul_eq_prime {a b p : DistinctionNat}
0203   (hp : primeOrbit p) (hmul : a * b = p) :
0204   unit a v unit b := by
0205   by_cases ha0 : a = zero
0206   · exfalso
0207     rcases hp with (hp0, _, _)
0208     apply hp0
0209     rw [← hmul, ha0, zero_mul_eq]
0210   · by_cases hb0 : b = zero
0211     · exfalso
0212       rcases hp with (hp0, _, _)
0213       apply hp0
0214       rw [← hmul, hb0, mul_zero_eq]
0215     · by_cases ha1 : unit a
0216       · exact Or.inl ha1
0217     · by_cases hb1 : unit b
0218       · exact Or.inr hb1
0219     · exfalso
0220       rcases hp with (_, _, hnf)
0221       exact hnf (a, b, ha0, hb0, ha1, hb1, hmul)
0222
0223 /-- Converse native factor theorem: if a nonzero non-unit orbit position has
0224 only unit factors, then it is a prime orbit. -/
0225 theorem primeOrbit_of_unit_or_unit
0226   {p : DistinctionNat}
0227   (hp0 : p ≠ zero)
0228   (hp1 : ¬ unit p)
0229   (hfac : ∀ a b : DistinctionNat, a * b = p → unit a v unit b) :
0230   primeOrbit p := by
0231   refine (hp0, hp1, ?_)
0232   intro hnon
0233   rcases hnon with (a, b, _ha0, _hb0, ha1, hb1, hmul)
0234   rcases hfac a b hmul with ha | hb
0235   · exact ha1 ha
0236   · exact hb1 hb
0237
0238 /-- Bundling certificate for the native divisibility surface. -/
0239 structure OrbitDivisibilityCertificate : Prop where
0240   divides_display :
0241     ∀ a b : DistinctionNat, divides a b ↔ a.toNat | b.toNat
0242   divides_reflexive :
0243     ∀ a : DistinctionNat, divides a a
0244   divides_transitive :
0245     ∀ {a b c : DistinctionNat}, divides a b → divides b c → divides a c
0246   one_divides_all :
0247     ∀ a : DistinctionNat, divides one a
0248   unit_display :
0249     ∀ a : DistinctionNat, unit a ↔ a.toNat = 1
0250   nontrivial_factorization_display :
0251     ∀ n : DistinctionNat,
0252       nontrivialFactorization n ↔
0253       ∃ a b : Nat,
0254         a ≠ 0 ∧ b ≠ 0 ∧ a ≠ 1 ∧ b ≠ 1 ∧ a * b = n.toNat
0255   prime_orbit_display :
0256     ∀ p : DistinctionNat,
0257       primeOrbit p ↔
0258       p.toNat ≠ 0 ∧ p.toNat ≠ 1 ∧
0259       ¬ ∃ a b : Nat,
0260         a ≠ 0 ∧ b ≠ 0 ∧ a ≠ 1 ∧ b ≠ 1 ∧ a * b = p.toNat
0261   prime_factor_property :
0262     ∀ {a b p : DistinctionNat},
0263       primeOrbit p → a * b = p → unit a v unit b
0264
0265 /-- The native orbit divisibility surface is closed. -/
0266 theorem orbit_divisibility_certificate : OrbitDivisibilityCertificate where
0267   divides_display := divides_iff_toNat_dvd
0268   divides_reflexive := divides_refl
0269   divides_transitive := by
0270     intro a b c hab hbc
0271     exact divides_trans hab hbc
0272   one_divides_all := one_divides
0273   unit_display := unit_iff_toNat_eq_one
0274   nontrivial_factorization_display := nontrivialFactorization_iff_toNat
0275   prime_orbit_display := primeOrbit_iff_toNat_no_nontrivial_factor
0276   prime_factor_property := by
0277     intro a b p hp hmul
0278     exact unit_or_unit_of_mul_eq_prime hp hmul
0279
0280 end DistinctionNat
0281 end PrimitiveRecognitionCalculus
0282 end Foundation
0283 end IndisputableMonolith

```

10. OrbitEuclidean.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/OrbitEuclidean.lean · Lines: 473

```

0001 /-
0002   PrimitiveRecognitionCalculus/OrbitEuclidean.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchors:
0008   Build Order step 3: Euclidean quotient/remainder, GCD, coprime, and
0009   rational normalization targets.
0010
0011   Strength: 6-only for definitions. Nat division, modulo, and gcd appear only
0012   in verifier transport theorems.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerRational
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitDivisibility
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022 namespace DistinctionNat
0023
0024 /-! ## Object-level quotient and remainder by repeated subtraction -/
0025
0026 /-- Fuelled quotient/remainder by repeated subtraction. The first argument is
0027 an orbit fuel, not verifier `Nat`. -/
0028 def divModFuel : DistinctionNat → DistinctionNat → DistinctionNat → DistinctionNat × DistinctionNat
0029 | zero, n, _ => (zero, n)
0030 | succ fuel, n, d =>
0031   if DistinctionNat.leq d n then
0032     let qr := divModFuel fuel (DistinctionNat.truncatedSub n d) d
0033     (succ qr.1, qr.2)
0034   else
0035     (zero, n)
0036
0037 /-- Euclidean quotient/remainder. The fuel `n` is enough when `d` is nonzero,
0038 because each successful subtraction lowers the dividend by at least one. -/
0039 def divMod (n d : DistinctionNat) (_hd : d ≠ zero) : DistinctionNat × DistinctionNat :=
0040   divModFuel n n d
0041
0042 /-- Object-level quotient. -/
0043 def quotient (n d : DistinctionNat) (hd : d ≠ zero) : DistinctionNat :=
0044   (divMod n d hd).1
0045
0046 /-- Object-level remainder. -/
0047 def remainder (n d : DistinctionNat) (hd : d ≠ zero) : DistinctionNat :=
0048   (divMod n d hd).2
0049
0050 private theorem divModFuel_toNat_aux (fuel n d : DistinctionNat)
0051   (hd : d.toNat ≠ 0)
0052   (hbound : n.toNat ≤ fuel.toNat) :
0053   let qr := divModFuel fuel n d
0054   qr.1.toNat = n.toNat / d.toNat ∧
0055   qr.2.toNat = n.toNat % d.toNat := by
0056   induction fuel generalizing n with
0057   | zero =>
0058     rw [toNat_zero] at hbound
0059     have hn0 : n.toNat = 0 := by omega
0060     simp [divModFuel, hn0]
0061   | succ fuel ih =>
0062     rw [toNat_succ] at hbound
0063     unfold divModFuel
0064     by_cases hleq : DistinctionNat.leq d n = true
0065     · have hle : d.toNat ≤ n.toNat :=
0066       (DistinctionNat.leq_eq_true_iff d n).mp hleq
0067       have hpos : 0 < d.toNat := by omega
0068       simp [hleq]
0069       have hbound' : (DistinctionNat.truncatedSub n d).toNat ≤ fuel.toNat := by
0070         rw [DistinctionNat.toNat_truncatedSub]
0071         omega
0072       have ih' := ih (DistinctionNat.truncatedSub n d) hbound'
0073       rcases ih' with (hq, hr)
0074       rw [hq, hr]
0075       constructor
0076       · rw [DistinctionNat.toNat_truncatedSub, Nat.div_eq_sub_div hpos hle]
0077       · rw [DistinctionNat.toNat_truncatedSub]
0078         exact (Nat.mod_eq_sub_mod hle).symm
0079     · have hlt : n.toNat < d.toNat := by
0080       have hf := (DistinctionNat.leq_eq_false_iff d n).mp (by
0081         cases h : DistinctionNat.leq d n with
0082         | false => rfl
0083         | true =>
0084           ex falso
0085           exact hleq h)
0086       exact hf
0087     simp [hleq]
0088     constructor
0089     · exact (Nat.div_eq_of_lt hlt).symm
0090     · exact (Nat.mod_eq_of_lt hlt).symm

```

```

0091
0092 /-- Euclidean quotient/remainder transports to verifier Nat division and
0093 modulus. -/
0094 theorem divMod_toNat (n d : DistinctionNat) (hd : d ≠ zero) :
0095   let qr := divMod n d hd
0096   qr.1.toNat = n.toNat / d.toNat ∧
0097   qr.2.toNat = n.toNat % d.toNat := by
0098   unfold divMod
0099   apply divModFuel_toNat_aux
0100   · intro hzero
0101     have : d = zero := by
0102       apply toNat_inj
0103       rw [hzero, toNat_zero]
0104     exact hd this
0105   · omega
0106
0107 theorem quotient_toNat (n d : DistinctionNat) (hd : d ≠ zero) :
0108   (quotient n d hd).toNat = n.toNat / d.toNat := by
0109   have h := divMod_toNat n d hd
0110   exact h.1
0111
0112 theorem remainder_toNat (n d : DistinctionNat) (hd : d ≠ zero) :
0113   (remainder n d hd).toNat = n.toNat % d.toNat := by
0114   have h := divMod_toNat n d hd
0115   exact h.2
0116
0117 theorem remainder_lt_divisor (n d : DistinctionNat) (hd : d ≠ zero) :
0118   (remainder n d hd).toNat < d.toNat := by
0119   rw [remainder_toNat]
0120   apply Nat.mod_lt
0121   exact Nat.pos_of_ne_zero (by
0122     intro hzero
0123     have : d = zero := by
0124       apply toNat_inj
0125       rw [hzero, toNat_zero]
0126     exact hd this)
0127
0128 /-! ## Object-level GCD by subtractive Euclidean descent -/
0129
0130 /-- Fuelled subtractive Euclidean GCD. -/
0131 def gcdFuel : DistinctionNat → DistinctionNat → DistinctionNat → DistinctionNat
0132 | zero, a, b => a + b
0133 | succ fuel, a, b =>
0134   if a = zero then
0135     b
0136   else if b = zero then
0137     a
0138   else if DistinctionNat.leq b a then
0139     gcdFuel fuel (DistinctionNat.truncatedSub a b) b
0140   else
0141     gcdFuel fuel a (DistinctionNat.truncatedSub b a)
0142
0143 /-- Object-level GCD by subtractive Euclidean descent. -/
0144 def gcd (a b : DistinctionNat) : DistinctionNat :=
0145   gcdFuel (a + b) a b
0146
0147 /-- Object-level coprimality. -/
0148 def coprime (a b : DistinctionNat) : Prop :=
0149   unit (gcd a b)
0150
0151 private theorem gcdFuel_toNat_aux (fuel a b : DistinctionNat)
0152   (hbound : a.toNat + b.toNat ≤ fuel.toNat) :
0153   (gcdFuel fuel a b).toNat = Nat.gcd a.toNat b.toNat := by
0154   induction fuel generalizing a b with
0155   | zero =>
0156     rw [toNat_zero] at hbound
0157     have ha0 : a.toNat = 0 := by omega
0158     have hb0 : b.toNat = 0 := by omega
0159     simp [gcdFuel, ha0, hb0, toNat_add]
0160   | succ fuel ih =>
0161     rw [toNat_succ] at hbound
0162     unfold gcdFuel
0163     by_cases ha : a = zero
0164     · simp [ha, Nat.gcd_zero_left]
0165     · by_cases hb : b = zero
0166     · simp [ha, hb, Nat.gcd_zero_right]
0167     · by_cases hleq : DistinctionNat.leq b a = true
0168     · have hle : b.toNat ≤ a.toNat :=
0169       (DistinctionNat.leq_eq_true_iff b a).mp hleq
0170       have hbpos : 0 < b.toNat := by
0171         have hbne : b.toNat ≠ 0 := by
0172           intro hzero
0173           have : b = zero := by
0174             apply toNat_inj
0175             rw [hzero, toNat_zero]
0176           exact hb this
0177         omega
0178       have hbound' :
0179         (DistinctionNat.truncatedSub a b).toNat + b.toNat ≤ fuel.toNat := by
0180         rw [DistinctionNat.toNat_truncatedSub]
0181         omega
0182       simp [ha, hb, hleq]
0183       rw [ih (DistinctionNat.truncatedSub a b) b hbound']
0184       rw [DistinctionNat.toNat_truncatedSub]
0185       exact Nat.gcd_sub_self_left hle
0186     · have hlt : a.toNat < b.toNat := by
0187       exact (DistinctionNat.leq_eq_false_iff b a).mp (by
0188         cases h : DistinctionNat.leq b a with
0189         | false => rfl
0190         | true =>
0191           exfalso

```



```

0192         exact hleq h)
0193     have hle : a.toNat ≤ b.toNat := by omega
0194     have hapos : 0 < a.toNat := by
0195       have hane : a.toNat ≠ 0 := by
0196         intro hzero
0197         have : a = zero := by
0198           apply toNat_inj
0199       rw [hzero, toNat_zero]
0200     exact ha this
0201     omega
0202     have hbound' :
0203       a.toNat + (DistinctionNat.truncatedSub b a).toNat ≤ fuel.toNat := by
0204       rw [DistinctionNat.toNat_truncatedSub]
0205       omega
0206     simp [ha, hb, hleq]
0207     rw [ih a (DistinctionNat.truncatedSub b a) hbound']
0208     rw [DistinctionNat.toNat_truncatedSub]
0209     exact Nat.gcd_sub_self_right hle
0210
0211 theorem gcd_toNat (a b : DistinctionNat) :
0212   (gcd a b).toNat = Nat.gcd a.toNat b.toNat := by
0213   unfold gcd
0214   apply gcdFuel_toNat_aux
0215   rw [toNat_add]
0216
0217 theorem coprime_iff_nat_coprime (a b : DistinctionNat) :
0218   coprime a b ↔ Nat.Coprime a.toNat b.toNat := by
0219   simp [coprime, gcd_toNat, unit_iff_toNat_eq_one]
0220
0221 /-- The native GCD divides the left input. -/
0222 theorem gcd_divides_left (a b : DistinctionNat) :
0223   divides (gcd a b) a := by
0224     rw [divides_iff_toNat_dvd, gcd_toNat]
0225     exact Nat.gcd_dvd_left a.toNat b.toNat
0226
0227 /-- The native GCD divides the right input. -/
0228 theorem gcd_divides_right (a b : DistinctionNat) :
0229   divides (gcd a b) b := by
0230     rw [divides_iff_toNat_dvd, gcd_toNat]
0231     exact Nat.gcd_dvd_right a.toNat b.toNat
0232
0233 theorem gcd_ne_zero_of_right_ne_zero (a b : DistinctionNat) (hb : b ≠ zero) :
0234   gcd a b ≠ zero := by
0235     intro h
0236     have hnata : (gcd a b).toNat = 0 := by
0237       rw [h, toNat_zero]
0238     rw [gcd_toNat] at hnata
0239     have hb0 : b.toNat = 0 := (Nat.gcd_eq_zero_iff.mp hnata).2
0240     apply hb
0241     apply toNat_inj
0242     rw [hb0, toNat_zero]
0243
0244 theorem quotient_mul_divisor_toNat_of_divides {n d : DistinctionNat}
0245   (hd : d ≠ zero) (hdiv : divides d n) :
0246   (quotient n d hd).toNat * d.toNat = n.toNat := by
0247     rw [quotient_toNat]
0248     exact Nat.div_mul_cancel ((divides_iff_toNat_dvd d n).mp hdiv)
0249
0250 theorem quotient_ne_zero_of_divides {n d : DistinctionNat}
0251   (hd : d ≠ zero) (hdiv : divides d n) (hn : n ≠ zero) :
0252   quotient n d hd ≠ zero := by
0253     intro hq
0254     have hqnata : (quotient n d hd).toNat = 0 := by
0255       rw [hq, toNat_zero]
0256     have hmul := quotient_mul_divisor_toNat_of_divides (n := n) (d := d) hd hdiv
0257     rw [hqnat, Nat.zero_mul] at hmul
0258     apply hn
0259     apply toNat_inj
0260     rw [hmul.symm, toNat_zero]
0261
0262 /-! ## Signed rational normalization by native orbit GCD -/
0263
0264 /-- Quotient a signed orbit by a nonzero orbit position, restoring the sign by
0265 the structural signed-orbit comparison. -/
0266 def signedQuotient (z : SignedOrbit) (d : DistinctionNat) (hd : d ≠ zero) :
0267   SignedOrbit :=
0268   let q := quotient z.abs d hd
0269   if z.nonnegFlag then
0270     SignedOrbit.ofOrbit q
0271   else
0272     SignedOrbit.negate (SignedOrbit.ofOrbit q)
0273
0274 theorem signedQuotient_abs_toNat (z : SignedOrbit)
0275   (d : DistinctionNat) (hd : d ≠ zero) :
0276   (signedQuotient z d hd).abs.toNat = z.abs.toNat / d.toNat := by
0277   unfold signedQuotient
0278   by_cases hflag : z.nonnegFlag = true
0279   · have hAbsQ :
0280     (SignedOrbit.ofOrbit (quotient z.abs d hd)).abs.toNat =
0281     (quotient z.abs d hd).toNat := by
0282     simp [SignedOrbit.abs_toNat, SignedOrbit.ofOrbit_toInt]
0283     simp [hflag, hAbsQ] using quotient_toNat z.abs d hd
0284   · have hflagFalse : z.nonnegFlag = false := by
0285     cases h : z.nonnegFlag with
0286     | false => rfl
0287     | true =>
0288       ex falso
0289       exact hflag h
0290   have hAbsQ :
0291     (SignedOrbit.negate (SignedOrbit.ofOrbit (quotient z.abs d hd))).abs.toNat =
0292     (quotient z.abs d hd).toNat := by

```

```

0293   simp [SignedOrbit.abs_toNat, SignedOrbit.ofOrbit_toInt,
0294         SignedOrbit.negate_toInt]
0295   simp [hflagFalse, hAbsQ] using quotient_toNat z.abs d hd
0296
0297 theorem signedQuotient_mul_divisor_toInt_of_divides
0298   (z : SignedOrbit) (d : DistinctionNat) (hd : d ≠ zero)
0299   (hdiv : divides d z.abs) :
0300   (signedQuotient z d hd).toInt * (d.toNat : ℤ) = z.toInt := by
0301   have hquotNat :
0302     (quotient z.abs d hd).toNat * d.toNat = z.abs.toNat :=
0303     quotient_mul_divisor_toNat_of_divides (n := z.abs) (d := d) hd hdiv
0304   have hquotInt :
0305     ((quotient z.abs d hd).toNat : ℤ) * (d.toNat : ℤ) =
0306     (z.abs.toNat : ℤ) := by
0307     exact_mod_cast hquotNat
0308   unfold signedQuotient
0309   by_cases hflag : z.nonnegFlag = true
0310   · have hnonneg : 0 ≤ z.toInt :=
0311     (SignedOrbit.nonnegFlag_eq_true_iff z).mp hflag
0312     have habs : (z.abs.toNat : ℤ) = z.toInt := by
0313       rw [SignedOrbit.abs_toNat]
0314     exact Int.ofNat_natAbs_of_nonneg hnonneg
0315   simp [hflag, SignedOrbit.ofOrbit_toInt]
0316   rw [hquotInt, habs]
0317   · have hflagFalse : z.nonnegFlag = false := by
0318     cases h : z.nonnegFlag with
0319     | false => rfl
0320     | true =>
0321       ex falso
0322       exact hflag h
0323   have hneg : z.toInt < 0 :=
0324     (SignedOrbit.nonnegFlag_eq_false_iff z).mp hflagFalse
0325   have habs : (z.abs.toNat : ℤ) = -z.toInt := by
0326     rw [SignedOrbit.abs_toNat]
0327     exact Int.ofNat_natAbs_of_nonpos (le_of_lt hneg)
0328   simp [hflagFalse, SignedOrbit.ofOrbit_toInt, SignedOrbit.negate_toInt]
0329   rw [hquotInt, habs]
0330   ring
0331
0332 /-- Normalize a ratio orbit by dividing numerator magnitude and denominator by
0333 their native orbit GCD. The signed numerator orientation is restored by
0334 'SignedOrbit.nonnegFlag'. -/
0335 def normalizeRatio (q : RatioOrbit) : RatioOrbit :=
0336   let g := gcd q.num.abs q.den
0337   have hg : g ≠ zero := gcd_ne_zero_of_right_ne_zero q.num.abs q.den q.den_ne_zero
0338   {
0339     num := signedQuotient q.num g hg
0340     den := quotient q.den g hg
0341     den_ne_zero :=
0342       quotient_ne_zero_of_divides
0343       (n := q.den) (d := g) hg
0344       (gcd_divides_right q.num.abs q.den)
0345       q.den_ne_zero
0346   }
0347
0348 theorem normalizeRatio_num_mul_gcd_toInt (q : RatioOrbit) :
0349   (normalizeRatio q).num.toInt *
0350   ((gcd q.num.abs q.den).toNat : ℤ) = q.num.toInt := by
0351   unfold normalizeRatio
0352   exact signedQuotient_mul_divisor_toInt_of_divides
0353     q.num (gcd q.num.abs q.den)
0354     (gcd_ne_zero_of_right_ne_zero q.num.abs q.den q.den_ne_zero)
0355     (gcd_divides_left q.num.abs q.den)
0356
0357 theorem normalizeRatio_den_mul_gcd_toNat (q : RatioOrbit) :
0358   (normalizeRatio q).den.toNat *
0359   (gcd q.num.abs q.den).toNat = q.den.toNat := by
0360   unfold normalizeRatio
0361   exact quotient_mul_divisor_toNat_of_divides
0362     (n := q.den) (d := gcd q.num.abs q.den)
0363     (gcd_ne_zero_of_right_ne_zero q.num.abs q.den q.den_ne_zero)
0364     (gcd_divides_right q.num.abs q.den)
0365
0366 theorem normalizeRatio_toRat (q : RatioOrbit) :
0367   (normalizeRatio q).toRat = q.toRat := by
0368   unfold RatioOrbit.toRat
0369   have hnumZ := normalizeRatio_num_mul_gcd_toInt q
0370   have hdenN := normalizeRatio_den_mul_gcd_toNat q
0371   have hnumQ :
0372     ((normalizeRatio q).num.toInt : ℚ) *
0373     ((gcd q.num.abs q.den).toNat : ℚ) =
0374     (q.num.toInt : ℚ) := by
0375     exact_mod_cast hnumZ
0376   have hdenQ :
0377     ((normalizeRatio q).den.toNat : ℚ) *
0378     ((gcd q.num.abs q.den).toNat : ℚ) =
0379     (q.den.toNat : ℚ) := by
0380     exact_mod_cast hdenN
0381   have hNormDen : ((normalizeRatio q).den.toNat : ℚ) ≠ 0 :=
0382     (normalizeRatio q).den_cast_ne_zero
0383   have hDen : (q.den.toNat : ℚ) ≠ 0 := q.den_cast_ne_zero
0384   field_simp [hNormDen, hDen]
0385   calc
0386     ((normalizeRatio q).num.toInt : ℚ) * (q.den.toNat : ℚ)
0387     = ((normalizeRatio q).num.toInt : ℚ) *
0388       ((normalizeRatio q).den.toNat : ℚ) *
0389       ((gcd q.num.abs q.den).toNat : ℚ) := by
0390       rw [hdenQ]
0391   _ = (((normalizeRatio q).num.toInt : ℚ) *
0392         ((gcd q.num.abs q.den).toNat : ℚ)) *
0393         ((normalizeRatio q).den.toNat : ℚ) := by ring

```

```

0394   _ = (q.num.toInt : Q) * ((normalizeRatio q).den.toNat : Q) := by
0395     rw [hnumQ]
0396   _ = ((normalizeRatio q).den.toNat : Q) * (q.num.toInt : Q) := by ring
0397
0398 theorem normalizeRatio_crossEq (q : RatioOrbit) :
0399   RatioOrbit.crossEq q (normalizeRatio q) := by
0400   rw [RatioOrbit.crossEq_iff_toNat_eq]
0401   exact (normalizeRatio_toNat q).symm
0402
0403 theorem normalizeRatio_coprime (q : RatioOrbit) :
0404   coprime (normalizeRatio q).num.abs (normalizeRatio q).den := by
0405   rw [coprime_iff_nat_coprime]
0406   unfold normalizeRatio
0407   rw [signedQuotient_abs_toNat, quotient_toNat]
0408   have hgpos : 0 < (gcd q.num.abs q.den).toNat := by
0409     rw [gcd_toNat]
0410     apply Nat.gcd_pos_of_pos_right
0411     exact Nat.pos_of_ne_zero (by
0412       intro hzero
0413       apply q.den_ne_zero
0414       apply toNat_inj)
0415     rw [hzero, toNat_zero]
0416   have hgposNat : 0 < Nat.gcd q.num.abs.toNat q.den.toNat := by
0417     rw [← gcd_toNat]
0418     exact hgpos
0419     rw [gcd_toNat]
0420   exact Nat.coprime_div_gcd_div_gcd
0421   (m := q.num.abs.toNat) (n := q.den.toNat) hgposNat
0422
0423 /-- After signed division by orbit GCD, every `RatioOrbit` admits a balanced
0424 equivalent representative whose numerator absolute value is coprime to the
0425 denominator. -/
0426 def RatioNormalizationTarget : Prop :=
0427   ∀ q : RatioOrbit,
0428     ∃ q' : RatioOrbit,
0429       RatioOrbit.crossEq q q' ∧
0430       coprime q'.num.abs q'.den
0431
0432 theorem ratio_normalization_target : RatioNormalizationTarget := by
0433   intro q
0434   exact (normalizeRatio q, normalizeRatio_crossEq q, normalizeRatio_coprime q)
0435
0436 /-- Bundling certificate for the Euclidean orbit surface closed in this pass. -/
0437 structure OrbitEuclideanCertificate : Prop where
0438   divmod_display :
0439     ∀ (n d : DistinctionNat) (hd : d ≠ zero),
0440       let qr := divMod n d hd
0441       qr.1.toNat = n.toNat / d.toNat ∧
0442       qr.2.toNat = n.toNat % d.toNat
0443   quotient_display :
0444     ∀ (n d : DistinctionNat) (hd : d ≠ zero),
0445       (quotient n d hd).toNat = n.toNat / d.toNat
0446   remainder_display :
0447     ∀ (n d : DistinctionNat) (hd : d ≠ zero),
0448       (remainder n d hd).toNat = n.toNat % d.toNat
0449   remainder_bound :
0450     ∀ (n d : DistinctionNat) (hd : d ≠ zero),
0451       (remainder n d hd).toNat < d.toNat
0452   gcd_display :
0453     ∀ a b : DistinctionNat, (gcd a b).toNat = Nat.gcd a.toNat b.toNat
0454   coprime_display :
0455     ∀ a b : DistinctionNat, coprime a b ↔ Nat.Coprime a.toNat b.toNat
0456   ratio_normalization :
0457     RatioNormalizationTarget
0458
0459 /-- The closed δ-only Euclidean orbit surface, including signed-rational
0460 normalization by native orbit GCD. -/
0461 theorem orbit_euclidean_certificate : OrbitEuclideanCertificate where
0462   divmod_display := divMod_toNat
0463   quotient_display := quotient_toNat
0464   remainder_display := remainder_toNat
0465   remainder_bound := remainder_lt_divisor
0466   gcd_display := gcd_toNat
0467   coprime_display := coprime_iff_nat_coprime
0468   ratio_normalization := ratio_normalization_target
0469
0470 end DistinctionNat
0471 end PrimitiveRecognitionCalculus
0472 end Foundation
0473 end IndisputableMonolith

```

11. RationalField.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RationalField.lean · Lines: 284

```

0001 /-
0002   PrimitiveRecognitionCalculus/RationalField.lean
0003
0004   Round-trip source:
0005     6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008     Build Order step 4: package the PRC rational quotient as the reusable
0009     field-like substrate needed by PRC real completion and cost.
0010
0011     Strength: 6-only for the object operations and positivity predicate.
0012     Verifier `Q` appears only in display theorems and law proofs.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCost
0017
0018 namespace IndisputableMonolith
0019 namespace Foundation
0020 namespace PrimitiveRecognitionCalculus
0021
0022 namespace RatioOrbit
0023
0024 /-- PRC-native positivity for a ratio orbit: positive signed numerator over a
0025 nonzero orbit denominator. The denominator is an orbit position, so nonzero
0026 means positive in the verifier display but is not part of the object definition. -/
0027 def positive (q : RatioOrbit) : Prop :=
0028   SignedOrbit.nonneg q.num  $\wedge$  SignedOrbit.balanced q.num SignedOrbit.zero
0029
0030 theorem positive_iff_toRat_pos (q : RatioOrbit) :
0031   positive q  $\leftrightarrow$  0 < q.toRat := by
0032   unfold positive RatioOrbit.toRat
0033   have hdenNat : 0 < q.den.toNat := Nat.pos_of_ne_zero q.den.toNat_ne_zero
0034   have hdenQ : 0 < (q.den.toNat : Q) := by exact_mod_cast hdenNat
0035   constructor
0036   · intro h
0037     have hnum_nonneg : 0  $\leq$  q.num.toInt :=
0038       (SignedOrbit.nonneg_iff_toInt_nonneg q.num).mp h.1
0039     have hnum_ne : q.num.toInt  $\neq$  0 := by
0040       intro hz
0041       exact h.2 ((SignedOrbit.balanced_iff_toInt_eq q.num SignedOrbit.zero).mpr (by
0042         rw [hz, SignedOrbit.zero_toInt]))
0043     have hnum_pos : 0 < q.num.toInt := by omega
0044     have hnumQ : 0 < (q.num.toInt : Q) := by exact_mod_cast hnum_pos
0045     positivity
0046   · intro h
0047     have hnum_pos : 0 < q.num.toInt := by
0048       have hden_ne : (q.den.toNat : Q)  $\neq$  0 := q.den_cast_ne_zero
0049       have hmul : 0 < ((q.num.toInt : Q) / (q.den.toNat : Q)) * (q.den.toNat : Q) :=
0050         mul_pos h hdenQ
0051       have hnumQ : 0 < (q.num.toInt : Q) := by
0052         field_simp [hden_ne] at hmul
0053         exact hmul
0054       exact_mod_cast hnumQ
0055     constructor
0056     · exact (SignedOrbit.nonneg_iff_toInt_nonneg q.num).mpr (by omega)
0057     · intro hbal
0058       have hnum_zero : q.num.toInt = 0 := by
0059         have := (SignedOrbit.balanced_iff_toInt_eq q.num SignedOrbit.zero).mp hbal
0060         simpa using this
0061       omega
0062
0063 theorem positive_normalize {q : RatioOrbit}
0064   (h : positive q) : positive (DistinctionNat.normalizeRatio q) := by
0065   rw [positive_iff_toRat_pos, DistinctionNat.normalizeRatio_toRat]
0066   exact (positive_iff_toRat_pos q).mp h
0067
0068 theorem positive_not_zero {q : RatioOrbit}
0069   (h : positive q) : q.toRat  $\neq$  0 := by
0070   exact ne_of_gt ((positive_iff_toRat_pos q).mp h)
0071
0072 end RatioOrbit
0073
0074 namespace PRCRat
0075
0076 /-! ## Division and positive rationals -/
0077
0078 /-- Division on PRC rationals, defined from PRC multiplication and reciprocal. -/
0079 def div (a b : PRCRat) : PRCRat :=
0080   a * b-1
0081
0082 instance instDiv : Div PRCRat := (div)
0083
0084 @[simp] theorem div_eq (a b : PRCRat) : a / b = div a b := rfl
0085
0086 theorem toRat_div (a b : PRCRat) :
0087   (a / b).toRat = a.toRat / b.toRat := by
0088   unfold HDiv.hDiv instHDiv Div.div instDiv div
0089   rw [toRat_mul', toRat_inv']
0090   rfl

```

```

0091
0092 /-- A PRC rational is positive if it has a positive ratio-orbit representative. -/
0093 def positive (q : PRCRat) : Prop :=
0094   ∃ r : RatioOrbit, q = mk r ∧ RatioOrbit.positive r
0095
0096 theorem positive_iff_toRat_pos (q : PRCRat) :
0097   positive q ↔ 0 < q.toRat := by
0098   constructor
0099   · intro h
0100     rcases h with (r, rfl, hr)
0101     rw [toRat_mk]
0102     exact (RatioOrbit.positive_iff_toRat_pos r).mp hr
0103   · refine Quot.induction_on q ?_
0104     intro r hr
0105     exact (r, rfl, (RatioOrbit.positive_iff_toRat_pos r).mpr (by simpa using hr))
0106
0107 theorem positive_ne_zero {q : PRCRat}
0108   (h : positive q) : q.toRat ≠ 0 :=
0109   ne_of_gt ((positive_iff_toRat_pos q).mp h)
0110
0111 /-! ## Operator-form field laws -/
0112
0113 theorem add_assoc' (a b c : PRCRat) :
0114   (a + b) + c = a + (b + c) :=
0115   add_assoc a b c
0116
0117 theorem zero_add' (a : PRCRat) :
0118   0 + a = a :=
0119   zero_add a
0120
0121 theorem add_zero' (a : PRCRat) :
0122   a + 0 = a :=
0123   add_zero a
0124
0125 theorem add_left_neg' (a : PRCRat) :
0126   -a + a = 0 := by
0127   apply toRat_injective
0128   simp
0129
0130 theorem add_right_neg' (a : PRCRat) :
0131   a + -a = 0 :=
0132   add_negate a
0133
0134 theorem mul_assoc' (a b c : PRCRat) :
0135   (a * b) * c = a * (b * c) :=
0136   mul_assoc a b c
0137
0138 theorem one_mul' (a : PRCRat) :
0139   1 * a = a :=
0140   one_mul a
0141
0142 theorem mul_one' (a : PRCRat) :
0143   a * 1 = a :=
0144   mul_one a
0145
0146 theorem zero_mul' (a : PRCRat) :
0147   0 * a = 0 := by
0148   apply toRat_injective
0149   simp
0150
0151 theorem mul_zero' (a : PRCRat) :
0152   a * 0 = 0 := by
0153   apply toRat_injective
0154   simp
0155
0156 theorem right_distrib' (a b c : PRCRat) :
0157   (a + b) * c = a * c + b * c := by
0158   apply toRat_injective
0159   simp [Rat.add_mul]
0160
0161 theorem left_distrib' (a b c : PRCRat) :
0162   a * (b + c) = a * b + a * c :=
0163   left_distrib a b c
0164
0165 theorem zero_ne_one : (0 : PRCRat) ≠ 1 := by
0166   intro h
0167   have hrat := congrArg toRat h
0168   simp at hrat
0169
0170 theorem inv_zero : (0 : PRCRat)-1 = 0 := by
0171   apply toRat_injective
0172   simp
0173
0174 theorem inv_mul_cancel {a : PRCRat} (h : a.toRat ≠ 0) :
0175   a-1 * a = 1 := by
0176   apply toRat_injective
0177   rw [toRat_mul', toRat_inv']
0178   simp [h]
0179
0180 theorem div_mul_cancel {a b : PRCRat} (h : b.toRat ≠ 0) :
0181   (a / b) * b = a := by
0182   apply toRat_injective
0183   rw [toRat_mul', toRat_div]
0184   field_simp [h]
0185
0186 theorem mul_div_cancel {a b : PRCRat} (h : b.toRat ≠ 0) :
0187   a * b / b = a := by
0188   apply toRat_injective
0189   rw [toRat_div, toRat_mul']
0190   field_simp [h]
0191

```

```

0192 end PRCRat
0193
0194 namespace PRCJCost
0195
0196 /-- PRC J-cost lifted from ratio-orbit representatives to the rational quotient. -/
0197 def onPRCRat : PRCRat → PRCRat :=
0198   Quot.lift
0199     (fun q => PRCRat.mk (onRatioOrbit q))
0200     (by
0201       intro a b h
0202       apply PRCRat.toRat_injective
0203       rw [PRCRat.toRat_mk, PRCRat.toRat_mk, onRatioOrbit_toRat, onRatioOrbit_toRat]
0204       have hrat : a.toRat = b.toRat := (ratioOrbitEquiv_iff_toRat_eq a b).mp h
0205       rw [hrat])
0206
0207 @[simp] theorem onPRCRat_mk (q : RatioOrbit) :
0208   onPRCRat (PRCRat.mk q) = PRCRat.mk (onRatioOrbit q) := by
0209   rfl
0210
0211 theorem onPRCRat_toRat (q : PRCRat) :
0212   (onPRCRat q).toRat = (q.toRat + q.toRat⁻¹) / 2 - 1 := by
0213   refine Quot.induction_on q ?_
0214   intro r
0215   change (PRCRat.mk (onRatioOrbit r)).toRat =
0216     ((PRCRat.mk r).toRat + (PRCRat.mk r).toRat⁻¹) / 2 - 1
0217   rw [PRCRat.toRat_mk, PRCRat.toRat_mk, onRatioOrbit_toRat]
0218
0219 theorem onPRCRat_normalized_representative (q : RatioOrbit) :
0220   onPRCRat (PRCRat.mk q) =
0221     onPRCRat (PRCRat.mk (DistinctionNat.normalizeRatio q)) := by
0222   apply PRCRat.toRat_injective
0223   rw [onPRCRat_toRat, onPRCRat_toRat, PRCRat.toRat_mk, PRCRat.toRat_mk,
0224     DistinctionNat.normalizeRatio_toRat]
0225
0226 end PRCJCost
0227
0228 /-! ## Bundled certificate -/
0229
0230 structure RationalFieldCertificate : Prop where
0231   add_comm : ∀ a b : PRCRat, a + b = b + a
0232   add_assoc : ∀ a b c : PRCRat, (a + b) + c = a + (b + c)
0233   zero_add : ∀ a : PRCRat, 0 + a = a
0234   add_zero : ∀ a : PRCRat, a + 0 = a
0235   add_left_neg : ∀ a : PRCRat, -a + a = 0
0236   mul_comm : ∀ a b : PRCRat, a * b = b * a
0237   mul_assoc : ∀ a b c : PRCRat, (a * b) * c = a * (b * c)
0238   one_mul : ∀ a : PRCRat, 1 * a = a
0239   mul_one : ∀ a : PRCRat, a * 1 = a
0240   left_distrib : ∀ a b c : PRCRat, a * (b + c) = a * b + a * c
0241   right_distrib : ∀ a b c : PRCRat, (a + b) * c = a * c + b * c
0242   zero_ne_one : (0 : PRCRat) ≠ 1
0243   inv_zero : (0 : PRCRat)⁻¹ = 0
0244   mul_inv_cancel : ∀ a : PRCRat, a.toRat ≠ 0 → a * a⁻¹ = 1
0245   inv_mul_cancel : ∀ a : PRCRat, a.toRat ≠ 0 → a⁻¹ * a = 1
0246   div_display : ∀ a b : PRCRat, (a / b).toRat = a.toRat / b.toRat
0247   positive_display : ∀ q : PRCRat, PRCRat.positive q → 0 < q.toRat
0248   ratio_positive_display : ∀ q : RatioOrbit, RatioOrbit.positive q → 0 < q.toRat
0249   jcost_display :
0250     ∀ q : PRCRat, (PRCJCost.onPRCRat q).toRat = (q.toRat + q.toRat⁻¹) / 2 - 1
0251   jcost_normalized_representative :
0252     ∀ q : RatioOrbit,
0253       PRCJCost.onPRCRat (PRCRat.mk q) =
0254       PRCJCost.onPRCRat (PRCRat.mk (DistinctionNat.normalizeRatio q))
0255
0256 theorem rational_field_certificate : RationalFieldCertificate where
0257   add_comm := PRCRat.add_comm
0258   add_assoc := PRCRat.add_assoc'
0259   zero_add := PRCRat.zero_add'
0260   add_zero := PRCRat.add_zero'
0261   add_left_neg := PRCRat.add_left_neg'
0262   mul_comm := PRCRat.mul_comm
0263   mul_assoc := PRCRat.mul_assoc'
0264   one_mul := PRCRat.one_mul'
0265   mul_one := PRCRat.mul_one'
0266   left_distrib := PRCRat.left_distrib'
0267   right_distrib := PRCRat.right_distrib'
0268   zero_ne_one := PRCRat.zero_ne_one
0269   inv_zero := PRCRat.inv_zero
0270   mul_inv_cancel := by
0271     intro a h
0272     exact PRCRat.mul_recip_cancel h
0273   inv_mul_cancel := by
0274     intro a h
0275     exact PRCRat.inv_mul_cancel h
0276   div_display := PRCRat.toRat_div
0277   positive_display := PRCRat.positive_iff_toRat_pos
0278   ratio_positive_display := RatioOrbit.positive_iff_toRat_pos
0279   jcost_display := PRCJCost.onPRCRat_toRat
0280   jcost_normalized_representative := PRCJCost.onPRCRat_normalized_representative
0281
0282 end PrimitiveRecognitionCalculus
0283 end Foundation
0284 end IndisputableMonolith

```

12. PRCJCost.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCost.lean · Lines: 242

```

0001 /-
0002   PrimitiveRecognitionCalculus/PRCJCost.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006   6/Logic_Functional_Equation.tex
0007
0008   Spec anchors:
0009   Build Order steps 5-7: ratio cost surface, RCL surface, and the strongest
0010   currently provable bridge to the existing J-cost uniqueness theorem.
0011
0012   Strength: 6-only for the rational cost object. The bridge to `Cost.Jcost`
0013   and `law_of_logic_forces_jcost` is a classical-extension transport surface
0014   because it lives on continuous positive real ratios.
0015 -/
0016
0017 import MathLib
0018 import IndisputableMonolith.Cost.FunctionalEquation
0019 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitEuclidean
0020
0021 namespace IndisputableMonolith
0022 namespace Foundation
0023 namespace PrimitiveRecognitionCalculus
0024
0025 namespace PRCJCost
0026
0027 /-! ## A rational PRC cost object -/
0028
0029 /-- The two-step orbit position. -/
0030 def twoOrbit : DistinctionNat :=
0031   DistinctionNat.succ DistinctionNat.one
0032
0033 @[simp] theorem twoOrbit_toNat :
0034   twoOrbit.toNat = 2 := by
0035   rfl
0036
0037 /-- The ratio orbit `2`. -/
0038 def two : RatioOrbit where
0039   num := SignedOrbit.ofOrbit twoOrbit
0040   den := DistinctionNat.one
0041   den_ne_zero := DistinctionNat.one_ne_zero
0042
0043 @[simp] theorem two_toRat :
0044   two.toRat = 2 := by
0045   unfold two RatioOrbit.toRat
0046   simp [twoOrbit_toNat, SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat]
0047
0048 /-- The ratio orbit `1/2`. -/
0049 def half : RatioOrbit where
0050   num := SignedOrbit.ofOrbit DistinctionNat.one
0051   den := twoOrbit
0052   den_ne_zero := by
0053     intro h
0054     have hnat := congrArg DistinctionNat.toNat h
0055     rw [twoOrbit_toNat, DistinctionNat.toNat_zero] at hnat
0056     norm_num at hnat
0057
0058 @[simp] theorem half_toRat :
0059   half.toRat = (1 / 2 : ℚ) := by
0060   unfold half RatioOrbit.toRat
0061   simp [twoOrbit_toNat, SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat]
0062
0063 /-- PRC's rational J-cost object on a ratio orbit:
0064   `J(q) = ((q + q⁻¹) / 2) - 1`.
0065
0066   This is a ratio-orbit object. It is not the real analytic uniqueness theorem;
0067   that theorem is bridged below. -/
0068 def onRatioOrbit (q : RatioOrbit) : RatioOrbit :=
0069   RatioOrbit.sub (RatioOrbit.mul (RatioOrbit.add q (RatioOrbit.recip q)) half) RatioOrbit.one
0070
0071 theorem onRatioOrbit_toRat (q : RatioOrbit) :
0072   (onRatioOrbit q).toRat = (q.toRat + q.toRat⁻¹) / 2 - 1 := by
0073   unfold onRatioOrbit
0074   rw [RatioOrbit.sub_toRat, RatioOrbit.mul_toRat, RatioOrbit.add_toRat,
0075     RatioOrbit.recip_toRat, half_toRat, RatioOrbit.one_toRat]
0076   ring
0077
0078 /-- The PRC rational cost transports to the existing real `Cost.Jcost`
0079   formula on the verifier display. -/
0080 theorem onRatioOrbit_toReal_jcost (q : RatioOrbit) :
0081   ((onRatioOrbit q).toRat : ℝ) = Cost.Jcost ((q.toRat : ℚ) : ℝ) := by
0082   rw [onRatioOrbit_toRat]
0083   unfold Cost.Jcost
0084   rw [Rat.cast_sub, Rat.cast_div, Rat.cast_add, Rat.cast_inv]
0085   norm_num
0086
0087 /-- Reciprocal symmetry of the PRC rational cost. -/
0088 theorem reciprocal_symmetric (q : RatioOrbit) :
0089   RatioOrbit.crossEq (onRatioOrbit q) (onRatioOrbit (RatioOrbit.recip q)) := by
0090   rw [RatioOrbit.crossEq_iff_toRat_eq]

```

```

0091 rw [onRatioOrbit_toRat, onRatioOrbit_toRat, RatioOrbit.recip_toRat]
0092 by_cases hq : q.toRat = 0
0093 · simp [hq]
0094 · field_simp [hq]
0095 ring
0096
0097 /-- Normalizing a ratio representative by native orbit GCD preserves the PRC
0098 cost. -/
0099 theorem normalized_invariant (q : RatioOrbit) :
0100   RatioOrbit.crossEq (onRatioOrbit q)
0101   (onRatioOrbit (DistinctionNat.normalizeRatio q)) := by
0102   rw [RatioOrbit.crossEq_iff_toRat_eq]
0103   rw [onRatioOrbit_toRat, onRatioOrbit_toRat, DistinctionNat.normalizeRatio_toRat]
0104
0105 /-- Division of ratio orbits, defined from multiplication and reciprocal. -/
0106 def div (q r : RatioOrbit) : RatioOrbit :=
0107   RatioOrbit.mul q (RatioOrbit.recip r)
0108
0109 theorem div_toRat (q r : RatioOrbit) :
0110   (div q r).toRat = q.toRat / r.toRat := by
0111   unfold div
0112   rw [RatioOrbit.mul_toRat, RatioOrbit.recip_toRat]
0113   rfl
0114
0115 private def rclLHS (x y : RatioOrbit) : RatioOrbit :=
0116   RatioOrbit.add (onRatioOrbit (RatioOrbit.mul x y)) (onRatioOrbit (div x y))
0117
0118 private def rclRHS (x y : RatioOrbit) : RatioOrbit :=
0119   RatioOrbit.add
0120     (RatioOrbit.add
0121       (RatioOrbit.mul two (RatioOrbit.mul (onRatioOrbit x) (onRatioOrbit y)))
0122       (RatioOrbit.mul two (onRatioOrbit x)))
0123     (RatioOrbit.mul two (onRatioOrbit y))
0124
0125 /-- Canonical PRC J-cost satisfies the RCL algebraically on nonzero ratio
0126 orbits. This is the rational surface of the composition law, not the
0127 continuous-real uniqueness theorem. -/
0128 theorem canonical_rcl_surface {x y : RatioOrbit}
0129   (hx : x.toRat ≠ 0) (hy : y.toRat ≠ 0) :
0130   RatioOrbit.crossEq (rclLHS x y) (rclRHS x y) := by
0131   rw [RatioOrbit.crossEq_iff_toRat_eq]
0132   unfold rclLHS rclRHS
0133   rw [RatioOrbit.add_toRat, RatioOrbit.add_toRat, RatioOrbit.add_toRat,
0134       RatioOrbit.mul_toRat, RatioOrbit.mul_toRat, RatioOrbit.mul_toRat,
0135       RatioOrbit.mul_toRat,
0136       onRatioOrbit_toRat, onRatioOrbit_toRat, onRatioOrbit_toRat,
0137       onRatioOrbit_toRat, div_toRat]
0138   simp [two_toRat]
0139   rw [RatioOrbit.mul_toRat]
0140   have hxy : x.toRat * y.toRat ≠ 0 := mul_ne_zero hx hy
0141   field_simp [hx, hy, hxy]
0142   ring_nf
0143
0144 /-! ## Bridge to the existing continuous positive-real uniqueness theorem -/
0145
0146 /-- Hypotheses for the later PRC-native cost-classification theorem. The
0147 'two_calibrated' field rules out the identically-zero cost on the discrete
0148 rational surface, playing the role of the continuous theorem's unit
0149 log-curvature calibration until the internal real completion exists. -/
0150 structure PRCNativeCostHypotheses (F : RatioOrbit → RatioOrbit) : Prop where
0151   reciprocal :
0152     ∀ q, RatioOrbit.crossEq (F q) (F (RatioOrbit.recip q))
0153   normalized_invariant :
0154     ∀ q, RatioOrbit.crossEq (F q) (F (DistinctionNat.normalizeRatio q))
0155   canonical_rcl :
0156     ∀ {x y : RatioOrbit}, x.toRat ≠ 0 → y.toRat ≠ 0 →
0157       RatioOrbit.crossEq
0158         (RatioOrbit.add (F (RatioOrbit.mul x y)) (F (div x y)))
0159         (RatioOrbit.add
0160           (RatioOrbit.add
0161             (RatioOrbit.mul two (RatioOrbit.mul (F x) (F y)))
0162             (RatioOrbit.mul two (F x)))
0163           (RatioOrbit.mul two (F y)))
0164   unit_zero :
0165     F RatioOrbit.one = RatioOrbit.zero
0166   two_calibrated :
0167     RatioOrbit.crossEq (F two) (onRatioOrbit two)
0168
0169 /-- Exact missing native theorem for a later pass: classify every admissible
0170 PRC cost on normalized ratio orbits, then transport to the continuous
0171 positive-real theorem as a corollary rather than using the real theorem as
0172 the premise. -/
0173 def PRCNativeCostUniquenessTarget : Prop :=
0174   ∀ F : RatioOrbit → RatioOrbit,
0175     PRCNativeCostHypotheses F →
0176     ∀ q : RatioOrbit, RatioOrbit.crossEq (F q) (onRatioOrbit q)
0177
0178 /-- The real-domain uniqueness theorem currently used by PRC. The quantified
0179 'AczelSmoothnessPackage' keeps the Aczel regularity commitment explicit. -/
0180 theorem bridge_to_existing_jcost_uniqueness
0181   (F : ℝ → ℝ)
0182   (hAczel : Cost.FunctionalEquation.AczelSmoothnessPackage)
0183   (hRecip : Cost.FunctionalEquation.IsReciprocalCost F)
0184   (hNorm : Cost.FunctionalEquation.IsNormalized F)
0185   (hComp : Cost.FunctionalEquation.SatisfiesCompositionLaw F)
0186   (hCalib : Cost.FunctionalEquation.IsCalibrated F)
0187   (hCont : ContinuousOn F (Set.Ioi 0)) :
0188   ∀ x : ℝ, 0 < x → F x = Cost.Jcost x := by
0189   let _ : Cost.FunctionalEquation.AczelSmoothnessPackage := hAczel
0190   exact Cost.FunctionalEquation.law_of_logic_forces_jcost
0191   F hRecip hNorm hComp hCalib hCont

```



```

0192
0193 /-- Certificate for the PRC cost pass. -/
0194 structure PRCJCostCertificate : Prop where
0195   rational_formula :
0196     ∀ q : RatioOrbit,
0197       (onRatioOrbit q).toRat = (q.toRat + q.toRat⁻¹) / 2 - 1
0198   real_jcost_bridge :
0199     ∀ q : RatioOrbit,
0200       ((onRatioOrbit q).toRat : ℝ) = Cost.Jcost ((q.toRat : ℚ) : ℝ)
0201   reciprocal :
0202     ∀ q : RatioOrbit,
0203       RatioOrbit.crossEq (onRatioOrbit q) (onRatioOrbit (RatioOrbit.recip q))
0204   normalization :
0205     ∀ q : RatioOrbit,
0206       RatioOrbit.crossEq (onRatioOrbit q)
0207         (onRatioOrbit (DistinctionNat.normalizeRatio q))
0208   canonical_rcl :
0209     ∀ {x y : RatioOrbit}, x.toRat ≠ 0 → y.toRat ≠ 0 →
0210       RatioOrbit.crossEq (rclLHS x y) (rclRHS x y)
0211   existing_real_uniqueness :
0212     ∀ (F : ℝ → ℝ),
0213       Cost.FunctionalEquation.AczelSmoothnessPackage →
0214       Cost.FunctionalEquation.IsReciprocalCost F →
0215       Cost.FunctionalEquation.IsNormalized F →
0216       Cost.FunctionalEquation.SatisfiesCompositionLaw F →
0217       Cost.FunctionalEquation.IsCalibrated F →
0218       ContinuousOn F (Set.Ioi 0) →
0219       ∀ x : ℝ, 0 < x → F x = Cost.Jcost x
0220   native_uniqueness_target_named :
0221     PRCNativeCostUniquenessTarget = PRCNativeCostUniquenessTarget
0222
0223 /-- The PRC rational cost surface is closed through canonical RCL and bridges
0224 honestly to the existing continuous-real uniqueness theorem. -/
0225 theorem prc_jcost_certificate : PRCJCostCertificate where
0226   rational_formula := onRatioOrbit.toRat
0227   real_jcost_bridge := onRatioOrbit.toReal_jcost
0228   reciprocal := reciprocal_symmetric
0229   normalization := normalized_invariant
0230   canonical_rcl := by
0231     intro x y hx hy
0232     exact canonical_rcl_surface hx hy
0233   existing_real_uniqueness := by
0234     intro F hA hR hN hC hCal hCont x hx
0235     exact bridge_to_existing_jcost_uniqueness F hA hR hN hC hCal hCont x hx
0236   native_uniqueness_target_named := rfl
0237
0238 end PRCJCost
0239
0240 end PrimitiveRecognitionCalculus
0241 end Foundation
0242 end IndisputableMonolith

```

13. PRCJCostDistanceIncrementTriangle.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceIncrementTriangle.lean · Lines: 261

```

0001 /-
0002   PrimitiveRecognitionCalculus/PRCJCostDistanceIncrementTriangle.lean
0003
0004   Round-trip source:
0005     6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008     Build Order step 9c: prove the one-dimensional additive increment modulus
0009     for the displayed rational J-cost distance.
0010 -/
0011
0012 import MathLib
0013 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceVerifierTriangle
0014
0015 namespace IndisputableMonolith
0016 namespace Foundation
0017 namespace PrimitiveRecognitionCalculus
0018
0019 theorem PRCJCostDistanceIncrementDisplay_formula (t : Q) :
0020   PRCJCostDistanceIncrementDisplay t =
0021     ((t * t) * (t * t)) / (2 * (1 + t * t)) := by
0022   unfold PRCJCostDistanceIncrementDisplay PRCJCostDistanceRatDisplay
0023   have hg_pos : (0 : Q) < 1 + t * t := by
0024     nlinarith [mul_self_nonneg t]
0025   have hg_ne : (1 + t * t : Q) ≠ 0 := ne_of_gt hg_pos
0026   field_simp [hg_ne]
0027   ring
0028
0029 private theorem increment_display_lt_of_sq_lt
0030   {t eta eps : Q} (heta_pos : 0 < eta) (heta_lt_one : eta < 1)
0031   (hsq : t * t < eta) (heta_sq_half_lt_eps : eta * eta / 2 < eps) :
0032   PRCJCostDistanceIncrementDisplay t < eps := by
0033   rw [PRCJCostDistanceIncrementDisplay_formula]
0034   have hs_nonneg : (0 : Q) ≤ t * t := mul_self_nonneg t
0035   have hden_pos : (0 : Q) < 2 * (1 + t * t) := by positivity
0036   have hden_ne : (2 * (1 + t * t) : Q) ≠ 0 := ne_of_gt hden_pos
0037   have hs_lt_one : t * t < 1 := lt_trans hsq heta_lt_one
0038   have hnum_lt : (t * t) * (t * t) < eta * eta := by nlinarith
0039   have hfrac_le : ((t * t) * (t * t)) / (2 * (1 + t * t)) ≤
0040     ((t * t) * (t * t)) / 2 := by
0041     have hden_ge_two : (2 : Q) ≤ 2 * (1 + t * t) := by nlinarith
0042     have hnum_nonneg : (0 : Q) ≤ (t * t) * (t * t) :=
0043       mul_nonneg hs_nonneg hs_nonneg
0044     exact div_le_div_of_nonneg_left hnum_nonneg (by norm_num) hden_ge_two
0045   have hnum_half_lt : ((t * t) * (t * t)) / 2 < eta * eta / 2 := by
0046     nlinarith
0047   exact lt_of_le_of_lt hfrac_le (lt_trans hnum_half_lt heta_sq_half_lt_eps)
0048
0049 private theorem sq_lt_of_display_lt_delta
0050   {t eta delta : Q} (heta_pos : 0 < eta) (hdelta_pos : 0 < delta)
0051   (hdelta_le : delta ≤ eta * eta / (4 * (1 + eta)))
0052   (hsmall : PRCJCostDistanceIncrementDisplay t < delta) :
0053   t * t < eta := by
0054   by_contra hnot
0055   have hge : eta ≤ t * t := by nlinarith
0056   rw [PRCJCostDistanceIncrementDisplay_formula] at hsmall
0057   have hs_nonneg : (0 : Q) ≤ t * t := mul_self_nonneg t
0058   have hs_pos : (0 : Q) < t * t := lt_of_lt_of_le heta_pos hge
0059   have hden_s_pos : (0 : Q) < 2 * (1 + t * t) := by positivity
0060   have hden_eta_pos : (0 : Q) < 4 * (1 + eta) := by positivity
0061   have hmono :
0062     eta * eta / (4 * (1 + eta)) ≤
0063       ((t * t) * (t * t)) / (2 * (1 + t * t)) := by
0064     let s : Q := t * t
0065     have hs_ge : eta ≤ s := by simpa [s] using hge
0066     have hs_nonneg' : (0 : Q) ≤ s := by simpa [s] using hs_nonneg
0067     have hs_den_pos : (0 : Q) < 2 * (1 + s) := by positivity
0068     have h_eta_den_two_pos : (0 : Q) < 2 * (1 + eta) := by positivity
0069     have h_eta_den_four_pos : (0 : Q) < 4 * (1 + eta) := by positivity
0070     have hhalf :
0071       eta * eta / (4 * (1 + eta)) ≤
0072       eta * eta / (2 * (1 + eta)) := by
0073       have hnum_nonneg : (0 : Q) ≤ eta * eta := by nlinarith
0074       have hden_le : 2 * (1 + eta) ≤ 4 * (1 + eta) := by nlinarith
0075       exact div_le_div_of_nonneg_left hnum_nonneg h_eta_den_two_pos hden_le
0076     have hmon :
0077       eta * eta / (2 * (1 + eta)) ≤
0078       (s * s) / (2 * (1 + s)) := by
0079       have hdiff_nonneg :
0080         0 ≤ s * s * (1 + eta) - eta * eta * (1 + s) := by
0081         have hleft : 0 ≤ s - eta := by nlinarith
0082         have heta_nonneg : 0 ≤ eta := le_of_lt heta_pos
0083         have hright : 0 ≤ s + eta + s * eta := by
0084           nlinarith [mul_nonneg hs_nonneg' heta_nonneg]
0085         have hprod : 0 ≤ (s - eta) * (s + eta + s * eta) :=
0086           mul_nonneg hleft hright
0087         nlinarith
0088       field_simp [ne_of_gt hs_den_pos, ne_of_gt h_eta_den_two_pos]
0089       nlinarith [hdiff_nonneg]
0090     exact le_trans hhalf (by simpa [s] using hmon)

```

```

0091 exact not_lt_of_ge (le_trans hdelta_le hmono) hsmall
0092
0093 theorem PRCJCostDistanceIncrementTriangleTarget_proved :
0094   PRCJCostDistanceIncrementTriangleTarget := by
0095   intro eps heps
0096   let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
0097   let four : PRCRat := two + two
0098   let rho : PRCRat := eps / (1 + eps)
0099   let eta : PRCRat := rho / four
0100   let delta : PRCRat := (eta * eta) / (four * (1 + eta))
0101   refine (delta, ?, ?)
0102   · rw [PRCRat.positive_iff_toRat_pos]
0103     have heps_pos : (0 : Q) < eps.toRat := (PRCRat.positive_iff_toRat_pos eps).mp heps
0104     have htwo : two.toRat = (2 : Q) := by
0105       dsimp [two]
0106       change (PRCRat.add PRCRat.one PRCRat.one).toRat = (2 : Q)
0107       rw [PRCRat.toRat_add]
0108       norm_num [PRCRat.one_toRat]
0109     have hfour : four.toRat = (4 : Q) := by
0110       dsimp [four]
0111       change (PRCRat.add two two).toRat = (4 : Q)
0112       rw [PRCRat.toRat_add]
0113       norm_num [htwo]
0114     have h_one_add_eps : ((1 : PRCRat) + eps).toRat = 1 + eps.toRat := by
0115       change (PRCRat.add PRCRat.one eps).toRat = 1 + eps.toRat
0116       rw [PRCRat.toRat_add, PRCRat.one_toRat]
0117     have h_one_add_eta : ((1 : PRCRat) + eta).toRat = 1 + eta.toRat := by
0118       change (PRCRat.add PRCRat.one eta).toRat = 1 + eta.toRat
0119       rw [PRCRat.toRat_add, PRCRat.one_toRat]
0120     have heta_pos : (0 : Q) < eta.toRat := by
0121       rw [PRCRat.toRat_div, hfour]
0122       have hrho_pos : (0 : Q) < rho.toRat := by
0123         rw [PRCRat.toRat_div, h_one_add_eps]
0124         positivity
0125       positivity
0126     rw [PRCRat.toRat_div, PRCRat.toRat_mul', PRCRat.toRat_mul',
0127         h_one_add_eta, hfour]
0128     positivity
0129   · intro p q hp hq
0130     have heps_pos : (0 : Q) < eps.toRat := (PRCRat.positive_iff_toRat_pos eps).mp heps
0131     have htwo : two.toRat = (2 : Q) := by
0132       dsimp [two]
0133       change (PRCRat.add PRCRat.one PRCRat.one).toRat = (2 : Q)
0134       rw [PRCRat.toRat_add]
0135     norm_num [PRCRat.one_toRat]
0136     have hfour : four.toRat = (4 : Q) := by
0137       dsimp [four]
0138       change (PRCRat.add two two).toRat = (4 : Q)
0139       rw [PRCRat.toRat_add]
0140     norm_num [htwo]
0141     have h_one_add_eps : ((1 : PRCRat) + eps).toRat = 1 + eps.toRat := by
0142       change (PRCRat.add PRCRat.one eps).toRat = 1 + eps.toRat
0143       rw [PRCRat.toRat_add, PRCRat.one_toRat]
0144     have h_one_add_eta : ((1 : PRCRat) + eta).toRat = 1 + eta.toRat := by
0145       change (PRCRat.add PRCRat.one eta).toRat = 1 + eta.toRat
0146       rw [PRCRat.toRat_add, PRCRat.one_toRat]
0147     have hrho : rho.toRat = eps.toRat / (1 + eps.toRat) := by
0148       rw [PRCRat.toRat_div, h_one_add_eps]
0149     have hrho_pos : (0 : Q) < rho.toRat := by
0150       rw [hrho]
0151       positivity
0152     have hrho_lt_one : rho.toRat < 1 := by
0153       rw [hrho]
0154       field_simp [ne_of_gt (by positivity : (0 : Q) < 1 + eps.toRat)]
0155       nlinarith
0156     have heta : eta.toRat = rho.toRat / 4 := by
0157       rw [PRCRat.toRat_div, hfour]
0158     have heta_pos : (0 : Q) < eta.toRat := by
0159       rw [heta]
0160       positivity
0161     have heta_lt_one : eta.toRat < 1 := by
0162       rw [heta]
0163       nlinarith
0164     have hdelta :
0165       delta.toRat = eta.toRat * eta.toRat / (4 * (1 + eta.toRat)) := by
0166       rw [PRCRat.toRat_div, PRCRat.toRat_mul', PRCRat.toRat_mul',
0167         h_one_add_eta, hfour]
0168     have hdelta_pos : (0 : Q) < delta.toRat := by
0169       rw [hdelta]
0170       positivity
0171     have hp_sq : p * p < eta.toRat := by
0172       exact sq_lt_of_display_lt_delta
0173     (t := p) (eta := eta.toRat) (delta := delta.toRat)
0174     heta_pos hdelta_pos (by rw [hdelta]) hp
0175     have hq_sq : q * q < eta.toRat := by
0176       exact sq_lt_of_display_lt_delta
0177     (t := q) (eta := eta.toRat) (delta := delta.toRat)
0178     heta_pos hdelta_pos (by rw [hdelta]) hq
0179     have hpq_sq_lt_rho : (p + q) * (p + q) < rho.toRat := by
0180       have hsq_bound : (p + q) * (p + q) ≤ 2 * (p * p) + 2 * (q * q) := by
0181         nlinarith [mul_self_nonneg (p - q)]
0182       rw [heta] at hp_sq hq_sq
0183       nlinarith
0184     have hrho_sq_half_lt_eps : rho.toRat * rho.toRat / 2 < eps.toRat := by
0185       have hrho_lt_eps : rho.toRat < eps.toRat := by
0186         rw [hrho]
0187         field_simp [ne_of_gt (by positivity : (0 : Q) < 1 + eps.toRat)]
0188         nlinarith
0189       nlinarith [hrho_pos, hrho_lt_one, hrho_lt_eps]
0190     exact increment_display_lt_of_sq_lt
0191     (t := p + q) (eta := rho.toRat) (eps := eps.toRat)

```

```

0192 hrho_pos hrho_lt_one hpq_sq_lt_rho hrho_sq_half_lt_eps
0193
0194 /-- Final closure for the rational increment modulus. -/
0195 theorem PRCJCostDistanceVerifierTriangleTarget_proved :
0196   PRCJCostDistanceVerifierTriangleTarget :=
0197     PRCJCostDistanceVerifierTriangleTarget_of_increment
0198     PRCJCostDistanceIncrementTriangleTarget_proved
0199
0200 /-- The PRC null-distance setoid target is now closed by the explicit rational
0201 increment modulus. -/
0202 theorem PRCNullDistanceSetoidTarget_proved :
0203   PRCNullDistanceSetoidTarget :=
0204     PRCNullDistanceSetoidTarget_of_increment_triangle
0205     PRCJCostDistanceIncrementTriangleTarget_proved
0206
0207 /-- The PRC triangle modulus is now closed by the explicit rational increment
0208 estimate. -/
0209 theorem PRCJCostDistanceTriangleModulusTarget_proved :
0210   PRCJCostDistanceTriangleModulusTarget :=
0211     PRCJCostDistanceTriangleModulusTarget_of_verifier
0212     PRCJCostDistanceVerifierTriangleTarget_proved
0213
0214 /-- The null-distance relation is transitive. -/
0215 theorem PRCNullDistanceTransitiveTarget_proved :
0216   PRCNullDistanceTransitiveTarget :=
0217     PRCNullDistanceTransitiveTarget_of_triangle_modulus
0218     PRCJCostDistanceTriangleModulusTarget_proved
0219
0220 /-- The final PRC real carrier: Cauchy ledgers quotiented by null distance. -/
0221 def PRCRealNullClosed : Type :=
0222   PRCRealNull PRCNullDistanceTransitiveTarget_proved
0223
0224 namespace PRCRealNullClosed
0225
0226 /-- Embed a PRC rational into the closed null-distance quotient carrier. -/
0227 def ofRat (q : PRCRat) : PRCRealNullClosed :=
0228   PRCRealNull.ofRat PRCNullDistanceTransitiveTarget_proved q
0229
0230 end PRCRealNullClosed
0231
0232 /-- Build Order step 9c closure certificate. -/
0233 structure PRCJCostDistanceIncrementTriangleCertificate : Prop where
0234   increment_formula :
0235      $\forall t : \mathbb{Q},$ 
0236        $\text{PRCJCostDistanceIncrementDisplay } t =$ 
0237        $((t * t) * (t * t)) / (2 * (1 + t * t))$ 
0238   increment_triangle : PRCJCostDistanceIncrementTriangleTarget
0239   verifier_triangle : PRCJCostDistanceVerifierTriangleTarget
0240   triangle_modulus : PRCJCostDistanceTriangleModulusTarget
0241   null_distance_transitive : PRCNullDistanceTransitiveTarget
0242   null_distance_setoid : PRCNullDistanceSetoidTarget
0243   real_null_carrier : Nonempty PRCRealNullClosed
0244   rat_embedding : Nonempty (PRCRat → PRCRealNullClosed)
0245
0246 /-- The explicit rational increment estimate closes the whole J-cost
0247 null-distance setoid chain. -/
0248 theorem prc_jcost_distance_increment_triangle_certificate :
0249   PRCJCostDistanceIncrementTriangleCertificate where
0250     increment_formula := PRCJCostDistanceIncrementDisplay_formula
0251     increment_triangle := PRCJCostDistanceIncrementTriangleTarget_proved
0252     verifier_triangle := PRCJCostDistanceVerifierTriangleTarget_proved
0253     triangle_modulus := PRCJCostDistanceTriangleModulusTarget_proved
0254     null_distance_transitive := PRCNullDistanceTransitiveTarget_proved
0255     null_distance_setoid := PRCNullDistanceSetoidTarget_proved
0256     real_null_carrier := (PRCRealNullClosed.ofRat 0)
0257     rat_embedding := (PRCRealNullClosed.ofRat)
0258
0259 end PrimitiveRecognitionCalculus
0260 end Foundation
0261 end IndisputableMonolith

```

14. PRCJCostDistanceVerifierTriangle.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceVerifierTriangle.lean · Lines: 104

```

0001 /-
0002   PrimitiveRecognitionCalculus/PRCJCostDistanceVerifierTriangle.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Build Order step 9b: prove or isolate the verifier-rational triangle
0009   inequality for the displayed J-cost distance.
0010
0011   This pass removes endpoint bookkeeping. The displayed distance depends only
0012   on the rational increment, so the remaining blocker is an additive two-leg
0013   modulus for increments `p` and `q`.
0014 -/
0015
0016 import Mathlib
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceTriangle
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 /-- One-increment display of the rational J-cost distance. -/
0024 def PRCJCostDistanceIncrementDisplay (t : Q) : Q :=
0025   PRCJCostDistanceRatDisplay 0 t
0026
0027 /-- The displayed distance is translation-invariant: it depends only on the
0028   increment between the endpoints. -/
0029 theorem PRCJCostDistanceRatDisplay_as_increment (x y : Q) :
0030   PRCJCostDistanceRatDisplay x y =
0031     PRCJCostDistanceIncrementDisplay (x - y) := by
0032   simp [PRCJCostDistanceIncrementDisplay, PRCJCostDistanceRatDisplay]
0033   ring_nf
0034
0035 /-- Sharper exact blocker: an additive two-leg modulus for rational increments.
0036   This is the mathematical core behind the three-endpoint verifier triangle
0037   target. -/
0038 def PRCJCostDistanceIncrementTriangleTarget : Prop :=
0039   ∀ eps : PRCRat, PRCRat.positive eps →
0040     ∃ delta : PRCRat, PRCRat.positive delta ∧
0041       ∀ p q : Q,
0042         PRCJCostDistanceIncrementDisplay p < delta.toRat →
0043         PRCJCostDistanceIncrementDisplay q < delta.toRat →
0044         PRCJCostDistanceIncrementDisplay (p + q) < eps.toRat
0045
0046 /-- The increment-only triangle target implies the verifier-rational
0047   three-endpoint triangle target. -/
0048 theorem PRCJCostDistanceVerifierTriangleTarget_of_increment
0049   (h : PRCJCostDistanceIncrementTriangleTarget) :
0050   PRCJCostDistanceVerifierTriangleTarget := by
0051   intro eps heps
0052   rcases h eps heps with (delta, hdelta_pos, hdelta)
0053   refine (delta, hdelta_pos, ?_)
0054   intro x y z hxy hyz
0055   have hxy' :
0056     PRCJCostDistanceIncrementDisplay (x - y) < delta.toRat := by
0057     rwa [PRCJCostDistanceRatDisplay_as_increment] at hxy
0058   have hyz' :
0059     PRCJCostDistanceIncrementDisplay (y - z) < delta.toRat := by
0060     rwa [PRCJCostDistanceRatDisplay_as_increment] at hyz
0061   have hsum :
0062     PRCJCostDistanceIncrementDisplay ((x - y) + (y - z)) < eps.toRat :=
0063     hdelta (x - y) (y - z) hxy' hyz'
0064   have hxz :
0065     PRCJCostDistanceRatDisplay x z =
0066     PRCJCostDistanceIncrementDisplay ((x - y) + (y - z)) := by
0067     rw [PRCJCostDistanceRatDisplay_as_increment]
0068     congr
0069     ring
0070     rwa [hxz]
0071
0072 /-- The increment-only blocker closes the whole PRC null-distance setoid chain. -/
0073 theorem PRCNulldistanceSetoidTarget_of_increment_triangle
0074   (h : PRCJCostDistanceIncrementTriangleTarget) :
0075   PRCNulldistanceSetoidTarget :=
0076   PRCNulldistanceSetoidTarget_of_verifier_triangle
0077   (PRCJCostDistanceVerifierTriangleTarget_of_increment h)
0078
0079 /-- Conditional certificate for step 9b. The only remaining theorem is now the
0080   increment-only modulus target. -/
0081 structure PRCJCostDistanceVerifierTriangleConditionalCertificate : Prop where
0082   translation_invariance :
0083     ∀ x y : Q,
0084       PRCJCostDistanceRatDisplay x y =
0085       PRCJCostDistanceIncrementDisplay (x - y)
0086   increment_triangle_target :
0087     PRCJCostDistanceIncrementTriangleTarget = PRCJCostDistanceIncrementTriangleTarget
0088   verifier_from_increment :
0089     PRCJCostDistanceIncrementTriangleTarget → PRCJCostDistanceVerifierTriangleTarget
0090   setoid_from_increment :

```

```
0091   PRCJCostDistanceIncrementTriangleTarget → PRNulldDistanceSetoidTarget
0092
0093   /-- Build Order step 9b conditional closure: the verifier triangle target is
0094   reduced to a one-dimensional additive increment estimate. -/
0095   theorem prc_jcost_distance_verifier_triangle_conditional_certificate :
0096     PRCJCostDistanceVerifierTriangleConditionalCertificate where
0097     translation_invariance := PRCJCostDistanceRatDisplay_as_increment
0098     increment_triangle_target := rfl
0099     verifier_from_increment := PRCJCostDistanceVerifierTriangleTarget_of_increment
0100     setoid_from_increment := PRNulldDistanceSetoidTarget_of_increment_triangle
0101
0102 end PrimitiveRecognitionCalculus
0103 end Foundation
0104 end IndisputableMonolith
```

15. PRCJCostDistanceTriangle.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCJCostDistanceTriangle.lean · Lines: 94

```

0001 /-
0002   PrimitiveRecognitionCalculus/PRCJCostDistanceTriangle.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Build Order step 9a: prove or isolate the local triangle modulus for
0009   'PRCJCostDistance'.
0010
0011   This pass translates the remaining PRC distance theorem into an exact
0012   verifier-rational inequality. No PRC object is redefined in verifier terms;
0013   the verifier formula is only a display theorem and blocker statement.
0014 -/
0015
0016 import MathLib
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealNullSetoid
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 /-- Verifier display of the J-cost square-gap distance. This is not the PRC
0024 definition; it is the rational formula shown by 'PRCJCostDistance_toRat'. -/
0025 def PRCJCostDistanceRatDisplay (x y : Q) : Q :=
0026   let g : Q := 1 + (x - y) * (x - y)
0027   (g + g⁻¹) / 2 - 1
0028
0029 /-- Display theorem for the PRC J-cost distance. -/
0030 theorem PRCJCostDistance_toRat (a b : PRCRat) :
0031   (PRCJCostDistance a b).toRat =
0032   PRCJCostDistanceRatDisplay a.toRat b.toRat := by
0033   unfold PRCJCostDistance PRCJCostDistanceRatDisplay
0034   rw [PRCJCost.onPRCRat_toRat, PRCSquareGap_toRat]
0035
0036 /-- Exact verifier-rational inequality still needed for the null-distance
0037 quotient. It supplies a PRC rational 'delta', but the analytic estimate itself
0038 is stated only on the conservative rational displays. -/
0039 def PRCJCostDistanceVerifierTriangleTarget : Prop :=
0040   ∀ eps : PRCRat, PRCRat.positive eps →
0041   ∃ delta : PRCRat, PRCRat.positive delta ∧
0042   ∀ x y z : Q,
0043     PRCJCostDistanceRatDisplay x y < delta.toRat →
0044     PRCJCostDistanceRatDisplay y z < delta.toRat →
0045     PRCJCostDistanceRatDisplay x z < eps.toRat
0046
0047 /-- The verifier-rational triangle inequality closes the PRC triangle-modulus
0048 target by display transport. -/
0049 theorem PRCJCostDistanceTriangleModulusTarget_of_verifier
0050   (h : PRCJCostDistanceVerifierTriangleTarget) :
0051   PRCJCostDistanceTriangleModulusTarget := by
0052   intro eps heps
0053   rcases h eps heps with (delta, hdelta_pos, hdelta)
0054   refine (delta, hdelta_pos, ?_)
0055   intro a b c hab hbc
0056   rw [PRCRat.lt_iff_toRat_lt] at hab hbc
0057   rw [PRCJCostDistance_toRat] at hab hbc
0058   exact hdelta a.toRat b.toRat c.toRat hab hbc
0059
0060 /-- Once the verifier-rational inequality is proved, the final null-distance
0061 setoid target follows. -/
0062 theorem PRCTNullDistanceSetoidTarget_of_verifier_triangle
0063   (h : PRCJCostDistanceVerifierTriangleTarget) :
0064   PRCTNullDistanceSetoidTarget :=
0065   PRCTNullDistanceSetoidTarget_of_triangle_modulus
0066   (PRCJCostDistanceTriangleModulusTarget_of_verifier h)
0067
0068 /-- Conditional certificate for step 9a. The only remaining mathematical
0069 problem is now the explicit rational inequality in
0070 'PRCJCostDistanceVerifierTriangleTarget'. -/
0071 structure PRCJCostDistanceTriangleConditionalCertificate : Prop where
0072   distance_display :
0073     ∀ a b : PRCRat,
0074       (PRCJCostDistance a b).toRat =
0075       PRCJCostDistanceRatDisplay a.toRat b.toRat
0076   verifier_triangle_target :
0077     PRCJCostDistanceVerifierTriangleTarget = PRCJCostDistanceVerifierTriangleTarget
0078   triangle_from_verifier :
0079     PRCJCostDistanceVerifierTriangleTarget → PRCJCostDistanceTriangleModulusTarget
0080   setoid_from_verifier :
0081     PRCJCostDistanceVerifierTriangleTarget → PRCTNullDistanceSetoidTarget
0082
0083 /-- Build Order step 9a conditional closure: PRC triangle transport is reduced
0084 to the displayed rational inequality. -/
0085 theorem prc_jcost_distance_triangle_conditional_certificate :
0086   PRCJCostDistanceTriangleConditionalCertificate where
0087   distance_display := PRCJCostDistance_toRat
0088   verifier_triangle_target := rfl
0089   triangle_from_verifier := PRCJCostDistanceTriangleModulusTarget_of_verifier
0090   setoid_from_verifier := PRCTNullDistanceSetoidTarget_of_verifier_triangle

```

```
0091  
0092 end PrimitiveRecognitionCalculus  
0093 end Foundation  
0094 end IndisputableMonolith
```


16. RealCauchy.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCauchy.lean · Lines: 236

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealCauchy.lean
0003
0004   Round-trip source:
0005     6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008     Build Order step 8: start the internal PRC real completion by forming
0009     Cauchy ledgers over `PRCRat`, a J-cost-derived closeness relation, and the
0010     first internal quotient carrier.
0011
0012   Strength: 6 + trace-closure. The completed sequence index is the completed
0013   orbit ledger from `TraceClosure`; verifier rationals appear only in display
0014   theorems and proofs.
0015 -/
0016
0017 import Mathlib
0018 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RationalField
0019 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.TraceClosure
0020
0021 namespace IndisputableMonolith
0022 namespace Foundation
0023 namespace PrimitiveRecognitionCalculus
0024
0025 namespace PRCRat
0026
0027 /-! ## Rational comparison surface used by Cauchy ledgers -/
0028
0029 /-- PRC-native strict order on rationals: `a < b` means the positive gap
0030 `b - a` has a positive ratio-orbit representative. -/
0031 def lt (a b : PRCRat) : Prop :=
0032   positive (b - a)
0033
0034 theorem lt_iff_toRat_lt (a b : PRCRat) :
0035   lt a b ↔ a.toRat < b.toRat := by
0036   unfold lt
0037   rw [positive_iff_toRat_pos]
0038   rw [PRCRat.sub_eq, PRCRat.toRat_sub]
0039   constructor
0040   · intro h
0041     linarith
0042   · intro h
0043     linarith
0044
0045 theorem zero_lt_of_positive {q : PRCRat}
0046   (h : positive q) : lt 0 q := by
0047   rw [lt_iff_toRat_lt]
0048   simp using (positive_iff_toRat_pos q).mp h
0049
0050 end PRCRat
0051
0052 /-! ## J-cost-derived distance on PRC rationals -/
0053
0054 /-- A positive comparison gap for additive rational separation. The square
0055 removes the need for a rational absolute value in this first Cauchy pass. -/
0056 def PRCSquareGap (a b : PRCRat) : PRCRat :=
0057   1 + (a - b) * (a - b)
0058
0059 theorem PRCSquareGap_toRat (a b : PRCRat) :
0060   (PRCSquareGap a b).toRat = 1 + (a.toRat - b.toRat) * (a.toRat - b.toRat) := by
0061   unfold PRCSquareGap
0062   rw [PRCRat.toRat_add', PRCRat.toRat_mul']
0063   simp [PRCRat.sub_eq]
0064
0065 /-- J-cost distance used by the first PRC Cauchy surface. It sends additive
0066 separation through the positive ratio `1 + (a-b)^2`, then applies the PRC
0067 rational J-cost. -/
0068 def PRCJCostDistance (a b : PRCRat) : PRCRat :=
0069   PRCJCost.onPRCRat (PRCSquareGap a b)
0070
0071 theorem PRCJCostDistance_self_zero (a : PRCRat) :
0072   PRCJCostDistance a a = 0 := by
0073   apply PRCRat.toRat_injective
0074   unfold PRCJCostDistance
0075   rw [PRCJCost.onPRCRat_toRat, PRCSquareGap_toRat]
0076   simp
0077
0078 theorem PRCJCostDistance_symmetric (a b : PRCRat) :
0079   PRCJCostDistance a b = PRCJCostDistance b a := by
0080   apply PRCRat.toRat_injective
0081   unfold PRCJCostDistance
0082   rw [PRCJCost.onPRCRat_toRat, PRCJCost.onPRCRat_toRat,
0083     PRCSquareGap_toRat, PRCSquareGap_toRat]
0084   ring
0085
0086 /-! ## PRC Cauchy ledgers -/
0087
0088 /-- A PRC Cauchy sequence is a completed orbit-indexed rational ledger whose
0089 J-cost distance eventually falls below every positive PRC rational tolerance. -/
0090 structure PRCCauchySeq where

```

```

0091 term : Nat → PRCRat
0092 cauchy :
0093   ∀ eps : PRCRat, PRCRat.positive eps →
0094   ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
0095     PRCRat.lt (PRCJCostDistance (term m) (term n)) eps
0096
0097 namespace PRCCauchySeq
0098
0099 /-- Constant rational ledgers are Cauchy. -/
0100 def constant (q : PRCRat) : PRCCauchySeq where
0101   term := fun _ => q
0102   cauchy := by
0103     intro eps heps
0104     refine (0, ?_)
0105     intro m n _hm _hn
0106     rw [PRCJCostDistance_self_zero]
0107     exact PRCRat.zero_lt_of_positive heps
0108
0109 @[simp] theorem constant_term (q : PRCRat) (n : Nat) :
0110   (constant q).term n = q := by
0111     rfl
0112
0113 end PRCCauchySeq
0114
0115 /-- The intended null-distance relation between two Cauchy ledgers. This is
0116 the relation that should become the final real quotient once transitivity is
0117 proved from the J-cost distance surface. -/
0118 def PRCNullEquivalent (u v : PRCCauchySeq) : Prop :=
0119   ∀ eps : PRCRat, PRCRat.positive eps →
0120   ∃ N : Nat, ∀ n : Nat, N ≤ n →
0121     PRCRat.lt (PRCJCostDistance (u.term n) (v.term n)) eps
0122
0123 theorem PRCNullEquivalent.refl (u : PRCCauchySeq) :
0124   PRCNullEquivalent u u := by
0125     intro eps heps
0126     refine (0, ?_)
0127     intro n _hn
0128     rw [PRCJCostDistance_self_zero]
0129     exact PRCRat.zero_lt_of_positive heps
0130
0131 theorem PRCNullEquivalent.symm {u v : PRCCauchySeq}
0132   (h : PRCNullEquivalent u v) : PRCNullEquivalent v u := by
0133     intro eps heps
0134     rcases h eps heps with (N, hN)
0135     refine (N, ?_)
0136     intro n hn
0137     rw [PRCJCostDistance_symmetric]
0138     exact hN n hn
0139
0140 /-- Exact blocker for the final null-distance quotient: prove triangle-style
0141 transitivity for the J-cost distance surface. -/
0142 def PRCNullDistanceTransitiveTarget : Prop :=
0143   ∀ u v w : PRCCauchySeq,
0144     PRCNullEquivalent u v →
0145     PRCNullEquivalent v w →
0146     PRCNullEquivalent u w
0147
0148 /-- Exact target that turns the intended null-distance relation into the real
0149 setoid. Reflexivity and symmetry are proved above; transitivity is the live
0150 mathematical obligation. -/
0151 def PRCNullDistanceSetoidTarget : Prop :=
0152   Equivalence PRCNullEquivalent
0153
0154 /-! ## First internal quotient carrier -/
0155
0156 /-- Sequence identity, used only as the first quotient carrier while the
0157 null-distance transitivity target remains open. -/
0158 def PRCSameTerm (u v : PRCCauchySeq) : Prop :=
0159   ∀ n : Nat, u.term n = v.term n
0160
0161 theorem PRCSameTerm.equivalence : Equivalence PRCSameTerm := by
0162   constructor
0163   · intro u n
0164     rfl
0165   · intro u v h n
0166     exact (h n).symm
0167   · intro u v w huv hnw n
0168     exact (huv n).trans (hwn n)
0169
0170 /-- The first internal setoid available without the null-distance triangle
0171 lemma. It is intentionally stronger than the final null-distance setoid. -/
0172 def PRCSameTermSetoid : Setoid PRCCauchySeq where
0173   r := PRCSameTerm
0174   iseqv := PRCSameTerm.equivalence
0175
0176 /-- First internal PRC real carrier. It is a Cauchy-ledger quotient, not Lean
0177 'ℝ'; the final quotient relation is recorded as 'PRCNullDistanceSetoidTarget'. -/
0178 def PRCReal : Type :=
0179   Quot PRCSameTermSetoid
0180
0181 namespace PRCReal
0182
0183 /-- Embed a PRC rational as a constant Cauchy ledger. -/
0184 def ofRat (q : PRCRat) : PRCReal :=
0185   Quot.mk PRCSameTermSetoid (PRCCauchySeq.constant q)
0186
0187 end PRCReal
0188
0189 /-- K1/R9. Audit record: internal Cauchy real ledgers require trace closure. -/
0190 def realCauchyClaim : StrengthClaim where
0191   label := "BuildOrder8_real_cauchy_internal_quotient"

```

```

0192 tag := StrengthTag.traceClosure
0193 statement :=
0194   "PRC Cauchy ledgers and their first internal quotient use completed orbit indexing."
0195
0196 /-- First-pass real Cauchy certificate. The carrier is internal and
0197 trace-closure tagged. The final null-distance quotient is left as an exact
0198 Lean target rather than hidden behind a classical real alias. -/
0199 structure PRCRealCauchyCertificate : Prop where
0200   cauchy_sequences : Nonempty PRCauchySeq
0201   constant_embedding_exists : Nonempty (PRCRat → PRCCauchySeq)
0202   jcost_distance_self_zero :
0203     ∀ q : PRCRat, PRCCostDistance q q = 0
0204   null_relation_reflexive :
0205     ∀ u : PRCCauchySeq, PRCHandleEquivalent u u
0206   null_relation_symmetric :
0207     ∀ u v : PRCCauchySeq, PRCHandleEquivalent u v → PRCHandleEquivalent v u
0208   same_term_setoid : Nonempty (Setoid PRCCauchySeq)
0209   real_quotient : Nonempty PRCReal
0210   rat_embedding : Nonempty (PRCRat → PRCReal)
0211   null_transitivity_target :
0212     PRCHandleDistanceTransitiveTarget = PRCHandleDistanceTransitiveTarget
0213   null_setoid_target :
0214     PRCHandleDistanceSetoidTarget = PRCHandleDistanceSetoidTarget
0215   strength_tag : realCauchyClaim.tag = StrengthTag.traceClosure
0216
0217 /-- Build Order step 8, first pass: internal Cauchy ledgers and an internal
0218 quotient carrier exist, with the exact null-distance setoid target named. -/
0219 theorem real_cauchy_certificate : PRCRealCauchyCertificate where
0220   cauchy_sequences := (PRCCauchySeq.constant 0)
0221   constant_embedding_exists := (PRCCauchySeq.constant)
0222   jcost_distance_self_zero := PRCCostDistance_self_zero
0223   null_relation_reflexive := PRCHandleEquivalent.refl
0224   null_relation_symmetric := by
0225     intro u v h
0226     exact PRCHandleEquivalent.symm h
0227   same_term_setoid := (PRCSameTermSetoid)
0228   real_quotient := (PRCReal.ofRat 0)
0229   rat_embedding := (PRCReal.ofRat)
0230   null_transitivity_target := rfl
0231   null_setoid_target := rfl
0232   strength_tag := rfl
0233
0234 end PrimitiveRecognitionCalculus
0235 end Foundation
0236 end IndisputableMonolith

```

17. RealNullSetoid.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealNullSetoid.lean · Lines: 135

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealNullSetoid.lean
0003
0004   Round-trip source:
0005     6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008     Build Order step 9: promote the first Cauchy-ledger carrier toward the
0009     final null-distance quotient.
0010
0011   This pass isolates the exact analytic blocker. The quotient bookkeeping is
0012   closed conditionally: a local triangle modulus for 'PRCJCostDistance' implies
0013   'PRCNullEquivalent' is transitive, hence a setoid.
0014 -/
0015
0016 import Mathlib
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCauchy
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 /-- Exact analytic blocker for the null-distance quotient. It says the
0024 J-cost-derived rational distance has a local triangle modulus: for each
0025 positive tolerance there is a positive smaller tolerance so that two small
0026 legs force the composed leg below the original tolerance. -/
0027 def PRCJCostDistanceTriangleModulusTarget : Prop :=
0028   ∀ eps : PRCRat, PRCRat.positive eps →
0029     ∃ delta : PRCRat, PRCRat.positive delta ∧
0030       ∀ a b c : PRCRat,
0031         PRCRat.lt (PRCJCostDistance a b) delta →
0032         PRCRat.lt (PRCJCostDistance b c) delta →
0033         PRCRat.lt (PRCJCostDistance a c) eps
0034
0035 /-- The analytic triangle modulus is sufficient for null-distance
0036 transitivity. All remaining work here is completed-orbit index bookkeeping. -/
0037 theorem PRCNullDistanceTransitiveTarget_of_triangle_modulus
0038   (htri : PRCJCostDistanceTriangleModulusTarget) :
0039   PRCNullDistanceTransitiveTarget := by
0040     intro u v w huv hvw eps heps
0041     rcases htri eps heps with (delta, hdelta_pos, hdelta)
0042     rcases huv delta hdelta_pos with (Nuv, hNuv)
0043     rcases hvw delta hdelta_pos with (Nvw, hNvw)
0044     refine (max Nuv Nvw, ?_)
0045     intro n hn
0046     have hn_uv : Nuv ≤ n := le_trans (Nat.le_max_left Nuv Nvw) hn
0047     have hn_vw : Nvw ≤ n := le_trans (Nat.le_max_right Nuv Nvw) hn
0048     exact hdelta (u.term n) (v.term n) (w.term n)
0049       (hNuv n hn_uv) (hNvw n hn_vw)
0050
0051 /-- A transitivity proof turns 'PRCNullEquivalent' into a setoid. -/
0052 def PRCNullDistanceSetoidOfTransitive
0053   (htrans : PRCNullDistanceTransitiveTarget) : Setoid PRCCauchySeq where
0054   r := PRCNullEquivalent
0055   iseqv := by
0056     constructor
0057     · exact PRCNullEquivalent.refl
0058     · intro u v
0059       exact PRCNullEquivalent.symm
0060     · intro u v w
0061       exact htrans u v w
0062
0063 /-- Conditional final real carrier: once the analytic triangle modulus is
0064 proved, this is the intended PRC real quotient by null distance. -/
0065 def PRCRealNull (htrans : PRCNullDistanceTransitiveTarget) : Type :=
0066   Quot (PRCNullDistanceSetoidOfTransitive htrans)
0067
0068 namespace PRCRealNull
0069
0070 /-- Embed a rational as a null-distance quotient class, conditional on the
0071 transitivity proof. -/
0072 def ofRat (htrans : PRCNullDistanceTransitiveTarget) (q : PRCRat) :
0073   PRCRealNull htrans :=
0074   Quot.mk (PRCNullDistanceSetoidOfTransitive htrans) (PRCCauchySeq.constant q)
0075
0076 end PRCRealNull
0077
0078 /-- The exact setoid target follows from transitivity. -/
0079 theorem PRCNullDistanceSetoidTarget_of_transitive
0080   (htrans : PRCNullDistanceTransitiveTarget) :
0081   PRCNullDistanceSetoidTarget := by
0082     exact (PRCNullDistanceSetoidOfTransitive htrans).iseqv
0083
0084 /-- The exact setoid target follows from the sharper triangle-modulus target. -/
0085 theorem PRCNullDistanceSetoidTarget_of_triangle_modulus
0086   (htri : PRCJCostDistanceTriangleModulusTarget) :
0087   PRCNullDistanceSetoidTarget :=
0088     PRCNullDistanceSetoidTarget_of_transitive
0089       (PRCNullDistanceTransitiveTarget_of_triangle_modulus htri)
0090

```

```

0091 /-- K1/R9. Audit record: the final null-distance quotient still lives under
0092 trace closure; the open obligation is analytic, not a new primitive. -/
0093 def realNullSetoidClaim : StrengthClaim where
0094   label := "BuildOrder9_real_null_distance_setoid"
0095   tag := StrengthTag.traceClosure
0096   statement :=
0097     "The PRC real null-distance setoid follows from the J-cost distance triangle modulus."
0098
0099 /-- Conditional certificate for Build Order step 9. It records the precise
0100 remaining theorem and proves that this theorem is sufficient to construct the
0101 null-distance setoid and quotient carrier. -/
0102 structure PRCRealNullSetoidConditionalCertificate : Prop where
0103   triangle_modulus_target :
0104     PRCJCostDistanceTriangleModulusTarget = PRCJCostDistanceTriangleModulusTarget
0105   transitive_from_triangle :
0106     PRCJCostDistanceTriangleModulusTarget → PRCNullDistanceTransitiveTarget
0107   setoid_from_transitive :
0108     PRCNullDistanceTransitiveTarget → PRCNullDistanceSetoidTarget
0109   setoid_from_triangle :
0110     PRCJCostDistanceTriangleModulusTarget → PRCNullDistanceSetoidTarget
0111   quotient_from_transitive :
0112     ∀ htrans : PRCNullDistanceTransitiveTarget, Nonempty (PRCRealNull htrans)
0113   rat_embedding_from_transitive :
0114     ∀ htrans : PRCNullDistanceTransitiveTarget, Nonempty (PRCRat → PRCRealNull htrans)
0115   strength_tag : realNullSetoidClaim.tag = StrengthTag.traceClosure
0116
0117 /-- Build Order step 9 conditional closure: no quotient mechanics remain once
0118 the local J-cost triangle modulus is proved. -/
0119 theorem real_null_setoid_conditional_certificate :
0120   PRCRealNullSetoidConditionalCertificate where
0121     triangle_modulus_target := rfl
0122     transitive_from_triangle := PRCNullDistanceTransitiveTarget_of_triangle_modulus
0123     setoid_from_transitive := PRCNullDistanceSetoidTarget_of_transitive
0124     setoid_from_triangle := PRCNullDistanceSetoidTarget_of_triangle_modulus
0125     quotient_from_transitive := by
0126       intro htrans
0127       exact (PRCRealNull.ofRat htrans 0)
0128     rat_embedding_from_transitive := by
0129       intro htrans
0130       exact (PRCRealNull.ofRat htrans)
0131     strength_tag := rfl
0132
0133 end PrimitiveRecognitionCalculus
0134 end Foundation
0135 end IndisputableMonolith

```

18. RealMulBoundedContinuity.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealMulBoundedContinuity.lean · Lines: 236

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealMulBoundedContinuity.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Build Order step 10a: reduce multiplication on `PRCRealMulClosed` to
0009   boundedness and bounded product-continuity moduli.
0010
0011   The object-level carrier remains the PRC null quotient. Verifier rationals
0012   appear only in display lemmas for analytic moduli.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteOrderedField
0017
0018 namespace IndisputableMonolith
0019 namespace Foundation
0020 namespace PrimitiveRecognitionCalculus
0021
0022 /-- Eventual PRC-native boundedness for a raw rational ledger. -/
0023 def PRCRawEventuallyBounded (s : PRCRawRatLedger) : Prop :=
0024   ∃ B : PRCRat, PRCRat.positive B ∧
0025   ∃ N : Nat, ∀ n : Nat, N ≤ n →
0026     PRCRat.lt (←B) (s n) ∧ PRCRat.lt (s n) B
0027
0028 /-- Exact boundedness target for Cauchy ledgers. -/
0029 def PRCCauchySeqEventuallyBoundedTarget : Prop :=
0030   ∀ u : PRCCauchySeq, PRCRawEventuallyBounded u.raw
0031
0032 /-- A rational lies inside the symmetric PRC interval `[-B,B]`. -/
0033 def PRCRat.InBound (B x : PRCRat) : Prop :=
0034   PRCRat.lt (←B) x ∧ PRCRat.lt x B
0035
0036 /-- Enlarging a symmetric PRC rational bound preserves membership in it. -/
0037 theorem PRCRat.InBound_mono {B C x : PRCRat}
0038   (hCB : C.toRat ≤ B.toRat) (hx : PRCRat.InBound C x) :
0039   PRCRat.InBound B x := by
0040   rcases hx with (hlo, hhi)
0041   constructor
0042   · rw [PRCRat.lt_iff_toRat_lt] at hlo ⊢
0043     have hnegB : (←B).toRat = -B.toRat := by simp
0044     have hnegC : (←C).toRat = -C.toRat := by simp
0045     rw [hnegC] at hlo
0046     rw [hnegB]
0047   · nlinarith
0048   · rw [PRCRat.lt_iff_toRat_lt] at hhi ⊢
0049     exact lt_of_lt_of_le hhi hCB
0050
0051 /-- Exact product-continuity modulus on bounded rational windows. This is the
0052   local analytic input needed by Cauchy multiplication and multiplication
0053   congruence. -/
0054 def PRCJCostDistanceMulBoundedContinuityTarget : Prop :=
0055   ∀ eps B : PRCRat, PRCRat.positive eps → PRCRat.positive B →
0056   ∃ delta : PRCRat, PRCRat.positive delta ∧
0057     ∀ a a' b b' : PRCRat,
0058       PRCRat.InBound B a →
0059       PRCRat.InBound B a' →
0060       PRCRat.InBound B b →
0061       PRCRat.InBound B b' →
0062       PRCRat.lt (PRCJCostDistance a a') delta →
0063       PRCRat.lt (PRCJCostDistance b b') delta →
0064       PRCRat.lt (PRCJCostDistance (a * b) (a' * b')) eps
0065
0066 /-- Conditional proof of product Cauchy closure from eventual boundedness and
0067   bounded product-continuity. -/
0068 theorem PRCRealMulClosureTarget_of_bounded_continuity
0069   (hbounded : PRCCauchySeqEventuallyBoundedTarget)
0070   (hmul_cont : PRCJCostDistanceMulBoundedContinuityTarget) :
0071   PRCRealMulClosureTarget := by
0072   intro u v eps heps
0073   rcases hbounded u with (Bu, hBu_pos, NuB, hNuB)
0074   rcases hbounded v with (Bv, hBv_pos, NvB, hNvB)
0075   let B : PRCRat := Bu + Bv + 1
0076   have hB_pos : PRCRat.positive B := by
0077     rw [PRCRat.positive_iff_toRat_pos]
0078     have hBu : (0 : ℚ) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
0079     have hBv : (0 : ℚ) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
0080     simp [B]
0081     nlinarith
0082   have hBu_le_B : Bu.toRat ≤ B.toRat := by
0083     have hBu : (0 : ℚ) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
0084     simp [B]
0085     nlinarith
0086   have hBv_le_B : Bv.toRat ≤ B.toRat := by
0087     have hBv : (0 : ℚ) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
0088     simp [B]
0089     nlinarith
0090   rcases hmul_cont eps B heps hB_pos with (delta, hdelta_pos, hdelta)

```

```

0091 rcases u.cauchy delta hdelta_pos with (NuC, hNuC)
0092 rcases v.cauchy delta hdelta_pos with (NvC, hNvC)
0093 let N := max (max NuB NvB) (max NuC NvC)
0094 refine (N, ?_)
0095 intro m n hm hn
0096 have hNuB_m : NuB ≤ m := le_trans (le_trans (Nat.le_max_left NuB NvB)
0097   (Nat.le_max_left (max NuB NvB) (max NuC NvC))) hm
0098 have hNuB_n : NuB ≤ n := le_trans (le_trans (Nat.le_max_left NuB NvB)
0099   (Nat.le_max_left (max NuB NvB) (max NuC NvC))) hn
0100 have hNvB_m : NvB ≤ m := le_trans (le_trans (Nat.le_max_right NuB NvB)
0101   (Nat.le_max_left (max NuB NvB) (max NuC NvC))) hm
0102 have hNvB_n : NvB ≤ n := le_trans (le_trans (Nat.le_max_right NuB NvB)
0103   (Nat.le_max_left (max NuB NvB) (max NuC NvC))) hn
0104 have hNuC_m : NuC ≤ m := le_trans (le_trans (Nat.le_max_left NuC NvC)
0105   (Nat.le_max_right (max NuB NvB) (max NuC NvC))) hm
0106 have hNuC_n : NuC ≤ n := le_trans (le_trans (Nat.le_max_left NuC NvC)
0107   (Nat.le_max_right (max NuB NvB) (max NuC NvC))) hn
0108 have hNvC_m : NvC ≤ m := le_trans (le_trans (Nat.le_max_right NuC NvC)
0109   (Nat.le_max_right (max NuB NvB) (max NuC NvC))) hm
0110 have hNvC_n : NvC ≤ n := le_trans (le_trans (Nat.le_max_right NuC NvC)
0111   (Nat.le_max_right (max NuB NvB) (max NuC NvC))) hn
0112 have hu_m_small : PRCRat.InBound B (u.term m) :=
0113   PRCRat.InBound_mono hBu_le_B (hNuB m hNuB_m)
0114 have hu_n_small : PRCRat.InBound B (u.term n) :=
0115   PRCRat.InBound_mono hBu_le_B (hNuB n hNuB_n)
0116 have hv_m_small : PRCRat.InBound B (v.term m) :=
0117   PRCRat.InBound_mono hBv_le_B (hNvB m hNvB_m)
0118 have hv_n_small : PRCRat.InBound B (v.term n) :=
0119   PRCRat.InBound_mono hBv_le_B (hNvB n hNvB_n)
0120 exact hdelta (u.term m) (u.term n) (v.term m) (v.term n)
0121 hu_m_small hu_n_small hv_m_small hv_n_small
0122 (hNuC m n hNuC_m hNuC_n)
0123 (hNvC m n hNvC_m hNvC_n)
0124
0125 /-- Conditional proof of product congruence from eventual boundedness and
0126 bounded product-continuity. -/
0127 theorem PRCRealMulCongruenceTarget_of_bounded_continuity
0128   (hbounded : PRCCauchySeqEventuallyBoundedTarget)
0129   (hmul_cont : PRCJCostDistanceMulBoundedContinuityTarget) :
0130   PRCRealMulCongruenceTarget := by
0131   intro u u' v v' huu hvv eps heps
0132   rcases hbounded u with (Bu, hBu_pos, NuB, hNuB)
0133   rcases hbounded u' with (Bu', hBu'_pos, Nu'B, hNu'B)
0134   rcases hbounded v with (Bv, hBv_pos, NvB, hNvB)
0135   rcases hbounded v' with (Bv', hBv'_pos, Nv'B, hNv'B)
0136   let B : PRCRat := Bu + Bu' + Bv + Bv' + 1
0137   have hB_pos : PRCRat.positive B := by
0138     rw [PRCRat.positive_iff_toRat_pos]
0139   have hBu : (0 : Q) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
0140   have hBu' : (0 : Q) < Bu'.toRat := (PRCRat.positive_iff_toRat_pos Bu').mp hBu'_pos
0141   have hBv : (0 : Q) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
0142   have hBv' : (0 : Q) < Bv'.toRat := (PRCRat.positive_iff_toRat_pos Bv').mp hBv'_pos
0143   simp [B]
0144   nlinarith
0145   have hBu_le_B : Bu.toRat ≤ B.toRat := by
0146     have hBu' : (0 : Q) < Bu'.toRat := (PRCRat.positive_iff_toRat_pos Bu').mp hBu'_pos
0147     have hBv : (0 : Q) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
0148     have hBv' : (0 : Q) < Bv'.toRat := (PRCRat.positive_iff_toRat_pos Bv').mp hBv'_pos
0149     simp [B]
0150     nlinarith
0151   have hBu' le_B : Bu'.toRat ≤ B.toRat := by
0152     have hBu : (0 : Q) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
0153     have hBv : (0 : Q) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
0154     have hBv' : (0 : Q) < Bv'.toRat := (PRCRat.positive_iff_toRat_pos Bv').mp hBv'_pos
0155     simp [B]
0156     nlinarith
0157   have hBv le_B : Bv.toRat ≤ B.toRat := by
0158     have hBu : (0 : Q) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
0159     have hBu' : (0 : Q) < Bu'.toRat := (PRCRat.positive_iff_toRat_pos Bu').mp hBu'_pos
0160     have hBv' : (0 : Q) < Bv'.toRat := (PRCRat.positive_iff_toRat_pos Bv').mp hBv'_pos
0161     simp [B]
0162     nlinarith
0163   have hBv' le_B : Bv'.toRat ≤ B.toRat := by
0164     have hBu : (0 : Q) < Bu.toRat := (PRCRat.positive_iff_toRat_pos Bu).mp hBu_pos
0165     have hBu' : (0 : Q) < Bu'.toRat := (PRCRat.positive_iff_toRat_pos Bu').mp hBu'_pos
0166     have hBv : (0 : Q) < Bv.toRat := (PRCRat.positive_iff_toRat_pos Bv).mp hBv_pos
0167     simp [B]
0168     nlinarith
0169   rcases hmul_cont eps B heps hB_pos with (delta, hdelta_pos, hdelta)
0170   rcases huu delta hdelta_pos with (NuC, hNuC)
0171   rcases hvv delta hdelta_pos with (NvC, hNvC)
0172   let N := max (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC)
0173   refine (N, ?_)
0174   intro n hn
0175   have hNuB_n : NuB ≤ n := le_trans
0176     (le_trans (Nat.le_max_left NuB Nu'B) (Nat.le_max_left (max NuB Nu'B) (max NvB Nv'B)))
0177     (le_trans (Nat.le_max_left (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC))) hn
0178   have hNu'B_n : Nu'B ≤ n := le_trans
0179     (le_trans (Nat.le_max_right NuB Nu'B) (Nat.le_max_left (max NuB Nu'B) (max NvB Nv'B)))
0180     (le_trans (Nat.le_max_left (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC))) hn
0181   have hNvB_n : NvB ≤ n := le_trans
0182     (le_trans (Nat.le_max_left NvB Nv'B) (Nat.le_max_right (max NuB Nu'B) (max NvB Nv'B)))
0183     (le_trans (Nat.le_max_left (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC))) hn
0184   have hNv'B_n : Nv'B ≤ n := le_trans
0185     (le_trans (Nat.le_max_right NvB Nv'B) (Nat.le_max_right (max NuB Nu'B) (max NvB Nv'B)))
0186     (le_trans (Nat.le_max_left (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC))) hn
0187   have hNuC_n : NuC ≤ n :=
0188     le_trans (Nat.le_max_left NuC NvC)
0189     (le_trans (Nat.le_max_right (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC))) hn
0190   have hNvC_n : NvC ≤ n :=
0191     le_trans (Nat.le_max_right NuC NvC)

```

```

0192 (le_trans (Nat.le_max_right (max (max NuB Nu'B) (max NvB Nv'B)) (max NuC NvC)) hn)
0193 exact hdelta (u.term n) (u'.term n) (v.term n) (v'.term n)
0194 (PRCRat.InBound_mono hBu_le_B (hNuB n hNuB_n))
0195 (PRCRat.InBound_mono hBu'_le_B (hNu'B n hNu'B_n))
0196 (PRCRat.InBound_mono hBv_le_B (hNvB n hNvB_n))
0197 (PRCRat.InBound_mono hBv'_le_B (hNv'B n hNv'B_n))
0198 (hNuC n hNuC_n)
0199 (hNvC n hNvC_n)
0200
0201 /-- Conditional certificate: the multiplication targets reduce to eventual
0202 boundedness plus bounded product continuity. -/
0203 structure PRCRealMulBoundedContinuityConditionalCertificate : Prop where
0204   boundedness_target :
0205     PRCCauchySeqEventuallyBoundedTarget = PRCCauchySeqEventuallyBoundedTarget
0206   product_continuity_target :
0207     PRCJCostDistanceMulBoundedContinuityTarget =
0208       PRCJCostDistanceMulBoundedContinuityTarget
0209   mul_closure_from_targets :
0210     PRCCauchySeqEventuallyBoundedTarget →
0211       PRCJCostDistanceMulBoundedContinuityTarget →
0212       PRCRealMulClosureTarget
0213   mul_congruence_from_targets :
0214     PRCCauchySeqEventuallyBoundedTarget →
0215       PRCJCostDistanceMulBoundedContinuityTarget →
0216       PRCRealMulCongruenceTarget
0217   mul_operation_from_targets :
0218     PRCCauchySeqEventuallyBoundedTarget →
0219       PRCJCostDistanceMulBoundedContinuityTarget →
0220       Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
0221
0222 theorem prc_real_mul_bounded_continuity_conditional_certificate :
0223   PRCRealMulBoundedContinuityConditionalCertificate where
0224     boundedness_target := rfl
0225     product_continuity_target := rfl
0226     mul_closure_from_targets := PRCRealMulClosureTarget_of_bounded_continuity
0227     mul_congruence_from_targets := PRCRealMulCongruenceTarget_of_bounded_continuity
0228     mul_operation_from_targets := by
0229       intro hbounded hcont
0230       exact (PRCRealNullClosed.mul0f
0231         (PRCRealMulClosureTarget_of_bounded_continuity hbounded hcont)
0232         (PRCRealMulCongruenceTarget_of_bounded_continuity hbounded hcont))
0233
0234 end PrimitiveRecognitionCalculus
0235 end Foundation
0236 end IndisputableMonolith

```


19. RealBoundednessModulus.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealBoundednessModulus.lean · Lines: 142

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealBoundednessModulus.lean
0003
0004   Round-trip source:
0005     6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008     Build Order step 10b: prove every J-cost Cauchy ledger is eventually
0009     PRC-bounded.
0010
0011   The proof uses the verifier rational display only to extract the ordinary
0012   square bound from small J-cost distance. The eventual bound itself is stated
0013   on the PRC rational ledger.
0014 -/
0015
0016 import Mathlib
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealMulBoundedContinuity
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 /-- A fixed PRC rational distance threshold small enough to force ordinary
0024 increment square below one. -/
0025 def PRCBoundednessDelta : PRCRat :=
0026   let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
0027   let four : PRCRat := two * two
0028   (1 : PRCRat) * ((four * two)^-1)
0029
0030 theorem PRCBoundednessDelta_toRat :
0031   PRCBoundednessDelta.toRat = (1 / 8 : Q) := by
0032   unfold PRCBoundednessDelta
0033   simp [PRCRat.toRat_mul, PRCRat.toRat_recip]
0034   norm_num
0035
0036 theorem PRCBoundednessDelta_positive :
0037   PRCRat.positive PRCBoundednessDelta := by
0038   rw [PRCRat.positive_iff_toRat_pos, PRCBoundednessDelta_toRat]
0039   norm_num
0040
0041 /-- Small increment display at the fixed threshold forces ordinary square
0042 increment below one. -/
0043 theorem PRCJCostDistanceIncrementDisplay_sq_lt_one {t : Q}
0044   (hsmall : PRCJCostDistanceIncrementDisplay t < (1 / 8 : Q)) :
0045   t * t < 1 := by
0046   by_contra hnot
0047   have hge : (1 : Q) ≤ t * t := by
0048     have hsq_nonneg : (0 : Q) ≤ t * t := mul_self_nonneg t
0049     nlinarith
0050   rw [PRCJCostDistanceIncrementDisplay_formula] at hsmall
0051   let s : Q := t * t
0052   have hs_ge : (1 : Q) ≤ s := by simpa [s] using hge
0053   have hs_den_pos : (0 : Q) < 2 * (1 + s) := by nlinarith
0054   have hmono : (1 / 8 : Q) ≤ (s * s) / (2 * (1 + s)) := by
0055     field_simp [ne_of_gt hs_den_pos]
0056     nlinarith [mul_self_nonneg s]
0057   exact not_lt_of_ge hmono (by simpa [s] using hsmall)
0058
0059 /-- Small PRC J-cost distance at the fixed threshold forces the ordinary
0060 rational display increment to have square below one. -/
0061 theorem PRCJCostDistance_sq_diff_lt_one_of_lt_boundedness_delta
0062   {a b : PRCRat}
0063   (hsmall : PRCRat.lt (PRCJCostDistance a b) PRCBoundednessDelta) :
0064   (a.toRat - b.toRat) * (a.toRat - b.toRat) < 1 := by
0065   rw [PRCRat.lt_iff_toRat_lt] at hsmall
0066   rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment,
0067     PRCBoundednessDelta_toRat] at hsmall
0068   exact PRCJCostDistanceIncrementDisplay_sq_lt_one hsmall
0069
0070 /-- A J-cost Cauchy ledger is eventually contained in a PRC symmetric rational
0071 interval. -/
0072 theorem PRCJCostCauchySeqEventuallyBoundedTarget_proved :
0073   PRCJCostCauchySeqEventuallyBoundedTarget := by
0074   intro u
0075   rcases u.cauchy PRCBoundednessDelta PRCBoundednessDelta_positive with
0076     (N, hN)
0077   let anchor : PRCRat := u.term N
0078   let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
0079   let B : PRCRat := anchor * anchor * two
0080   have hB_pos : PRCRat.positive B := by
0081     rw [PRCRat.positive_iff_toRat_pos]
0082     have hsq : (0 : Q) ≤ anchor.toRat * anchor.toRat :=
0083       mul_self_nonneg anchor.toRat
0084     simp [B, two]
0085     nlinarith
0086   refine (B, hB_pos, N, ?_)
0087   intro n hn
0088   have hdist : PRCRat.lt (PRCJCostDistance (u.term n) anchor) PRCBoundednessDelta := by
0089     simpa [anchor] using hn n N hn (Nat.le_refl N)
0090   have hsquare :

```

```

0091      ((u.term n).toRat - anchor.toRat) *
0092      ((u.term n).toRat - anchor.toRat) < 1 :=
0093      PRCJCostDistance_sq_diff_lt_one_of_lt_boundedness_delta hdist
0094      let x : ℚ := (u.term n).toRat
0095      let q : ℚ := anchor.toRat
0096      have hsquare_xq : (x - q) * (x - q) < 1 := by
0097      simp [x, q] using hsquare
0098      have hdiff_lt_one : x - q < 1 := by
0099      nlinarith [mul_self_nonneg ((x - q) - 1)]
0100      have hdiff_gt_neg_one : -1 < x - q := by
0101      nlinarith [mul_self_nonneg ((x - q) + 1)]
0102      constructor
0103      · rw [PRCRat.lt_iff_toRat_lt]
0104      simp [B, two, anchor]
0105      nlinarith [mul_self_nonneg (2 * q + 1)]
0106      · rw [PRCRat.lt_iff_toRat_lt]
0107      simp [B, two, anchor]
0108      nlinarith [mul_self_nonneg (2 * q - 1)]
0109
0110      /-- Step 10b closure certificate: eventual boundedness is proved, so the
0111      remaining multiplication blocker is only bounded product-continuity. -/
0112      structure PRCRealBoundednessModulusCertificate : Prop where
0113      boundedness_delta_positive : PRCRat.positive PRCBoundednessDelta
0114      distance_sq_bound :
0115      ∀ a b : PRCRat,
0116      PRCRat.lt (PRCJCostDistance a b) PRCBoundednessDelta →
0117      (a.toRat - b.toRat) * (a.toRat - b.toRat) < 1
0118      eventual_boundedness : PRCCauchySeqEventuallyBoundedTarget
0119      mul_closure_from_product_continuity :
0120      PRCJCostDistanceMulBoundedContinuityTarget → PRCRealMulClosureTarget
0121      mul_congruence_from_product_continuity :
0122      PRCJCostDistanceMulBoundedContinuityTarget → PRCRealMulCongruenceTarget
0123
0124      theorem prc_real_boundedness_modulus_certificate :
0125      PRCRealBoundednessModulusCertificate where
0126      boundedness_delta_positive := PRCBoundednessDelta_positive
0127      distance_sq_bound := by
0128      intro a b h
0129      exact PRCJCostDistance_sq_diff_lt_one_of_lt_boundedness_delta h
0130      eventual_boundedness := PRCCauchySeqEventuallyBoundedTarget_proved
0131      mul_closure_from_product_continuity := by
0132      intro hcont
0133      exact PRCRealMulClosureTarget_of_bounded_continuity
0134      PRCCauchySeqEventuallyBoundedTarget_proved hcont
0135      mul_congruence_from_product_continuity := by
0136      intro hcont
0137      exact PRCRealMulCongruenceTarget_of_bounded_continuity
0138      PRCCauchySeqEventuallyBoundedTarget_proved hcont
0139
0140      end PrimitiveRecognitionCalculus
0141      end Foundation
0142      end IndisputableMonolith

```

20. RealProductContinuity.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealProductContinuity.lean · Lines: 304

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealProductContinuity.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Build Order step 10c: prove the bounded product-continuity modulus for
0009   'PRCJCostDistance'.
0010
0011   The object-level statement remains PRC-rational. The proof uses verifier
0012   rationals only as display transport for the analytic inequality.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealBoundednessModulus
0017
0018 namespace IndisputableMonolith
0019 namespace Foundation
0020 namespace PrimitiveRecognitionCalculus
0021
0022 theorem PRCJCostDistanceIncrementDisplay_lt_of_sq_lt
0023   {t eta eps : ℚ} (heta_pos : 0 < eta) (heta_lt_one : eta < 1)
0024   (hsq : t * t < eta) (heta_sq_half_lt_eps : eta * eta / 2 < eps) :
0025   PRCJCostDistanceIncrementDisplay t < eps := by
0026   rw [PRCJCostDistanceIncrementDisplay_formula]
0027   have hs_nonneg : (0 : ℚ) ≤ t * t := mul_self_nonneg t
0028   have hden_pos : (0 : ℚ) < 2 * (1 + t * t) := by positivity
0029   have hs_lt_one : t * t < 1 := lt_trans hsq heta_lt_one
0030   have hnum_lt : (t * t) * (t * t) < eta * eta := by nlinarith
0031   have hfrac_le : ((t * t) * (t * t)) / (2 * (1 + t * t)) ≤
0032     ((t * t) * (t * t)) / 2 := by
0033     have hden_ge_two : (2 : ℚ) ≤ 2 * (1 + t * t) := by nlinarith
0034     have hnum_nonneg : (0 : ℚ) ≤ (t * t) * (t * t) :=
0035       mul_nonneg hs_nonneg hs_nonneg
0036     exact div_le_div_of_nonneg_left hnum_nonneg (by norm_num) hden_ge_two
0037   have hnum_half_lt : ((t * t) * (t * t)) / 2 < eta * eta / 2 := by
0038     nlinarith
0039   exact lt_of_le_of_lt hfrac_le (lt_trans hnum_half_lt heta_sq_half_lt_eps)
0040
0041 theorem PRCJCostDistance_sq_lt_of_display_lt_delta
0042   {t eta delta : ℚ} (heta_pos : 0 < eta) (hdelta_pos : 0 < delta)
0043   (hdelta_le : delta ≤ eta * eta / (4 * (1 + eta)))
0044   (hsmall : PRCJCostDistanceIncrementDisplay t < delta) :
0045   t * t < eta := by
0046   by_contra hnot
0047   have hge : eta ≤ t * t := by nlinarith
0048   rw [PRCJCostDistanceIncrementDisplay_formula] at hsmall
0049   have hs_nonneg : (0 : ℚ) ≤ t * t := mul_self_nonneg t
0050   have hs_pos : (0 : ℚ) < t * t := lt_of_lt_of_le heta_pos hge
0051   have hden_s_pos : (0 : ℚ) < 2 * (1 + t * t) := by positivity
0052   have hmono :
0053     eta * eta / (4 * (1 + eta)) ≤
0054     ((t * t) * (t * t)) / (2 * (1 + t * t)) := by
0055     let s : ℚ := t * t
0056     have hs_ge : eta ≤ s := by simpa [s] using hge
0057     have hs_nonneg' : (0 : ℚ) ≤ s := by simpa [s] using hs_nonneg
0058     have hs_den_pos : (0 : ℚ) < 2 * (1 + s) := by positivity
0059     have h_eta_den_two_pos : (0 : ℚ) < 2 * (1 + eta) := by positivity
0060     have h_eta_den_four_pos : (0 : ℚ) < 4 * (1 + eta) := by positivity
0061     have hhalf :
0062       eta * eta / (4 * (1 + eta)) ≤
0063       eta * eta / (2 * (1 + eta)) := by
0064       have hnum_nonneg : (0 : ℚ) ≤ eta * eta := by nlinarith
0065       have hden_le : 2 * (1 + eta) ≤ 4 * (1 + eta) := by nlinarith
0066       exact div_le_div_of_nonneg_left hnum_nonneg h_eta_den_two_pos hden_le
0067     have hmon :
0068       eta * eta / (2 * (1 + eta)) ≤
0069       (s * s) / (2 * (1 + s)) := by
0070       have hdiff_nonneg :
0071         0 ≤ s * s * (1 + eta) - eta * eta * (1 + s) := by
0072         have hleft : 0 ≤ s - eta := by nlinarith
0073         have heta_nonneg : 0 ≤ eta := le_of_lt heta_pos
0074         have hright : 0 ≤ s + eta + s * eta := by
0075           nlinarith [mul_nonneg hs_nonneg' heta_nonneg]
0076         have hprod : 0 ≤ (s - eta) * (s + eta + s * eta) :=
0077           mul_nonneg hleft hright
0078         nlinarith
0079       field_simp [ne_of_gt hs_den_pos, ne_of_gt h_eta_den_two_pos]
0080       nlinarith [hdiff_nonneg]
0081     exact le_trans hhalf (by simpa [s] using hmon)
0082   exact not_lt_of_ge (le_trans hdelta_le hmono) hsmall
0083
0084 theorem PRCRat.InBound_sq_lt {B x : PRCRat}
0085   (hB : PRCRat.positive B) (hx : PRCRat.InBound B x) :
0086   x.toRat * x.toRat < B.toRat * B.toRat := by
0087   rcases hx with (hlo, hhi)
0088   rw [PRCRat.lt_iff_toRat_lt] at hlo hhi
0089   have hB_pos : (0 : ℚ) < B.toRat := (PRCRat.positive_iff_toRat_pos B).mp hB
0090   have hneg : (-B).toRat = -B.toRat := by simp

```

```

0091 rw [hneg] at hlo
0092 have hleft : 0 < B.toRat - x.toRat := by nlinarith
0093 have hright : 0 < B.toRat + x.toRat := by nlinarith
0094 have hprod : 0 < (B.toRat - x.toRat) * (B.toRat + x.toRat) :=
0095   mul_pos hleft hright
0096   nlinarith
0097
0098 private theorem product_factor_sq_lt
0099   {da b eta M : ℚ}
0100   (hda : da * da < eta)
0101   (hb : b * b < M * M)
0102   (hM_pos : 0 < M) :
0103   (da * b) * (da * b) < eta * (M * M) := by
0104   have hda_nonneg : 0 ≤ da * da := mul_self_nonneg da
0105   have hb_nonneg : 0 ≤ b * b := mul_self_nonneg b
0106   have hM_sq_pos : 0 < M * M := mul_pos hM_pos hM_pos
0107   have hgap1 : 0 < eta - da * da := by nlinarith
0108   have hgap2 : 0 < M * M - b * b := by nlinarith
0109   have hterm1 : 0 < (eta - da * da) * (M * M) :=
0110     mul_pos hgap1 hM_sq_pos
0111   have hterm2 : 0 ≤ (da * da) * (M * M - b * b) :=
0112     mul_nonneg hda_nonneg (le_of_lt hgap2)
0113   have hgap :
0114     0 < eta * (M * M) - (da * da) * (b * b) := by
0115     nlinarith
0116   have hsq : (da * b) * (da * b) = (da * da) * (b * b) := by ring
0117   rw [hsq]
0118   nlinarith
0119
0120 private theorem rational_product_increment_sq_lt
0121   {a a' b b' M eta rho : ℚ}
0122   (hM_pos : 0 < M) (hrho_pos : 0 < rho)
0123   (heta_eq : eta = rho / (4 * (1 + M * M)))
0124   (ha' : a' * a' < M * M)
0125   (hb : b * b < M * M)
0126   (hda : (a - a') * (a - a') < eta)
0127   (hdb : (b - b') * (b - b') < eta) :
0128   (a * b - a' * b') * (a * b - a' * b') < rho := by
0129   let u : ℚ := (a - a') * b
0130   let v : ℚ := a' * (b - b')
0131   have hu : u * u < eta * (M * M) := by
0132     simpa [u] using product_factor_sq_lt
0133   (da := a - a') (b := b) (eta := eta) (M := M)
0134   hda hb hM_pos
0135   have hv : v * v < eta * (M * M) := by
0136     simpa [v, mul_comm, mul_left_comm, mul_assoc] using product_factor_sq_lt
0137   (da := b - b') (b := a') (eta := eta) (M := M)
0138   hdb ha' hM_pos
0139   have hsum_le : (u + v) * (u + v) ≤ 2 * (u * u) + 2 * (v * v) := by
0140     nlinarith [mul_self_nonneg (u - v)]
0141   have hsum_lt : (u + v) * (u + v) < 4 * eta * (M * M) := by
0142     nlinarith
0143   have hscale : 4 * eta * (M * M) < rho := by
0144     rw [heta_eq]
0145     have hden_pos : (0 : ℚ) < 4 * (1 + M * M) := by positivity
0146     field_simp [ne_of_gt hden_pos]
0147     have hM_sq_pos : 0 < M * M := mul_pos hM_pos hM_pos
0148     nlinarith
0149   have hidentity : a * b - a' * b' = u + v := by
0150     dsimp [u, v]
0151     ring
0152   rw [hidentity]
0153   exact lt_trans hsum_lt hscale
0154
0155 theorem PRCJCostDistanceMulBoundedContinuityTarget_proved :
0156   PRCJCostDistanceMulBoundedContinuityTarget := by
0157   intro eps B heps hB
0158   let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
0159   let four : PRCRat := two * two
0160   let rho : PRCRat := eps * (((1 : PRCRat) + eps)⁻¹)
0161   let K : PRCRat := (1 : PRCRat) + (B * B)
0162   let eta : PRCRat := rho * ((four * K)⁻¹)
0163   let delta : PRCRat := (eta * eta) * ((four * ((1 : PRCRat) + eta))⁻¹)
0164   have heps_pos : (0 : ℚ) < eps.toRat :=
0165     (PRCRat.positive_iff_toRat_pos eps).mp heps
0166   have hB_pos : (0 : ℚ) < B.toRat :=
0167     (PRCRat.positive_iff_toRat_pos B).mp hB
0168   have htwo : two.toRat = (2 : ℚ) := by
0169     dsimp [two]
0170     change (PRCRat.add PRCRat.one PRCRat.one).toRat = (2 : ℚ)
0171   rw [PRCRat.toRat_add]
0172   norm_num [PRCRat.one_toRat]
0173   have hfour : four.toRat = (4 : ℚ) := by
0174     dsimp [four]
0175     change (PRCRat.mul two two).toRat = (4 : ℚ)
0176   rw [PRCRat.toRat_mul]
0177   norm_num [htwo]
0178   have h_one_add_eps :
0179     (((1 : PRCRat) + eps).toRat) = 1 + eps.toRat := by
0180     change (PRCRat.add PRCRat.one eps).toRat = 1 + eps.toRat
0181   rw [PRCRat.toRat_add, PRCRat.one_toRat]
0182   have hK : K.toRat = 1 + B.toRat * B.toRat := by
0183     dsimp [K]
0184     change (PRCRat.add PRCRat.one (PRCRat.mul B B)).toRat =
0185       1 + B.toRat * B.toRat
0186   rw [PRCRat.toRat_add, PRCRat.one_toRat, PRCRat.toRat_mul]
0187   have hK_pos : (0 : ℚ) < K.toRat := by
0188     rw [hK]
0189     nlinarith [mul_self_nonneg B.toRat]
0190   have hrho : rho.toRat = eps.toRat / (1 + eps.toRat) := by
0191     dsimp [rho]

```

```

0192 simp [PRCRat.toRat_mul, PRCRat.toRat_recip, PRCRat.toRat_add,
0193 PRCRat.one_toRat]
0194 ring
0195 have hrho_pos : (0 : Q) < rho.toRat := by
0196   rw [hrho]
0197   positivity
0198 have hrho_lt_one : rho.toRat < 1 := by
0199   rw [hrho]
0200   field_simp [ne_of_gt (by positivity : (0 : Q) < 1 + eps.toRat)]
0201   nlinarith
0202 have hrho_sq_half_lt_eps : rho.toRat * rho.toRat / 2 < eps.toRat := by
0203   have hrho_lt_eps : rho.toRat < eps.toRat := by
0204     rw [hrho]
0205     field_simp [ne_of_gt (by positivity : (0 : Q) < 1 + eps.toRat)]
0206     nlinarith
0207   nlinarith [hrho_pos, hrho_lt_one, hrho_lt_eps]
0208 have heta : eta.toRat = rho.toRat / (4 * K.toRat) := by
0209   dsimp [eta]
0210   rw [PRCRat.toRat_mul, PRCRat.toRat_recip, PRCRat.toRat_mul, hfour]
0211   ring
0212 have heta_pos : (0 : Q) < eta.toRat := by
0213   rw [heta]
0214   positivity
0215 have heta_lt_one : eta.toRat < 1 := by
0216   rw [heta]
0217   have hden_pos : (0 : Q) < 4 * K.toRat := by positivity
0218   field_simp [ne_of_gt hden_pos]
0219   have hK_gt_zero : 0 < 4 * K.toRat := by positivity
0220   nlinarith [hrho_lt_one, hrho_pos, hK_pos]
0221 have h_one_add_eta :
0222   ((1 : PRCRat) + eta).toRat = 1 + eta.toRat := by
0223   change (PRCRat.add PRCRat.one eta).toRat = 1 + eta.toRat
0224   rw [PRCRat.toRat_add, PRCRat.one_toRat]
0225 have hdelta :
0226   delta.toRat = eta.toRat * eta.toRat / (4 * (1 + eta.toRat)) := by
0227   dsimp [delta]
0228   simp [PRCRat.toRat_mul, PRCRat.toRat_recip, PRCRat.toRat_add,
0229     PRCRat.one_toRat, hfour]
0230   have hden_pos : (0 : Q) < 4 * (1 + eta.toRat) := by positivity
0231   field_simp [ne_of_gt hden_pos]
0232   have hdelta_pos_rat : (0 : Q) < delta.toRat := by
0233     rw [hdelta]
0234     positivity
0235   have hdelta_pos : PRCRat.positive delta := by
0236     rw [PRCRat.positive_iff_toRat_pos]
0237     exact hdelta_pos_rat
0238   refine (delta, hdelta_pos, ?)
0239   intro a a' b b' ha ha' hb hb' haa hbb
0240   have ha_sq : a'.toRat * a'.toRat < B.toRat * B.toRat :=
0241     PRCRat.InBound_sq_lt hB ha'
0242   have hb_sq : b.toRat * b.toRat < B.toRat * B.toRat :=
0243     PRCRat.InBound_sq_lt hB hb
0244   have haa_rat : PRCJCostDistanceIncrementDisplay (a.toRat - a'.toRat) < delta.toRat := by
0245     rw [PRCRat.lt_iff_toRat_lt] at haa
0246     rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment] at haa
0247     exact haa
0248   have hbb_rat : PRCJCostDistanceIncrementDisplay (b.toRat - b'.toRat) < delta.toRat := by
0249     rw [PRCRat.lt_iff_toRat_lt] at hbb
0250     rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment] at hbb
0251     exact hbb
0252   have hda_sq : (a.toRat - a'.toRat) * (a.toRat - a'.toRat) < eta.toRat :=
0253     PRCJCostDistance_sq_lt_of_display_lt_delta
0254   (t := a.toRat - a'.toRat) (eta := eta.toRat) (delta := delta.toRat)
0255   heta_pos hdelta_pos_rat (by rw [hdelta]) haa_rat
0256   have hdb_sq : (b.toRat - b'.toRat) * (b.toRat - b'.toRat) < eta.toRat :=
0257     PRCJCostDistance_sq_lt_of_display_lt_delta
0258   (t := b.toRat - b'.toRat) (eta := eta.toRat) (delta := delta.toRat)
0259   heta_pos hdelta_pos_rat (by rw [hdelta]) hbb_rat
0260   have hprod_sq :
0261     ((a * b).toRat - (a' * b').toRat) *
0262     ((a * b).toRat - (a' * b').toRat) < rho.toRat := by
0263     simp [PRCRat.toRat_mul]
0264     exact rational_product_increment_sq_lt
0265     (a := a.toRat) (a' := a'.toRat) (b := b.toRat) (b' := b'.toRat)
0266     (M := B.toRat) (eta := eta.toRat) (rho := rho.toRat)
0267     hB_pos hrho_pos (by rw [heta, hK]) ha_sq hb_sq hda_sq hdb_sq
0268   rw [PRCRat.lt_iff_toRat_lt]
0269   rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment]
0270   exact PRCJCostDistanceIncrementDisplay_lt_of_sq_lt
0271   (t := (a * b).toRat - (a' * b').toRat)
0272   (eta := rho.toRat) (eps := eps.toRat)
0273   hrho_pos hrho_lt_one hprod_sq hrho_sq_half_lt_eps
0274
0275 structure PRCRealProductContinuityCertificate : Prop where
0276   product_continuity : PRCJCostDistanceMulBoundedContinuityTarget
0277   mul_closure : PRCRealMulClosureTarget
0278   mul_congruence : PRCRealMulCongruenceTarget
0279   mul_operation :
0280     Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
0281
0282 theorem prc_real_product_continuity_certificate :
0283   PRCRealProductContinuityCertificate where
0284   product_continuity := PRCJCostDistanceMulBoundedContinuityTarget_proved
0285   mul_closure :=
0286     PRCRealMulClosureTarget_of_bounded_continuity
0287     PRCCauchySeqEventuallyBoundedTarget_proved
0288     PRCJCostDistanceMulBoundedContinuityTarget_proved
0289   mul_congruence :=
0290     PRCRealMulCongruenceTarget_of_bounded_continuity
0291     PRCCauchySeqEventuallyBoundedTarget_proved
0292     PRCJCostDistanceMulBoundedContinuityTarget_proved

```

```
0293   mul_operation := by
0294     exact (PRCRealMulClosed.mul0f
0295       (PRCRealMulClosureTarget_of_bounded_continuity
0296         PRCCauchySeqEventuallyBoundedTarget_proved
0297         PRCJCostDistanceMulBoundedContinuityTarget_proved)
0298       (PRCRealMulCongruenceTarget_of_bounded_continuity
0299         PRCCauchySeqEventuallyBoundedTarget_proved
0300         PRCJCostDistanceMulBoundedContinuityTarget_proved))
0301
0302 end PrimitiveRecognitionCalculus
0303 end Foundation
0304 end IndisputableMonolith
```

21. RealOrderCongruence.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealOrderCongruence.lean · Lines: 206

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealOrderCongruence.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Build Order step 10d: prove that the eventual non-strict order candidate
0009   descends to the null-distance quotient.
0010
0011   The proof uses verifier rationals only to read the J-cost distance as an
0012   ordinary small increment. The order statement itself remains over PRC raw
0013   ledgers.
0014 -/
0015
0016 import Mathlib
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealProductContinuity
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 private theorem rat_sq_lt_sq_bounds {x gamma : ℚ}
0024   (hgamma : 0 < gamma) (hsq : x * x < gamma * gamma) :
0025   -gamma < x ∧ x < gamma := by
0026   constructor
0027   · by_contra hnot
0028     have hxle : x ≤ -gamma := by linarith
0029     have hnonneg : 0 ≤ -x - gamma := by linarith
0030     have hprod : 0 ≤ (-x - gamma) * (-x + gamma) := by
0031       have hright : 0 ≤ -x + gamma := by linarith
0032       exact mul_nonneg hnonneg hright
0033     nlinarith
0034   · by_contra hnot
0035     have hxge : gamma ≤ x := by linarith
0036     have hnonneg : 0 ≤ x - gamma := by linarith
0037     have hprod : 0 ≤ (x - gamma) * (x + gamma) := by
0038       have hright : 0 ≤ x + gamma := by linarith
0039     exact mul_nonneg hnonneg hright
0040   nlinarith
0041
0042 theorem PRCJCostDistance_sq_diff_lt_of_lt_modulus
0043   {a b eta delta : PRCRat}
0044   (heta : PRCRat.positive eta)
0045   (hdelta_pos : PRCRat.positive delta)
0046   (hdelta_le :
0047     delta.toRat ≤ eta.toRat * eta.toRat / (4 * (1 + eta.toRat)))
0048   (hsmall : PRCRat.lt (PRCJCostDistance a b) delta) :
0049   (a.toRat - b.toRat) * (a.toRat - b.toRat) < eta.toRat := by
0050   have heta_pos : 0 < eta.toRat := (PRCRat.positive_iff_toRat_pos eta).mp heta
0051   have hdelta_pos_rat : 0 < delta.toRat :=
0052     (PRCRat.positive_iff_toRat_pos delta).mp hdelta_pos
0053   have hdisplay :
0054     PRCJCostDistanceIncrementDisplay (a.toRat - b.toRat) < delta.toRat := by
0055     rw [PRCRat.lt_iff_toRat_lt] at hsmall
0056     rw [PRCJCostDistance_toRat, PRCJCostDistanceRatDisplay_as_increment] at hsmall
0057     exact hsmall
0058   exact PRCJCostDistance_sq_lt_of_display_lt_delta
0059   (t := a.toRat - b.toRat) (eta := eta.toRat) (delta := delta.toRat)
0060   heta_pos hdelta_pos_rat hdelta_le hdisplay
0061
0062 theorem PRCJCostDistance_abs_diff_lt_of_lt_order_delta
0063   {a b gamma delta : PRCRat}
0064   (hgamma : PRCRat.positive gamma)
0065   (hdelta_pos : PRCRat.positive delta)
0066   (hdelta_le :
0067     delta.toRat ≤
0068       (gamma.toRat * gamma.toRat) * (gamma.toRat * gamma.toRat) /
0069       (4 * (1 + gamma.toRat * gamma.toRat)))
0070   (hsmall : PRCRat.lt (PRCJCostDistance a b) delta) :
0071   -gamma.toRat < a.toRat - b.toRat ∧
0072   a.toRat - b.toRat < gamma.toRat := by
0073   let eta : PRCRat := gamma * gamma
0074   have heta : PRCRat.positive eta := by
0075     rw [PRCRat.positive_iff_toRat_pos]
0076     have hgamma_pos : 0 < gamma.toRat :=
0077       (PRCRat.positive_iff_toRat_pos gamma).mp hgamma
0078     simp [eta, PRCRat.toRat_mul]
0079     nlinarith
0080   have heta_toRat : eta.toRat = gamma.toRat * gamma.toRat := by
0081     simp [eta, PRCRat.toRat_mul]
0082   have hsq :
0083     (a.toRat - b.toRat) * (a.toRat - b.toRat) <
0084     gamma.toRat * gamma.toRat := by
0085     have hcore := PRCJCostDistance_sq_diff_lt_of_lt_modulus
0086     (a := a) (b := b) (eta := eta) (delta := delta)
0087     heta hdelta_pos (by simp [heta_toRat] using hdelta_le) hsmall
0088     simp [heta_toRat] using hcore
0089     exact rat_sq_lt_sq_bounds
0090     ((PRCRat.positive_iff_toRat_pos gamma).mp hgamma) hsq

```

```

0091 theorem PRCRawEventuallyLe_of_null_equiv
0092 {u u' v v' : PRCauchySeq}
0093 (huu : PRCNullEquivalent u u')
0094 (hvv : PRCNullEquivalent v v')
0095 (hle : PRCRawEventuallyLe u.raw v.raw) :
0096   PRCRawEventuallyLe u'.raw v'.raw := by
0097   intro eps heps
0098   let two : PRCRat := (1 : PRCRat) + (1 : PRCRat)
0099   let four : PRCRat := two * two
0100   let gamma : PRCRat := eps * (four⁻¹)
0101   let eta : PRCRat := gamma * gamma
0102   let delta : PRCRat := (eta * eta) * ((four * ((1 : PRCRat) + eta))⁻¹)
0103   have heps_pos : 0 < eps.toRat :=
0104     (PRCRat.positive_iff_toRat_pos eps).mp heps
0105   have htwo : two.toRat = (2 : ℚ) := by
0106     dsimp [two]
0107     change (PRCRat.add PRCRat.one PRCRat.one).toRat = (2 : ℚ)
0108     rw [PRCRat.toRat_add, PRCRat.one_toRat]
0109   norm_num
0110   have hfour : four.toRat = (4 : ℚ) := by
0111     dsimp [four]
0112     change (PRCRat.mul two two).toRat = (4 : ℚ)
0113     rw [PRCRat.toRat_mul]
0114   norm_num [htwo]
0115   have hgamma_toRat : gamma.toRat = eps.toRat / 4 := by
0116     dsimp [gamma]
0117     rw [PRCRat.toRat_mul, PRCRat.toRat_recip, hfour]
0118   ring
0119   have hgamma_pos_rat : 0 < gamma.toRat := by
0120     rw [hgamma_toRat]
0121     positivity
0122   have hgamma_pos : PRCRat.positive gamma := by
0123     rw [PRCRat.positive_iff_toRat_pos]
0124     exact hgamma_pos_rat
0125   have heta_toRat : eta.toRat = gamma.toRat * gamma.toRat := by
0126     simp [eta, PRCRat.toRat_mul]
0127   have heta_pos_rat : 0 < eta.toRat := by
0128     rw [heta_toRat]
0129     nlinarith
0130   have h_one_add_eta :
0131     ((1 : PRCRat) + eta).toRat = 1 + eta.toRat := by
0132     change (PRCRat.add PRCRat.one eta).toRat = 1 + eta.toRat
0133     rw [PRCRat.toRat_add, PRCRat.one_toRat]
0134   have hdelta_toRat :
0135     delta.toRat =
0136       eta.toRat * eta.toRat / (4 * (1 + eta.toRat)) := by
0137     dsimp [delta]
0138     simp [PRCRat.toRat_mul, PRCRat.toRat_recip, PRCRat.toRat_add,
0139           PRCRat.one_toRat, hfour]
0140   have hden_pos : (0 : ℚ) < 4 * (1 + eta.toRat) := by positivity
0141   field_simp [ne_of_gt hden_pos]
0142   have hdelta_pos_rat : 0 < delta.toRat := by
0143     rw [hdelta_toRat]
0144     positivity
0145   have hdelta_pos : PRCRat.positive delta := by
0146     rw [PRCRat.positive_iff_toRat_pos]
0147     exact hdelta_pos_rat
0148   rcases huu delta hdelta_pos with (Nu, hNu)
0149   rcases hvv delta hdelta_pos with (Nv, hNv)
0150   rcases hle gamma hgamma_pos with (Nle, hNle)
0151   refine (max (max Nu Nv) Nle, ?_)
0152   intro n hn
0153   have hNu_n : Nu ≤ n :=
0154     le_trans (Nat.le_max_left Nu Nv)
0155     (le_trans (Nat.le_max_left (max Nu Nv) Nle) hn)
0156   have hNv_n : Nv ≤ n :=
0157     le_trans (Nat.le_max_right Nu Nv)
0158     (le_trans (Nat.le_max_left (max Nu Nv) Nle) hn)
0159   have hNle_n : Nle ≤ n :=
0160     le_trans (Nat.le_max_right (max Nu Nv) Nle) hn
0161   have hu_close := PRCJCostDistance_abs_diff_lt_of_lt_order_delta
0162   (a := u.term n) (b := u'.term n) (gamma := gamma) (delta := delta)
0163   hgamma_pos hdelta_pos (by simp [hdelta_toRat, heta_toRat])
0164   (hNu n hNu_n)
0165   have hv_close := PRCJCostDistance_abs_diff_lt_of_lt_order_delta
0166   (a := v.term n) (b := v'.term n) (gamma := gamma) (delta := delta)
0167   hgamma_pos hdelta_pos (by simp [hdelta_toRat, heta_toRat])
0168   (hNv n hNv_n)
0169   have hu'_lt : (u'.term n).toRat < (u.term n).toRat + gamma.toRat := by
0170     rcases hu_close with (hlo, _hhi)
0171     nlinarith
0172   have hv'_lt : (v'.term n).toRat < (v.term n).toRat + gamma.toRat := by
0173     rcases hv_close with (hlo, _hhi)
0174     nlinarith
0175   have huv'_lt : (u'.term n).toRat < (v'.term n).toRat + gamma.toRat := by
0176     have hle_n := hNle n hNle_n
0177     rw [PRCRat.lt_iff_toRat_lt] at hle_n
0178     simp [PRCcauchySeq.raw, PRCRat.toRat_add] using hle_n
0179     rw [PRCRat.lt_iff_toRat_lt]
0180     simp [PRCcauchySeq.raw, PRCRat.toRat_add]
0181   have hthree_gamma_lt_eps : 3 * gamma.toRat < eps.toRat := by
0182     rw [hgamma_toRat]
0183     nlinarith
0184   nlinarith
0185   nlinarith
0186
0187 theorem PRCRealOrderCongruenceTarget_proved :
0188   PRCRealOrderCongruenceTarget := by
0189     intro u u' v v' huu hvv
0190     constructor
0191     · intro hle

```



```
0192   exact PRCRawEventuallyLe_of_null_equiv huu hvv hle
0193   · intro hle
0194   exact PRCRawEventuallyLe_of_null_equiv
0195     (PRCNullEquivalent.symm huu) (PRCNullEquivalent.symm hvv) hle
0196
0197   structure PRCRealOrderCongruenceCertificate : Prop where
0198     order_congruence : PRCRealOrderCongruenceTarget
0199
0200   theorem prc_real_order_congruence_certificate :
0201     PRCRealOrderCongruenceCertificate where
0202     order_congruence := PRCRealOrderCongruenceTarget_proved
0203
0204   end PrimitiveRecognitionCalculus
0205   end Foundation
0206   end IndisputableMonolith
```

22. RealCompleteness.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteness.lean · Lines: 592

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealCompleteness.lean
0003
0004   Round-trip source:
0005     6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008     Build Order step 10e: replace the placeholder completeness target with a
0009     theorem-shaped internal Cauchy-of-Cauchy statement and isolate the exact
0010     diagonal blocker.
0011
0012   This file closes the construction fact that every raw PRC rational Cauchy
0013   ledger already determines a point of the null quotient, then closes the
0014   explicit `PRCRealCompletenessTarget` diagonal theorem.
0015 -/
0016
0017 import MathLib
0018 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealOrderCongruence
0019
0020 namespace IndisputableMonolith
0021 namespace Foundation
0022 namespace PrimitiveRecognitionCalculus
0023
0024 /-- Every raw rational Cauchy ledger can be packaged as a `PRCCauchySeq`. -/
0025 def PRCRawCauchyRealizationTarget : Prop :=
0026   ∀ s : PRCRawRatLedger,
0027     PRCRawCauchy s →
0028       ∃ u : PRCCauchySeq, u.raw = s
0029
0030 theorem PRCRawCauchyRealizationTarget_proved :
0031   PRCRawCauchyRealizationTarget := by
0032     intro s hs
0033     refine ({ term := s, cauchy := hs }, ?_)
0034     rfl
0035
0036 /-- Every raw rational Cauchy ledger determines a point of the final
0037 null-distance quotient. -/
0038 def PRCRawCauchyQuotientPointTarget : Prop :=
0039   ∀ s : PRCRawRatLedger,
0040     PRCRawCauchy s →
0041       Nonempty PRCRealNullClosed
0042
0043 theorem PRCRawCauchyQuotientPointTarget_proved :
0044   PRCRawCauchyQuotientPointTarget := by
0045     intro s hs
0046     rcases PRCRawCauchyRealizationTarget_proved s hs with (u, _hu)
0047     exact {Quot.mk
0048       (PRCNullDistanceSetoidOfTransitive PRCNullDistanceTransitiveTarget_proved)
0049       u}
0050
0051 /-- Named diagonal selection blocker for internal completeness. It asks for an
0052 actual Cauchy ledger limit for every representative-Cauchy sequence of Cauchy
0053 ledgers. -/
0054 def PRCRealDiagonalSelectionTarget : Prop :=
0055   ∀ U : Nat → PRCRawCauchySeq,
0056     PRCRealRepresentativeCauchy U →
0057       ∃ L : PRCRawCauchySeq, PRCRealRepresentativeLimit U L
0058
0059 /-- Sharper diagonal blocker: construct the raw rational ledger underneath the
0060 limit and prove both its Cauchy property and its limit property. This is the
0061 mathematical work left after the quotient and packaging bookkeeping is removed. -/
0062 def PRCRealRawDiagonalLedgerTarget : Prop :=
0063   ∀ U : Nat → PRCRawCauchySeq,
0064     PRCRealRepresentativeCauchy U →
0065       ∃ s : PRCRawRatLedger,
0066         PRCRawCauchy s ∧
0067         ∀ eps : PRCRat, PRCRat.positive eps →
0068           ∃ N : Nat, ∀ n : Nat, N ≤ n →
0069             PRCRawEventuallyClose (U n).raw s eps
0070
0071 /-- Tail-selection version of the raw diagonal theorem. It asks for an explicit
0072 choice of a sufficiently deep raw index in each representative, so the diagonal
0073 ledger is made from actual terms of the input Cauchy ledgers. -/
0074 def PRCRealTailSelectionTarget : Prop :=
0075   ∀ U : Nat → PRCRawCauchySeq,
0076     PRCRealRepresentativeCauchy U →
0077       ∃ pick : Nat → Nat,
0078         let s : PRCRawRatLedger := fun n => (U n).term (pick n)
0079         PRCRawCauchy s ∧
0080         ∀ eps : PRCRat, PRCRat.positive eps →
0081           ∃ N : Nat, ∀ n : Nat, N ≤ n →
0082             PRCRawEventuallyClose (U n).raw s eps
0083
0084 /-- The PRC rational unit-fraction tolerance `1 / (n+1)`. This is verifier
0085 display machinery for the completeness proof, not a new PRC primitive. -/
0086 def PRCUnitFraction (n : Nat) : PRCRat :=
0087   let den : DistinctionNat := DistinctionNat.ofNat (n + 1)
0088   have hden : den ≠ DistinctionNat.zero := by
0089     intro h
0090     have hnat := congrArg DistinctionNat.toNat h

```

```

0091   rw [DistinctionNat.toNat_ofNat, DistinctionNat.toNat_zero] at hnat
0092   omega
0093   PRCRat.mk {
0094     num := SignedOrbit.one
0095     den := den
0096     den_ne_zero := hden
0097   }
0098
0099   theorem PRCUnitFraction_toRat (n : Nat) :
0100     (PRCUnitFraction n).toRat = (1 : Q) / (n + 1 : Nat) := by
0101     unfold PRCUnitFraction
0102     rw [PRCRat.toRat_mk]
0103     unfold RatioOrbit.toRat
0104     simp [SignedOrbit.one_toInt, DistinctionNat.toNat_ofNat]
0105
0106   theorem PRCUnitFraction_positive (n : Nat) :
0107     PRCRat.positive (PRCUnitFraction n) := by
0108     rw [PRCRat.positive_iff_toRat_pos, PRCUnitFraction_toRat]
0109     positivity
0110
0111   /-- Exact cofinal tolerance-schedule target needed by the tail-selection
0112   diagonal proof. -/
0113   def PRCRealCofinalToleranceScheduleTarget : Prop :=
0114     ∃ tau : Nat → PRCRat,
0115     (∀ n : Nat, PRCRat.positive (tau n)) ∧
0116     ∀ eps : PRCRat, PRCRat.positive eps →
0117       ∃ N : Nat, ∀ n : Nat, N ≤ n → PRCRat.lt (tau n) eps
0118
0119   theorem PRCUnitFraction_eventually_lt
0120     {eps : PRCRat} (heps : PRCRat.positive eps) :
0121     ∃ N : Nat, ∀ n : Nat, N ≤ n →
0122       PRCRat.lt (PRCUnitFraction n) eps := by
0123     have heps_pos : (0 : Q) < eps.toRat :=
0124       (PRCRat.positive_iff_toRat_pos eps).mp heps
0125     rcases exists_nat_gt (1 / eps.toRat) with (N, hN)
0126     refine (N, ?_)
0127     intro n hn
0128     rw [PRCRat.lt_iff_toRat_lt, PRCUnitFraction_toRat]
0129     have hN_le_n : (N : Q) ≤ (n : Q) := by exact_mod_cast hn
0130     have hn_lt_succ : (n : Q) < (n + 1 : Nat) := by
0131       norm_num
0132     have hgt : (1 / eps.toRat : Q) < (n + 1 : Nat) := by
0133       exact lt_of_lt_of_le hN (le_trans hN_le_n (le_of_lt hn_lt_succ))
0134     have hprod : (1 : Q) < eps.toRat * (n + 1 : Nat) := by
0135       have hmul := mul_lt_mul_of_pos_left hgt heps_pos
0136       have hone : eps.toRat * (1 / eps.toRat) = (1 : Q) := by
0137         field_simp [ne_of_gt heps_pos]
0138       rw [hone] at hmul
0139       simpa [mul_comm] using hmul
0140     have hden_pos : (0 : Q) < (n + 1 : Nat) := by positivity
0141     field_simp [ne_of_gt hden_pos]
0142     linarith
0143
0144   theorem PRCRealCofinalToleranceScheduleTarget_proved :
0145     PRCRealCofinalToleranceScheduleTarget := by
0146     refine (PRCUnitFraction, PRCUnitFraction_positive, ?_)
0147     intro eps heps
0148     exact PRCUnitFraction_eventually_lt heps
0149
0150   theorem PRCRat.lt_trans {a b c : PRCRat}
0151     (hab : PRCRat.lt a b) (hbc : PRCRat.lt b c) :
0152     PRCRat.lt a c := by
0153     rw [PRCRat.lt_iff_toRat_lt] at hab hbc
0154     exact _root_.lt_trans hab hbc
0155
0156   /-- Three-leg form of the J-cost distance modulus. The diagonal proof naturally
0157   travels from a selected diagonal point to an intermediate raw point, then across
0158   representatives, then back down another selected diagonal point. -/
0159   def PRCJCostDistanceThreeLegModulusTarget : Prop :=
0160     ∀ eps : PRCRat, PRCRat.positive eps →
0161     ∃ delta : PRCRat, PRCRat.positive delta ∧
0162     ∀ a b c d : PRCRat,
0163       PRCRat.lt (PRCJCostDistance a b) delta →
0164       PRCRat.lt (PRCJCostDistance b c) delta →
0165       PRCRat.lt (PRCJCostDistance c d) delta →
0166       PRCRat.lt (PRCJCostDistance a d) eps
0167
0168   theorem PRCJCostDistanceThreeLegModulusTarget_proved :
0169     PRCJCostDistanceThreeLegModulusTarget := by
0170     intro eps heps
0171     rcases PRCJCostDistanceTriangleModulusTarget_proved eps heps with
0172       (eta heta_pos, heta_tri)
0173     rcases PRCJCostDistanceTriangleModulusTarget_proved eta heta_pos with
0174       (theta, htheta_pos, htheta_tri)
0175     rcases PRCUnitFraction_eventually_lt heta_pos with (Neta, hNeta)
0176     rcases PRCUnitFraction_eventually_lt htheta_pos with (Ntheta, hNtheta)
0177     let delta := PRCUnitFraction (max Neta Ntheta)
0178     have hdelta_pos : PRCRat.positive delta := PRCUnitFraction_positive _
0179     have hdelta_lt_eta : PRCRat.lt delta eta := by
0180       exact hNeta (max Neta Ntheta) (Nat.le_max_left Neta Ntheta)
0181     have hdelta_lt_theta : PRCRat.lt delta theta := by
0182       exact hNtheta (max Neta Ntheta) (Nat.le_max_right Neta Ntheta)
0183     refine (delta, hdelta_pos, ?_)
0184     intro a b c d hab hbc hcd
0185     have hab_eta : PRCRat.lt (PRCJCostDistance a b) eta :=
0186       PRCRat.lt_trans hab hdelta_lt_eta
0187     have hbc_theta : PRCRat.lt (PRCJCostDistance b c) theta :=
0188       PRCRat.lt_trans hbc hdelta_lt_theta
0189     have hcd_theta : PRCRat.lt (PRCJCostDistance c d) theta :=
0190       PRCRat.lt_trans hcd hdelta_lt_theta
0191     have hbd_eta : PRCRat.lt (PRCJCostDistance b d) eta :=

```

```

0192   htheta_tri b c d hbc_theta hcd_theta
0193   exact heta_tri a b d hab_eta hbd_eta
0194
0195 /-- A candidate index is deep enough for one raw Cauchy ledger at one
0196 tolerance. -/
0197 def PRCRowTailBound (u : PRCCauchySeq) (eps : PRCRat) (N : Nat) : Prop :=
0198   ∀ m n : Nat, N ≤ m → N ≤ n →
0199     PRCRat.lt (PRCJCostDistance (u.term m) (u.term n)) eps
0200
0201 theorem PRCRowTailBound_mono {u : PRCCauchySeq} {eps : PRCRat}
0202   {N M : Nat} (hNM : N ≤ M) (hN : PRCRowTailBound u eps N) :
0203   PRCRowTailBound u eps M := by
0204   intro m n hm hn
0205   exact hN m n (le_trans hNM hm) (le_trans hNM hn)
0206
0207 theorem PRCRealFiniteRowTailBound_exists
0208   (U : Nat → PRCCauchySeq) (eps : PRCRat)
0209   (heps : PRCRat.positive eps) :
0210   ∀ r : Nat,
0211     ∃ N : Nat, ∀ i : Nat, i ≤ r → PRCRowTailBound (U i) eps N
0212 | 0 => by
0213   rcases (U 0).cauchy eps heps with (N, hN)
0214   refine (N, ?_)
0215   intro i hi
0216   have hi0 : i = 0 := by omega
0217   subst hi0
0218   exact hN
0219 | Nat.succ r => by
0220   rcases PRCRealFiniteRowTailBound_exists U eps heps r with
0221     (Nprev, hprev)
0222   rcases (U (Nat.succ r)).cauchy eps heps with (Nlast, hlast)
0223   refine (max Nprev Nlast, ?_)
0224   intro i hi
0225   by_cases hir : i ≤ r
0226   · exact PRCRowTailBound_mono
0227     (Nat.le_max_left Nprev Nlast) (hprev i hir)
0228   · have hi_last : i = Nat.succ r := by omega
0229     subst hi_last
0230     exact PRCRowTailBound_mono
0231       (Nat.le_max_right Nprev Nlast) hlast
0232
0233 /-- Exact finite-row scheduler needed by the diagonal tail-selection proof:
0234 for every diagonal row `r`, choose one raw index deep enough for all rows
0235 `0,...,r` at the tolerance `1/(r+1)`. -/
0236 def PRCRealFiniteRowTailSelectionTarget : Prop :=
0237   ∀ U : Nat → PRCCauchySeq,
0238     ∃ pick : Nat → Nat,
0239     ∀ r i : Nat, i ≤ r →
0240       PRCRowTailBound (U i) (PRCUnitFraction r) (pick r)
0241
0242 theorem PRCRealFiniteRowTailSelectionTarget_proved :
0243   PRCRealFiniteRowTailSelectionTarget := by
0244   intro U
0245   choose pick hpick using
0246   fun r => PRCRealFiniteRowTailBound_exists
0247     U (PRCUnitFraction r) (PRCUnitFraction_positive r) r
0248   exact (pick, hpick)
0249
0250 /-- A raw index is deep enough for every eligible representative row
0251 `i ≤ r` to be close to the target row `r`, once the outer representative
0252 threshold has been crossed. -/
0253 def PRCRepresentativeFiniteTailBound
0254   (U : Nat → PRCCauchySeq) (eps : PRCRat)
0255   (outer r N : Nat) : Prop :=
0256   ∀ i k : Nat, outer ≤ i → i ≤ r → N ≤ k →
0257     PRCRat.lt (PRCJCostDistance ((U i).term k) ((U r).term k)) eps
0258
0259 theorem PRCRepresentativeFiniteTailBound_mono
0260   {U : Nat → PRCCauchySeq} {eps : PRCRat}
0261   {outer r N M : Nat} (hNM : N ≤ M)
0262   (hN : PRCRepresentativeFiniteTailBound U eps outer r N) :
0263   PRCRepresentativeFiniteTailBound U eps outer r M := by
0264   intro i k hoi hir hMk
0265   exact hN i k hoi hir (le_trans hNM hMk)
0266
0267 theorem PRCRepresentativeFiniteTailBound_exists
0268   (U : Nat → PRCCauchySeq) (eps : PRCRat)
0269   (outer r : Nat)
0270   (houter : ∀ m n : Nat, outer ≤ m → outer ≤ n →
0271     PRCRawEventuallyClose (U m).raw (U n).raw eps) :
0272   ∃ N : Nat, PRCRepresentativeFiniteTailBound U eps outer r N := by
0273   suffices hfinite :
0274     ∀ limit : Nat,
0275       limit ≤ r →
0276       ∃ N : Nat,
0277         ∀ i k : Nat, outer ≤ i → i ≤ limit → N ≤ k →
0278           PRCRat.lt
0279             (PRCJCostDistance ((U i).term k) ((U r).term k)) eps by
0280   rcases hfinite r (Nat.le_refl r) with (N, hN)
0281   exact (N, hN)
0282   intro limit
0283   induction limit with
0284   | zero =>
0285     intro _hlim
0286     by_cases houter_zero : outer ≤ 0
0287     · have houter_r : outer ≤ r := by omega
0288       rcases houter 0 r houter_zero houter_r with (N, hN)
0289       refine (N, ?_)
0290       intro i k hoi hir hNk
0291       have hi0 : i = 0 := by omega
0292       subst hi0

```

```

0293   simpα [PRCCauchySeq.raw] using hN k hNk
0294   · refine (0, ?_)
0295   intro i _k hoi hir _h0k
0296   have : outer ≤ 0 := by omega
0297   exact False.elim (houter_zero this)
0298 | succ limit ih =>
0299   intro hlim
0300   rcases ih (Nat.le_of_succ.le hlim) with (Nprev, hprev)
0301   by_cases hlast : outer ≤ Nat.succ limit
0302   · rcases houter (Nat.succ limit) r hlast
0303     (le_trans hlast hlim) with
0304     (Nlast, hNlast)
0305     refine (max Nprev Nlast, ?_)
0306     intro i k hoi hir hmaxk
0307     by_cases hir_prev : i ≤ limit
0308     · exact hprev i k hoi hir_prev
0309     (le_trans (Nat.le_max_left Nprev Nlast) hmaxk)
0310     · have hi_last : i = Nat.succ limit := by omega
0311       subst hi_last
0312       simpα [PRCCauchySeq.raw] using
0313       hNlast k (le_trans (Nat.le_max_right Nprev Nlast) hmaxk)
0314     · refine (Nprev, ?_)
0315     intro i k hoi hir hNprevk
0316     have hir_prev : i ≤ limit := by omega
0317     exact hprev i k hoi hir_prev hNprevk
0318
0319 /-- Exact finite representative scheduler needed by the diagonal proof. For
0320 each tolerance rung it chooses the outer Cauchy-representative threshold and a
0321 raw depth that realizes all finite representative-tail comparisons up to that
0322 rung. -/
0323 def PRCRealFiniteRepresentativeTailSelectionTarget : Prop :=
0324   ∀ U : Nat → PRCCauchySeq,
0325   PRCRealRepresentativeCauchy U →
0326   ∃ outer pick : Nat → Nat,
0327   (∀ r m n : Nat, outer r ≤ m → outer r ≤ n →
0328     PRCRawEventuallyClose (U m).raw (U n).raw (PRCUnitFraction r)) ∧
0329   ∀ r : Nat,
0330     PRCRepresentativeFiniteTailBound
0331     U (PRCUnitFraction r) (outer r) r (pick r)
0332
0333 theorem PRCRealFiniteRepresentativeTailSelectionTarget_proved :
0334   PRCRealFiniteRepresentativeTailSelectionTarget := by
0335   intro U hU
0336   choose outer houter using
0337   fun r => hU (PRCUnitFraction r) (PRCUnitFraction_positive r)
0338   choose pick hpick using
0339   fun r => PRCRepresentativeFiniteTailBound_exists
0340   U (PRCUnitFraction r) (outer r) r (houter r)
0341   exact (outer, pick, houter, hpick)
0342
0343 /-- Single finite diagonal scheduler: one raw choice function satisfies both
0344 finite row-tail and finite representative-tail constraints at each tolerance
0345 rung. This is still finite-rung scheduling, not yet the completed global
0346 tail-selection theorem. -/
0347 def PRCRealFiniteDiagonalScheduleTarget : Prop :=
0348   ∀ U : Nat → PRCCauchySeq,
0349   PRCRealRepresentativeCauchy U →
0350   ∃ outer pick : Nat → Nat,
0351   (∀ r m n : Nat, outer r ≤ m → outer r ≤ n →
0352     PRCRawEventuallyClose (U m).raw (U n).raw (PRCUnitFraction r)) ∧
0353   (∀ r i : Nat, i ≤ r →
0354     PRCRowTailBound (U i) (PRCUnitFraction r) (pick r)) ∧
0355   ∀ r : Nat,
0356     PRCRepresentativeFiniteTailBound
0357     U (PRCUnitFraction r) (outer r) r (pick r)
0358
0359 theorem PRCRealFiniteDiagonalScheduleTarget_proved :
0360   PRCRealFiniteDiagonalScheduleTarget := by
0361   intro U hU
0362   rcases PRCRealFiniteRowTailSelectionTarget_proved U with
0363   (rowPick, hrowPick)
0364   rcases PRCRealFiniteRepresentativeTailSelectionTarget_proved U hU with
0365   (outer, repPick, houter, hrepPick)
0366   refine (outer, fun r => max (rowPick r) (repPick r), houter, ?_, ?_)
0367   · intro r i hir
0368     exact PRCRowTailBound_mono
0369     (Nat.le_max_left (rowPick r) (repPick r))
0370     (hrowPick r i hir)
0371   · intro r
0372     exact PRCRepresentativeFiniteTailBound_mono
0373     (Nat.le_max_right (rowPick r) (repPick r))
0374     (hrepPick r)
0375
0376 theorem PRCRealTailSelectionTarget_proved :
0377   PRCRealTailSelectionTarget := by
0378   intro U hU
0379   rcases PRCRealFiniteDiagonalScheduleTarget_proved U hU with
0380   (_outer, pick, _houterTau, hrow, _hrep)
0381   refine (pick, ?_, ?_)
0382   · intro eps heps
0383   rcases PRCJCostDistanceThreeLegModulusTarget_proved eps heps with
0384   (delta, hdelta_pos, hthree)
0385   rcases hU delta hdelta_pos with (Nrep, hNrep)
0386   rcases PRCUnitFraction_eventually_lt hdelta_pos with (Ntau, hNtau)
0387   refine (max Nrep Ntau, ?_)
0388   intro m hm hn
0389   have hm_rep : Nrep ≤ m := le_trans (Nat.le_max_left Nrep Ntau) hm
0390   have hn_rep : Nrep ≤ n := le_trans (Nat.le_max_left Nrep Ntau) hn
0391   have hm_tauN : Ntau ≤ m := le_trans (Nat.le_max_right Nrep Ntau) hm
0392   have hn_tauN : Ntau ≤ n := le_trans (Nat.le_max_right Nrep Ntau) hn
0393   have htaum_delta : PRCRat.lt (PRCUnitFraction m) delta :=

```

```

0394 hNtau m hm_tauN
0395 have htaun_delta : PRCRat.lt (PRCUnitFraction n) delta :=
0396 hNtau n hn_tauN
0397 rcases hNrep m n hm_rep hn_rep with (Npair, hNpair)
0398 let K : Nat := max (pick m) (max (pick n) Npair)
0399 have hpickmK : pick m ≤ K := Nat.le_max_left (pick m) (max (pick n) Npair)
0400 have hpicknK : pick n ≤ K :=
0401   le_trans (Nat.le_max_left (pick n) Npair)
0402   (Nat.le_max_right (pick m) (max (pick n) Npair))
0403 have hpairK : Npair ≤ K :=
0404   le_trans (Nat.le_max_right (pick n) Npair)
0405   (Nat.le_max_right (pick m) (max (pick n) Npair))
0406 have hleg1_tau :
0407   PRCRat.lt
0408     (PRCJCostDistance ((U m).term (pick m)) ((U m).term K))
0409     (PRCUnitFraction m) :=
0410     (hrow m m (Nat.le_refl m)) (pick m) K (Nat.le_refl (pick m)) hpickmK
0411 have hleg1 :
0412   PRCRat.lt
0413     (PRCJCostDistance ((U m).term (pick m)) ((U m).term K))
0414     delta :=
0415     PRCRat.lt_trans hleg1_tau htaun_delta
0416 have hleg2 :
0417   PRCRat.lt
0418     (PRCJCostDistance ((U m).term K) ((U n).term K))
0419     delta := by
0420     simp [PRCCauchySeq.raw] using hNpair K hpairK
0421 have hleg3_tau :
0422   PRCRat.lt
0423     (PRCJCostDistance ((U n).term K) ((U n).term (pick n)))
0424     (PRCUnitFraction n) :=
0425     (hrow n n (Nat.le_refl n)) K (pick n) hpicknK (Nat.le_refl (pick n))
0426 have hleg3 :
0427   PRCRat.lt
0428     (PRCJCostDistance ((U n).term K) ((U n).term (pick n)))
0429     delta :=
0430     PRCRat.lt_trans hleg3_tau htaun_delta
0431 exact hthree
0432 ((U m).term (pick m)) ((U m).term K)
0433 ((U n).term K) ((U n).term (pick n))
0434 hleg1 hleg2 hleg3
0435 · intro eps heps
0436 rcases PRCJCostDistanceThreeLegModulusTarget_proved eps heps with
0437   (delta, hdelta_pos, hthree)
0438 rcases hU delta hdelta_pos with (Nrep, hNrep)
0439 rcases PRCUnitFraction_eventually_lt hdelta_pos with (Ntau, hNtau)
0440 refine (max Nrep Ntau, ?_)
0441 intro n hn
0442 have hn_rep : Nrep ≤ n := le_trans (Nat.le_max_left Nrep Ntau) hn
0443 rcases (U n).cauchy delta hdelta_pos with (NrowN, hNrowN)
0444 refine (max (max Nrep Ntau) NrowN, ?_)
0445 intro l hl
0446 have hl_rep : Nrep ≤ l :=
0447   le_trans (Nat.le_max_left Nrep Ntau)
0448   (le_trans (Nat.le_max_left (max Nrep Ntau) NrowN) hl)
0449 have hl_tauN : Ntau ≤ l :=
0450   le_trans (Nat.le_max_right Nrep Ntau)
0451   (le_trans (Nat.le_max_left (max Nrep Ntau) NrowN) hl)
0452 have hl_rowN : NrowN ≤ l :=
0453   le_trans (Nat.le_max_right (max Nrep Ntau) NrowN) hl
0454 have htaul_delta : PRCRat.lt (PRCUnitFraction l) delta :=
0455   hNtau l hl_tauN
0456 rcases hNrep n l hn_rep hl_rep with (Npair, hNpair)
0457 let K : Nat := max l (max (pick l) Npair)
0458 have hK : l ≤ K := Nat.le_max_left l (max (pick l) Npair)
0459 have hpicklK : pick l ≤ K :=
0460   le_trans (Nat.le_max_left (pick l) Npair)
0461   (Nat.le_max_right l (max (pick l) Npair))
0462 have hpairK : Npair ≤ K :=
0463   le_trans (Nat.le_max_right (pick l) Npair)
0464   (Nat.le_max_right l (max (pick l) Npair))
0465 have hrowNK : NrowN ≤ K := le_trans hl_rowN hK
0466 have hleg1 :
0467   PRCRat.lt
0468     (PRCJCostDistance ((U n).term l) ((U n).term K))
0469     delta :=
0470     hNrowN l K hl_rowN hrowNK
0471 have hleg2 :
0472   PRCRat.lt
0473     (PRCJCostDistance ((U n).term K) ((U l).term K))
0474     delta := by
0475     simp [PRCCauchySeq.raw] using hNpair K hpairK
0476 have hleg3_tau :
0477   PRCRat.lt
0478     (PRCJCostDistance ((U l).term K) ((U l).term (pick l)))
0479     (PRCUnitFraction l) :=
0480     (hrow l l (Nat.le_refl l)) K (pick l) hpicklK (Nat.le_refl (pick l))
0481 have hleg3 :
0482   PRCRat.lt
0483     (PRCJCostDistance ((U l).term K) ((U l).term (pick l)))
0484     delta :=
0485     PRCRat.lt_trans hleg3_tau htaul_delta
0486 exact hthree
0487 ((U n).term l) ((U n).term K)
0488 ((U l).term K) ((U l).term (pick l))
0489 hleg1 hleg2 hleg3
0490
0491 /-- An explicit tail-selection diagonal is enough to build the raw diagonal
0492 ledger target. -/
0493 theorem PRCRealRawDiagonalLedgerTarget_of_tail_selection
0494   (htail : PRCRealTailSelectionTarget) :

```

```

0495   PRCRealRawDiagonalLedgeTarget := by
0496   intro U hU
0497   rcases htail U hU with (pick, hs_cauchy, hs_limit)
0498   exact (fun n => (U n).term (pick n), hs_cauchy, hs_limit)
0499
0500 theorem PRCRealRawDiagonalLedgeTarget_proved :
0501   PRCRealRawDiagonalLedgeTarget :=
0502   PRCRealRawDiagonalLedgeTarget_of_tail_selection
0503   PRCRealTailSelectionTarget_proved
0504
0505 /-- A raw diagonal ledger packages immediately as the representative limit
0506 needed by the quotient-level diagonal selection target. -/
0507 theorem PRCRealDiagonalSelectionTarget_of_raw_diagonal_ledger
0508   (hraw : PRCRealRawDiagonalLedgeTarget) :
0509   PRCRealDiagonalSelectionTarget := by
0510   intro U hU
0511   rcases hraw U hU with (s, hs_cauchy, hs_limit)
0512   refine ({ term := s, cauchy := hs_cauchy }, ?_)
0513   simpa [PRCRealRepresentativeLimit, PRCCauchySeq.raw] using hs_limit
0514
0515 theorem PRCRealDiagonalSelectionTarget_proved :
0516   PRCRealDiagonalSelectionTarget :=
0517   PRCRealDiagonalSelectionTarget_of_raw_diagonal_ledger
0518   PRCRealRawDiagonalLedgeTarget_proved
0519
0520 /-- The diagonal selection target is exactly the sharpened completeness target. -/
0521 theorem PRCRealCompletenessTarget_of_diagonal_selection
0522   (hdiag : PRCRealDiagonalSelectionTarget) :
0523   PRCRealCompletenessTarget := by
0524   exact hdiag
0525
0526 theorem PRCRealCompletenessTarget_proved :
0527   PRCRealCompletenessTarget :=
0528   PRCRealCompletenessTarget_of_diagonal_selection
0529   PRCRealDiagonalSelectionTarget_proved
0530
0531 /-- The full internal completeness theorem is now the concrete representative
0532 Cauchy-of-Cauchy diagonal target. -/
0533 theorem PRCRealCompletenessTarget_sharpened :
0534   PRCRealCompletenessTarget = PRCRealCompletenessTarget := rfl
0535
0536 /-- Step 10e certificate. It records the closed raw-ledger realization fact,
0537 the tail-selection diagonal, and the representative-completeness theorem for
0538 'PRCRealNullClosed'. -/
0539 structure PRCRealCompletenessSharpenedCertificate : Prop where
0540   raw_cauchy_realization : PRCRawCauchyRealizationTarget
0541   raw_cauchy_quotient_point : PRCRawCauchyQuotientPointTarget
0542   diagonal_selection_from_raw_diagonal :
0543     PRCRealRawDiagonalLedgeTarget → PRCRealDiagonalSelectionTarget
0544   raw_diagonal_from_tail_selection :
0545     PRCRealTailSelectionTarget → PRCRealRawDiagonalLedgeTarget
0546   cofinal_tolerance_schedule : PRCRealCofinalToleranceScheduleTarget
0547   three_leg_distance_modulus : PRCJCostDistanceThreeLegModulusTarget
0548   finite_row_tail_selection : PRCRealFiniteRowTailSelectionTarget
0549   finite_representative_tail_selection :
0550     PRCRealFiniteRepresentativeTailSelectionTarget
0551   finite_diagonal_schedule : PRCRealFiniteDiagonalScheduleTarget
0552   tail_selection : PRCRealTailSelectionTarget
0553   raw_diagonal : PRCRealRawDiagonalLedgeTarget
0554   diagonal_selection : PRCRealDiagonalSelectionTarget
0555   completeness_from_diagonal_selection :
0556     PRCRealDiagonalSelectionTarget → PRCRealCompletenessTarget
0557   completeness : PRCRealCompletenessTarget
0558   completeness_target : PRCRealCompletenessTarget = PRCRealCompletenessTarget
0559
0560 theorem prc_real_completeness_sharpened_certificate :
0561   PRCRealCompletenessSharpenedCertificate where
0562   raw_cauchy_realization := PRCRawCauchyRealizationTarget_proved
0563   raw_cauchy_quotient_point := PRCRawCauchyQuotientPointTarget_proved
0564   diagonal_selection_from_raw_diagonal :=
0565     PRCRealDiagonalSelectionTarget_of_raw_diagonal_ledger
0566   raw_diagonal_from_tail_selection :=
0567     PRCRealRawDiagonalLedgeTarget_of_tail_selection
0568   cofinal_tolerance_schedule :=
0569     PRCRealCofinalToleranceScheduleTarget_proved
0570   three_leg_distance_modulus :=
0571     PRCJCostDistanceThreeLegModulusTarget_proved
0572   finite_row_tail_selection :=
0573     PRCRealFiniteRowTailSelectionTarget_proved
0574   finite_representative_tail_selection :=
0575     PRCRealFiniteRepresentativeTailSelectionTarget_proved
0576   finite_diagonal_schedule :=
0577     PRCRealFiniteDiagonalScheduleTarget_proved
0578   tail_selection :=
0579     PRCRealTailSelectionTarget_proved
0580   raw_diagonal :=
0581     PRCRealRawDiagonalLedgeTarget_proved
0582   diagonal_selection :=
0583     PRCRealDiagonalSelectionTarget_proved
0584   completeness_from_diagonal_selection :=
0585     PRCRealCompletenessTarget_of_diagonal_selection
0586   completeness :=
0587     PRCRealCompletenessTarget_proved
0588   completeness_target := PRCRealCompletenessTarget_sharpened
0589
0590 end PrimitiveRecognitionCalculus
0591 end Foundation
0592 end IndisputableMonolith

```

23. RealCompleteOrderedField.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteOrderedField.lean · Lines: 370

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealCompleteOrderedField.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Build Order step 10: start the complete ordered field surface over the
0009   closed null-distance quotient `PRCRealNullClosed`.
0010
0011   This pass does not alias the carrier to Lean `ℝ`. It exposes the exact
0012   quotient-algebra blockers for addition, multiplication, negation, and order.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceIncrementTriangle
0017
0018 namespace IndisputableMonolith
0019 namespace Foundation
0020 namespace PrimitiveRecognitionCalculus
0021
0022 /-- Raw completed-orbit rational ledger, before a Cauchy proof is attached. -/
0023 abbrev PRCRawRatLedger := Nat → PRCRat
0024
0025 /-- Cauchy predicate on raw ledgers, using the same J-cost distance as
0026 `PRCCauchySeq`. -/
0027 def PRCRawCauchy (s : PRCRawRatLedger) : Prop :=
0028   ∀ eps : PRCRat, PRCRat.positive eps →
0029     ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
0030       PRCRat.lt (PRCJCostDistance (s m) (s n)) eps
0031
0032 /-- Null equivalence on raw ledgers. -/
0033 def PRCRawNullEquivalent (s t : PRCRawRatLedger) : Prop :=
0034   ∀ eps : PRCRat, PRCRat.positive eps →
0035     ∃ N : Nat, ∀ n : Nat, N ≤ n →
0036       PRCRat.lt (PRCJCostDistance (s n) (t n)) eps
0037
0038 namespace PRCCauchySeq
0039
0040 /-- Forget a Cauchy ledger to its raw rational ledger. -/
0041 def raw (u : PRCCauchySeq) : PRCRawRatLedger :=
0042   u.term
0043
0044 theorem raw_cauchy (u : PRCCauchySeq) : PRCRawCauchy u.raw :=
0045   u.cauchy
0046
0047 @[simp] theorem raw_apply (u : PRCCauchySeq) (n : Nat) :
0048   u.raw n = u.term n := rfl
0049
0050 end PRCCauchySeq
0051
0052 /-- Pointwise addition of raw ledgers. -/
0053 def PRCRawAdd (u v : PRCRawRatLedger) : PRCRawRatLedger :=
0054   fun n => u n + v n
0055
0056 /-- Pointwise negation of raw ledgers. -/
0057 def PRCRawNeg (u : PRCRawRatLedger) : PRCRawRatLedger :=
0058   fun n => -u n
0059
0060 /-- Pointwise multiplication of raw ledgers. -/
0061 def PRCRawMul (u v : PRCRawRatLedger) : PRCRawRatLedger :=
0062   fun n => u n * v n
0063
0064 /-- Pointwise non-strict order candidate for raw ledgers. -/
0065 def PRCRawEventuallyLe (u v : PRCRawRatLedger) : Prop :=
0066   ∀ eps : PRCRat, PRCRat.positive eps →
0067     ∃ N : Nat, ∀ n : Nat, N ≤ n →
0068       PRCRat.lt (u n) (v n + eps)
0069
0070 /-- J-cost distance is invariant under translating both endpoints on the
0071 right. -/
0072 theorem PRCJCostDistance_add_right (a b c : PRCRat) :
0073   PRCJCostDistance (a + c) (b + c) = PRCJCostDistance a b := by
0074   apply PRCRat.toRat_injective
0075   rw [PRCJCostDistance_toRat, PRCJCostDistance_toRat]
0076   simp [PRCJCostDistanceRatDisplay]
0077
0078 /-- J-cost distance is invariant under translating both endpoints on the left. -/
0079 theorem PRCJCostDistance_add_left (a b c : PRCRat) :
0080   PRCJCostDistance (c + a) (c + b) = PRCJCostDistance a b := by
0081   apply PRCRat.toRat_injective
0082   rw [PRCJCostDistance_toRat, PRCJCostDistance_toRat]
0083   simp [PRCJCostDistanceRatDisplay]
0084
0085 /-- J-cost distance is invariant under negating both endpoints. -/
0086 theorem PRCJCostDistance_neg_neg (a b : PRCRat) :
0087   PRCJCostDistance (-a) (-b) = PRCJCostDistance a b := by
0088   apply PRCRat.toRat_injective
0089   rw [PRCJCostDistance_toRat, PRCJCostDistance_toRat]
0090   simp [PRCJCostDistanceRatDisplay]

```



```

0091 ring_nf
0092
0093 /-- Exact blocker for addition: pointwise sums of Cauchy ledgers are Cauchy. -/
0094 def PRCRealAddClosureTarget : Prop :=
0095   ∀ u v : PRCCauchySeq, PRCRawCauchy (PRCRawAdd u.raw v.raw)
0096
0097 /-- Exact blocker for additive quotient well-definedness under null distance. -/
0098 def PRCRealAddCongruenceTarget : Prop :=
0099   ∀ u' v v' : PRCCauchySeq,
0100     PRCNullEquivalent u u' →
0101     PRCNullEquivalent v v' →
0102     PRCRawNullEquivalent
0103       (PRCRawAdd u.raw v.raw)
0104       (PRCRawAdd u'.raw v'.raw)
0105
0106 /-- Exact blocker for negation: pointwise negations of Cauchy ledgers are
0107 Cauchy. -/
0108 def PRCRealNegClosureTarget : Prop :=
0109   ∀ u : PRCCauchySeq, PRCRawCauchy (PRCRawNeg u.raw)
0110
0111 /-- Exact blocker for negation quotient well-definedness. -/
0112 def PRCRealNegCongruenceTarget : Prop :=
0113   ∀ u v : PRCCauchySeq,
0114     PRCNullEquivalent u v →
0115     PRCRawNullEquivalent (PRCRawNeg u.raw) (PRCRawNeg v.raw)
0116
0117 /-- Exact blocker for multiplication: pointwise products of Cauchy ledgers are
0118 Cauchy. This is expected to require a boundedness lemma for Cauchy ledgers. -/
0119 def PRCRealMulClosureTarget : Prop :=
0120   ∀ u v : PRCCauchySeq, PRCRawCauchy (PRCRawMul u.raw v.raw)
0121
0122 /-- Exact blocker for multiplicative quotient well-definedness under null
0123 distance. This is also expected to require eventual boundedness. -/
0124 def PRCRealMulCongruenceTarget : Prop :=
0125   ∀ u u' v v' : PRCCauchySeq,
0126     PRCNullEquivalent u u' →
0127     PRCNullEquivalent v v' →
0128     PRCRawNullEquivalent
0129       (PRCRawMul u.raw v.raw)
0130       (PRCRawMul u'.raw v'.raw)
0131
0132 /-- Exact blocker for the order relation descending to the null-distance
0133 quotient. -/
0134 def PRCRealOrderCongruenceTarget : Prop :=
0135   ∀ u u' v v' : PRCCauchySeq,
0136     PRCNullEquivalent u u' →
0137     PRCNullEquivalent v v' →
0138     (PRCRawEventuallyLe u.raw v.raw ↔
0139      PRCRawEventuallyLe u'.raw v'.raw)
0140
0141 /-- Quantitative closeness for two raw ledgers at a fixed PRC tolerance. -/
0142 def PRCRawEventuallyClose (s t : PRCRawRatLedger) (eps : PRCRat) : Prop :=
0143   ∃ N : Nat, ∀ n : Nat, N ≤ n →
0144     PRCRat.lt (PRCJCostDistance (s n) (t n)) eps
0145
0146 /-- A sequence of Cauchy ledgers is Cauchy as a sequence of null-quotient
0147 representatives when its representative tails are eventually close at every
0148 positive PRC tolerance. -/
0149 def PRCRealRepresentativeCauchy (U : Nat → PRCCauchySeq) : Prop :=
0150   ∀ eps : PRCRat, PRCRat.positive eps →
0151     ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
0152       PRCRawEventuallyClose (U m).raw (U n).raw eps
0153
0154 /-- A Cauchy ledger `L` is a representative limit of a sequence of null-quotient
0155 representatives. -/
0156 def PRCRealRepresentativeLimit (U : Nat → PRCCauchySeq)
0157   (L : PRCCauchySeq) : Prop :=
0158   ∀ eps : PRCRat, PRCRat.positive eps →
0159     ∃ N : Nat, ∀ n : Nat, N ≤ n →
0160       PRCRawEventuallyClose (U n).raw L.raw eps
0161
0162 /-- Exact blocker for completeness of the internal null quotient. This is the
0163 diagonal theorem: every Cauchy sequence of Cauchy-ledger representatives has a
0164 Cauchy-ledger representative limit. -/
0165 def PRCRealCompletenessTarget : Prop :=
0166   ∀ U : Nat → PRCCauchySeq,
0167     PRCRealRepresentativeCauchy U →
0168     ∃ L : PRCCauchySeq, PRCRealRepresentativeLimit U L
0169
0170 /-- Pointwise sums of Cauchy ledgers are Cauchy. -/
0171 theorem PRCRealAddClosureTarget_proved : PRCRealAddClosureTarget := by
0172   intro u v eps heps
0173   rcases PRCJCostDistanceTriangleModulusTarget_proved eps heps with
0174     (delta, hdelta_pos, hdelta)
0175   rcases u.cauchy delta hdelta_pos with (Nu, hNu)
0176   rcases v.cauchy delta hdelta_pos with (Nv, hNv)
0177   refine (max Nu Nv, ?_)
0178   intro m n hm hn
0179   have hmu : Nu ≤ m := le_trans (Nat.le_max_left Nu Nv) hm
0180   have hnu : Nu ≤ n := le_trans (Nat.le_max_left Nu Nv) hn
0181   have hmv : Nv ≤ m := le_trans (Nat.le_max_right Nu Nv) hm
0182   have hnv : Nv ≤ n := le_trans (Nat.le_max_right Nu Nv) hn
0183   exact hdelta
0184     ((u.term m) + (v.term m))
0185     ((u.term n) + (v.term m))
0186     ((u.term n) + (v.term n))
0187   (by
0188     rw [PRCJCostDistance_add_right]
0189     exact hNu m n hmu hnu)
0190   (by
0191     rw [PRCJCostDistance_add_left]

```

```

0192     exact hNv m n hmv hnv)
0193
0194 /-- Pointwise negations of Cauchy ledgers are Cauchy. -/
0195 theorem PRCRealNegClosureTarget_proved : PRCRealNegClosureTarget := by
0196   intro u eps heps
0197   rcases u.cauchy eps heps with (N, hN)
0198   refine (N, ?_)
0199   intro m n hm hn
0200   change PRCRat.lt (PRCJCostDistance (-(u.term m)) (-(u.term n))) eps
0201   rw [PRCJCostDistance_neg_neg]
0202   exact hN m n hm hn
0203
0204 /-- Addition respects null equivalence. -/
0205 theorem PRCRealAddCongruenceTarget_proved :
0206   PRCRealAddCongruenceTarget := by
0207   intro u u' v v' huu hvv eps heps
0208   rcases PRCJCostDistanceTriangleModulusTarget_proved eps heps with
0209     (delta, hdelta_pos, hdelta)
0210   rcases huu delta hdelta_pos with (Nu, hNu)
0211   rcases hvv delta hdelta_pos with (Nv, hNv)
0212   refine (max Nu Nv, ?_)
0213   intro n hn
0214   have hNu : Nu ≤ n := le_trans (Nat.le_max_left Nu Nv) hn
0215   have hNv : Nv ≤ n := le_trans (Nat.le_max_right Nu Nv) hn
0216   exact hdelta
0217   ((u.term n) + (v.term n))
0218   ((u'.term n) + (v'.term n))
0219   ((u'.term n) + (v'.term n))
0220   (by
0221     rw [PRCJCostDistance_add_right]
0222     exact hNu n hNu)
0223   (by
0224     rw [PRCJCostDistance_add_left]
0225     exact hNv n hnv)
0226
0227 /-- Negation respects null equivalence. -/
0228 theorem PRCRealNegCongruenceTarget_proved :
0229   PRCRealNegCongruenceTarget := by
0230   intro u v huv eps heps
0231   rcases huv eps heps with (N, hN)
0232   refine (N, ?_)
0233   intro n hn
0234   change PRCRat.lt (PRCJCostDistance (-(u.term n)) (-(v.term n))) eps
0235   rw [PRCJCostDistance_neg_neg]
0236   exact hN n hn
0237
0238 /-- Conditional construction of a pointwise-sum Cauchy ledger from the addition
0239 closure target. -/
0240 def PRCCauchySeq.addOf
0241   (hadd : PRCRealAddClosureTarget) (u v : PRCCauchySeq) : PRCCauchySeq where
0242   term := PRCRawAdd u.raw v.raw
0243   cauchy := hadd u v
0244
0245 /-- Conditional construction of a pointwise-negation Cauchy ledger from the
0246 negation closure target. -/
0247 def PRCCauchySeq.negOf
0248   (hneg : PRCRealNegClosureTarget) (u : PRCCauchySeq) : PRCCauchySeq where
0249   term := PRCRawNeg u.raw
0250   cauchy := hneg u
0251
0252 /-- Conditional construction of a pointwise-product Cauchy ledger from the
0253 multiplication closure target. -/
0254 def PRCCauchySeq.mulOf
0255   (hmul : PRCRealMulClosureTarget) (u v : PRCCauchySeq) : PRCCauchySeq where
0256   term := PRCRawMul u.raw v.raw
0257   cauchy := hmul u v
0258
0259 /-- Conditional addition on the closed null quotient. -/
0260 noncomputable def PRCRealNullClosed.addOf
0261   (hadd : PRCRealAddClosureTarget)
0262   (hcong : PRCRealAddCongruenceTarget) :
0263   PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed :=
0264   Quot.lift
0265     (fun u v =>
0266       Quot.mk (PRCNullDistanceSetoidOfTransitive PRCNullDistanceTransitiveTarget_proved)
0267         (PRCCauchySeq.addOf hadd u v))
0268   (by
0269     intro u v₁ v₂ hv
0270     apply Quot.sound
0271     exact hcong u u v₁ v₂ (PRCNullEquivalent.refl u) hv)
0272   (by
0273     intro u₁ u₂ v hu
0274     apply Quot.sound
0275     exact hcong u₁ u₂ v v hu (PRCNullEquivalent.refl v))
0276
0277 /-- Conditional negation on the closed null quotient. -/
0278 noncomputable def PRCRealNullClosed.negOf
0279   (hneg : PRCRealNegClosureTarget)
0280   (hcong : PRCRealNegCongruenceTarget) :
0281   PRCRealNullClosed → PRCRealNullClosed :=
0282   Quot.lift
0283     (fun u =>
0284       Quot.mk (PRCNullDistanceSetoidOfTransitive PRCNullDistanceTransitiveTarget_proved)
0285         (PRCCauchySeq.negOf hneg u))
0286   (by
0287     intro u v huv
0288     apply Quot.sound
0289     exact hcong u v huv)
0290
0291 /-- Conditional multiplication on the closed null quotient. -/
0292 noncomputable def PRCRealNullClosed.mulOf

```

```

0293 (hmul : PRCRealMulClosureTarget)
0294 (hcong : PRCRealMulCongruenceTarget) :
0295 PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed :=
0296 Quot.lift₂
0297 (fun u v =>
0298   Quot.mk (PRCNullDistanceSetoidOfTransitive PRCNullDistanceTransitiveTarget_proved)
0299   (PRCCauchySeq.mulOf hmul u v))
0300 (by
0301   intro u v₁ v₂ hv
0302   apply Quot.sound
0303   exact hcong u u v₁ v₂ (PRCNullEquivalent.refl u) hv)
0304 (by
0305   intro u₁ u₂ v hu
0306   apply Quot.sound
0307   exact hcong u₁ u₂ v v hu (PRCNullEquivalent.refl v))
0308
0309 /-- Bundle of exact blockers for the next real-completion phase. -/
0310 structure PRCRealCompleteOrderedFieldTargets : Prop where
0311   add_closure : PRCRealAddClosureTarget
0312   add_congruence : PRCRealAddCongruenceTarget
0313   neg_closure : PRCRealNegClosureTarget
0314   neg_congruence : PRCRealNegCongruenceTarget
0315   mul_closure : PRCRealMulClosureTarget = PRCRealMulClosureTarget
0316   mul_congruence : PRCRealMulCongruenceTarget = PRCRealMulCongruenceTarget
0317   order_congruence : PRCRealOrderCongruenceTarget = PRCRealOrderCongruenceTarget
0318   completeness : PRCRealCompletenessTarget = PRCRealCompletenessTarget
0319
0320 /-- Conditional first complete-ordered-field surface. It records that the
0321 carrier and rational embedding are closed, while algebra/order/completeness are
0322 reduced to named exact targets. -/
0323 structure PRCRealCompleteOrderedFieldConditionalCertificate : Prop where
0324   carrier : Nonempty PRCRealNullClosed
0325   rat_embedding : Nonempty (PRCRat → PRCRealNullClosed)
0326   targets : PRCRealCompleteOrderedFieldTargets
0327   add_operation_from_targets :
0328     PRCRealAddClosureTarget →
0329     PRCRealAddCongruenceTarget →
0330     Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
0331   neg_operation_from_targets :
0332     PRCRealNegClosureTarget →
0333     PRCRealNegCongruenceTarget →
0334     Nonempty (PRCRealNullClosed → PRCRealNullClosed)
0335   mul_operation_from_targets :
0336     PRCRealMulClosureTarget →
0337     PRCRealMulCongruenceTarget →
0338     Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
0339   strength_tag : StrengthTag.traceClosure = StrengthTag.traceClosure
0340
0341 /-- Build Order step 10, first pass: quotient algebra is reduced to exact
0342 closure and congruence targets. -/
0343 theorem prc_real_complete_ordered_field_conditional_certificate :
0344   PRCRealCompleteOrderedFieldConditionalCertificate where
0345   carrier := (PRCRealNullClosed.ofRat 0)
0346   rat_embedding := (PRCRealNullClosed.ofRat)
0347   targets := {
0348     add_closure := PRCRealAddClosureTarget_proved
0349     add_congruence := PRCRealAddCongruenceTarget_proved
0350     neg_closure := PRCRealNegClosureTarget_proved
0351     neg_congruence := PRCRealNegCongruenceTarget_proved
0352     mul_closure := rfl
0353     mul_congruence := rfl
0354     order_congruence := rfl
0355     completeness := rfl
0356   }
0357   add_operation_from_targets := by
0358     intro hadd hcong
0359     exact (PRCRealNullClosed.addOf hadd hcong)
0360   neg_operation_from_targets := by
0361     intro hneg hcong
0362     exact (PRCRealNullClosed.negOf hneg hcong)
0363   mul_operation_from_targets := by
0364     intro hmul hcong
0365     exact (PRCRealNullClosed.mulOf hmul hcong)
0366   strength_tag := rfl
0367
0368 end PrimitiveRecognitionCalculus
0369 end Foundation
0370 end IndisputableMonolith

```

24. RealCompleteOrderedFieldPromoted.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompleteOrderedFieldPromoted.lean · Lines: 88

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealCompleteOrderedFieldPromoted.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Build Order step 10: promote the proved add, neg, mul, order, and
0009   representative-completeness targets into one complete ordered-field
0010   certificate surface over `PRCRealNullClosed`.
0011
0012   This is a certificate-promotion layer. It does not alias the internal carrier
0013   to Lean `R`; it bundles the theorem surfaces already proved for the PRC null
0014   quotient.
0015 -/
0016
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteness
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 /-- Promoted Step 10 certificate: the internal null quotient has the closed
0024 operations and theorem surfaces needed by the current complete ordered-field
0025 layer. Full Mathlib typeclass instances remain a later packaging pass. -/
0026 structure PRCRealCompleteOrderedFieldPromotedCertificate : Prop where
0027   carrier : Nonempty PRCRealNullClosed
0028   rat_embedding : Nonempty (PRCRat → PRCRealNullClosed)
0029   add_closure : PRCRealAddClosureTarget
0030   add_congruence : PRCRealAddCongruenceTarget
0031   add_operation :
0032     Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
0033   neg_closure : PRCRealNegClosureTarget
0034   neg_congruence : PRCRealNegCongruenceTarget
0035   neg_operation : Nonempty (PRCRealNullClosed → PRCRealNullClosed)
0036   mul_closure : PRCRealMulClosureTarget
0037   mul_congruence : PRCRealMulCongruenceTarget
0038   mul_operation :
0039     Nonempty (PRCRealNullClosed → PRCRealNullClosed → PRCRealNullClosed)
0040   order_congruence : PRCRealOrderCongruenceTarget
0041   representative_completeness : PRCRealCompletenessTarget
0042   first_pass_certificate : PRCRealCompleteOrderedFieldConditionalCertificate
0043   product_continuity_certificate : PRCRealProductContinuityCertificate
0044   order_congruence_certificate : PRCRealOrderCongruenceCertificate
0045   completeness_certificate : PRCRealCompletenessSharpenedCertificate
0046   strength_tag : StrengthTag.traceClosure = StrengthTag.traceClosure
0047
0048 theorem prc_real_complete_ordered_field_promoted_certificate :
0049   PRCRealCompleteOrderedFieldPromotedCertificate where
0050     carrier := (PRCRealNullClosed.ofRat 0)
0051     rat_embedding := (PRCRealNullClosed.ofRat)
0052     add_closure := PRCRealAddClosureTarget_proved
0053     add_congruence := PRCRealAddCongruenceTarget_proved
0054     add_operation :=
0055       (PRCRealNullClosed.addOf
0056         PRCRealAddClosureTarget_proved
0057         PRCRealAddCongruenceTarget_proved)
0058     neg_closure := PRCRealNegClosureTarget_proved
0059     neg_congruence := PRCRealNegCongruenceTarget_proved
0060     neg_operation :=
0061       (PRCRealNullClosed.negOf
0062         PRCRealNegClosureTarget_proved
0063         PRCRealNegCongruenceTarget_proved)
0064     mul_closure := PRCRealMulClosureTarget_of_bounded_continuity
0065     PRCRealCauchySeqEventuallyBoundedTarget_proved
0066     PRCJCostDistanceMulBoundedContinuityTarget_proved
0067     mul_congruence := PRCRealMulCongruenceTarget_of_bounded_continuity
0068     PRCRealCauchySeqEventuallyBoundedTarget_proved
0069     PRCJCostDistanceMulBoundedContinuityTarget_proved
0070     mul_operation :=
0071       (PRCRealNullClosed.mulOf
0072         (PRCRealMulClosureTarget_of_bounded_continuity
0073         PRCRealCauchySeqEventuallyBoundedTarget_proved
0074         PRCJCostDistanceMulBoundedContinuityTarget_proved)
0075         (PRCRealMulCongruenceTarget_of_bounded_continuity
0076         PRCRealCauchySeqEventuallyBoundedTarget_proved
0077         PRCJCostDistanceMulBoundedContinuityTarget_proved))
0078     order_congruence := PRCRealOrderCongruenceTarget_proved
0079     representative_completeness := PRCRealCompletenessTarget_proved
0080     first_pass_certificate := prc_real_complete_ordered_field_conditional_certificate
0081     product_continuity_certificate := prc_real_product_continuity_certificate
0082     order_congruence_certificate := prc_real_order_congruence_certificate
0083     completeness_certificate := prc_real_completeness_sharpened_certificate
0084     strength_tag := rfl
0085
0086 end PrimitiveRecognitionCalculus
0087 end Foundation
0088 end IndisputableMonolith

```

25. RealCompletion.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RealCompletion.lean · Lines: 97

```

0001 /-
0002   PrimitiveRecognitionCalculus/RealCompletion.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Spec anchors:
0008     K1 ('classical extension'), K4.14, A5
0009
0010   This is the honest real-completion boundary. It embeds the quotient-native
0011   PRC rational surface into Lean's 'ℝ' and records completeness as a
0012   classical-extension fact. It is not a claim that the PRC-internal Cauchy
0013   quotient has already been built.
0014 -/
0015
0016 import Mathlib
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerRational
0018 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Strength
0019
0020 namespace IndisputableMonolith
0021 namespace Foundation
0022 namespace PrimitiveRecognitionCalculus
0023
0024 /-- K4.14. The first real boundary: Lean's complete real line, reached only
0025 under the classical-extension tag. -/
0026 abbrev PRCRealBoundary : Type := ℝ
0027
0028 namespace PRCRealBoundary
0029
0030 /-- K4.14/A5. Embed a PRC rational into the real boundary through its
0031 conservative rational display. -/
0032 def ofRat (q : PRCRat) : PRCRealBoundary :=
0033   (q.toRat : ℝ)
0034
0035 @[simp] theorem ofRat_add (a b : PRCRat) :
0036   ofRat (a + b) = ofRat a + ofRat b := by
0037   unfold ofRat
0038   rw [PRCRat.toRat_add']
0039   norm_num
0040
0041 @[simp] theorem ofRat_mul (a b : PRCRat) :
0042   ofRat (a * b) = ofRat a * ofRat b := by
0043   unfold ofRat
0044   rw [PRCRat.toRat_mul']
0045   norm_num
0046
0047 @[simp] theorem ofRat_neg (a : PRCRat) :
0048   ofRat (-a) = -ofRat a := by
0049   unfold ofRat
0050   rw [PRCRat.toRat_neg']
0051   norm_num
0052
0053 @[simp] theorem ofRat_inv (a : PRCRat) :
0054   ofRat (a⁻¹) = (ofRat a)⁻¹ := by
0055   unfold ofRat
0056   rw [PRCRat.toRat_inv']
0057   norm_num
0058
0059 /-- K4.14. The real boundary carries Lean's complete-space structure. -/
0060 theorem complete_space : CompleteSpace PRCRealBoundary := by
0061   infer_instance
0062
0063 end PRCRealBoundary
0064
0065 /-- K1/K4.14. Audit record: the real-completion boundary is a classical
0066 extension until an internal PRC Cauchy quotient is built. -/
0067 def realCompletionClaim : StrengthClaim where
0068   label := "K4.14_real_completion_boundary"
0069   tag := StrengthTag.classicalExtension
0070   statement := "The first PRC real boundary embeds PRCRat into Lean Real and uses classical completeness."
0071
0072 /-- K4.14. First real-completion boundary certificate. -/
0073 structure RealCompletionBoundaryCertificate : Prop where
0074   real_boundary_exists : Nonempty PRCRealBoundary
0075   rational_embedding_exists : Nonempty (PRCRat → PRCRealBoundary)
0076   preserves_add : ∀ a b : PRCRat,
0077     PRCRealBoundary.ofRat (a + b) =
0078       PRCRealBoundary.ofRat a + PRCRealBoundary.ofRat b
0079   preserves_mul : ∀ a b : PRCRat,
0080     PRCRealBoundary.ofRat (a * b) =
0081       PRCRealBoundary.ofRat a * PRCRealBoundary.ofRat b
0082   complete : CompleteSpace PRCRealBoundary
0083   strength_tag : realCompletionClaim.tag = StrengthTag.classicalExtension
0084
0085 /-- K4.14. The classical real boundary is available and tagged honestly. -/
0086 theorem real_completion_boundary_certificate :
0087   RealCompletionBoundaryCertificate where
0088     real_boundary_exists := (0)
0089     rational_embedding_exists := (PRCRealBoundary.ofRat)
0090     preserves_add := PRCRealBoundary.ofRat_add

```

```
0091 preserves_mul := PRCRealBoundary.ofRat_mul
0092 complete      := PRCRealBoundary.complete_space
0093 strength_tag  := rfl
0094
0095 end PrimitiveRecognitionCalculus
0096 end Foundation
0097 end IndisputableMonolith
```

26. TraceClosure.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/TraceClosure.lean · Lines: 107

```

0001 /-
0002   PrimitiveRecognitionCalculus/TraceClosure.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Spec anchors:
0008     K1 ('δ + trace-closure'), R9, K4.13
0009
0010   This module records the first completed-trace boundary. It does not claim
0011   that completed infinity is δ-only. It explicitly carries the
0012   'traceClosure' strength tag.
0013 -/
0014
0015 import Mathlib
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Basic
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Orbit
0018 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Strength
0019
0020 namespace IndisputableMonolith
0021 namespace Foundation
0022 namespace PrimitiveRecognitionCalculus
0023
0024 /-- K4.13/R9. A completed trace is an infinite ledger of distinction acts.
0025 This is a trace-closure object, not a finite δ-only trace. -/
0026 structure CompletedTrace where
0027   actAt : Nat → DistinctionAct
0028
0029 namespace CompletedTrace
0030
0031 /-- The finite prefix of length 'n' cut out of a completed trace. -/
0032 def finitePrefix (S : CompletedTrace) : Nat → Trace
0033   | 0 => Trace.empty
0034   | Nat.succ n => Trace.extend (finitePrefix S n) (S.actAt n)
0035
0036 @[simp] theorem prefix_zero (S : CompletedTrace) :
0037   S.finitePrefix 0 = Trace.empty := by
0038   rfl
0039
0040 @[simp] theorem prefix_succ (S : CompletedTrace) (n : Nat) :
0041   S.finitePrefix (Nat.succ n) = Trace.extend (S.finitePrefix n) (S.actAt n) := by
0042   rfl
0043
0044 /-- The canonical completed trace repeats the primitive distinction act. -/
0045 def canonical : CompletedTrace where
0046   actAt := fun _ => DistinctionAct.delta
0047
0048 @[simp] theorem canonical_actAt (n : Nat) :
0049   canonical.actAt n = DistinctionAct.delta := by
0050   rfl
0051
0052 /-- Every prefix of the canonical completed trace is a finite trace. -/
0053 theorem canonical_prefix_exists (n : Nat) :
0054   Nonempty Trace := by
0055   exact (canonical.finitePrefix n)
0056
0057 end CompletedTrace
0058
0059 /-- K4.13. A completed orbit ledger is the infinite sequence of finite
0060 δ-orbit positions. This is the natural-number side of trace closure. -/
0061 structure CompletedOrbitLedger where
0062   positionAt : Nat → DistinctionNat
0063
0064 namespace CompletedOrbitLedger
0065
0066 /-- The canonical completed orbit sends verifier index 'n' to the 'n'th
0067 δ-orbit position. -/
0068 def canonical : CompletedOrbitLedger where
0069   positionAt := DistinctionNat.ofNat
0070
0071 @[simp] theorem canonical_toNat (n : Nat) :
0072   (canonical.positionAt n).toNat = n := by
0073   exact DistinctionNat.toNat_ofNat n
0074
0075 theorem canonical_succ (n : Nat) :
0076   canonical.positionAt (Nat.succ n) =
0077     DistinctionNat.succ (canonical.positionAt n) := by
0078   rfl
0079
0080 end CompletedOrbitLedger
0081
0082 /-- K1/R9. Audit record: completed traces require the trace-closure tag. -/
0083 def traceClosureClaim : StrengthClaim where
0084   label := "K4.13_trace_closure_boundary"
0085   tag := StrengthTag.traceClosure
0086   statement := "Completed traces and completed orbit ledgers extend finite PRC by trace closure."
0087
0088 /-- K4.13. First trace-closure certificate. -/
0089 structure TraceClosureCertificate : Prop where
0090   completed_trace_exists : Nonempty CompletedTrace

```

```
0091 canonical_completed_trace_exists : Nonempty CompletedTrace
0092 completed_orbit_ledger_exists : Nonempty CompletedOrbitLedger
0093 canonical_orbit_verifier_faithful :
0094   ∀ n : Nat, (CompletedOrbitLedger.canonical.positionAt n).toNat = n
0095 strength_tag : traceClosureClaim.tag = StrengthTag.traceClosure
0096
0097 /-- K4.13. The trace-closure boundary is inhabited and tagged honestly. -/
0098 theorem trace_closure_certificate : TraceClosureCertificate where
0099   completed_trace_exists := (CompletedTrace.canonical)
0100   canonical_completed_trace_exists := (CompletedTrace.canonical)
0101   completed_orbit_ledger_exists := (CompletedOrbitLedger.canonical)
0102   canonical_orbit_verifier_faithful := CompletedOrbitLedger.canonical_toNat
0103   strength_tag := rfl
0104
0105 end PrimitiveRecognitionCalculus
0106 end Foundation
0107 end IndisputableMonolith
```


27. TraceLogic.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/TraceLogic.lean · Lines: 237

```

0001 /-
0002   PrimitiveRecognitionCalculus/TraceLogic.lean
0003
0004   Round-trip source:
0005     6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008     Build Order step 11: build logical connectives and quantifier surfaces from
0009     stable trace predicates.
0010
0011     This first pass keeps the logic surface over finite PRC traces. Completed
0012     trace families and model-theoretic inevitability are separate later layers.
0013 -/
0014
0015 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.SameDiff
0016
0017 namespace IndisputableMonolith
0018 namespace Foundation
0019 namespace PrimitiveRecognitionCalculus
0020
0021 /-- A proposition in the first PRC logic pass is a predicate on finite traces
0022 that persists under trace extension. -/
0023 structure TracePredicate where
0024   holds : Trace → Prop
0025   stable :
0026     ∀ {T U : Trace}, Trace.Extends T U → holds T → holds U
0027
0028 namespace TracePredicate
0029
0030 /-- Truth is the stable predicate that holds at every trace. -/
0031 def top : TracePredicate where
0032   holds := fun _ => True
0033   stable := by
0034     intro _T _U _hTU _h
0035     trivial
0036
0037 /-- Falsehood is the stable predicate that holds at no trace. -/
0038 def bottom : TracePredicate where
0039   holds := fun _ => False
0040   stable := by
0041     intro _T _U _hTU h
0042     exact False.elim h
0043
0044 /-- Conjunction of stable trace predicates. -/
0045 def and (P Q : TracePredicate) : TracePredicate where
0046   holds := fun T => P.holds T ∧ Q.holds T
0047   stable := by
0048     intro T U hTU h
0049     exact (P.stable hTU h.1, Q.stable hTU h.2)
0050
0051 /-- Disjunction of stable trace predicates. -/
0052 def or (P Q : TracePredicate) : TracePredicate where
0053   holds := fun T => P.holds T ∨ Q.holds T
0054   stable := by
0055     intro T U hTU h
0056     cases h with
0057     | inl hP => exact Or.inl (P.stable hTU hP)
0058     | inr hQ => exact Or.inr (Q.stable hTU hQ)
0059
0060 /-- Implication is persistence along every future extension of the current
0061 trace. This makes implication itself stable under extension. -/
0062 def imp (P Q : TracePredicate) : TracePredicate where
0063   holds := fun T =>
0064     ∀ U : Trace, Trace.Extends T U → P.holds U → Q.holds U
0065   stable := by
0066     intro T U hTU h V hUV hPV
0067     exact h V (Trace.extends_trans hTU hUV) hPV
0068
0069 /-- Negation is implication into falsehood. -/
0070 def not (P : TracePredicate) : TracePredicate :=
0071   imp P bottom
0072
0073 /-- Universal quantification over a verifier-indexed family of stable trace
0074 predicates. The family parameter is verifier bookkeeping; stability is still a
0075 finite-trace theorem. -/
0076 def all {α : Type} (P : α → TracePredicate) : TracePredicate where
0077   holds := fun T => ∀ a : α, (P a).holds T
0078   stable := by
0079     intro T U hTU h a
0080     exact (P a).stable hTU (h a)
0081
0082 /-- Existential quantification over a verifier-indexed family of stable trace
0083 predicates. The witness is preserved while the trace is extended. -/
0084 def exists_ {α : Type} (P : α → TracePredicate) : TracePredicate where
0085   holds := fun T => ∃ a : α, (P a).holds T
0086   stable := by
0087     intro T U hTU h
0088     rcases h with (a, ha)
0089     exact (a, (P a).stable hTU ha)
0090

```

```

0091 theorem top_intro (T : Trace) :
0092   top.holds T := by
0093     trivial
0094
0095 theorem and_intro {P Q : TracePredicate} {T : Trace}
0096   (hP : P.holds T) (hQ : Q.holds T) :
0097   (and P Q).holds T := by
0098     exact (hP, hQ)
0099
0100 theorem and_left {P Q : TracePredicate} {T : Trace}
0101   (h : (and P Q).holds T) :
0102   P.holds T := by
0103     exact h.1
0104
0105 theorem and_right {P Q : TracePredicate} {T : Trace}
0106   (h : (and P Q).holds T) :
0107   Q.holds T := by
0108     exact h.2
0109
0110 theorem or_inl {P Q : TracePredicate} {T : Trace}
0111   (hP : P.holds T) :
0112   (or P Q).holds T := by
0113     exact Or.inl hP
0114
0115 theorem or_inr {P Q : TracePredicate} {T : Trace}
0116   (hQ : Q.holds T) :
0117   (or P Q).holds T := by
0118     exact Or.inr hQ
0119
0120 theorem imp_elim {P Q : TracePredicate} {T U : Trace}
0121   (himp : (imp P Q).holds T)
0122   (hTU : Trace.Extends T U)
0123   (hP : P.holds U) :
0124   Q.holds U := by
0125     exact himp U hTU hP
0126
0127 theorem not_elim {P : TracePredicate} {T U : Trace}
0128   (hn : (not P).holds T)
0129   (hTU : Trace.Extends T U)
0130   (hP : P.holds U) :
0131   False := by
0132     exact hn U hTU hP
0133
0134 theorem all_intro {α : Type} {P : α → TracePredicate} {T : Trace}
0135   (h : ∀ a : α, (P a).holds T) :
0136   (all P).holds T := by
0137     exact h
0138
0139 theorem all_elim {α : Type} {P : α → TracePredicate} {T : Trace}
0140   (h : (all P).holds T) (a : α) :
0141   (P a).holds T := by
0142     exact h a
0143
0144 theorem exists_intro {α : Type} {P : α → TracePredicate} {T : Trace}
0145   (a : α) (h : (P a).holds T) :
0146   (exists_ P).holds T := by
0147     exact (a, h)
0148
0149 theorem persists {P : TracePredicate} {T U : Trace}
0150   (hTU : Trace.Extends T U) (hP : P.holds T) :
0151   P.holds U :=
0152   P.stable hTU hP
0153
0154 end TracePredicate
0155
0156 /-- Headline target for the first trace-logic pass. -/
0157 structure TraceLogicCertificate : Prop where
0158   proposition_surface : Nonempty TracePredicate
0159   truth_intro : ∀ T : Trace, TracePredicate.top.holds T
0160   conjunction_intro :
0161     ∀ {P Q : TracePredicate} {T : Trace},
0162     P.holds T → Q.holds T → (TracePredicate.and P Q).holds T
0163   conjunction_left :
0164     ∀ {P Q : TracePredicate} {T : Trace},
0165     (TracePredicate.and P Q).holds T → P.holds T
0166   conjunction_right :
0167     ∀ {P Q : TracePredicate} {T : Trace},
0168     (TracePredicate.and P Q).holds T → Q.holds T
0169   disjunction_left :
0170     ∀ {P Q : TracePredicate} {T : Trace},
0171     P.holds T → (TracePredicate.or P Q).holds T
0172   disjunction_right :
0173     ∀ {P Q : TracePredicate} {T : Trace},
0174     Q.holds T → (TracePredicate.or P Q).holds T
0175   implication_elim :
0176     ∀ {P Q : TracePredicate} {T U : Trace},
0177     (TracePredicate.imp P Q).holds T →
0178     Trace.Extends T U → P.holds U → Q.holds U
0179   negation_elim :
0180     ∀ {P : TracePredicate} {T U : Trace},
0181     (TracePredicate.not P).holds T →
0182     Trace.Extends T U → P.holds U → False
0183   universal_intro :
0184     ∀ {α : Type} {P : α → TracePredicate} {T : Trace},
0185     (∀ a : α, (P a).holds T) → (TracePredicate.all P).holds T
0186   universal_elim :
0187     ∀ {α : Type} {P : α → TracePredicate} {T : Trace},
0188     (TracePredicate.all P).holds T → ∀ a : α, (P a).holds T
0189   existential_intro :
0190     ∀ {α : Type} {P : α → TracePredicate} {T : Trace},
0191     ∀ a : α, (P a).holds T → (TracePredicate.exists_ P).holds T

```

```

0192 persistence :
0193   V {P : TracePredicate} {T U : Trace},
0194     Trace.Extends T U → P.holds T → P.holds U
0195 strength_tag : StrengthTag.deltaOnly = StrengthTag.deltaOnly
0196
0197 theorem trace_logic_certificate : TraceLogicCertificate where
0198   proposition_surface := (TracePredicate.top)
0199   truth_intro := TracePredicate.top_intro
0200   conjunction_intro := by
0201     intro P Q T hP hQ
0202     exact TracePredicate.and_intro hP hQ
0203   conjunction_left := by
0204     intro P Q T h
0205     exact TracePredicate.and_left h
0206   conjunction_right := by
0207     intro P Q T h
0208     exact TracePredicate.and_right h
0209   disjunction_left := by
0210     intro P Q T hP
0211     exact TracePredicate.or_inl hP
0212   disjunction_right := by
0213     intro P Q T hQ
0214     exact TracePredicate.or_inr hQ
0215   implication_elim := by
0216     intro P Q T U himp hTU hP
0217     exact TracePredicate.imp_elim himp hTU hP
0218   negation_elim := by
0219     intro P T U hn hTU hP
0220     exact TracePredicate.not_elim hn hTU hP
0221   universal_intro := by
0222     intro α P T h
0223     exact TracePredicate.all_intro h
0224   universal_elim := by
0225     intro α P T h a
0226     exact TracePredicate.all_elim h a
0227   existential_intro := by
0228     intro α P T a h
0229     exact TracePredicate.exists_intro a h
0230   persistence := by
0231     intro P T U hTU hP
0232     exact TracePredicate.persists hTU hP
0233   strength_tag := rfl
0234
0235 end PrimitiveRecognitionCalculus
0236 end Foundation
0237 end IndisputableMonolith

```

28. FormalSystem.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/FormalSystem.lean · Lines: 123

```

0001 /-
0002   PrimitiveRecognitionCalculus/FormalSystem.lean
0003
0004   Round-trip source:
0005     6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008     Build Order step 12: define expressivity and embeddings in enough
0009     generality to prove the inevitability surface.
0010
0011   This module is not claiming that every historical foundation has already
0012   been parsed into the interface. It closes the exact theorem-shaped interface:
0013   any formal system that supplies distinguishable endpoint tokens and preserves
0014   trace extension admits a PRC trace embedding.
0015 -/
0016
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.TraceLogic
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 /-- Minimal formal-system interface needed by the PRC inevitability theorem.
0024 `Token` and `Expr` are verifier-side carriers for an arbitrary formal system,
0025 while `distinguishes` and `exprExtends` record the structure that makes 6
0026 visible inside it. -/
0027 structure FormalSystem where
0028   Token : Type
0029   Expr : Type
0030   distinguishes : Token → Token → Prop
0031   exprExtends : Expr → Expr → Prop
0032   endpointToken : Endpoint → Token
0033   traceExpr : Trace → Expr
0034   traceExpr_extends :
0035     ∀ {T U : Trace}, Trace.Extends T U → exprExtends (traceExpr T) (traceExpr U)
0036
0037 namespace FormalSystem
0038
0039 /-- A formal system is expressive for the first inevitability pass when it can
0040 distinguish the two endpoints of the primitive distinction. -/
0041 def Expressive (F : FormalSystem) : Prop :=
0042   F.distinguishes (F.endpointToken Endpoint.left) (F.endpointToken Endpoint.right)
0043
0044 end FormalSystem
0045
0046 /-- A PRC embedding into a formal system preserves the primitive endpoint
0047 distinction and finite trace extension. -/
0048 structure PRCEmbeddingInto (F : FormalSystem) where
0049   endpointMap : Endpoint → F.Token
0050   traceMap : Trace → F.Expr
0051   preserves_distinction :
0052     F.distinguishes (endpointMap Endpoint.left) (endpointMap Endpoint.right)
0053   preserves_trace_extension :
0054     ∀ {T U : Trace}, Trace.Extends T U → F.exprExtends (traceMap T) (traceMap U)
0055
0056 /-- The canonical embedding supplied by an expressive formal system's own trace
0057 and endpoint interpretation fields. -/
0058 def PRCEmbeddingInto.ofExpressive
0059   (F : FormalSystem) (hF : F.Expressive) : PRCEmbeddingInto F where
0060   endpointMap := F.endpointToken
0061   traceMap := F.traceExpr
0062   preserves_distinction := hF
0063   preserves_trace_extension := F.traceExpr_extends
0064
0065 /-- Exact target for Build Order step 12. -/
0066 def FormalSystemEmbeddingTarget : Prop :=
0067   ∀ F : FormalSystem, F.Expressive → Nonempty (PRCEmbeddingInto F)
0068
0069 theorem FormalSystemEmbeddingTarget_proved :
0070   FormalSystemEmbeddingTarget := by
0071   intro F hF
0072   exact (PRCEmbeddingInto.ofExpressive F hF)
0073
0074 theorem Endpoint.left_ne_right :
0075   Endpoint.left ≠ Endpoint.right := by
0076   intro h
0077   have hside := congrArg Endpoint.side h
0078   cases hside
0079
0080 /-- PRC itself as the minimal formal system: endpoint tokens are endpoints,
0081 expressions are finite traces, and expression extension is trace extension. -/
0082 def PRCFormalSystem : FormalSystem where
0083   Token := Endpoint
0084   Expr := Trace
0085   distinguishes := fun a b => a ≠ b
0086   exprExtends := Trace.Extends
0087   endpointToken := id
0088   traceExpr := id
0089   traceExpr_extends := by
0090     intro T U hTU

```

```

0091     exact hTU
0092
0093 theorem PRCFormalSystem_expressive :
0094   PRCFormalSystem.Expressive := by
0095     exact Endpoint.left_ne_right
0096
0097 theorem PRCFormalSystem_embedding :
0098   Nonempty (PRCEmbeddingInto PRCFormalSystem) :=
0099     FormalSystemEmbeddingTarget_proved PRCFormalSystem PRCFormalSystem_expressive
0100
0101 /-- Step 12 certificate: the formal-system surface and embedding theorem are
0102 closed. The broader claim that every external foundation satisfies this
0103 interface is the next inevitability layer, not hidden here. -/
0104 structure FormalSystemCertificate : Prop where
0105   formal_system_surface : Nonempty FormalSystem
0106   prc_system_expressive : PRCFormalSystem.Expressive
0107   prc_system_embedding : Nonempty (PRCEmbeddingInto PRCFormalSystem)
0108   embedding_target : FormalSystemEmbeddingTarget
0109   embedding_from_expressive :
0110      $\forall F : \text{FormalSystem}, F.\text{Expressive} \rightarrow \text{Nonempty } (\text{PRCEmbeddingInto } F)$ 
0111   strength_tag : StrengthTag.deltaOnly = StrengthTag.deltaOnly
0112
0113 theorem formal_system_certificate : FormalSystemCertificate where
0114   formal_system_surface := (PRCFormalSystem)
0115   prc_system_expressive := PRCFormalSystem_expressive
0116   prc_system_embedding := PRCFormalSystem_embedding
0117   embedding_target := FormalSystemEmbeddingTarget_proved
0118   embedding_from_expressive := FormalSystemEmbeddingTarget_proved
0119   strength_tag := rfl
0120
0121 end PrimitiveRecognitionCalculus
0122 end Foundation
0123 end IndisputableMonolith

```

29. Inevitability.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Inevitability.lean · Lines: 90

```

0001 /-
0002   PrimitiveRecognitionCalculus/Inevitability.lean
0003
0004   Round-trip source:
0005     6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008     Build Order step 13: prove that every admissible expressive foundation
0009     admits a PRC trace core.
0010
0011   "Admissible" here is deliberately exact: the foundation has been parsed into
0012   the `FormalSystem` interface and supplies endpoint distinction. Parsing
0013   external historical foundations into this interface is a later corpus task,
0014   not an implicit axiom in this theorem.
0015 -/
0016
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.FormalSystem
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 /-- An admissible foundation for the first inevitability theorem is a formal
0024 system that is expressive enough to distinguish the two endpoints of  $\delta$ . -/
0025 structure AdmissibleFoundation where
0026   system : FormalSystem
0027   expressive : system.Expressive
0028
0029 /-- The exact first inevitability target. -/
0030 def PRCInevitabilityTarget : Prop :=
0031    $\forall A : \text{AdmissibleFoundation}, \text{Nonempty } (\text{PRCEmbeddingInto } A.\text{system})$ 
0032
0033 /-- Any foundation already parsed into the admissible interface presupposes a
0034 PRC trace core. -/
0035 theorem any_foundation_presupposes_distinction :
0036   PRCInevitabilityTarget := by
0037     intro A
0038     exact FormalSystemEmbeddingTarget_proved A.system A.expressive
0039
0040 /-- PRC itself is an admissible foundation in this interface. -/
0041 def PRCAdmissibleFoundation : AdmissibleFoundation where
0042   system := PRCFormalSystem
0043   expressive := PRCFormalSystem_expressive
0044
0045 theorem PRCAdmissibleFoundation_embeds :
0046   Nonempty (PRCEmbeddingInto PRCAdmissibleFoundation.system) :=
0047   any_foundation_presupposes_distinction PRCAdmissibleFoundation
0048
0049 /-- The exact remaining external parsing target schema: once a corpus of
0050 external foundations and a faithful-parse relation are fixed, every external
0051 object in that corpus must parse to an expressive formal system before the
0052 inevitability theorem applies to it. -/
0053 def ExternalFoundationParsingTarget
0054   (ExternalFoundation : Type)
0055   (FaithfulParse : ExternalFoundation  $\rightarrow$  FormalSystem  $\rightarrow$  Prop) : Prop :=
0056    $\forall E : \text{ExternalFoundation},$ 
0057      $\exists F : \text{FormalSystem}, \text{FaithfulParse } E F \wedge F.\text{Expressive}$ 
0058
0059 /-- Step 13 certificate. The admissible-interface theorem is closed; the
0060 external parsing workload is named separately so it is not hidden inside the
0061 theorem. -/
0062 structure PRCInevitabilityCertificate : Prop where
0063   admissible_foundation_surface : Nonempty AdmissibleFoundation
0064   prc_admissible : Nonempty AdmissibleFoundation
0065   inevitability_target : PRCInevitabilityTarget
0066   any_foundation_embedding :
0067      $\forall A : \text{AdmissibleFoundation}, \text{Nonempty } (\text{PRCEmbeddingInto } A.\text{system})$ 
0068   prc_embedding : Nonempty (PRCEmbeddingInto PRCAdmissibleFoundation.system)
0069   external_parsing_target_schema :
0070      $\forall (\text{ExternalFoundation} : \text{Type})$ 
0071       (FaithfulParse : ExternalFoundation  $\rightarrow$  FormalSystem  $\rightarrow$  Prop),
0072       ExternalFoundationParsingTarget ExternalFoundation FaithfulParse =
0073       ExternalFoundationParsingTarget ExternalFoundation FaithfulParse
0074   strength_tag : StrengthTag.deltaOnly = StrengthTag.deltaOnly
0075
0076 theorem prc_inevitability_certificate :
0077   PRCInevitabilityCertificate where
0078     admissible_foundation_surface := (PRCAdmissibleFoundation)
0079     prc_admissible := (PRCAdmissibleFoundation)
0080     inevitability_target := any_foundation_presupposes_distinction
0081     any_foundation_embedding := any_foundation_presupposes_distinction
0082     prc_embedding := PRCAdmissibleFoundation_embeds
0083     external_parsing_target_schema := by
0084       intro ExternalFoundation FaithfulParse
0085       rfl
0086     strength_tag := rfl
0087
0088 end PrimitiveRecognitionCalculus

```

```
0089 end Foundation
0090 end IndisputableMonolith
```

30. RecognizerBridge.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/RecognizerBridge.lean · Lines: 117

```

0001 /-
0002   PrimitiveRecognitionCalculus/RecognizerBridge.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Build Order step 14: produce the positive-ratio recognizer surface from
0009   PRC cost theory and map it into the existing Law-of-Logic J-cost chain.
0010
0011   The positive-ratio recognizer is PRC-native. The existing continuous
0012   uniqueness theorem is still recorded as a classical-extension bridge because
0013   it quantifies over positive real ratios and continuity.
0014 -/
0015
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Inevitability
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RationalField
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 /-- Positive PRC ratios are the input surface for recognizer comparisons. -/
0024 structure PRCPositiveRatio where
0025   value : PRCRat
0026   positive : PRCRat.positive value
0027
0028 namespace PRCPositiveRatio
0029
0030 /-- The canonical positive unit ratio. -/
0031 def one : PRCPositiveRatio where
0032   value := 1
0033   positive := by
0034     rw [PRCRat.positive_iff_toRat_pos]
0035     change 0 < PRCRat.one.toRat
0036     rw [PRCRat.one_toRat]
0037     norm_num
0038
0039 /-- The quotient-level PRC J-cost assigned to a positive ratio. -/
0040 def cost (r : PRCPositiveRatio) : PRCRat :=
0041   PRCJCost.onPRCRat r.value
0042
0043 theorem cost_toRat (r : PRCPositiveRatio) :
0044   r.cost.toRat = (r.value.toRat + r.value.toRat⁻¹) / 2 - 1 :=
0045   PRCJCost.onPRCRat_toRat r.value
0046
0047 theorem cost_toReal_jcost (r : PRCPositiveRatio) :
0048   (r.cost.toRat : ℝ) = Cost.Jcost ((r.value.toRat : ℚ) : ℝ) := by
0049   rw [cost_toRat]
0050   unfold Cost.Jcost
0051   rw [Rat.cast_sub, Rat.cast_div, Rat.cast_add, Rat.cast_inv]
0052   norm_num
0053
0054 end PRCPositiveRatio
0055
0056 /-- Recognition cost on a positive PRC ratio. -/
0057 def PRCRecognitionCost (r : PRCPositiveRatio) : PRCRat :=
0058   r.cost
0059
0060 theorem PRCRecognitionCost_display (r : PRCPositiveRatio) :
0061   (PRCRecognitionCost r).toRat =
0062     (r.value.toRat + r.value.toRat⁻¹) / 2 - 1 :=
0063   PRCPositiveRatio.cost_toRat r
0064
0065 /-- Exact bridge target from PRC recognizer costs into the existing
0066 Law-of-Logic uniqueness theorem. -/
0067 def PRCRecognizerLawOfLogicBridgeTarget : Prop :=
0068   ∀ (F : ℝ → ℝ),
0069     Cost.FunctionalEquation.AczelSmoothnessPackage →
0070     Cost.FunctionalEquation.IsReciprocalCost F →
0071     Cost.FunctionalEquation.IsNormalized F →
0072     Cost.FunctionalEquation.SatisfiesCompositionLaw F →
0073     Cost.FunctionalEquation.IsCalibrated F →
0074     ContinuousOn F (Set.Ioi 0) →
0075     ∀ x : ℝ, 0 < x → F x = Cost.Jcost x
0076
0077 theorem PRCRecognizerLawOfLogicBridgeTarget_proved :
0078   PRCRecognizerLawOfLogicBridgeTarget := by
0079   intro F hA hR hN hC hCal hCont x hx
0080   exact PRCJCost.bridge_to_existing_jcost_uniqueness
0081     F hA hR hN hC hCal hCont x hx
0082
0083 /-- Step 14 certificate. The recognizer surface is closed through the existing
0084 continuous Law-of-Logic bridge; fully native arbitrary-cost uniqueness remains
0085 the named PRC target from "PRCJCost.lean". -/
0086 structure PRCRecognizerBridgeCertificate : Prop where
0087   positive_ratio_surface : Nonempty PRCPositiveRatio
0088   recognition_cost_surface : Nonempty (PRCPositiveRatio → PRCRat)
0089   cost_display :
0090     ∀ r : PRCPositiveRatio,

```



```

0091      (PRCRognitionCost r).toRat =
0092      (r.value.toRat + r.value.toRat-1) / 2 - 1
0093  real_jcost_bridge :
0094    ∀ r : PRCPositiveRatio,
0095    ((PRCRognitionCost r).toRat : ℝ) =
0096    Cost.Jcost ((r.value.toRat : ℚ) : ℝ)
0097  law_of_logic_bridge : PRCRecognizerLawOfLogicBridgeTarget
0098  native_uniqueness_target_named :
0099    PRCJCost.PRCNativeCostUniquenessTarget =
0100    PRCJCost.PRCNativeCostUniquenessTarget
0101  strength_tag : StrengthTag.classicalExtension = StrengthTag.classicalExtension
0102
0103  theorem prc_recognizer_bridge_certificate :
0104    PRCRecognizerBridgeCertificate where
0105    positive_ratio_surface := (PRCPositiveRatio.one)
0106    recognition_cost_surface := (PRCRognitionCost)
0107    cost_display := PRCRecognitionCost_display
0108    real_jcost_bridge := by
0109      intro r
0110      exact PRCPositiveRatio.cost.toReal_jcost r
0111    law_of_logic_bridge := PRCRecognizerLawOfLogicBridgeTarget_proved
0112    native_uniqueness_target_named := rfl
0113    strength_tag := rfl
0114
0115  end PrimitiveRecognitionCalculus
0116  end Foundation
0117  end IndisputableMonolith

```

31. PRCNativeCostUniqueness.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/PRCNativeCostUniqueness.lean · Lines: 3798

```

0001 /-
0002   PrimitiveRecognitionCalculus/PRCNativeCostUniqueness.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Immediate target after pass 24: attack `PRCNativeCostUniquenessTarget`
0009   or isolate the smallest exact blocker.
0010
0011   The current PRC cost hypotheses are too weak to prove uniqueness directly:
0012   after setting `g = F + 1`, the RCL has the d'Alembert form
0013   `g(xy) + g(x/y) = 2 g(x) g(y)`. On a rational multiplicative group,
0014   one-point calibration at `2` does not by itself control all prime directions.
0015   This file names the exact missing character-factorization and calibrated
0016   character-rigidity targets and proves that together they imply native
0017   cost uniqueness.
0018 -/
0019
0020 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCost
0021
0022 namespace IndisputableMonolith
0023 namespace Foundation
0024 namespace PrimitiveRecognitionCalculus
0025 namespace PRCJCost
0026
0027 /-- A ratio-character candidate for the d'Alembert factorization of a PRC cost.
0028 It is stated at the ratio-orbit level and uses cross-equivalence rather than
0029 definitional equality, so it remains quotient-native. -/
0030 structure PRCRatioCharacter (χ : RatioOrbit → RatioOrbit) : Prop where
0031   unit :
0032     RatioOrbit.crossEq (χ RatioOrbit.one) RatioOrbit.one
0033   multiplicative :
0034     ∀ x y : RatioOrbit,
0035       (RatioOrbit.crossEq (χ (RatioOrbit.mul x y))
0036         (RatioOrbit.mul (χ x) (χ y)))
0037   reciprocal :
0038     ∀ x : RatioOrbit,
0039       RatioOrbit.crossEq (χ (RatioOrbit.recip x))
0040         (RatioOrbit.recip (χ x))
0041   normalized_invariant :
0042     ∀ q : RatioOrbit,
0043       RatioOrbit.crossEq (χ q) (χ (DistinctionNat.normalizeRatio q))
0044   nonzero_preserving :
0045     ∀ {q : RatioOrbit}, q.toRat ≠ 0 → (χ q).toRat ≠ 0
0046
0047 /-- Cost generated from a rational character. The canonical PRC cost is the
0048 identity-character case. -/
0049 def costFromCharacter (χ : RatioOrbit → RatioOrbit) (q : RatioOrbit) :
0050   RatioOrbit :=
0051   onRatioOrbit (χ q)
0052
0053 theorem costFromCharacter_toRat
0054   (χ : RatioOrbit → RatioOrbit) (q : RatioOrbit) :
0055   (costFromCharacter χ q).toRat =
0056   ((χ q).toRat + (χ q).toRat⁻¹) / 2 - 1 := by
0057   exact onRatioOrbit_toRat (χ q)
0058
0059 /-- First exact blocker: every admissible PRC-native RCL cost should factor
0060 through a ratio character. This is the discrete d'Alembert factorization step. -/
0061 def PRCNativeCostCharacterFactorizationTarget : Prop :=
0062   ∀ F : RatioOrbit → RatioOrbit,
0063     PRCNativeCostHypotheses F →
0064     ∃ χ : RatioOrbit → RatioOrbit,
0065       PRCRatioCharacter χ ∧
0066       ∀ q : RatioOrbit,
0067         RatioOrbit.crossEq (F q) (costFromCharacter χ q)
0068
0069 /-- Second exact blocker: a calibrated rational character cost must be the
0070 canonical identity-character cost. This is where prime-direction freedom has
0071 to be eliminated. -/
0072 def PRCNativeCostCharacterRigidityTarget : Prop :=
0073   ∀ χ : RatioOrbit → RatioOrbit,
0074     PRCRatioCharacter χ →
0075     RatioOrbit.crossEq (costFromCharacter χ two) (onRatioOrbit two) →
0076     ∀ q : RatioOrbit,
0077       RatioOrbit.crossEq (costFromCharacter χ q) (onRatioOrbit q)
0078
0079 /-- The ratio direction associated to a nonzero orbit position. -/
0080 def orbitDirection (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero) :
0081   RatioOrbit where
0082   num := SignedOrbit.ofOrbit p
0083   den := DistinctionNat.one
0084   den_ne_zero := DistinctionNat.one_ne_zero
0085
0086 /-- The ratio direction associated to a native prime orbit. -/
0087 def primeDirection (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p) :
0088   RatioOrbit :=
0089   orbitDirection p hp.1
0090

```

```

0091 /-- A ratio character is calibrated on every native prime direction when its
0092 generated cost agrees with canonical J-cost on each prime orbit. -/
0093 def PRCCharacterPrimeDirectionCalibrated
0094   (χ : RatioOrbit → RatioOrbit) : Prop :=
0095   ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
0096     RatioOrbit.crossEq
0097       (costFromCharacter χ (primeDirection p hp))
0098       (onRatioOrbit (primeDirection p hp))
0099
0100 /-- Sharper target A: the two-point calibration at orbit `2` must force
0101 calibration on every prime direction. This is the exact place where the present
0102 surface lacks control of independent prime axes. -/
0103 def PRCTwoCalibrationForcesPrimeCalibrationTarget : Prop :=
0104   ∀ χ : RatioOrbit → RatioOrbit,
0105     PRCRatioCharacter χ →
0106     RatioOrbit.crossEq (costFromCharacter χ two) (onRatioOrbit two) →
0107     PRCCharacterPrimeDirectionCalibrated χ
0108
0109 /-- Sharper target B: once every prime direction is calibrated, the character
0110 cost propagates to every rational direction. This is the unique-factorization
0111 side of the rigidity problem. -/
0112 def PRCPrimeCalibrationPropagationTarget : Prop :=
0113   ∀ χ : RatioOrbit → RatioOrbit,
0114     PRCRatioCharacter χ →
0115     PRCCharacterPrimeDirectionCalibrated χ →
0116     ∀ q : RatioOrbit,
0117       RatioOrbit.crossEq (costFromCharacter χ q) (onRatioOrbit q)
0118
0119 theorem onRatioOrbit_congr {a b : RatioOrbit}
0120   (h : RatioOrbit.crossEq a b) :
0121     RatioOrbit.crossEq (onRatioOrbit a) (onRatioOrbit b) := by
0122   rw [RatioOrbit.crossEq_iff_toRat_eq] at h
0123   rw [onRatioOrbit_toRat, onRatioOrbit_toRat, h]
0124
0125 theorem jcost_eq_forces_same_or_reciprocal {a b : RatioOrbit}
0126   (ha : a.toRat ≠ 0) (hb : b.toRat ≠ 0)
0127   (h : RatioOrbit.crossEq (onRatioOrbit a) (onRatioOrbit b)) :
0128     RatioOrbit.crossEq a b
0129   RatioOrbit.crossEq a (RatioOrbit.recip b) := by
0130   rw [RatioOrbit.crossEq_iff_toRat_eq] at h
0131   rw [onRatioOrbit_toRat, onRatioOrbit_toRat] at h
0132   rw [RatioOrbit.crossEq_iff_toRat_eq,
0133     RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat]
0134   have hsum : a.toRat + a.toRat⁻¹ = b.toRat + b.toRat⁻¹ := by
0135     linarith
0136   have hprod :
0137     (a.toRat - b.toRat) * (a.toRat * b.toRat - 1) = 0 := by
0138     have hmul :=
0139       congrArg (fun t : ℚ => t * (a.toRat * b.toRat)) hsum
0140     field_simp [ha, hb] at hmul
0141     ring_nf at hmul
0142     linarith
0143   rcases mul_eq_zero.mp hprod with hcase | hrec
0144   · left
0145     linarith
0146   · right
0147     have hmul : a.toRat * b.toRat = 1 := by
0148       linarith
0149     have hb_inv : b.toRat * b.toRat⁻¹ = 1 := by
0150       field_simp [hb]
0151     calc
0152       a.toRat = a.toRat * (b.toRat * b.toRat⁻¹) := by
0153         rw [hb_inv, mul_one]
0154       = (a.toRat * b.toRat) * b.toRat⁻¹ := by
0155         ring
0156       = b.toRat⁻¹ := by
0157         rw [hmul, one_mul]
0158
0159 theorem primeDirection_toRat_ne_zero
0160   (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p) :
0161     (primeDirection p hp).toRat ≠ 0 := by
0162     have hpNat : p.toNat ≠ 0 := by
0163       intro hzero
0164       apply hp.1
0165       apply DistinctionNat.toNat_inj
0166       rw [hzero, DistinctionNat.toNat_zero]
0167     unfold primeDirection orbitDirection RatioOrbit.toRat
0168     simp [SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat, hpNat]
0169
0170 theorem primeDirection_toRat
0171   (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p) :
0172     (primeDirection p hp).toRat = (p.toNat : ℚ) := by
0173     unfold primeDirection orbitDirection RatioOrbit.toRat
0174     simp [SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat]
0175
0176 theorem orbitDirection_toRat
0177   (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero) :
0178     (orbitDirection p hp).toRat = (p.toNat : ℚ) := by
0179     unfold orbitDirection RatioOrbit.toRat
0180     simp [SignedOrbit.ofOrbit_toInt, DistinctionNat.one_toNat]
0181
0182 theorem primeDirection_not_crossEq_recip
0183   (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p) :
0184     ~ RatioOrbit.crossEq
0185       (primeDirection p hp)
0186       (RatioOrbit.recip (primeDirection p hp)) := by
0187     intro h
0188     rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
0189       primeDirection_toRat] at h
0189     have hpNat0 : p.toNat ≠ 0 := by
0190       intro hzero

```

```

0192   apply hp.1
0193   apply DistinctionNat.toNat_inj
0194   rw [hzero, DistinctionNat.toNat_zero]
0195   have hpNat00 : (p.toNat : Q) ≠ 0 := by
0196     exact_mod_cast hpNat0
0197   have hsqQ : (p.toNat : Q) * (p.toNat : Q) = 1 := by
0198     have hmul := congrArg (fun t : Q => t * (p.toNat : Q)) h
0199     field_simp [hpNat00] at hmul
0200     ring_nf at hmul
0201     exact hmul
0202   have hsqNat : p.toNat * p.toNat = 1 := by
0203     exact_mod_cast hsqQ
0204   have hle : p.toNat ≤ 1 := by
0205     by_contra hnot
0206     have hge : 2 ≤ p.toNat := by omega
0207     have hprodge : 2 ≤ p.toNat * p.toNat := by
0208       calc
0209         2 ≤ 2 * 2 := by norm_num
0210         _ ≤ p.toNat * p.toNat := Nat.mul_le_mul hge hge
0211     omega
0212   have hpOne : p.toNat = 1 := by omega
0213   exact hp.2.1 ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hpOne)
0214
0215 theorem orbitDirection_nonunit_not_crossEq_recip
0216   (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero)
0217   (hunit : ¬ DistinctionNat.unit p) :
0218   ¬ RatioOrbit.crossEq
0219     (orbitDirection p hp)
0220     (RatioOrbit.recip (orbitDirection p hp)) := by
0221   intro h
0222   rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
0223     orbitDirection_toRat] at h
0224   have hpNat0 : p.toNat ≠ 0 := by
0225     intro hzero
0226     apply hp
0227     apply DistinctionNat.toNat_inj
0228     rw [hzero, DistinctionNat.toNat_zero]
0229     have hpNat00 : (p.toNat : Q) ≠ 0 := by
0230       exact_mod_cast hpNat0
0231     have hsqQ : (p.toNat : Q) * (p.toNat : Q) = 1 := by
0232       have hmul := congrArg (fun t : Q => t * (p.toNat : Q)) h
0233       field_simp [hpNat00] at hmul
0234       ring_nf at hmul
0235       exact hmul
0236     have hsqNat : p.toNat * p.toNat = 1 := by
0237       exact_mod_cast hsqQ
0238     have hle : p.toNat ≤ 1 := by
0239       by_contra hnot
0240       have hge : 2 ≤ p.toNat := by omega
0241       have hprodge : 2 ≤ p.toNat * p.toNat := by
0242         calc
0243           2 ≤ 2 * 2 := by norm_num
0244           _ ≤ p.toNat * p.toNat := Nat.mul_le_mul hge hge
0245     omega
0246     have hpOne : p.toNat = 1 := by omega
0247     exact hunit ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hpOne)
0248
0249 theorem orbit_succ_ne_zero (p : DistinctionNat) :
0250   DistinctionNat.succ p ≠ DistinctionNat.zero := by
0251   intro h
0252   exact DistinctionNat.zero_ne_succ p h.symm
0253
0254 theorem orbitDirection_succ_crossEq_add_one
0255   (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero) :
0256   RatioOrbit.crossEq
0257     (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
0258     (RatioOrbit.add (orbitDirection p hp) RatioOrbit.one) := by
0259   rw [RatioOrbit.crossEq_iff_toRat_eq]
0260   rw [RatioOrbit.add_toRat, RatioOrbit.one_toRat,
0261     orbitDirection_toRat, orbitDirection_toRat, DistinctionNat.toNat_succ]
0262   norm_num
0263
0264 theorem RatioOrbit.add_right_one_cancel {a b : RatioOrbit}
0265   (h : RatioOrbit.crossEq
0266     (RatioOrbit.add a RatioOrbit.one)
0267     (RatioOrbit.add b RatioOrbit.one)) :
0268   RatioOrbit.crossEq a b := by
0269   rw [RatioOrbit.crossEq_iff_toRat_eq] at h
0270   rw [RatioOrbit.add_toRat, RatioOrbit.add_toRat, RatioOrbit.one_toRat] at h
0271   linarith
0272
0273 /-- Native trace carried by an orbit position, defined by recursion on the
0274 6-orbit rather than by infier 'Nat' as object theory. -/
0275 def orbitPositionTrace : DistinctionNat → Trace
0276 | DistinctionNat.zero => Trace.empty
0277 | DistinctionNat.succ n => Trace.step (orbitPositionTrace n)
0278
0279 theorem orbitPositionTrace_add_extends_left
0280   (p r : DistinctionNat) :
0281   Trace.Extends (orbitPositionTrace p) (orbitPositionTrace (p + r)) := by
0282   induction r with
0283   | zero =>
0284     rw [DistinctionNat.add_zero_eq]
0285     exact Trace.extends_refl (orbitPositionTrace p)
0286   | succ r ih =>
0287     rw [DistinctionNat.add_succ_eq]
0288     rcases ih with (suffix, hsuffix)
0289     refine (Trace.step suffix, ?_)
0290     simp [Trace.step, orbitPositionTrace, hsuffix]
0291
0292 theorem orbitPositionTrace_add_extends_right

```

```

0293 (p r : DistinctionNat) :
0294 Trace.Extends (orbitPositionTrace r) (orbitPositionTrace (p + r)) := by
0295   rw [DistinctionNat.add_comm p r]
0296   exact orbitPositionTrace_add_extends_left r p
0297
0298 theorem orbitPositionTrace_extends_of_toNat_le
0299 {p r : DistinctionNat} (hpr : p.toNat ≤ r.toNat) :
0300   Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) := by
0301   let k : DistinctionNat := DistinctionNat.ofNat (r.toNat - p.toNat)
0302   have hsum : p + k = r := by
0303     apply DistinctionNat.toNat_inj
0304     rw [DistinctionNat.toNat_add, DistinctionNat.toNat_ofNat]
0305     omega
0306   rw [← hsum]
0307   exact orbitPositionTrace_add_extends_left p k
0308
0309 theorem orbitPositionTrace_comparable
0310 (p r : DistinctionNat) :
0311   Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) ∨
0312   Trace.Extends (orbitPositionTrace r) (orbitPositionTrace p) := by
0313   by_cases hpr : p.toNat ≤ r.toNat
0314   · exact Or.inl (orbitPositionTrace_extends_of_toNat_le hpr)
0315   · have hrp : r.toNat ≤ p.toNat := by omega
0316     exact Or.inr (orbitPositionTrace_extends_of_toNat_le hrp)
0317
0318 /-- Two prime axes are trace-connected when their native orbit traces admit a
0319 common finite 6-extension. -/
0320 def PRCPrimeAxisTraceConnected
0321 (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p)
0322 (r : DistinctionNat) (hr : DistinctionNat.primeOrbit r) : Prop :=
0323   ∃ T : Trace,
0324     Trace.Extends (orbitPositionTrace p) T ∧
0325     Trace.Extends (orbitPositionTrace r) T
0326
0327 theorem PRCPrimeAxisTraceConnected_proved
0328 (p : DistinctionNat) (hp : DistinctionNat.primeOrbit p)
0329 (r : DistinctionNat) (hr : DistinctionNat.primeOrbit r) :
0330   PRCPrimeAxisTraceConnected p hp r hr := by
0331   exact (orbitPositionTrace (p + r),
0332     orbitPositionTrace_add_extends_left p r,
0333     orbitPositionTrace_add_extends_right p r)
0334
0335 /-- A character has global cost orientation when every rational direction is
0336 sent either to itself or to its reciprocal. Since J-cost is reciprocal-symmetric,
0337 this is exactly the orientation information needed for cost propagation. -/
0338 def PRCCharacterGlobalCostOrientation
0339 (χ : RatioOrbit → RatioOrbit) : Prop :=
0340   ∀ q : RatioOrbit,
0341     RatioOrbit.crossEq (χ q) q ∨
0342     RatioOrbit.crossEq (χ q) (RatioOrbit.recip q)
0343
0344 /-- Sharper propagation blocker: prime calibration must force a coherent global
0345 orientation. Without this, independent prime inversions can preserve prime
0346 costs while breaking composite costs. -/
0347 def PRCPrimeCalibrationForcesGlobalOrientationTarget : Prop :=
0348   ∀ χ : RatioOrbit → RatioOrbit,
0349     PRCRatioCharacter χ →
0350     PRCCharacterPrimeDirectionCalibrated χ →
0351     PRCCharacterGlobalCostOrientation χ
0352
0353 /-- Prime-axis orientation is coherent when the character chooses the same
0354 orientation on every native prime direction: all identity or all reciprocal. -/
0355 def PRCCharacterPrimeOrientationCoherent
0356 (χ : RatioOrbit → RatioOrbit) : Prop :=
0357   (∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
0358     RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp)) ∨
0359   (∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
0360     RatioOrbit.crossEq (χ (primeDirection p hp))
0361       (RatioOrbit.recip (primeDirection p hp)))
0362
0363 /-- Local prime orientation says each prime axis is individually sent to itself
0364 or to its reciprocal. This is the algebraic content of equality of J-costs on a
0365 single prime direction. -/
0366 def PRCCharacterPrimeLocalOrientation
0367 (χ : RatioOrbit → RatioOrbit) : Prop :=
0368   ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
0369     RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp) ∨
0370     RatioOrbit.crossEq (χ (primeDirection p hp))
0371       (RatioOrbit.recip (primeDirection p hp))
0372
0373 /-- No mixed prime orientation says a character cannot choose identity on one
0374 prime axis and reciprocal on another. This is the trace-coherence condition that
0375 rules out independent prime-axis inversions. -/
0376 def PRCCharacterNoMixedPrimeOrientation
0377 (χ : RatioOrbit → RatioOrbit) : Prop :=
0378   ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
0379   ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
0380     RatioOrbit.crossEq (χ (primeDirection p hp)) (primeDirection p hp) →
0381     RatioOrbit.crossEq (χ (primeDirection r hr))
0382       (RatioOrbit.recip (primeDirection r hr)) →
0383     False
0384
0385 /-- Prime identity orientation is trace-coherent when identity orientation at
0386 one calibrated prime forces identity orientation at every calibrated prime. This
0387 is the missing cross-prime relation; the current ratio-character laws are local
0388 to multiplication and reciprocal and do not by themselves connect the orientation
0389 choices of different prime axes. -/
0390 def PRCCharacterPrimeIdentityTraceCoherent
0391 (χ : RatioOrbit → RatioOrbit) : Prop :=
0392   ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
0393   ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,

```

```

0394 RatioOrbit.crossEq (x (primeDirection p hp)) (primeDirection p hp) →
0395 RatioOrbit.crossEq (x (primeDirection r hr)) (primeDirection r hr)
0396
0397 /-- A character respects prime-axis trace connection when identity orientation
0398 transports along the finite  $\delta$ -trace component relating two prime axes. -/
0399 def PRCCharacterPrimeIdentityRespectsTraceConnected
0400   (x : RatioOrbit → RatioOrbit) : Prop :=
0401   ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
0402   ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
0403   PRCPrimeAxisTraceConnected p hp r hr →
0404   RatioOrbit.crossEq (x (primeDirection p hp)) (primeDirection p hp) →
0405   RatioOrbit.crossEq (x (primeDirection r hr)) (primeDirection r hr)
0406
0407 /-- A more explicit form of the trace-transport rule: identity orientation
0408 transports when two prime-axis traces are witnessed inside the same finite
0409  $\delta$ -trace extension. -/
0410 def PRCCharacterPrimeIdentityRespectsCommonTraceExtension
0411   (x : RatioOrbit → RatioOrbit) : Prop :=
0412   ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
0413   ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
0414   T : Trace,
0415   Trace.Extends (orbitPositionTrace p) T →
0416   Trace.Extends (orbitPositionTrace r) T →
0417   RatioOrbit.crossEq (x (primeDirection p hp)) (primeDirection p hp) →
0418   RatioOrbit.crossEq (x (primeDirection r hr)) (primeDirection r hr)
0419
0420 /-- The exact trace-order law: identity orientation transports between prime axes
0421 whose finite  $\delta$ -orbit traces are comparable by extension. The structural
0422 comparability of any two orbit traces is proved above, so this is the part that
0423 must come from the ratio character respecting trace order. -/
0424 def PRCCharacterPrimeIdentityRespectsComparableTrace
0425   (x : RatioOrbit → RatioOrbit) : Prop :=
0426   ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
0427   ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
0428   (Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) v
0429   Trace.Extends (orbitPositionTrace r) (orbitPositionTrace p)) →
0430   RatioOrbit.crossEq (x (primeDirection p hp)) (primeDirection p hp) →
0431   RatioOrbit.crossEq (x (primeDirection r hr)) (primeDirection r hr)
0432
0433 /-- Identity orientation for an arbitrary nonzero orbit direction, not only for
0434 prime axes. This lets trace transport pass through composite orbit positions. -/
0435 def PRCCharacterOrbitDirectionIdentity
0436   (x : RatioOrbit → RatioOrbit)
0437   (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero) : Prop :=
0438   RatioOrbit.crossEq (x (orbitDirection p hp)) (orbitDirection p hp)
0439
0440 /-- Reciprocal orientation for an arbitrary nonzero orbit direction. -/
0441 def PRCCharacterOrbitDirectionReciprocal
0442   (x : RatioOrbit → RatioOrbit)
0443   (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero) : Prop :=
0444   RatioOrbit.crossEq
0445     (x (orbitDirection p hp)) (RatioOrbit.recip (orbitDirection p hp))
0446
0447 /-- The one-step trace-order law: identity orientation is invariant under one
0448 successor step of the nonzero  $\delta$ -orbit. This is smaller than prime-to-prime
0449 transport, because it acts before primality is imposed. -/
0450 def PRCCharacterOrbitIdentityRespectsSuccessorStep
0451   (x : RatioOrbit → RatioOrbit) : Prop :=
0452   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0453   PRCCharacterOrbitDirectionIdentity x p hp →
0454   PRCCharacterOrbitDirectionIdentity x
0455     (DistinctionNat.succ p) (orbit_succ_ne_zero p)
0456
0457 /-- Forward one-step successor law for identity orientation on nonzero orbit
0458 directions. -/
0459 def PRCCharacterOrbitIdentityExtendsSuccessorStep
0460   (x : RatioOrbit → RatioOrbit) : Prop :=
0461   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0462   PRCCharacterOrbitDirectionIdentity x p hp →
0463   PRCCharacterOrbitDirectionIdentity x
0464     (DistinctionNat.succ p) (orbit_succ_ne_zero p)
0465
0466 /-- Backward one-step successor law for identity orientation on nonzero orbit
0467 directions. -/
0468 def PRCCharacterOrbitIdentityContractsSuccessorStep
0469   (x : RatioOrbit → RatioOrbit) : Prop :=
0470   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0471   PRCCharacterOrbitDirectionIdentity x
0472     (DistinctionNat.succ p) (orbit_succ_ne_zero p) →
0473   PRCCharacterOrbitDirectionIdentity x p hp
0474
0475 /-- The successor-step transport needed for trace coherence is exactly the
0476 forward and backward one-step laws bundled together. -/
0477 def PRCCharacterOrbitIdentitySuccessorTransport
0478   (x : RatioOrbit → RatioOrbit) : Prop :=
0479   PRCCharacterOrbitIdentityExtendsSuccessorStep x ∧
0480   PRCCharacterOrbitIdentityContractsSuccessorStep x
0481
0482 /-- Additive compatibility with the  $\delta$ -successor operation on nonzero orbit
0483 directions. This is the missing bridge between multiplicative ratio characters
0484 and the trace/additive structure of the orbit. -/
0485 def PRCCharacterOrbitSuccessorAdditiveCompatible
0486   (x : RatioOrbit → RatioOrbit) : Prop :=
0487   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0488   RatioOrbit.crossEq
0489     (x (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p)))
0490     (RatioOrbit.add (x (orbitDirection p hp)) RatioOrbit.one)
0491
0492 theorem PRCCharacterOrbitIdentityExtendsSuccessorStep_of_additive_compat
0493   {x : RatioOrbit → RatioOrbit}
0494   (hadd : PRCCharacterOrbitSuccessorAdditiveCompatible x) :

```

```

0495   PRCCharacterOrbitIdentityExtendsSuccessorStep x := by
0496   intro p hp hpId
0497   have hysucc := hadd p hp
0498   rw [PRCCharacterOrbitDirectionIdentity] at hpId ⊢
0499   rw [RatioOrbit.crossEq_iff_toNat_eq] at hysucc hpId ⊢
0500   rw [RatioOrbit.add_toNat, RatioOrbit.one_toNat] at hysucc
0501   rw [hysucc, hpId]
0502   rw [orbitDirection_toNat, orbitDirection_toNat, DistinctionNat.toNat_succ]
0503   norm_num
0504
0505   theorem PRCCharacterOrbitIdentityContractsSuccessorStep_of_additive_compat
0506   {x : RatioOrbit → RatioOrbit}
0507   (hadd : PRCCharacterOrbitSuccessorAdditiveCompatible x) :
0508   PRCCharacterOrbitIdentityContractsSuccessorStep x := by
0509   intro p hp hsuccId
0510   have hysucc := hadd p hp
0511   rw [PRCCharacterOrbitDirectionIdentity] at hsuccId ⊢
0512   rw [RatioOrbit.crossEq_iff_toNat_eq] at hysucc hsuccId ⊢
0513   rw [RatioOrbit.add_toNat, RatioOrbit.one_toNat] at hysucc
0514   rw [hysucc] at hsuccId
0515   rw [orbitDirection_toNat, DistinctionNat.toNat_succ] at hsuccId
0516   rw [orbitDirection_toNat]
0517   have hsuccCast :
0518     ((Nat.succ p.toNat : Nat) : Q) = (p.toNat : Q) + 1 := by
0519     norm_num
0520   rw [hsuccCast] at hsuccId
0521   linarith
0522
0523   theorem PRCCharacterOrbitIdentitySuccessorTransport_of_additive_compat
0524   {x : RatioOrbit → RatioOrbit}
0525   (hadd : PRCCharacterOrbitSuccessorAdditiveCompatible x) :
0526   PRCCharacterOrbitIdentitySuccessorTransport x :=
0527   (PRCCharacterOrbitIdentityExtendsSuccessorStep_of_additive_compat hadd,
0528    PRCCharacterOrbitIdentityContractsSuccessorStep_of_additive_compat hadd)
0529
0530   theorem orbit_succ_not_unit_of_nonzero_not_unit
0531   (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero)
0532   (hunit : ¬ DistinctionNat.unit p) :
0533   ¬ DistinctionNat.unit (DistinctionNat.succ p) := by
0534   intro hsuccUnit
0535   have hpNat0 : p.toNat ≠ 0 := by
0536     intro hz
0537     apply hp
0538     apply DistinctionNat.toNat_inj
0539     rw [hz, DistinctionNat.toNat_zero]
0540     have hpNat1 : p.toNat = 1 := by
0541       intro hone
0542       exact hunit ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hone)
0543     have hsuccNat1 : (DistinctionNat.succ p).toNat = 1 :=
0544       (DistinctionNat.unit_iff_toNat_eq_one (DistinctionNat.succ p)).mp hsuccUnit
0545     rw [DistinctionNat.toNat_succ] at hsuccNat1
0546     omega
0547
0548   /-- Every nonunit orbit direction is locally oriented: identity or reciprocal.
0549   This is the nonprime analogue of the already proved local-prime orientation
0550   alternative. -/
0551   def PRCCharacterNonunitOrbitLocalOrientation
0552   {x : RatioOrbit → RatioOrbit} : Prop :=
0553   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0554   ¬ DistinctionNat.unit p →
0555     PRCCharacterOrbitDirectionIdentity x p hp →
0556     PRCCharacterOrbitDirectionReciprocal x p hp
0557
0558   /-- Product-factor propagation for local orientation. If two nonunit factors are
0559   locally identity-or-reciprocal oriented, their product is locally oriented too.
0560   This is the exact multiplicative step needed to move from prime-axis
0561   orientation to composite orbit directions. -/
0562   def PRCCharacterOrbitProductLocalOrientationPropagates
0563   {x : RatioOrbit → RatioOrbit} : Prop :=
0564   ∀ a b p : DistinctionNat,
0565   ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
0566   ¬ DistinctionNat.unit a →
0567     ¬ DistinctionNat.unit b →
0568     ∀ hp : p ≠ DistinctionNat.zero,
0569     ¬ DistinctionNat.unit p →
0570     a * b = p →
0571     (PRCCharacterOrbitDirectionIdentity x a ha →
0572      PRCCharacterOrbitDirectionReciprocal x a ha) →
0573     (PRCCharacterOrbitDirectionIdentity x b hb →
0574      PRCCharacterOrbitDirectionReciprocal x b hb) →
0575     PRCCharacterOrbitDirectionIdentity x p hp →
0576     PRCCharacterOrbitDirectionReciprocal x p hp
0577
0578   theorem ratioOrbit_mul_congr {a₁ a₂ b₁ b₂ : RatioOrbit}
0579   (ha : RatioOrbit.crossEq a₁ a₂) (hb : RatioOrbit.crossEq b₁ b₂) :
0580   RatioOrbit.crossEq (RatioOrbit.mul a₁ b₁) (RatioOrbit.mul a₂ b₂) := by
0581   rw [RatioOrbit.crossEq_iff_toNat_eq] at ha hb ⊢
0582   rw [RatioOrbit.mul_toNat, RatioOrbit.mul_toNat, ha, hb]
0583
0584   theorem ratioOrbit_recip_congr {a b : RatioOrbit}
0585   (h : RatioOrbit.crossEq a b) :
0586   RatioOrbit.crossEq (RatioOrbit.recip a) (RatioOrbit.recip b) := by
0587   rw [RatioOrbit.crossEq_iff_toNat_eq] at h ⊢
0588   rw [RatioOrbit.recip_toNat, RatioOrbit.recip_toNat, h]
0589
0590   theorem ratioOrbit_mul_recip_recip_crossEq_recip_mul
0591   (a b : RatioOrbit) :
0592   RatioOrbit.crossEq
0593   (RatioOrbit.mul (RatioOrbit.recip a) (RatioOrbit.recip b))
0594   (RatioOrbit.recip (RatioOrbit.mul a b)) := by
0595   rw [RatioOrbit.crossEq_iff_toNat_eq, RatioOrbit.mul_toNat,

```

```

0596   RatioOrbit.recip_toRat, RatioOrbit.recip_toRat, RatioOrbit.recip_toRat,
0597   RatioOrbit.mul_toRat]
0598   by_cases ha : a.toRat = 0
0599   · simp [ha]
0600   · by_cases hb : b.toRat = 0
0601     · simp [hb]
0602     · field_simp [ha, hb]
0603
0604 theorem orbitDirection_mul_crossEq
0605   (a b p : DistinctionNat)
0606   (ha : a ≠ DistinctionNat.zero) (hb : b ≠ DistinctionNat.zero)
0607   (hp : p ≠ DistinctionNat.zero)
0608   (hmul : a * b = p) :
0609   RatioOrbit.crossEq (orbitDirection p hp)
0610   (RatioOrbit.mul (orbitDirection a ha) (orbitDirection b hb)) := by
0611   rw [RatioOrbit.crossEq_iff_toRat_eq, orbitDirection_toRat,
0612     RatioOrbit.mul_toRat, orbitDirection_toRat, orbitDirection_toRat]
0613   have hnata := congrArg DistinctionNat.toNat hmul
0614   rw [DistinctionNat.toNat_mul] at hnata
0615   exact_mod_cast hnata.symm
0616
0617 /-- A character is compatible with the native display of an orbit product when
0618 the character value on the product orbit agrees with the character value on the
0619 ratio product of the factor orbits. This is not automatic from
0620 cross-equivalence; it is the quotient-respect step missing from the bare
0621 character interface. -/
0622 def PRCCharacterOrbitProductDisplayCompatible
0623   (χ : RatioOrbit → RatioOrbit) : Prop :=
0624   ∀ a b p : DistinctionNat,
0625   ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
0626   ∀ hp : p ≠ DistinctionNat.zero,
0627   a * b = p →
0628     RatioOrbit.crossEq (χ (orbitDirection p hp))
0629     (χ (RatioOrbit.mul (orbitDirection a ha) (orbitDirection b hb)))
0630
0631 /-- Quotient-respect for a ratio character: equivalent ratio-orbit displays
0632 must receive equivalent character values. This is the missing map-respects-setoid
0633 condition for using a raw `RatioOrbit → RatioOrbit` function as a quotient-native
0634 PRC character. -/
0635 def PRCCharacterRespectsCrossEq (χ : RatioOrbit → RatioOrbit) : Prop :=
0636   ∀ q r : RatioOrbit,
0637     RatioOrbit.crossEq q r → RatioOrbit.crossEq (χ q) (χ r)
0638
0639 /-- Canonical-normalization target for ratio orbits. If two raw ratio displays
0640 are cross-equivalent, native GCD normalization should return the same raw
0641 representative. This is the exact quotient-normalization uniqueness statement
0642 needed to turn `normalized_invariant` into general quotient respect. -/
0643 def PRCNormalizeRatioCanonicalTarget : Prop :=
0644   ∀ q r : RatioOrbit,
0645     RatioOrbit.crossEq q r →
0646     DistinctionNat.normalizeRatio q = DistinctionNat.normalizeRatio r
0647
0648 /-- A signed-orbit display is sign-canonical when it is literally the
0649 nonnegative orbit display of its absolute value, or literally the negated
0650 nonnegative display of its absolute value. This records the raw representative
0651 condition supplied by `signedQuotient`, not just balanced integer equality. -/
0652 def PRCSignedOrbitSignCanonical (z : SignedOrbit) : Prop :=
0653   (0 ≤ z.toInt ∧ z = SignedOrbit.ofOrbit z.abs) ∨
0654   (z.toInt < 0 ∧ z = SignedOrbit.negate (SignedOrbit.ofOrbit z.abs))
0655
0656 /-- A raw ratio display is reduced and sign-canonical when its numerator
0657 absolute value is coprime to the positive denominator and the signed numerator
0658 itself is in the canonical raw signed-orbit form. -/
0659 def PRCRatioReducedSignCanonical (q : RatioOrbit) : Prop :=
0660   DistinctionNat.coprime q.num.abs q.den ∧
0661   PRCSignedOrbitSignCanonical q.num
0662
0663 theorem signedOrbit_ofOrbit_abs_self (n : DistinctionNat) :
0664   (SignedOrbit.ofOrbit n).abs = n := by
0665   apply DistinctionNat.toNat_inj
0666   rw [SignedOrbit.abs_toNat, SignedOrbit.ofOrbit_toInt]
0667   simp
0668
0669 theorem signedOrbit_neg_ofOrbit_abs_self (n : DistinctionNat) :
0670   (SignedOrbit.negate (SignedOrbit.ofOrbit n)).abs = n := by
0671   apply DistinctionNat.toNat_inj
0672   rw [SignedOrbit.abs_toNat, SignedOrbit.negate_toInt,
0673     SignedOrbit.ofOrbit_toInt]
0674   simp
0675
0676 theorem signedQuotient_signCanonical_of_divides
0677   (z : SignedOrbit) (d : DistinctionNat) (hd : d ≠ DistinctionNat.zero)
0678   (hdiv : DistinctionNat.divides d z.abs) :
0679   PRCSignedOrbitSignCanonical (DistinctionNat.signedQuotient z d hd) := by
0680   unfold DistinctionNat.signedQuotient
0681   by_cases hflag : z.nonnegFlag = true
0682   · left
0683     constructor
0684     · simp [hflag, SignedOrbit.ofOrbit_toInt]
0685     · rw [if_pos hflag]
0686     · rw [signedOrbit_ofOrbit_abs_self]
0687   · have hflagFalse : z.nonnegFlag = false := by
0688     cases h : z.nonnegFlag with
0689     | false => rfl
0690     | true =>
0691       exfalso
0692       exact hflag h
0693   right
0694   have hzneg : z.toInt < 0 :=
0695     (SignedOrbit.nonnegFlag_eq_false_iff z).mp hflagFalse
0696   have hzabs_ne : z.abs ≠ DistinctionNat.zero := by

```



```

0697   apply SignedOrbit.abs_ne_zero_of_toInt_ne_zero
0698   omega
0699   have hq_ne :
0700     DistinctionNat.quotient z.abs d hd ≠ DistinctionNat.zero :=
0701     DistinctionNat.quotient_ne_zero_of_divides
0702     (n := z.abs) (d := d) hd hdiv hzabs_ne
0703   have hq_pos : 0 < (DistinctionNat.quotient z.abs d hd).toNat := by
0704     have hq_nat_ne : (DistinctionNat.quotient z.abs d hd).toNat ≠ 0 := by
0705       intro hzero
0706       apply hq_ne
0707       apply DistinctionNat.toNat_inj
0708       rw [hzero, DistinctionNat.toNat_zero]
0709       omega
0710   constructor
0711   · simp [hflagFalse, SignedOrbit.negate_toInt,
0712     SignedOrbit.ofOrbit_toInt]
0713   exact hq_pos
0714   · rw [if_neg hflag]
0715   rw [signedOrbit_neg_ofOrbit_abs_self]
0716
0717   theorem normalizeRatio_reduced_signCanonical (q : RatioOrbit) :
0718     PRCRatioReducedSignCanonical (DistinctionNat.normalizeRatio q) := by
0719     constructor
0720     · exact DistinctionNat.normalizeRatio_coprime q
0721     · unfold DistinctionNat.normalizeRatio
0722       exact signedQuotient_signCanonical_of_divides
0723         q.num (DistinctionNat.gcd q.num.abs q.den)
0724         (DistinctionNat.gcd_ne_zero_of_right_ne_zero
0725           q.num.abs q.den q.den_ne_zero)
0726         (DistinctionNat.gcd_divides_left q.num.abs q.den)
0727
0728   theorem PRCSignedOrbitSignCanonical.eq_of_toInt_eq
0729     (z w : SignedOrbit)
0730     (hz : PRCSignedOrbitSignCanonical z)
0731     (hw : PRCSignedOrbitSignCanonical w)
0732     (hzw : z.toInt = w.toInt) :
0733     z = w := by
0734     rcases hz with (hzNonneg, hzCanon) | (hzNeg, hzCanon)
0735     · rcases hw with (_hwNonneg, hwCanon) | (hwNeg, _hwCanon)
0736       · calc
0737         z = SignedOrbit.ofOrbit z.abs := hzCanon
0738         _ = SignedOrbit.ofOrbit w.abs := by
0739           have habs : z.abs = w.abs := by
0740             apply DistinctionNat.toNat_inj
0741             rw [SignedOrbit.abs_toNat, SignedOrbit.abs_toNat, hzw]
0742             exact congrArg SignedOrbit.ofOrbit habs
0743             _ = w := hwCanon.symm
0744       · rw [hzw] at hzNonneg
0745       omega
0746     · rcases hw with (hwNonneg, _hwCanon) | (_hwNeg, hwCanon)
0747     · rw [hzw] at hzNeg
0748     omega
0749     · calc
0750       z = SignedOrbit.negate (SignedOrbit.ofOrbit z.abs) := hzCanon
0751       _ = SignedOrbit.negate (SignedOrbit.ofOrbit w.abs) := by
0752         have habs : z.abs = w.abs := by
0753           apply DistinctionNat.toNat_inj
0754           rw [SignedOrbit.abs_toNat, SignedOrbit.abs_toNat, hzw]
0755           exact congrArg (fun n => SignedOrbit.negate (SignedOrbit.ofOrbit n)) habs
0756           _ = w := hwCanon.symm
0757
0758   theorem PRCReducedSignCanonical_den_dvd_of_crossEq
0759     {q r : RatioOrbit}
0760     (hq : PRCRatioReducedSignCanonical q)
0761     (_hr : PRCRatioReducedSignCanonical r)
0762     (hqr : RatioOrbit.crossEq q r) :
0763     q.den.toNat | r.den.toNat := by
0764     have hcrossZ : q.num.toInt * (r.den.toNat : ℤ) =
0765       r.num.toInt * (q.den.toNat : ℤ) := by
0766       unfold RatioOrbit.crossEq at hqr
0767       have hdisplay :=
0768         (SignedOrbit.balanced_iff_toInt_eq
0769           (q.num.scaleByNat r.den) (r.num.scaleByNat q.den)).mp hqr
0770       rw [SignedOrbit.scaleByNat_toInt, SignedOrbit.scaleByNat_toInt] at hdisplay
0771       exact hdisplay
0772     have hcrossNat :
0773       q.num.abs.toNat * r.den.toNat =
0774       r.num.abs.toNat * q.den.toNat := by
0775       have h := congrArg Int.natAbs hcrossZ
0776       rw [Int.natAbs_mul, Int.natAbs_mul,
0777         ← SignedOrbit.abs_toNat q.num, ← SignedOrbit.abs_toNat r.num,
0778         Int.natAbs_natCast, Int.natAbs_natCast] at h
0779       exact h
0780     have hcop :
0781       Nat.Coprime q.num.abs.toNat q.den.toNat :=
0782       (DistinctionNat.coprime_iff_nat_coprime q.num.abs q.den).mp hq.1
0783     have hdivMul : q.den.toNat | q.num.abs.toNat * r.den.toNat := by
0784       rw [hcrossNat]
0785     exact Nat.dvd_mul_left q.den.toNat r.num.abs.toNat
0786     exact hcop.symm.dvd_of_dvd_mul_left hdivMul
0787
0788   theorem PRCReducedSignCanonical_den_eq_of_crossEq
0789     {q r : RatioOrbit}
0790     (hq : PRCRatioReducedSignCanonical q)
0791     (hr : PRCRatioReducedSignCanonical r)
0792     (hqr : RatioOrbit.crossEq q r) :
0793     q.den = r.den := by
0794     apply DistinctionNat.toNat_inj
0795     exact Nat.dvd_antisymm
0796       (PRCReducedSignCanonical_den_dvd_of_crossEq hq hr hqr)
0797       (PRCReducedSignCanonical_den_dvd_of_crossEq hr hq

```

```

0798 (RatioOrbit.crossEq_symm hqr))
0799
0800 theorem PRCReducedSignCanonical_num_eq_of_crossEq
0801 {q r : RatioOrbit}
0802 (hq : PRCRatioReducedSignCanonical q)
0803 (hr : PRCRatioReducedSignCanonical r)
0804 (hqr : RatioOrbit.crossEq q r) :
0805   q.num = r.num := by
0806   have hden : q.den = r.den :=
0807     PRCReducedSignCanonical_den_eq_of_crossEq hq hr hqr
0808   have hcrossZ : q.num.toInt * (r.den.toNat : Z) =
0809     r.num.toInt * (q.den.toNat : Z) := by
0810     unfold RatioOrbit.crossEq at hqr
0811     have hdisplay :=
0812       (SignedOrbit.balanced_iff_toInt_eq
0813         (q.num.scaleByNat r.den) (r.num.scaleByNat q.den)).mp hqr
0814     rw [SignedOrbit.scaleByNat_toInt, SignedOrbit.scaleByNat_toInt] at hdisplay
0815     exact hdisplay
0816   have hdenInt : (q.den.toNat : Z) ≠ 0 := by
0817     exact_mod_cast q.den.toNat_ne_zero
0818   have hnum : q.num.toInt = r.num.toInt := by
0819     rw [← hden] at hcrossZ
0820     exact mul_right_cancel hdenInt hcrossZ
0821   exact PRCSignedOrbitSignCanonical.eq_of_toInt_eq hq.2 hr.2 hnum
0822
0823 /-- Reduced sign-canonical uniqueness is the exact remaining raw-display
0824 number-theory blocker for canonical normalization. It says two reduced,
0825 sign-canonical ratio displays with the same cross-multiplication class are
0826 definitionally the same raw ratio orbit. -/
0827 def PRCReducedSignCanonicalRatioUniqueTarget : Prop :=
0828   ∀ q r : RatioOrbit,
0829     PRCRatioReducedSignCanonical q →
0830     PRCRatioReducedSignCanonical r →
0831     RatioOrbit.crossEq q r →
0832     q = r
0833
0834 theorem PRCReducedSignCanonicalRatioUniqueTarget_proved :
0835   PRCReducedSignCanonicalRatioUniqueTarget := by
0836   intro q r hq hr hqr
0837   cases q with
0838   | mk qnum qden qden_ne_zero =>
0839     cases r with
0840     | mk rnum rden rden_ne_zero =>
0841       have hnum :
0842         qnum = rnum :=
0843         PRCReducedSignCanonical_num_eq_of_crossEq
0844           (q := (qnum, qden, qden_ne_zero))
0845           (r := (rnum, rden, rden_ne_zero)) hq hr hqr
0846       have hden :
0847         qden = rden :=
0848         PRCReducedSignCanonical_den_eq_of_crossEq
0849           (q := (qnum, qden, qden_ne_zero))
0850           (r := (rnum, rden, rden_ne_zero)) hq hr hqr
0851       subst hnum
0852       subst hden
0853       rfl
0854
0855 theorem PRCNormalizeRatioCanonicalTarget_of_reduced_signCanonical_unique
0856 (hunique : PRCReducedSignCanonicalRatioUniqueTarget) :
0857   PRCNormalizeRatioCanonicalTarget := by
0858   intro q r hqr
0859   apply hunique
0860   · exact normalizeRatio_reduced_signCanonical q
0861   · exact normalizeRatio_reduced_signCanonical r
0862   · exact RatioOrbit.crossEq_trans
0863     (RatioOrbit.crossEq_symm (DistinctionNat.normalizeRatio_crossEq q))
0864     (RatioOrbit.crossEq_trans hqr (DistinctionNat.normalizeRatio_crossEq r))
0865
0866 theorem PRCNormalizeRatioCanonicalTarget_proved :
0867   PRCNormalizeRatioCanonicalTarget :=
0868   PRCNormalizeRatioCanonicalTarget_of_reduced_signCanonical_unique
0869   PRCReducedSignCanonicalRatioUniqueTarget_proved
0870
0871 theorem PRCCharacterRespectsCrossEq_of_normalizeRatio_canonical
0872 {χ : RatioOrbit → RatioOrbit}
0873 (hχ : PRCRatioCharacter χ)
0874 (hcanon : PRCNormalizeRatioCanonicalTarget) :
0875   PRCCharacterRespectsCrossEq χ := by
0876   intro q r hqr
0877   exact RatioOrbit.crossEq_trans
0878     (hχ.normalized_invariant q)
0879     (RatioOrbit.crossEq_trans
0880       (by
0881         rw [hcanon q r hqr]
0882         exact RatioOrbit.crossEq_refl (χ (DistinctionNat.normalizeRatio r)))
0883       (RatioOrbit.crossEq_symm (hχ.normalized_invariant r))))
0884
0885 theorem PRCCharacterOrbitProductDisplayCompatible_of_crossEq_respect
0886 {χ : RatioOrbit → RatioOrbit}
0887 (hrespect : PRCCharacterRespectsCrossEq χ) :
0888   PRCCharacterOrbitProductDisplayCompatible χ := by
0889   intro a b p ha hb hp hmul
0890   exact hrespect (orbitDirection p hp)
0891     (RatioOrbit.mul (orbitDirection a ha) (orbitDirection b hb))
0892     (orbitDirection_mul_crossEq a b p ha hb hp hmul)
0893
0894 /-- Product factors cannot be mixed identity/reciprocal oriented. This is the
0895 exact obstruction left after the pure same-orientation product algebra is
0896 discharged. -/
0897 def PRCCharacterOrbitProductNoMixedOrientation
0898 (χ : RatioOrbit → RatioOrbit) : Prop :=

```

```

0899   ∀ a b p : DistinctionNat,
0900   ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
0901   ~ DistinctionNat.unit a →
0902   ~ DistinctionNat.unit b →
0903   ∀ _hp : p ≠ DistinctionNat.zero,
0904   ~ DistinctionNat.unit p →
0905   a * b = p →
0906   (¬ (PRCCharacterOrbitDirectionIdentity x a ha ∧
0907     PRCCharacterOrbitDirectionReciprocal x b hb)) ∧
0908   (¬ (PRCCharacterOrbitDirectionReciprocal x a ha ∧
0909     PRCCharacterOrbitDirectionIdentity x b hb))
0910
0911 /-- Nonunit orbit orientation is coherent when every nonunit orbit direction
0912 chooses the same branch: all identity or all reciprocal. This is the exact
0913 coherence statement strong enough to rule out mixed product factors. -/
0914 def PRCCharacterNonunitOrbitOrientationCoherent
0915   (x : RatioOrbit → RatioOrbit) : Prop :=
0916   (∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0917     ~ DistinctionNat.unit p →
0918     PRCCharacterOrbitDirectionIdentity x p hp) ∨
0919   (∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0920     ~ DistinctionNat.unit p →
0921     PRCCharacterOrbitDirectionReciprocal x p hp)
0922
0923 /-- Cross-nonunit no-mixing: identity orientation at one nonunit orbit direction
0924 cannot coexist with reciprocal orientation at another. This is the branch-coupling
0925 part of global nonunit coherence, separated from local orientation existence. -/
0926 def PRCCharacterNoMixedNonunitOrbitOrientation
0927   (x : RatioOrbit → RatioOrbit) : Prop :=
0928   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0929   ~ DistinctionNat.unit p →
0930   ∀ r : DistinctionNat, ∀ hr : r ≠ DistinctionNat.zero,
0931   ~ DistinctionNat.unit r →
0932   PRCCharacterOrbitDirectionIdentity x p hp →
0933   PRCCharacterOrbitDirectionReciprocal x r hr →
0934   False
0935
0936 /-- Positive branch transport form of nonunit coherence: if one nonunit orbit
0937 direction is identity-oriented, every nonunit orbit direction is identity-oriented.
0938 This is the same branch-coupling law as no-mixing once local orientation is known,
0939 but it states the missing transport direction directly. -/
0940 def PRCCharacterNonunitIdentityBranchTransport
0941   (x : RatioOrbit → RatioOrbit) : Prop :=
0942   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0943   ~ DistinctionNat.unit p →
0944   PRCCharacterOrbitDirectionIdentity x p hp →
0945   ∀ r : DistinctionNat, ∀ hr : r ≠ DistinctionNat.zero,
0946   ~ DistinctionNat.unit r →
0947   PRCCharacterOrbitDirectionIdentity x r hr
0948
0949 /-- Trace-order form of nonunit identity transport: identity orientation at one
0950 nonunit orbit direction transports to another nonunit direction when their
0951 finite δ-orbit traces are comparable. Since orbit traces are structurally
0952 comparable, this is equivalent to global nonunit identity-branch transport, but
0953 it exposes the next proof obligation as a trace-order law. -/
0954 def PRCCharacterNonunitIdentityRespectsComparableTrace
0955   (x : RatioOrbit → RatioOrbit) : Prop :=
0956   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
0957   ~ DistinctionNat.unit p →
0958   ∀ r : DistinctionNat, ∀ hr : r ≠ DistinctionNat.zero,
0959   ~ DistinctionNat.unit r →
0960   (Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) ∨
0961     Trace.Extends (orbitPositionTrace r) (orbitPositionTrace p)) →
0962   PRCCharacterOrbitDirectionIdentity x p hp →
0963   PRCCharacterOrbitDirectionIdentity x r hr
0964
0965 theorem orbit_mul_not_unit_of_left_not_unit
0966   {p r : DistinctionNat} (hunit : ~ DistinctionNat.unit p) :
0967   ~ DistinctionNat.unit (p * r) := by
0968   intro hprodUnit
0969   have hprodNat : (p * r).toNat = 1 :=
0970     (DistinctionNat.unit_iff_toNat_eq_one (p * r)).mp hprodUnit
0971   have hpNat1 : p.toNat ≠ 1 := by
0972     intro hOne
0973     exact hunit ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hOne)
0974   rw [DistinctionNat.toNat_mul] at hprodNat
0975   have hpOne : p.toNat = 1 := Nat.eq_one_of_mul_eq_one_right hprodNat
0976   exact hpNat1 hpOne
0977
0978 theorem PRCCharacterNonunitOrbitLocalOrientation_of_coherent
0979   (x : RatioOrbit → RatioOrbit)
0980   (hcoh : PRCCharacterNonunitOrbitOrientationCoherent x) :
0981   PRCCharacterNonunitOrbitLocalOrientation x := by
0982   intro p hp hunit
0983   rcases hcoh with hallId | hallRec
0984   · exact Or.inl (hallId p hp hunit)
0985   · exact Or.inr (hallRec p hp hunit)
0986
0987 theorem PRCCharacterNoMixedNonunitOrbitOrientation_of_coherent
0988   (x : RatioOrbit → RatioOrbit)
0989   (hcoh : PRCCharacterNonunitOrbitOrientationCoherent x) :
0990   PRCCharacterNoMixedNonunitOrbitOrientation x := by
0991   intro p hp hunit r hr hUnit hpId hrRec
0992   rcases hcoh with hallId | hallRec
0993   · have hrId := hallId r hr hUnit
0994     have hself :
0995       RatioOrbit.crossEq (orbitDirection r hr)
0996       (RatioOrbit.recip (orbitDirection r hr)) :=
0997       RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hrId) hrRec
0998     exact orbitDirection_nonunit_not_crossEq_recip r hr hUnit hself
0999   · have hpRec := hallRec p hp hunit

```

```

1000   have hself :
1001     RatioOrbit.crossEq (orbitDirection p hp)
1002     (RatioOrbit.recip (orbitDirection p hp)) :=
1003     RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
1004     exact orbitDirection_nonunit_not_crossEq_recip p hp hunit hself
1005
1006 theorem PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
1007   {x : RatioOrbit → RatioOrbit}
1008   (hnomix : PRCCharacterOrbitProductNoMixedOrientation x) :
1009   PRCCharacterNoMixedNonunitOrbitOrientation x := by
1010   intro p hp hpUnit r hr hrUnit hpId hrRec
1011   exact ((hnomix p r (p * r) hp hr hpUnit hrUnit
1012     (DistinctionNat.mul_ne_zero hp hr)
1013     (orbit_mul_not_unit_of_left_not_unit hpUnit) rfl).1
1014     (hpId, hrRec))
1015
1016 theorem PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit
1017   {x : RatioOrbit → RatioOrbit}
1018   (hnomix : PRCCharacterNoMixedNonunitOrbitOrientation x) :
1019   PRCCharacterOrbitProductNoMixedOrientation x := by
1020   intro a b p ha hb haUnit hbUnit _hp _hpUnit _hmul
1021   constructor
1022   · rintro (haId, hbRec)
1023     exact hnomix a ha haUnit b hb hbUnit haId hbRec
1024   · rintro (haRec, hbId)
1025     exact hnomix b hb hbUnit a ha haUnit hbId haRec
1026
1027 theorem PRCCharacterOrbitProductNoMixedOrientation_iff_no_mixed_nonunit
1028   {x : RatioOrbit → RatioOrbit} :
1029   PRCCharacterOrbitProductNoMixedOrientation x ↔
1030   PRCCharacterNoMixedNonunitOrbitOrientation x :=
1031   (PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed,
1032     PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit)
1033
1034 theorem PRCCharacterNoMixedNonunitOrbitOrientation_of_identity_branch_transport
1035   {x : RatioOrbit → RatioOrbit}
1036   (htransport : PRCCharacterNonunitIdentityBranchTransport x) :
1037   PRCCharacterNoMixedNonunitOrbitOrientation x := by
1038   intro p hp hpUnit r hr hrUnit hpId hrRec
1039   have hrId := htransport p hp hpUnit hpId r hr hrUnit
1040   have hself :
1041     RatioOrbit.crossEq (orbitDirection r hr)
1042     (RatioOrbit.recip (orbitDirection r hr)) :=
1043     RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hrId) hrRec
1044     exact orbitDirection_nonunit_not_crossEq_recip r hr hrUnit hself
1045
1046 theorem PRCCharacterOrbitProductNoMixedOrientation_of_identity_branch_transport
1047   {x : RatioOrbit → RatioOrbit}
1048   (htransport : PRCCharacterNonunitIdentityBranchTransport x) :
1049   PRCCharacterOrbitProductNoMixedOrientation x :=
1050   PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit
1051   (PRCCharacterNoMixedNonunitOrbitOrientation_of_identity_branch_transport
1052     htransport)
1053
1054 theorem PRCCharacterNonunitIdentityBranchTransport_of_local_no_mixed
1055   {x : RatioOrbit → RatioOrbit}
1056   (hlocal : PRCCharacterNonunitOrbitLocalOrientation x)
1057   (hnomix : PRCCharacterNoMixedNonunitOrbitOrientation x) :
1058   PRCCharacterNonunitIdentityBranchTransport x := by
1059   intro p hp hpUnit hpId r hr hrUnit
1060   rcases hlocal r hr hrUnit with hrId | hrRec
1061   · exact hrId
1062   · exact False.elim (hnomix p hp hpUnit r hr hrUnit hpId hrRec)
1063
1064 theorem PRCCharacterNonunitIdentityBranchTransport_of_coherent
1065   {x : RatioOrbit → RatioOrbit}
1066   (hcoh : PRCCharacterNonunitOrbitOrientationCoherent x) :
1067   PRCCharacterNonunitIdentityBranchTransport x := by
1068   intro p hp hpUnit hpId r hr hrUnit
1069   rcases hcoh with hallId | hallRec
1070   · exact hallId r hr hrUnit
1071   · have hpRec := hallRec p hp hpUnit
1072     have hself :
1073       RatioOrbit.crossEq (orbitDirection p hp)
1074       (RatioOrbit.recip (orbitDirection p hp)) :=
1075       RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
1076       exact False.elim
1077       (orbitDirection_nonunit_not_crossEq_recip p hp hpUnit hself)
1078
1079 theorem PRCCharacterNonunitIdentityBranchTransport_of_comparable_trace
1080   {x : RatioOrbit → RatioOrbit}
1081   (hcomp : PRCCharacterNonunitIdentityRespectsComparableTrace x) :
1082   PRCCharacterNonunitIdentityBranchTransport x := by
1083   intro p hp hpUnit hpId r hr hrUnit
1084   exact hcomp p hp hpUnit r hr hrUnit
1085   (orbitPositionTrace_comparable p r) hpId
1086
1087 theorem PRCCharacterNonunitIdentityRespectsComparableTrace_of_branch_transport
1088   {x : RatioOrbit → RatioOrbit}
1089   (htransport : PRCCharacterNonunitIdentityBranchTransport x) :
1090   PRCCharacterNonunitIdentityRespectsComparableTrace x := by
1091   intro p hp hpUnit r hr hrUnit _hcomp hpId
1092   exact htransport p hp hpUnit hpId r hr hrUnit
1093
1094 theorem PRCCharacterNonunitIdentityRespectsComparableTrace_iff_branch_transport
1095   {x : RatioOrbit → RatioOrbit} :
1096   PRCCharacterNonunitIdentityRespectsComparableTrace x ↔
1097   PRCCharacterNonunitIdentityBranchTransport x :=
1098   (PRCCharacterNonunitIdentityBranchTransport_of_comparable_trace,
1099     PRCCharacterNonunitIdentityRespectsComparableTrace_of_branch_transport)
1100

```

```

1101 /-- Two-branch version of the global branch-coupling law: any nonunit
1102 identity-oriented direction transports identity to every nonunit direction, and
1103 any reciprocal-oriented direction transports reciprocal to every nonunit
1104 direction. -/
1105 def PRCCharacterNonunitBranchAgreement
1106   (x : RatioOrbit → RatioOrbit) : Prop :=
1107   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
1108     ~ DistinctionNat.unit p →
1109     ∀ r : DistinctionNat, ∀ hr : r ≠ DistinctionNat.zero,
1110       ~ DistinctionNat.unit r →
1111       (PRCCharacterOrbitDirectionIdentity x p hp →
1112        PRCCharacterOrbitDirectionIdentity x r hr) ∧
1113       (PRCCharacterOrbitDirectionReciprocal x p hp →
1114        PRCCharacterOrbitDirectionReciprocal x r hr)
1115
1116 theorem PRCCharacterNonunitBranchAgreement_of_coherent
1117   (x : RatioOrbit → RatioOrbit)
1118   (hcoh : PRCCharacterNonunitOrbitOrientationCoherent x) :
1119   PRCCharacterNonunitBranchAgreement x := by
1120   intro p hp hpUnit r hr hrUnit
1121   rcases hcoh with hallId | hallRec
1122   · constructor
1123     · intro hpId
1124       exact hallId r hr hrUnit
1125     · intro hpRec
1126       have hpId := hallId p hp hpUnit
1127       have hself :
1128         RatioOrbit.crossEq (orbitDirection p hp)
1129         (RatioOrbit.recip (orbitDirection p hp)) :=
1130         RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
1131       exact False.elim
1132         (orbitDirection_nonunit_not_crossEq_recip p hp hpUnit hself)
1133   · constructor
1134     · intro hpId
1135       have hpRec := hallRec p hp hpUnit
1136       have hself :
1137         RatioOrbit.crossEq (orbitDirection p hp)
1138         (RatioOrbit.recip (orbitDirection p hp)) :=
1139         RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
1140       exact False.elim
1141         (orbitDirection_nonunit_not_crossEq_recip p hp hpUnit hself)
1142     · intro hpRec
1143       exact hallRec r hr hrUnit
1144
1145 theorem PRCCharacterNonunitOrbitOrientationCoherent_of_local_branch_agreement
1146   (x : RatioOrbit → RatioOrbit)
1147   (hlocal : PRCCharacterNonunitOrbitLocalOrientation x)
1148   (hagree : PRCCharacterNonunitBranchAgreement x) :
1149   PRCCharacterNonunitOrbitOrientationCoherent x := by
1150   by_cases hId :
1151     ∃ p : DistinctionNat, ∃ hp : p ≠ DistinctionNat.zero,
1152     ∃ hunit : ~ DistinctionNat.unit p,
1153     PRCCharacterOrbitDirectionIdentity x p hp
1154   · rcases hId with (p0, hp0, hunit0, hp0Id)
1155     exact Or.inl (by
1156       intro q hq hqUnit
1157       exact (hagree p0 hp0 hunit0 q hq hqUnit).1 hp0Id)
1158   · exact Or.inr (by
1159     intro q hq hqUnit
1160     rcases hlocal q hq hqUnit with hqId | hqRec
1161     · exact False.elim (hId (q, hq, hqUnit, hqId))
1162     · exact hqRec)
1163
1164 theorem PRCCharacterNonunitBranchAgreement_iff_coherent_of_local
1165   (x : RatioOrbit → RatioOrbit)
1166   (hlocal : PRCCharacterNonunitOrbitLocalOrientation x) :
1167   PRCCharacterNonunitBranchAgreement x ↔
1168   PRCCharacterNonunitOrbitOrientationCoherent x :=
1169   (PRCCharacterNonunitOrbitOrientationCoherent_of_local_branch_agreement
1170    hlocal,
1171    PRCCharacterNonunitBranchAgreement_of_coherent)
1172
1173 theorem PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
1174   (x : RatioOrbit → RatioOrbit)
1175   (hlocal : PRCCharacterNonunitOrbitLocalOrientation x)
1176   (hnomix : PRCCharacterNoMixedNonunitOrbitOrientation x) :
1177   PRCCharacterNonunitOrbitOrientationCoherent x := by
1178   by_cases hId :
1179     ∃ p : DistinctionNat, ∃ hp : p ≠ DistinctionNat.zero,
1180     ∃ hunit : ~ DistinctionNat.unit p,
1181     PRCCharacterOrbitDirectionIdentity x p hp
1182   · rcases hId with (p0, hp0, hunit0, hp0Id)
1183     exact Or.inl (by
1184       intro q hq hqUnit
1185       rcases hlocal q hq hqUnit with hqId | hqRec
1186       · exact hqId
1187       · exact False.elim (hnomix p0 hp0 hunit0 q hq hqUnit hp0Id hqRec))
1188   · exact Or.inr (by
1189     intro q hq hqUnit
1190     rcases hlocal q hq hqUnit with hqId | hqRec
1191     · exact False.elim (hId (q, hq, hqUnit, hqId))
1192     · exact hqRec)
1193
1194 theorem PRCCharacterNonunitOrbitOrientationCoherent_of_local_identity_branch_transport
1195   (x : RatioOrbit → RatioOrbit)
1196   (hlocal : PRCCharacterNonunitOrbitLocalOrientation x)
1197   (htransport : PRCCharacterNonunitIdentityBranchTransport x) :
1198   PRCCharacterNonunitOrbitOrientationCoherent x :=
1199   PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed hlocal
1200   (PRCCharacterNoMixedNonunitOrbitOrientation_of_identity_branch_transport
1201    htransport)

```

```

1202 theorem PRCCharacterOrbitProductNoMixedOrientation_of_nonunit_coherent
1203 {x : RatioOrbit → RatioOrbit}
1204 (hcoh : PRCCharacterNonunitOrbitOrientationCoherent x) :
1205 PRCCharacterOrbitProductNoMixedOrientation x := by
1206   intro a b p ha hb haUnit hbUnit _hp _hmul
1207   rcases hcoh with haId | hbId | haUnit | hbUnit | hmul
1208   · constructor
1209     · rintro (haId, hbRec)
1210       have hbId := haId b hb hbUnit
1211       have hself :
1212         RatioOrbit.crossEq (orbitDirection b hb)
1213         (RatioOrbit.recip (orbitDirection b hb)) :=
1214           RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hbId) hbRec
1215       exact orbitDirection_nonunit_not_crossEq_recip b hb hbUnit hself
1216     · rintro (haRec, _hbId)
1217       have haId := haId a ha haUnit
1218       have hself :
1219         RatioOrbit.crossEq (orbitDirection a ha)
1220         (RatioOrbit.recip (orbitDirection a ha)) :=
1221           RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm haId) haRec
1222       exact orbitDirection_nonunit_not_crossEq_recip a ha haUnit hself
1223   · constructor
1224     · rintro (haId, _hbRec)
1225       have haRec := haId a ha haUnit
1226       have hself :
1227         RatioOrbit.crossEq (orbitDirection a ha)
1228         (RatioOrbit.recip (orbitDirection a ha)) :=
1229           RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm haId) haRec
1230       exact orbitDirection_nonunit_not_crossEq_recip a ha haUnit hself
1231     · rintro (_haRec, hbId)
1232       have hbRec := hbId b hb hbUnit
1233       have hself :
1234         RatioOrbit.crossEq (orbitDirection b hb)
1235         (RatioOrbit.recip (orbitDirection b hb)) :=
1236           RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hbId) hbRec
1237       exact orbitDirection_nonunit_not_crossEq_recip b hb hbUnit hself
1238   · exact orbitDirection_nonunit_not_crossEq_recip b hb hbUnit hself
1239
1240 theorem PRCCharacterOrbitProductIdentityIdentity
1241 {x : RatioOrbit → RatioOrbit}
1242 (hx : PRCRatioCharacter x)
1243 (hcompat : PRCCharacterOrbitProductDisplayCompatible x)
1244 {a b p : DistinctionNat}
1245 (ha : a ≠ DistinctionNat.zero) (hb : b ≠ DistinctionNat.zero)
1246 (hp : p ≠ DistinctionNat.zero)
1247 (hmul : a * b = p)
1248 (haId : PRCCharacterOrbitDirectionIdentity x a ha)
1249 (hbId : PRCCharacterOrbitDirectionIdentity x b hb) :
1250 PRCCharacterOrbitDirectionIdentity x p hp := by
1251   unfold PRCCharacterOrbitDirectionIdentity at *
1252   exact RatioOrbit.crossEq_trans
1253     (hcompat a b p ha hb hp hmul)
1254     (RatioOrbit.crossEq_trans
1255       (hx.multiplicative (orbitDirection a ha) (orbitDirection b hb))
1256       (RatioOrbit.crossEq_trans
1257         (ratioOrbit_mul_congr haId hbId)
1258         (RatioOrbit.crossEq_symm
1259           (orbitDirection_mul_crossEq a b p ha hb hp hmul))))
1260
1261 theorem PRCCharacterOrbitProductReciprocalReciprocal
1262 {x : RatioOrbit → RatioOrbit}
1263 (hx : PRCRatioCharacter x)
1264 (hcompat : PRCCharacterOrbitProductDisplayCompatible x)
1265 {a b p : DistinctionNat}
1266 (ha : a ≠ DistinctionNat.zero) (hb : b ≠ DistinctionNat.zero)
1267 (hp : p ≠ DistinctionNat.zero)
1268 (hmul : a * b = p)
1269 (haRec : PRCCharacterOrbitDirectionReciprocal x a ha)
1270 (hbRec : PRCCharacterOrbitDirectionReciprocal x b hb) :
1271 PRCCharacterOrbitDirectionReciprocal x p hp := by
1272   unfold PRCCharacterOrbitDirectionReciprocal at *
1273   exact RatioOrbit.crossEq_trans
1274     (hcompat a b p ha hb hp hmul)
1275     (RatioOrbit.crossEq_trans
1276       (hx.multiplicative (orbitDirection a ha) (orbitDirection b hb))
1277       (RatioOrbit.crossEq_trans
1278         (ratioOrbit_mul_congr haRec hbRec)
1279         (RatioOrbit.crossEq_trans
1280           (ratioOrbit_mul_recip_recip_crossEq_recip_mul
1281             (orbitDirection a ha) (orbitDirection b hb))
1282           (ratioOrbit_recip_congr
1283             (RatioOrbit.crossEq_symm
1284               (orbitDirection_mul_crossEq a b p ha hb hp hmul))))))
1285
1286 theorem PRCCharacterOrbitProductLocalOrientationPropagates_of_display_compatible_nomix
1287 {x : RatioOrbit → RatioOrbit}
1288 (hx : PRCRatioCharacter x)
1289 (hcompat : PRCCharacterOrbitProductDisplayCompatible x)
1290 (hnomix : PRCCharacterOrbitProductNoMixedOrientation x) :
1291 PRCCharacterOrbitProductLocalOrientationPropagates x := by
1292   intro a b p ha hb haUnit hbUnit hp hpUnit hmul haLocal hbLocal
1293   rcases haLocal with haId | haRec
1294   · rcases hbLocal with hbId | hbRec
1295     · exact Or.inl
1296       (PRCCharacterOrbitProductIdentityIdentity hx hcompat
1297         ha hb hp hmul haId hbId)
1298   · exact False.elim
1299     ((hnomix a b p ha hb haUnit hbUnit hp hpUnit hmul).1
1300      (haId, hbRec))
1301   · rcases hbLocal with hbId | hbRec
1302     · exact False.elim

```

```

1303 ((hnomix a b p ha hb haUnit hbUnit hpUnit hmul).2
1304 (haRec, hbId))
1305 · exact Or.inr
1306 (PRCCharacterOrbitProductReciprocalReciprocal hx hcompat
1307 ha hb hp hmul haRec hbRec)
1308
1309 theorem PRCCharacterNonunitOrbitLocalOrientation_of_prime_and_product_local
1310 {x : RatioOrbit → RatioOrbit}
1311 (hprimeLocal : PRCCharacterPrimeLocalOrientation x)
1312 (hprod : PRCCharacterOrbitProductLocalOrientationPropagates x) :
1313 PRCCharacterNonunitOrbitLocalOrientation x := by
1314   intro p hp hunit
1315   let P : Nat → Prop := fun n =>
1316     ∀ q : DistinctionNat, q.toNat = n →
1317     (hq : q ≠ DistinctionNat.zero) →
1318     ¬ DistinctionNat.unit q →
1319     PRCCharacterOrbitDirectionIdentity x q hq v
1320     PRCCharacterOrbitDirectionReciprocal x q hq
1321   have hP : ∀ n : Nat, (∀ m : Nat, m < n → P m) → P n := by
1322     intro n ih q hqNat hq0 hqUnit
1323     by_cases hqPrime : DistinctionNat.primeOrbit q
1324     · simp [primeDirection] using hprimeLocal q hqPrime
1325     · have hfacs : DistinctionNat.nontrivialFactorization q := by
1326       by_contra hnotFac
1327       exact hqPrime (hq0, hqUnit, hnotFac)
1328     rcases hfacs with (a, b, ha0, hb0, haUnit, hbUnit, hmul)
1329     have hmulNat : a.toNat * b.toNat = n := by
1330       have hnats := congrArg DistinctionNat.toNat hmul
1331       rw [DistinctionNat.toNat_mul, hqNat] at hnats
1332       exact hnats
1333     have haNat0 : a.toNat ≠ 0 := by
1334       intro hz
1335       apply ha0
1336       apply DistinctionNat.toNat_inj
1337       rw [hz, DistinctionNat.toNat_zero]
1338     have hbNat0 : b.toNat ≠ 0 := by
1339       intro hz
1340       apply hb0
1341       apply DistinctionNat.toNat_inj
1342       rw [hz, DistinctionNat.toNat_zero]
1343     have haNat1 : a.toNat ≠ 1 := by
1344       intro hne
1345       exact haUnit ((DistinctionNat.unit_iff_toNat_eq_one a).mpr hne)
1346     have hbNat1 : b.toNat ≠ 1 := by
1347       intro hne
1348       exact hbUnit ((DistinctionNat.unit_iff_toNat_eq_one b).mpr hne)
1349     have haPos : 0 < a.toNat := by omega
1350     have hbPos : 0 < b.toNat := by omega
1351     have haGtOne : 1 < a.toNat := by omega
1352     have hbGtOne : 1 < b.toNat := by omega
1353     have haLt : a.toNat < n := by
1354       calc
1355         a.toNat = a.toNat * 1 := by rw [Nat.mul_one]
1356         < a.toNat * b.toNat :=
1357           Nat.mul_lt_mul_of_pos_left hbGtOne haPos
1358         = n := hmulNat
1359     have hbLt : b.toNat < n := by
1360       calc
1361         b.toNat = 1 * b.toNat := by rw [Nat.one_mul]
1362         < a.toNat * b.toNat :=
1363           Nat.mul_lt_mul_of_pos_right haGtOne hbPos
1364         = n := hmulNat
1365     have haLocal :
1366       PRCCharacterOrbitDirectionIdentity x a ha0 v
1367       PRCCharacterOrbitDirectionReciprocal x a ha0 :=
1368       ih a.toNat haLt a rfl ha0 haUnit
1369     have hbLocal :
1370       PRCCharacterOrbitDirectionIdentity x b hb0 v
1371       PRCCharacterOrbitDirectionReciprocal x b hb0 :=
1372       ih b.toNat hbLt b rfl hb0 hbUnit
1373     exact hprod a b q ha0 hb0 haUnit hbUnit hq0 hqUnit hmul
1374     haLocal hbLocal
1375   have hmain : P p.toNat :=
1376     Nat.strong_induction_on (p := P) p.toNat hP
1377   exact hmain p rfl hp hunit
1378
1379 /-- Adjacent nonunit orbit steps cannot mix identity on one side with reciprocal
1380 orientation on the other. This is the prime-floor version of the no-mixed
1381 orientation law. -/
1382 def PRCCharacterPrimeFloorNoAdjacentMixedOrientation
1383 {x : RatioOrbit → RatioOrbit} : Prop :=
1384   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
1385   ¬ DistinctionNat.unit p →
1386   (¬ (PRCCharacterOrbitDirectionIdentity x p hp ∧
1387     PRCCharacterOrbitDirectionReciprocal x
1388     (DistinctionNat.succ p) (orbit_succ_ne_zero p))) ∧
1389   (¬ (PRCCharacterOrbitDirectionReciprocal x p hp ∧
1390     PRCCharacterOrbitDirectionIdentity x
1391     (DistinctionNat.succ p) (orbit_succ_ne_zero p)))
1392
1393 theorem PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_nonunit_coherent
1394 {x : RatioOrbit → RatioOrbit}
1395 (hcoh : PRCCharacterNonunitOrbitOrientationCoherent x) :
1396 PRCCharacterPrimeFloorNoAdjacentMixedOrientation x := by
1397   intro p hp hunit
1398   have hsuccUnit :
1399     ¬ DistinctionNat.unit (DistinctionNat.succ p) :=
1400     orbit_succ_not_unit_of_nonzero_not_unit p hp hunit
1401   rcases hcoh with hllId | hllRec
1402   · constructor
1403   · rintro (_hpId, hsuccRec)

```

```

1404   have hsuccId :=
1405     hallId (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit
1406   have hself :
1407     RatioOrbit.crossEq
1408       (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
1409       (RatioOrbit.recip
1410         (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))) :=
1411     RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hsuccId) hsuccRec
1412   exact orbitDirection_nonunit_not_crossEq_recip
1413     (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit hself
1414   · rintro (hpRec, _hsuccId)
1415     have hpId := hallId p hp hunit
1416     have hself :
1417       RatioOrbit.crossEq (orbitDirection p hp)
1418       (RatioOrbit.recip (orbitDirection p hp)) :=
1419       RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
1420     exact orbitDirection_nonunit_not_crossEq_recip p hp hunit hself
1421   · constructor
1422   · rintro (hpId, _hsuccRec)
1423     have hpRec := hallRec p hp hunit
1424     have hself :
1425       RatioOrbit.crossEq (orbitDirection p hp)
1426       (RatioOrbit.recip (orbitDirection p hp)) :=
1427       RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
1428     exact orbitDirection_nonunit_not_crossEq_recip p hp hunit hself
1429   · rintro (_hpRec, hsuccId)
1430     have hsuccRec :=
1431       hallRec (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit
1432     have hself :
1433       RatioOrbit.crossEq
1434         (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
1435         (RatioOrbit.recip
1436           (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))) :=
1437       RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hsuccId) hsuccRec
1438     exact orbitDirection_nonunit_not_crossEq_recip
1439       (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit hself
1440
1441   /-- Forward one-step identity transport above the unit floor. The unit orbit is
1442   self-reciprocal, so forcing transport out of `1` would wrongly exclude the
1443   globally reciprocal character branch. -/
1444   def PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep
1445     (x : RatioOrbit → RatioOrbit) : Prop :=
1446     ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
1447     ~ DistinctionNat.unit p →
1448     PRCCharacterOrbitDirectionIdentity x p hp →
1449     PRCCharacterOrbitDirectionIdentity x
1450       (DistinctionNat.succ p) (orbit_succ_ne_zero p)
1451
1452   /-- Backward one-step identity transport above the unit floor. -/
1453   def PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep
1454     (x : RatioOrbit → RatioOrbit) : Prop :=
1455     ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
1456     ~ DistinctionNat.unit p →
1457     PRCCharacterOrbitDirectionIdentity x
1458       (DistinctionNat.succ p) (orbit_succ_ne_zero p) →
1459     PRCCharacterOrbitDirectionIdentity x p hp
1460
1461   /-- The corrected successor-transport rule: identity orientation transports
1462   along one 6-successor step only once the path is above the self-reciprocal unit
1463   orbit. This is the exact layer needed for prime-to-prime trace coherence. -/
1464   def PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport
1465     (x : RatioOrbit → RatioOrbit) : Prop :=
1466     PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep x ∧
1467     PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep x
1468
1469   theorem PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep_of_local_adjacent_nomix
1470     {x : RatioOrbit → RatioOrbit}
1471     (hlocal : PRCCharacterNonunitOrbitLocalOrientation x)
1472     (hnomix : PRCCharacterPrimeFloorNoAdjacentMixedOrientation x) :
1473     PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep x := by
1474     intro p hp hunit hpId
1475     have hsuccUnit :
1476       ~ DistinctionNat.unit (DistinctionNat.succ p) :=
1477       orbit_succ_not_unit_of_nonzero_not_unit p hp hunit
1478     rcases hlocal (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit
1479     with hsuccId | hsuccRec
1480     · exact hsuccId
1481     · exact False.elim ((hnomix p hp hunit).1 (hpId, hsuccRec))
1482
1483   theorem PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep_of_local_adjacent_nomix
1484     {x : RatioOrbit → RatioOrbit}
1485     (hlocal : PRCCharacterNonunitOrbitLocalOrientation x)
1486     (hnomix : PRCCharacterPrimeFloorNoAdjacentMixedOrientation x) :
1487     PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep x := by
1488     intro p hp hunit hsuccId
1489     rcases hlocal p hp hunit with hpId | hpRec
1490     · exact hpId
1491     · exact False.elim ((hnomix p hp hunit).2 (hpRec, hsuccId))
1492
1493   theorem PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
1494     {x : RatioOrbit → RatioOrbit}
1495     (hlocal : PRCCharacterNonunitOrbitLocalOrientation x)
1496     (hnomix : PRCCharacterPrimeFloorNoAdjacentMixedOrientation x) :
1497     PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport x :=
1498     (PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep_of_local_adjacent_nomix
1499       hlocal hnomix,
1500     PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep_of_local_adjacent_nomix
1501       hlocal hnomix)
1502
1503   theorem PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_successor_transport
1504     {x : RatioOrbit → RatioOrbit}

```



```

1505 (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport x) :
1506   PRCCharacterPrimeFloorNoAdjacentMixedOrientation x := by
1507   intro p hp hunt
1508   have hsuccUnit :
1509     ~ DistinctionNat.unit (DistinctionNat.succ p) :=
1510     orbit_succ_not_unit_of_nonzero_not_unit p hp hunt
1511   constructor
1512   · rintro (hpId, hsuccRec)
1513     have hsuccId :
1514       PRCCharacterOrbitDirectionIdentity x
1515         (DistinctionNat.succ p) (orbit_succ_ne_zero p) :=
1516       hstep.1 p hp hunt hpId
1517     have hself :
1518       RatioOrbit.crossEq
1519         (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
1520         (RatioOrbit.recip
1521           (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))) :=
1522       RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hsuccId) hsuccRec
1523     exact orbitDirection_nonunit_not_crossEq_recip
1524     (DistinctionNat.succ p) (orbit_succ_ne_zero p) hsuccUnit hself
1525   · rintro (hpRec, hsuccId)
1526     have hpId : PRCCharacterOrbitDirectionIdentity x p hp :=
1527     hstep.2 p hp hunt hsuccId
1528     have hself :
1529       RatioOrbit.crossEq (orbitDirection p hp)
1530       (RatioOrbit.recip (orbitDirection p hp)) :=
1531       RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hpId) hpRec
1532     exact orbitDirection_nonunit_not_crossEq_recip p hp hunt hself
1533
1534 theorem PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_iff_local_adjacent_nomix
1535   {x : RatioOrbit → RatioOrbit}
1536   (hlocal : PRCCharacterNonunitOrbitLocalOrientation x) :
1537   PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport x ↔
1538   PRCCharacterPrimeFloorNoAdjacentMixedOrientation x :=
1539   (PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_successor_transport,
1540    PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
1541     hlocal)
1542
1543 theorem PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_nonunit_identity_comparable_trace
1544   {x : RatioOrbit → RatioOrbit}
1545   (hcomp : PRCCharacterNonunitIdentityRespectsComparableTrace x) :
1546   PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport x := by
1547   constructor
1548   · intro p hp hunt hpId
1549     have hsuccUnit :
1550       ~ DistinctionNat.unit (DistinctionNat.succ p) :=
1551       orbit_succ_not_unit_of_nonzero_not_unit p hp hunt
1552     exact hcomp p hp hunt (DistinctionNat.succ p)
1553     (orbit_succ_ne_zero p) hsuccUnit
1554     (orbitPositionTrace_comparable p (DistinctionNat.succ p)) hpId
1555   · intro p hp hunt hsuccId
1556     have hsuccUnit :
1557       ~ DistinctionNat.unit (DistinctionNat.succ p) :=
1558       orbit_succ_not_unit_of_nonzero_not_unit p hp hunt
1559     exact hcomp (DistinctionNat.succ p) (orbit_succ_ne_zero p)
1560     hsuccUnit p hp hunt
1561     (orbitPositionTrace_comparable (DistinctionNat.succ p) p) hsuccId
1562
1563 theorem PRCCharacterOrbitIdentity_of_le_of_prime_floor_successor_transport
1564   {x : RatioOrbit → RatioOrbit}
1565   (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport x) :
1566   ∀ {p r : DistinctionNat}, (hp : p ≠ DistinctionNat.zero) →
1567     ~ DistinctionNat.unit p →
1568     (hr : r ≠ DistinctionNat.zero) →
1569     p.toNat ≤ r.toNat →
1570     PRCCharacterOrbitDirectionIdentity x p hp →
1571     PRCCharacterOrbitDirectionIdentity x r hr := by
1572   intro p r
1573   revert p
1574   induction r with
1575   | zero =>
1576     intro p hp _hpu hr _hle _hpId
1577     exact False.elim (hr rfl)
1578   | succ n ih =>
1579     intro p hp _hpu _hr _hle _hpId
1580     by_cases hEq : p = DistinctionNat.succ n
1581     · subst p
1582       simpa [PRCCharacterOrbitDirectionIdentity, orbitDirection] using hpId
1583     · have hle_n : p.toNat ≤ n.toNat := by
1584       have hnotNat : p.toNat ≠ Nat.succ n.toNat := by
1585         intro hnat
1586         apply hEq
1587         apply DistinctionNat.toNat_inj
1588         simpa [DistinctionNat.toNat_succ] using hnat
1589       rw [DistinctionNat.toNat_succ] at hle
1590       omega
1591     have hpNat0 : p.toNat ≠ 0 := by
1592     intro hz
1593     apply hp
1594     apply DistinctionNat.toNat_inj
1595     rw [hz, DistinctionNat.toNat_zero]
1596     have hpNat1 : p.toNat ≠ 1 := by
1597     intro hone
1598     exact hpu ((DistinctionNat.unit_iff_toNat_eq_one p).mpr hone)
1599     have hn0 : n ≠ DistinctionNat.zero := by
1600     intro hn
1601     have hnNat : n.toNat = 0 := by rw [hn, DistinctionNat.toNat_zero]
1602     omega
1603     have hnunit : ~ DistinctionNat.unit n := by
1604     intro hunt
1605     have hnNat1 : n.toNat = 1 :=

```

```

1606 (DistinctionNat.unit_iff_toNat_eq_one n).mp huni
1607 omega
1608 have hnId :
1609   PRCCharacterOrbitDirectionIdentity χ n hn0 :=
1610   ih hp hpu hn0 hle_n hpId
1611   exact hstep.1 n hn0 hnunit hnId
1612
1613 theorem PRCCharacterOrbitIdentity_of_ge_of_prime_floor_successor_transport
1614   {χ : RatioOrbit → RatioOrbit}
1615   (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
1616   ∀ {p r : DistinctionNat}, (hp : p ≠ DistinctionNat.zero) →
1617   ¬ DistinctionNat.unit p →
1618   (hr : r ≠ DistinctionNat.zero) →
1619   ¬ DistinctionNat.unit r →
1620   r.toNat ≤ p.toNat →
1621   PRCCharacterOrbitDirectionIdentity χ p hp →
1622   PRCCharacterOrbitDirectionIdentity χ r hr := by
1623   intro p r
1624   revert r
1625   induction p with
1626   | zero =>
1627     intro r hp _hpu _hr _hru _hge _hpId
1628     exact False.elim (hp rfl)
1629   | succ n ih =>
1630     intro r _hp _hpu hr hru hge hpId
1631     by_cases hEq : r = DistinctionNat.succ n
1632     · subst r
1633       simpa [PRCCharacterOrbitDirectionIdentity, orbitDirection] using hpId
1634     · have hge_n : r.toNat ≤ n.toNat := by
1635       have hnotNat : r.toNat ≠ Nat.succ n.toNat := by
1636         intro hnat
1637         apply hEq
1638         apply DistinctionNat.toNat_inj
1639       simpa [DistinctionNat.toNat_succ] using hnat
1640       rw [DistinctionNat.toNat_succ] at hge
1641       omega
1642       have hrNat0 : r.toNat ≠ 0 := by
1643         intro hz
1644         apply hr
1645         apply DistinctionNat.toNat_inj
1646         rw [hz, DistinctionNat.toNat_zero]
1647       have hrNat1 : r.toNat ≠ 1 := by
1648         intro hone
1649         exact hru ((DistinctionNat.unit_iff_toNat_eq_one r).mpr hone)
1650       have hn0 : n ≠ DistinctionNat.zero := by
1651         intro hn
1652         have hnNat : n.toNat = 0 := by rw [hn, DistinctionNat.toNat_zero]
1653         omega
1654       have hnunit : ¬ DistinctionNat.unit n := by
1655         intro huni
1656         have hnNat1 : n.toNat = 1 :=
1657           (DistinctionNat.unit_iff_toNat_eq_one n).mp huni
1658         omega
1659       have hsuccId :
1660         PRCCharacterOrbitDirectionIdentity χ
1661         (DistinctionNat.succ n) (orbit_succ_ne_zero n) := by
1662         simpa [PRCCharacterOrbitDirectionIdentity, orbitDirection] using hpId
1663       have hnId :
1664         PRCCharacterOrbitDirectionIdentity χ n hn0 :=
1665         hstep.2 n hn0 hnunit hsuccId
1666       exact ih hn0 hnunit hr hru hge_n hnId
1667
1668 theorem PRCCharacterPrimeIdentityRespectsComparableTrace_of_prime_floor_successor_transport
1669   {χ : RatioOrbit → RatioOrbit}
1670   (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
1671   PRCCharacterPrimeIdentityRespectsComparableTrace χ := by
1672   intro p hp r hr _hcomp hpId
1673   have hpOrbitId :
1674     PRCCharacterOrbitDirectionIdentity χ p hp.1 := by
1675     exact hpId
1676   rcases Nat.le_total p.toNat r.toNat with hle | hge
1677   · exact PRCCharacterOrbitIdentity_of_le_of_prime_floor_successor_transport
1678     hstep hp.1 hp.2.1 hr.1 hle hpOrbitId
1679   · exact PRCCharacterOrbitIdentity_of_ge_of_prime_floor_successor_transport
1680     hstep hp.1 hp.2.1 hr.1 hr.2.1 hge hpOrbitId
1681
1682 theorem PRCCharacterNonunitIdentityRespectsComparableTrace_of_prime_floor_successor_transport
1683   {χ : RatioOrbit → RatioOrbit}
1684   (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
1685   PRCCharacterNonunitIdentityRespectsComparableTrace χ := by
1686   intro p hp hpUnit r hr hrUnit _hcomp hpId
1687   rcases Nat.le_total p.toNat r.toNat with hle | hge
1688   · exact PRCCharacterOrbitIdentity_of_le_of_prime_floor_successor_transport
1689     hstep hp hpUnit hr hle hpId
1690   · exact PRCCharacterOrbitIdentity_of_ge_of_prime_floor_successor_transport
1691     hstep hp hpUnit hr hrUnit hge hpId
1692
1693 theorem PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_prime_floor_successor_transport
1694   {χ : RatioOrbit → RatioOrbit}
1695   (hlocal : PRCCharacterNonunitOrbitLocalOrientation χ)
1696   (hstep : PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport χ) :
1697   PRCCharacterNonunitOrbitOrientationCoherent χ := by
1698   by_cases hId :
1699   | p : DistinctionNat, ∃ hp : p ≠ DistinctionNat.zero,
1700     ∃ huni : ¬ DistinctionNat.unit p,
1701     PRCCharacterOrbitDirectionIdentity χ p hp
1702   · rcases hId with (p0, hp0, huni0, hp0Id)
1703     exact Or.inl (by
1704       intro q hq hqUnit
1705       rcases Nat.le_total p0.toNat q.toNat with hle | hge
1706       · exact PRCCharacterOrbitIdentity_of_le_of_prime_floor_successor_transport

```

```

1707   hstep hp0 hunit0 hq hle hp0Id
1708   · exact PRCCharacterOrbitIdentity_of_ge_of_prime_floor_successor_transport
1709   hstep hp0 hunit0 hq hqUnit hge hp0Id)
1710   · exact Or.inr (by
1711     intro q hq hqUnit
1712     rcases hlocal q hq hqUnit with hqId | hqRec
1713     · exact False.elim (hId (q, hq, hqUnit, hqId))
1714     · exact hqRec)
1715
1716 theorem PRCCharacterOrbitIdentityRespectsSuccessorStep_of_transport
1717   {x : RatioOrbit → RatioOrbit}
1718   (htransport : PRCCharacterOrbitIdentitySuccessorTransport x) :
1719   PRCCharacterOrbitIdentityRespectsSuccessorStep x := by
1720   intro p hp
1721   exact (htransport.1 p hp, htransport.2 p hp)
1722
1723 theorem PRCCharacterOrbitIdentity_one_of_identity
1724   {x : RatioOrbit → RatioOrbit}
1725   (hstep : PRCCharacterOrbitIdentityRespectsSuccessorStep x)
1726   (p : DistinctionNat) (hp : p ≠ DistinctionNat.zero)
1727   (hpId : PRCCharacterOrbitDirectionIdentity x p hp) :
1728   PRCCharacterOrbitDirectionIdentity x
1729   DistinctionNat.one DistinctionNat.one_ne_zero := by
1730   induction p with
1731   | zero =>
1732     exact False.elim (hp rfl)
1733   | succ n ih =>
1734     cases n with
1735     | zero =>
1736       exact hpId
1737     | succ m =>
1738       have hn : DistinctionNat.succ m ≠ DistinctionNat.zero :=
1739         orbit_succ_ne_zero m
1740       have hnId :
1741         PRCCharacterOrbitDirectionIdentity x
1742         (DistinctionNat.succ m) hn :=
1743         (hstep (DistinctionNat.succ m) hn).2 hpId
1744       exact ih hn hnId
1745
1746 theorem PRCCharacterOrbitIdentity_of_one
1747   {x : RatioOrbit → RatioOrbit}
1748   (hstep : PRCCharacterOrbitIdentityRespectsSuccessorStep x)
1749   {r : DistinctionNat} (hr : r ≠ DistinctionNat.zero)
1750   (honeId : PRCCharacterOrbitDirectionIdentity x
1751     DistinctionNat.one DistinctionNat.one_ne_zero) :
1752   PRCCharacterOrbitDirectionIdentity x r hr := by
1753   induction r with
1754   | zero =>
1755     exact False.elim (hr rfl)
1756   | succ n ih =>
1757     cases n with
1758     | zero =>
1759       exact honeId
1760     | succ m =>
1761       have hn : DistinctionNat.succ m ≠ DistinctionNat.zero :=
1762         orbit_succ_ne_zero m
1763       have hnId :
1764         PRCCharacterOrbitDirectionIdentity x
1765         (DistinctionNat.succ m) hn :=
1766         ih hn
1767       exact (hstep (DistinctionNat.succ m) hn).1 hnId
1768
1769 theorem PRCCharacterPrimeIdentityRespectsComparableTrace_of_successor_step
1770   {x : RatioOrbit → RatioOrbit}
1771   (hstep : PRCCharacterOrbitIdentityRespectsSuccessorStep x) :
1772   PRCCharacterPrimeIdentityRespectsComparableTrace x := by
1773   intro p hp r hr _hcomp hpId
1774   have hpOrbitId :
1775     PRCCharacterOrbitDirectionIdentity x p hp.1 := by
1776     exact hpId
1777   have honeId :
1778     PRCCharacterOrbitDirectionIdentity x
1779     DistinctionNat.one DistinctionNat.one_ne_zero :=
1780     PRCCharacterOrbitIdentity_one_of_identity hstep hp.1 hpOrbitId
1781   exact PRCCharacterOrbitIdentity_of_one hstep hr.1 honeId
1782
1783 theorem PRCCharacterPrimeIdentityRespectsCommonTraceExtension_of_comparable_trace
1784   {x : RatioOrbit → RatioOrbit}
1785   (hcomp : PRCCharacterPrimeIdentityRespectsComparableTrace x) :
1786   PRCCharacterPrimeIdentityRespectsCommonTraceExtension x := by
1787   intro p hp r hr _T _hpT _hrT hpId
1788   exact hcomp p hp r hr (orbitPositionTrace_comparable p r) hpId
1789
1790 theorem PRCCharacterPrimeIdentityRespectsTraceConnected_of_common_trace_extension
1791   {x : RatioOrbit → RatioOrbit}
1792   (hcommon : PRCCharacterPrimeIdentityRespectsCommonTraceExtension x) :
1793   PRCCharacterPrimeIdentityRespectsTraceConnected x := by
1794   intro p hp r hr hconn hpId
1795   rcases hconn with (T, hpT, hrT)
1796   exact hcommon p hp r hr T hpT hrT hpId
1797
1798 /-- Local orientation target: prime cost calibration must at least orient each
1799 prime axis as identity or reciprocal. -/
1800 def PRCPrimeCalibrationForcesLocalPrimeOrientationTarget : Prop :=
1801   ∀ x : RatioOrbit → RatioOrbit,
1802     PRCRatioCharacter x →
1803     PRCCharacterPrimeDirectionCalibrated x →
1804     PRCCharacterPrimeLocalOrientation x
1805
1806 theorem PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved :
1807   PRCPrimeCalibrationForcesLocalPrimeOrientationTarget := by

```

```

1808 intro x hx hprime p hp
1809 have hq : (primeDirection p hp).toRat ≠ 0 :=
1810   primeDirection_toRat_ne_zero p hp
1811 have hxq : (x (primeDirection p hp)).toRat ≠ 0 :=
1812   hx.nonzero_preserving hq
1813 have hcal :
1814   RatioOrbit.crossEq
1815     (onRatioOrbit (x (primeDirection p hp)))
1816     (onRatioOrbit (primeDirection p hp)) := by
1817   simpa [costFromCharacter] using hprime p hp
1818 exact jcost_eq_forces_same_or_reciprocal hxq hq hcal
1819
1820 /-- No-mixing target: prime cost calibration must forbid independent mixed
1821 identity/reciprocal choices on different prime axes. -/
1822 def PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget : Prop :=
1823   ∀ x : RatioOrbit → RatioOrbit,
1824   PRCRatioCharacter x →
1825   PRCCharacterPrimeDirectionCalibrated x →
1826   PRCCharacterNoMixedPrimeOrientation x
1827
1828 /-- Exact remaining trace-coherence target: prime calibration must make identity
1829 orientation propagate across prime axes. -/
1830 def PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget : Prop :=
1831   ∀ x : RatioOrbit → RatioOrbit,
1832   PRCRatioCharacter x →
1833   PRCCharacterPrimeDirectionCalibrated x →
1834   PRCCharacterPrimeIdentityTraceCoherent x
1835
1836 /-- Smaller trace-transport target: prime calibration should make identity
1837 orientation invariant along native prime-axis trace connections. The structural
1838 connectivity of the prime-axis trace graph is already proved above. -/
1839 def PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget : Prop :=
1840   ∀ x : RatioOrbit → RatioOrbit,
1841   PRCRatioCharacter x →
1842   PRCCharacterPrimeDirectionCalibrated x →
1843   PRCCharacterPrimeIdentityRespectsTraceConnected x
1844
1845 /-- Sharper form of the trace-transport target: prime calibration must force
1846 identity orientation to respect an explicitly witnessed common finite 6-trace
1847 extension. -/
1848 def PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget : Prop :=
1849   ∀ x : RatioOrbit → RatioOrbit,
1850   PRCRatioCharacter x →
1851   PRCCharacterPrimeDirectionCalibrated x →
1852   PRCCharacterPrimeIdentityRespectsCommonTraceExtension x
1853
1854 /-- Sharper trace-order target: prime calibration should force identity
1855 orientation to respect comparability of finite 6-orbit traces. -/
1856 def PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget : Prop :=
1857   ∀ x : RatioOrbit → RatioOrbit,
1858   PRCRatioCharacter x →
1859   PRCCharacterPrimeDirectionCalibrated x →
1860   PRCCharacterPrimeIdentityRespectsComparableTrace x
1861
1862 /-- Sharper one-step target: prime calibration should force identity orientation
1863 to be invariant under one successor step on every nonzero orbit direction. -/
1864 def PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget : Prop :=
1865   ∀ x : RatioOrbit → RatioOrbit,
1866   PRCRatioCharacter x →
1867   PRCCharacterPrimeDirectionCalibrated x →
1868   PRCCharacterOrbitIdentityRespectsSuccessorStep x
1869
1870 /-- Sharper directional successor target: prime calibration should force both
1871 one-step directions separately. This exposes the exact additive-trace
1872 compatibility missing from the purely multiplicative ratio-character laws. -/
1873 def PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget : Prop :=
1874   ∀ x : RatioOrbit → RatioOrbit,
1875   PRCRatioCharacter x →
1876   PRCCharacterPrimeDirectionCalibrated x →
1877   PRCCharacterOrbitIdentitySuccessorTransport x
1878
1879 /-- Sharper additive successor target: prime calibration should force the ratio
1880 character to respect the additive successor operation on nonzero orbit
1881 directions. -/
1882 def PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget : Prop :=
1883   ∀ x : RatioOrbit → RatioOrbit,
1884   PRCRatioCharacter x →
1885   PRCCharacterPrimeDirectionCalibrated x →
1886   PRCCharacterOrbitSuccessorAdditiveCompatible x
1887
1888 /-- Corrected successor target after the reciprocal-character check: prime
1889 calibration should force successor transport above the self-reciprocal unit
1890 floor, not additive transport out of the unit orbit itself. -/
1891 def PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget : Prop :=
1892   ∀ x : RatioOrbit → RatioOrbit,
1893   PRCRatioCharacter x →
1894   PRCCharacterPrimeDirectionCalibrated x →
1895   PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport x
1896
1897 /-- First component of the prime-floor successor blocker: prime calibration
1898 should orient every nonunit orbit direction, not only prime axes. -/
1899 def PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget : Prop :=
1900   ∀ x : RatioOrbit → RatioOrbit,
1901   PRCRatioCharacter x →
1902   PRCCharacterPrimeDirectionCalibrated x →
1903   PRCCharacterNonunitOrbitLocalOrientation x
1904
1905 /-- Sharper source of nonunit local orientation: prime calibration should force
1906 the product-factor propagation step that carries prime-axis orientation through
1907 composite orbit positions. -/
1908 def PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget : Prop :=

```

```

1909   ∀ x : RatioOrbit → RatioOrbit,
1910   PRCRatioCharacter x →
1911   PRCCharacterPrimeDirectionCalibrated x →
1912   PRCCharacterOrbitProductLocalOrientationPropagates x
1913
1914   /-- Product-display compatibility target: prime calibration should force the
1915   character to respect the native equality between product orbit directions and
1916   ratio products of factor directions. -/
1917   def PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget : Prop :=
1918     ∀ x : RatioOrbit → RatioOrbit,
1919     PRCRatioCharacter x →
1920     PRCCharacterPrimeDirectionCalibrated x →
1921     PRCCharacterOrbitProductDisplayCompatible x
1922
1923   /-- Sharper source of product-display compatibility: prime calibration should
1924   force the raw character to respect ratio cross-equivalence. Without this,
1925   `x : RatioOrbit → RatioOrbit` is not yet a quotient-native character. -/
1926   def PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget : Prop :=
1927     ∀ x : RatioOrbit → RatioOrbit,
1928     PRCRatioCharacter x →
1929     PRCCharacterPrimeDirectionCalibrated x →
1930     PRCCharacterRespectsCrossEq x
1931
1932   theorem PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_normalizeRatio_canonical
1933     (hcanon : PRCNormalizeRatioCanonicalTarget) :
1934     PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget := by
1935     intro x hx _hprime
1936     exact PRCCharacterRespectsCrossEq_of_normalizeRatio_canonical hx hcanon
1937
1938   theorem PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_reduced_signCanonical_unique
1939     (huniqueness : PRCReducedSignCanonicalRatioUniqueTarget) :
1940     PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget :=
1941     PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_normalizeRatio_canonical
1942     (PRCNormalizeRatioCanonicalTarget_of_reduced_signCanonical_unique huniqueness)
1943
1944   theorem PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_proved :
1945     PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget :=
1946     PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_reduced_signCanonical_unique
1947     PRCReducedSignCanonicalRatioUniqueTarget_proved
1948
1949   /-- Product no-mixing target: prime calibration should rule out mixed
1950   identity/reciprocal factor orientations under native multiplication. -/
1951   def PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget : Prop :=
1952     ∀ x : RatioOrbit → RatioOrbit,
1953     PRCRatioCharacter x →
1954     PRCCharacterPrimeDirectionCalibrated x →
1955     PRCCharacterOrbitProductNoMixedOrientation x
1956
1957   /-- Stronger replacement for product no-mixing: prime calibration should force a
1958   single coherent orientation across all nonunit orbit directions. Once this is
1959   available, mixed product factors are impossible by nonunit non-self-reciprocity. -/
1960   def PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget : Prop :=
1961     ∀ x : RatioOrbit → RatioOrbit,
1962     PRCRatioCharacter x →
1963     PRCCharacterPrimeDirectionCalibrated x →
1964     PRCCharacterNonunitOrbitOrientationCoherent x
1965
1966   /-- Branch-coupling target: prime calibration should prevent any identity-oriented
1967   nonunit direction from coexisting with any reciprocal-oriented nonunit direction.
1968   Together with local nonunit orientation this is exactly global nonunit
1969   orientation coherence. -/
1970   def PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget : Prop :=
1971     ∀ x : RatioOrbit → RatioOrbit,
1972     PRCRatioCharacter x →
1973     PRCCharacterPrimeDirectionCalibrated x →
1974     PRCCharacterNoMixedNonunitOrbitOrientation x
1975
1976   /-- Positive transport form of the same branch-coupling blocker: if prime
1977   calibration allows one nonunit direction to remain identity-oriented, that
1978   identity branch must transport to every nonunit direction. -/
1979   def PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget : Prop :=
1980     ∀ x : RatioOrbit → RatioOrbit,
1981     PRCRatioCharacter x →
1982     PRCCharacterPrimeDirectionCalibrated x →
1983     PRCCharacterNonunitIdentityBranchTransport x
1984
1985   /-- Trace-order sharpening of nonunit identity-branch transport: prime
1986   calibration should force identity orientation to respect comparability of finite
1987   6-orbit traces on nonunit directions. -/
1988   def PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget : Prop :=
1989     ∀ x : RatioOrbit → RatioOrbit,
1990     PRCRatioCharacter x →
1991     PRCCharacterPrimeDirectionCalibrated x →
1992     PRCCharacterNonunitIdentityRespectsComparableTrace x
1993
1994   /-- Two-branch agreement target: prime calibration should force a nonunit branch
1995   choice at one direction to agree with every other nonunit direction, for both
1996   identity and reciprocal branches. -/
1997   def PRCPrimeCalibrationForcesNonunitBranchAgreementTarget : Prop :=
1998     ∀ x : RatioOrbit → RatioOrbit,
1999     PRCRatioCharacter x →
2000     PRCCharacterPrimeDirectionCalibrated x →
2001     PRCCharacterNonunitBranchAgreement x
2002
2003   /-- Local orientation plus two-branch agreement is the positive normal form of
2004   the global nonunit branch-coupling blocker. -/
2005   def PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget :
2006     Prop :=
2007     PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
2008     PRCPrimeCalibrationForcesNonunitBranchAgreementTarget
2009

```

```

2010 /-- Sharpened source of global nonunit coherence: first prove every nonunit orbit
2011 direction has a local branch, then prove the cross-nonunit no-mixing law. -/
2012 def PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget : Prop :=
2013   PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
2014   PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
2015
2016 /-- Product-layer sharpening of global nonunit coherence: local nonunit
2017 orientation is already available, so the remaining branch-coupling obligation can
2018 be carried by the product no-mixing law. -/
2019 def PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget : Prop :=
2020   PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
2021   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
2022
2023 theorem PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_coherent
2024   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2025   PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget := by
2026     intro x hx hprime
2027     exact PRCCharacterNoMixedNonunitOrbitOrientation_of_coherent
2028       (hcoh x hx hprime)
2029
2030 theorem PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed
2031   (hprod : PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget) :
2032   PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget := by
2033     intro x hx hprime
2034     exact PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
2035       (hprod x hx hprime)
2036
2037 theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_no_mixed_nonunit
2038   (hnomix : PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget) :
2039   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget := by
2040     intro x hx hprime
2041     exact PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit
2042       (hnomix x hx hprime)
2043
2044 theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_no_mixed_nonunit :
2045   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
2046   PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget :=
2047   (PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed,
2048    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_no_mixed_nonunit)
2049
2050 theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport
2051   (htransport : PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget) :
2052   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget := by
2053     intro x hx hprime
2054     exact PRCCharacterOrbitProductNoMixedOrientation_of_identity_branch_transport
2055       (htransport x hx hprime)
2056
2057 theorem PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace
2058   (hcomp : PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget) :
2059   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget := by
2060     intro x hx hprime
2061     exact PRCCharacterNonunitIdentityBranchTransport_of_comparable_trace
2062       (hcomp x hx hprime)
2063
2064 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_local_product_no_mixed
2065   (hsharp : PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget) :
2066   PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget :=
2067   (hsharp.1,
2068    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed
2069      hsharp.2)
2070
2071 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_product_no_mixed
2072   (hsharp : PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget) :
2073   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
2074     intro x hx hprime
2075     exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
2076       (hsharp.1 x hx hprime)
2077     (PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
2078      (hsharp.2 x hx hprime))
2079
2080 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_coherent
2081   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2082   PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget :=
2083   ((by
2084     intro x hx hprime
2085     exact PRCCharacterNonunitOrbitLocalOrientation_of_coherent
2086       (hcoh x hx hprime)),
2087    PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_coherent hcoh)
2088
2089 theorem PRCPrimeCalibrationForcesNonunitBranchAgreementTarget_of_coherent
2090   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2091   PRCPrimeCalibrationForcesNonunitBranchAgreementTarget := by
2092     intro x hx hprime
2093     exact PRCCharacterNonunitBranchAgreement_of_coherent (hcoh x hx hprime)
2094
2095 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget_of_coherent
2096   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2097   PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget :=
2098   ((by
2099     intro x hx hprime
2100     exact PRCCharacterNonunitOrbitLocalOrientation_of_coherent
2101       (hcoh x hx hprime)),
2102    PRCPrimeCalibrationForcesNonunitBranchAgreementTarget_of_coherent hcoh)
2103
2104 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_branch_agreement
2105   (hsharp :
2106     PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget) :
2107   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
2108     intro x hx hprime
2109     exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_branch_agreement
2110       (hsharp.1 x hx hprime) (hsharp.2 x hx hprime)

```

```

2111 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_local_branch_agreement :
2112   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
2113     PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget :=
2114   (PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalBranchAgreementTarget_of_coherent,
2115     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_branch_agreement)
2116
2117 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_no_mixed
2118   (hsharp : PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget) :
2119   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
2120     intro x hy hprime
2121     exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
2122       (hsharp.1 x hy hprime) (hsharp.2 x hy hprime)
2123
2124 theorem PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_coherent
2125   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2126   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget := by
2127     intro x hy hprime
2128     exact PRCCharacterNonunitIdentityBranchTransport_of_coherent
2129       (hcoh x hy hprime)
2130
2131 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_local_no_mixed :
2132   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
2133     PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget :=
2134   (PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_coherent,
2135     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_no_mixed)
2136
2137 /-- Sharpened source of nonunit orientation coherence: local nonunit orientation
2138 plus prime-floor successor transport force every nonunit orbit direction onto
2139 one coherent identity/reciprocal branch. -/
2140 def PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget : Prop :=
2141   PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
2142   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
2143
2144 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_and_prime_floor_successor_transport
2145   (hsharp : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget) :
2146   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
2147     intro x hy hprime
2148     exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_prime_floor_successor_transport
2149       (hsharp.1 x hy hprime) (hsharp.2 x hy hprime)
2150
2151 theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_nonunit_coherent
2152   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2153   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget := by
2154     intro x hy hprime
2155     exact PRCCharacterOrbitProductNoMixedOrientation_of_nonunit_coherent
2156       (hcoh x hy hprime)
2157
2158 theorem PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_nonunit_coherent
2159   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2160   PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget := by
2161     intro x hy hprime
2162     exact PRCCharacterNonunitOrbitLocalOrientation_of_coherent
2163       (hcoh x hy hprime)
2164
2165 theorem PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_nonunit_coherent
2166   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2167   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget := by
2168     intro x hy hprime
2169     exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
2170       (PRCCharacterNonunitOrbitLocalOrientation_of_coherent (hcoh x hy hprime))
2171     (PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_nonunit_coherent
2172       (hcoh x hy hprime))
2173
2174 theorem PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_prime_floor_successor_transport
2175   (hstep : PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :
2176   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget := by
2177     intro x hy hprime
2178     exact PRCCharacterNonunitIdentityRespectsComparableTrace_of_prime_floor_successor_transport
2179       (hstep x hy hprime)
2180
2181 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget_of_nonunit_coherent
2182   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2183   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget :=
2184   (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_nonunit_coherent hcoh,
2185     PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_nonunit_coherent hcoh)
2186
2187 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_sharpened :
2188   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
2189     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget :=
2190   (PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget_of_nonunit_coherent,
2191     PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_and_prime_floor_successor_transport)
2192
2193 theorem PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_identity_comparable_trace
2194   (hcomp : PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget) :
2195   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget := by
2196     intro x hy hprime
2197     exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_nonunit_identity_comparable_trace
2198       (hcomp x hy hprime)
2199
2200 theorem PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_iff_prime_floor_successor_transport :
2201   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget ↔
2202     PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget :=
2203   (PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_identity_comparable_trace,
2204     PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_prime_floor_successor_transport)
2205
2206 /-- Pass-45 sharpening of product-local orientation: same-orientation products
2207 are algebraic and product-display compatibility is proved through canonical
2208 normalization. The remaining product commitment is nonunit orientation
2209 coherence, which implies product no-mixing. -/
2210 def PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget : Prop :=

```

```

2212 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
2213
2214 theorem PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_of_crossEq_respect
2215   (hrespect : PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget) :
2216   PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget := by
2217     intro x hx hprime
2218     exact PRCCharacterOrbitProductDisplayCompatible_of_crossEq_respect
2219     (hrespect x hx hprime)
2220
2221 theorem PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_proved :
2222   PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget :=
2223   PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_of_crossEq_respect
2224   PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_proved
2225
2226 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed
2227   (hprod : PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget) :
2228   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
2229     intro x hx hprime
2230     have hprodLocal : PRCCharacterOrbitProductLocalOrientationPropagates x :=
2231       PRCCharacterOrbitProductLocalOrientationPropagates_of_display_compatible_nomix
2232       hx (PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_proved
2233         x hx hprime) (hprod x hx hprime)
2234     exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
2235     (PRCCharacterNonunitOrbitLocalOrientation_of_prime_and_product_local
2236      (PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved x hx hprime)
2237      hprodLocal)
2238     (PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
2239      (hprod x hx hprime))
2240
2241 theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_nonunit_coherent :
2242   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
2243   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget :=
2244   (PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed,
2245    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_nonunit_coherent)
2246
2247 theorem PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed
2248   (hprod : PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget) :
2249   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget :=
2250   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_coherent
2251   (PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed
2252    hprod)
2253
2254 theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_branch_transport :
2255   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
2256   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget :=
2257   (PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed,
2258    PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport)
2259
2260 theorem PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_branch_transport
2261   (htransport : PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget) :
2262   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget := by
2263     intro x hx hprime
2264     exact PRCCharacterNonunitIdentityRespectsComparableTrace_of_branch_transport
2265     (htransport x hx hprime)
2266
2267 theorem PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_product_no_mixed
2268   (hprod : PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget) :
2269   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget :=
2270   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_branch_transport
2271   (PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed
2272    hprod)
2273
2274 theorem PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_iff_comparable_trace :
2275   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget ↔
2276   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget :=
2277   (PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_branch_transport,
2278    PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace)
2279
2280 theorem PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_comparable_trace :
2281   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
2282   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget :=
2283   (PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_product_no_mixed,
2284    fun hcomp =>
2285      PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport
2286      (PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace
2287       hcomp))
2288
2289 /-- Second component of the prime-floor successor blocker: adjacent nonunit
2290 orbit directions cannot carry opposite identity/reciprocal orientations. -/
2291 def PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget : Prop :=
2292   ∀ x : RatioOrbit → RatioOrbit,
2293   PRCRatioCharacter x →
2294   PRCCharacterPrimeFloorDirectionCalibrated x →
2295   PRCCharacterPrimeFloorNoAdjacentMixedOrientation x
2296
2297 theorem PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_nonunit_coherent
2298   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2299   PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget := by
2300     intro x hx hprime
2301     exact PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_nonunit_coherent
2302     (hcoh x hx hprime)
2303
2304 theorem PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_successor_transport
2305   (hstep : PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :
2306   PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget := by
2307     intro x hx hprime
2308     exact PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_successor_transport
2309     (hstep x hx hprime)
2310
2311 /-- The exact local version of the corrected successor blocker: local nonunit
2312 orientation plus adjacent no-mixing is equivalent to local nonunit orientation

```



```

2313 plus prime-floor successor transport. -/
2314 def PRCPrimeFloorSuccessorTransportLocalAdjacentTarget : Prop :=
2315   PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
2316   PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
2317
2318 /-- Pass-39 refinement of the prime-floor successor target. It separates local
2319 nonunit orientation from adjacent no-mixing instead of bundling both facts under
2320 successor transport. -/
2321 def PRCPrimeFloorSuccessorTransportSharpenedTarget : Prop :=
2322   PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget ∧
2323   PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
2324
2325 theorem PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget_of_additive_compat
2326   (hadd : PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget) :
2327   PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget := by
2328     intro x hx hprime
2329     exact PRCCharacterOrbitIdentitySuccessorTransport_of_additive_compat
2330       (hadd x hx hprime)
2331
2332 theorem PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_nomix
2333   (hsharp : PRCPrimeFloorSuccessorTransportSharpenedTarget) :
2334   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget := by
2335     intro x hx hprime
2336     exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
2337       (PRCCharacterNonunitOrbitLocalOrientation_of_coherent
2338        (hsharp.1 x hx hprime))
2339     (hsharp.2 x hx hprime)
2340
2341 theorem PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_target
2342   (hsharp : PRCPrimeFloorSuccessorTransportLocalAdjacentTarget) :
2343   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget := by
2344     intro x hx hprime
2345     exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
2346       (hsharp.1 x hx hprime) (hsharp.2 x hx hprime)
2347
2348 theorem PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_local_successor_transport
2349   (hsharp :
2350     PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
2351     PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :
2352   PRCPrimeFloorSuccessorTransportLocalAdjacentTarget :=
2353     (hsharp.1,
2354      PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_successor_transport
2355       hsharp.2)
2356
2357 theorem PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_local_successor_transport :
2358   PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
2359     (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
2360      PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :=
2361     ((fun hsharp =>
2362       (hsharp.1,
2363        PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_target
2364         hsharp)),
2365      PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_local_successor_transport)
2366
2367 theorem PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_nonunit_coherent
2368   (hcoh : PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget) :
2369   PRCPrimeFloorSuccessorTransportLocalAdjacentTarget :=
2370     (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_nonunit_coherent hcoh,
2371      PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_nonunit_coherent
2372       hcoh)
2373
2374 theorem PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_adjacent
2375   (hsharp : PRCPrimeFloorSuccessorTransportLocalAdjacentTarget) :
2376   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget := by
2377     exact PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_and_prime_floor_successor_transport
2378       (hsharp.1,
2379        PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_target
2380         hsharp)
2381
2382 theorem PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_nonunit_coherent :
2383   PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
2384     (PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget :=
2385      (PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_adjacent,
2386       PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_nonunit_coherent))
2387
2388 theorem PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_product_local_orientation
2389   (hprod : PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget) :
2390   PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget := by
2391     intro x hx hprime
2392     exact PRCCharacterNonunitOrbitLocalOrientation_of_prime_and_product_local
2393       (PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved x hx hprime)
2394     (hprod x hx hprime)
2395
2396 theorem PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget_of_display_compatible_nomix
2397   (hsharp : PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget) :
2398   PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget := by
2399     intro x hx hprime
2400     exact PRCCharacterOrbitProductLocalOrientationPropagates_of_display_compatible_nomix
2401       hx (PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_proved
2402          x hx hprime)
2403     (PRCCharacterOrbitProductNoMixedOrientation_of_nonunit_coherent
2404      (hsharp x hx hprime))
2405
2406 theorem PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_prime_floor_successor_transport
2407   (hstep : PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) :
2408   PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget := by
2409     intro x hy hprime
2410     exact PRCCharacterPrimeIdentityRespectsComparableTrace_of_prime_floor_successor_transport
2411       (hstep x hx hprime)
2412
2413 theorem PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget_of_transport

```

```

2414 (htransport : PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget) :
2415   PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget := by
2416   intro x hx hprime
2417   exact PRCCharacterOrbitIdentityRespectsSuccessorStep_of_transport
2418   (htransport x hx hprime)
2419
2420 theorem PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_successor_step
2421   (hstep : PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget) :
2422   PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget := by
2423   intro x hx hprime
2424   exact PRCCharacterPrimeIdentityRespectsComparableTrace_of_successor_step
2425   (hstep x hx hprime)
2426
2427 theorem PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget_of_comparable_trace
2428   (hcomp : PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget) :
2429   PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget := by
2430   intro x hx hprime
2431   exact PRCCharacterPrimeIdentityRespectsCommonTraceExtension_of_comparable_trace
2432   (hcomp x hx hprime)
2433
2434 theorem PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget_of_common_trace_extension
2435   (hcommon : PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget) :
2436   PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget := by
2437   intro x hx hprime
2438   exact PRCCharacterPrimeIdentityRespectsTraceConnected_of_common_trace_extension
2439   (hcommon x hx hprime)
2440
2441 theorem PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget_of_trace_transport
2442   (htransport : PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget) :
2443   PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget := by
2444   intro x hx hprime p hp r hr hpId
2445   exact htransport x hx hprime p hp r hr
2446   (PRCPrimeAxisTraceConnected_proved p hp r hr) hpId
2447
2448 theorem PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget_of_trace_coherence
2449   (htrace : PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget) :
2450   PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget := by
2451   intro x hx hprime p hp r hr hpId hrRec
2452   have hrId := htrace x hx hprime p hp r hr hpId
2453   have hself :
2454     RatioOrbit.crossEq
2455       (primeDirection r hr)
2456       (RatioOrbit.recip (primeDirection r hr)) :=
2457     RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm hrId) hrRec
2458   exact primeDirection_not_crossEq_recip r hr hself
2459
2460 /-- Sharper orientation blocker A: prime cost calibration must choose one
2461 coherent orientation across all native prime axes. This is the place where
2462 mixed independent prime inversions must be ruled out. -/
2463 def PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget : Prop :=
2464   ∀ x : RatioOrbit → RatioOrbit,
2465   PRCRatioCharacter x →
2466   PRCCharacterPrimeDirectionCalibrated x →
2467   PRCCharacterPrimeOrientationCoherent x
2468
2469 theorem PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget_of_local_and_nomixed
2470   (hlocal : PRCPrimeCalibrationForcesLocalPrimeOrientationTarget)
2471   (hnomix : PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget) :
2472   PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget := by
2473   intro x hx hprime
2474   have hloc := hlocal x hx hprime
2475   have hno := hnomix x hx hprime
2476   by_cases hId :
2477     ∃ p : DistinctionNat, ∃ hp : DistinctionNat.primeOrbit p,
2478     RatioOrbit.crossEq (x (primeDirection p hp)) (primeDirection p hp)
2479   · rcases hId with (p0, hp0, hid0)
2480     exact Or.inl (by
2481       intro p hp
2482       rcases hloc p hp with hid | hrec
2483       · exact hid
2484       · exact False.elim (hno p0 hp0 p hp hid0 hrec))
2485   · exact Or.inr (by
2486     intro p hp
2487     rcases hloc p hp with hid | hrec
2488     · exact False.elim (hId (p, hp, hid))
2489     · exact hrec)
2490
2491 /-- Sharper orientation blocker B: once prime orientation is coherent, the
2492 multiplicative character law and native rational factorization must propagate
2493 that orientation to every ratio direction. -/
2494 def PRCCoherentPrimeOrientationPropagatesToGlobalTarget : Prop :=
2495   ∀ x : RatioOrbit → RatioOrbit,
2496   PRCRatioCharacter x →
2497   PRCCharacterPrimeOrientationCoherent x →
2498   PRCCharacterGlobalCostOrientation x
2499
2500 theorem PRCPrimeCalibrationForcesGlobalOrientationTarget_of_prime_orientation_targets
2501   (hcoherent : PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget)
2502   (hprop : PRCCoherentPrimeOrientationPropagatesToGlobalTarget) :
2503   PRCPrimeCalibrationForcesGlobalOrientationTarget := by
2504   intro x hx hprime
2505   exact hprop x hx (hcoherent x hx hprime)
2506
2507 theorem PRCPrimeCalibrationPropagationTarget_of_global_orientation
2508   (horient : PRCPrimeCalibrationForcesGlobalOrientationTarget) :
2509   PRCPrimeCalibrationPropagationTarget := by
2510   intro x hx hprime q
2511   rcases horient x hx hprime q with hsame | hinv
2512   · exact onRatioOrbit_congr hsame
2513   · exact RatioOrbit.crossEq_trans
2514     (onRatioOrbit_congr hinv)

```

```

2515 (RatioOrbit.crossEq_symm (reciprocal_symmetric q))
2516
2517 /-- Pass-27 refinement of prime propagation. -/
2518 def PRCPrimeCalibrationPropagationSharpenedTarget : Prop :=
2519   PRCPrimeFloorSuccessorTransportSharpenedTarget ∧
2520   PRCCoherentPrimeOrientationPropagatesToGlobalTarget
2521
2522 theorem PRCPrimeCalibrationPropagationTarget_of_sharpened_orientation
2523   (hsharp : PRCPrimeCalibrationPropagationSharpenedTarget) :
2524   PRCPrimeCalibrationPropagationTarget :=
2525   PRCPrimeCalibrationPropagationTarget_of_global_orientation
2526   (PRCPrimeCalibrationForcesGlobalOrientationTarget_of_prime_orientation_targets
2527     (PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget_of_local_and_nomixed
2528       (PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved
2529         (PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget_of_trace_coherence
2530           (PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget_of_trace_transport
2531             (PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget_of_common_trace_extension
2532               (PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget_of_comparable_trace
2533                 (PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_prime_floor_successor_transport
2534                   (PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_nomix
2535                     hsharp.1)))))))
2536   hsharp.2)
2537
2538 theorem PRCNativeCostCharacterRigidityTarget_of_prime_targets
2539   (htwo : PRCTwoCalibrationForcesPrimeCalibrationTarget)
2540   (hprop : PRCPrimeCalibrationPropagationTarget) :
2541   PRCNativeCostCharacterRigidityTarget := by
2542     intro x hx htwoCal q
2543     exact hprop x hx (htwo x hx htwoCal) q
2544
2545 theorem PRCNativeCostUniquenessTarget_of_prime_character_targets
2546   (hfactor : PRCNativeCostCharacterFactorizationTarget)
2547   (htwo : PRCTwoCalibrationForcesPrimeCalibrationTarget)
2548   (hprop : PRCPrimeCalibrationPropagationTarget) :
2549   PRCNativeCostUniquenessTarget := by
2550     intro F hF q
2551     rcases hfactor F hF with (x, hx, hFh)
2552     have hcal :
2553       RatioOrbit.crossEq (costFromCharacter x two) (onRatioOrbit two) :=
2554       RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm (hFh two)) hF.two_calibrated
2555     have hrigid :=
2556       PRCNativeCostCharacterRigidityTarget_of_prime_targets htwo hprop
2557     exact RatioOrbit.crossEq_trans (hFh q) (hrigid x hx hcal q)
2558
2559 /-- The identity map is a ratio character. This sanity-check anchors the
2560 character interface to the canonical cost. -/
2561 theorem identity_ratio_character :
2562   PRCRatioCharacter (fun q : RatioOrbit => q) where
2563     unit := RatioOrbit.crossEq_refl RatioOrbit.one
2564     multiplicative := by
2565       intro x y
2566       exact RatioOrbit.crossEq_refl (RatioOrbit.mul x y)
2567     reciprocal := by
2568       intro x
2569       exact RatioOrbit.crossEq_refl (RatioOrbit.recip x)
2570     normalized_invariant := by
2571       intro q
2572       exact DistinctionNat.normalizeRatio_crossEq q
2573     nonzero_preserving := by
2574       intro q hq
2575       exact hq
2576
2577 theorem identity_character_rigid :
2578   ∀ q : RatioOrbit,
2579   RatioOrbit.crossEq
2580     (costFromCharacter (fun q : RatioOrbit => q) q)
2581     (onRatioOrbit q) := by
2582     intro q
2583     exact RatioOrbit.crossEq_refl (onRatioOrbit q)
2584
2585 theorem identity_character_global_orientation :
2586   PRCCharacterGlobalCostOrientation (fun q : RatioOrbit => q) := by
2587     intro q
2588     exact Or.inl (RatioOrbit.crossEq_refl q)
2589
2590 theorem identity_character_prime_orientation_coherent :
2591   PRCCharacterPrimeOrientationCoherent (fun q : RatioOrbit => q) := by
2592     exact Or.inl (by
2593       intro p hp
2594       exact RatioOrbit.crossEq_refl (primeDirection p hp))
2595
2596 /-- The global reciprocal map is also a ratio character. This is the first
2597 explicit witness that the multiplicative character laws alone do not choose the
2598 identity orientation. -/
2599 theorem reciprocal_ratio_character :
2600   PRCRatioCharacter (fun q : RatioOrbit => RatioOrbit.recip q) where
2601     unit := by
2602       rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
2603         RatioOrbit.one_toRat]
2604     norm_num
2605     multiplicative := by
2606       intro x y
2607       rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
2608         RatioOrbit.mul_toRat, RatioOrbit.mul_toRat, RatioOrbit.recip_toRat,
2609         RatioOrbit.recip_toRat]
2610       by_cases hx : x.toRat = 0
2611       · simp [hx]
2612       · by_cases hy : y.toRat = 0
2613         · simp [hy]
2614         · field_simp [hx, hy]
2615     reciprocal := by

```

```

2616   intro x
2617   exact RatioOrbit.crossEq_refl (RatioOrbit.recip (RatioOrbit.recip x))
2618   normalized_invariant := by
2619   intro q
2620   rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
2621       RatioOrbit.recip_toRat]
2622   have hnorm :
2623     q.toRat = (DistinctionNat.normalizeRatio q).toRat :=
2624     (RatioOrbit.crossEq_iff_toRat_eq q (DistinctionNat.normalizeRatio q)).mp
2625     (DistinctionNat.normalizeRatio_crossEq q)
2626   exact congrArg Inv.inv hnorm
2627   nonzero_preserving := by
2628   intro q hq
2629   rw [RatioOrbit.recip_toRat]
2630   exact inv_ne_zero hq
2631
2632   theorem reciprocal_character_prime_calibrated :
2633     PRCCharacterPrimeDirectionCalibrated
2634     (fun q : RatioOrbit => RatioOrbit.recip q) := by
2635     intro p hp
2636     simp [costFromCharacter] using
2637       RatioOrbit.crossEq_symm (reciprocal_symmetric (primeDirection p hp))
2638
2639   theorem reciprocal_character_global_orientation :
2640     PRCCharacterGlobalCostOrientation
2641     (fun q : RatioOrbit => RatioOrbit.recip q) := by
2642     intro q
2643     exact Or.inr (RatioOrbit.crossEq_refl (RatioOrbit.recip q))
2644
2645   theorem reciprocal_character_prime_orientation_coherent :
2646     PRCCharacterPrimeOrientationCoherent
2647     (fun q : RatioOrbit => RatioOrbit.recip q) := by
2648     exact Or.inr (by
2649       intro p hp
2650       exact RatioOrbit.crossEq_refl (RatioOrbit.recip (primeDirection p hp)))
2651
2652   theorem reciprocal_character_not_successor_additive_compatible :
2653     ~ PRCCharacterOrbitSuccessorAdditiveCompatible
2654     (fun q : RatioOrbit => RatioOrbit.recip q) := by
2655     intro hcompat
2656     have hstep := hcompat DistinctionNat.one DistinctionNat.one_ne_zero
2657     rw [RatioOrbit.crossEq_iff_toRat_eq, RatioOrbit.recip_toRat,
2658         RatioOrbit.add_toRat, RatioOrbit.recip_toRat, RatioOrbit.one_toRat,
2659         orbitDirection_toRat, orbitDirection_toRat, DistinctionNat.toNat_succ,
2660         DistinctionNat.one_toNat] at hstep
2661     norm_num at hstep
2662
2663   theorem PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget_refuted :
2664     ~ PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget := by
2665     intro htarget
2666     exact reciprocal_character_not_successor_additive_compatible
2667       (htarget (fun q : RatioOrbit => RatioOrbit.recip q))
2668     reciprocal_ratio_character reciprocal_character_prime_calibrated)
2669
2670   /-- The sharpened replacement for the opaque native uniqueness blocker. -/
2671   def PRCNativeCostUniquenessSharpenedTarget : Prop :=
2672     PRCNativeCostCharacterFactorizationTarget ∧
2673     PRCNativeCostCharacterRigidityTarget
2674
2675   /-- Pass-26 refinement of the rigidity target. -/
2676   def PRCNativeCostCharacterRigiditySharpenedTarget : Prop :=
2677     PRCTwoCalibrationForcesPrimeCalibrationTarget ∧
2678     PRCPrimeCalibrationPropagationTarget
2679
2680   theorem PRCNativeCostUniquenessTarget_of_character_targets
2681     (hfactor : PRCNativeCostCharacterFactorizationTarget)
2682     (hrigid : PRCNativeCostCharacterRigidityTarget) :
2683     PRCNativeCostUniquenessTarget := by
2684     intro F hF q
2685     rcases hfactor F hF with (χ, hχ, hFχ)
2686     have hcal :
2687       RatioOrbit.crossEq (costFromCharacter χ two) (onRatioOrbit two) :=
2688       RatioOrbit.crossEq_trans (RatioOrbit.crossEq_symm (hFχ two)) hF.two_calibrated
2689     exact RatioOrbit.crossEq_trans (hFχ q) (hrigid χ hχ hcal q)
2690
2691   /-- Pass-25 certificate: native cost uniqueness is not closed, but the missing
2692   mathematics is now split into exact Lean targets. -/
2693   structure PRCNativeCostUniquenessBlockerCertificate : Prop where
2694     factorization_target :
2695       PRCNativeCostCharacterFactorizationTarget =
2696       PRCNativeCostCharacterFactorizationTarget
2697     rigidity_target :
2698       PRCNativeCostCharacterRigidityTarget =
2699       PRCNativeCostCharacterRigidityTarget
2700     two_to_prime_target :
2701       PRCTwoCalibrationForcesPrimeCalibrationTarget =
2702       PRCTwoCalibrationForcesPrimeCalibrationTarget
2703     prime_propagation_target :
2704       PRCPrimeCalibrationPropagationTarget =
2705       PRCPrimeCalibrationPropagationTarget
2706     global_orientation_target :
2707       PRCPrimeCalibrationForcesGlobalOrientationTarget =
2708       PRCPrimeCalibrationForcesGlobalOrientationTarget
2709     coherent_prime_orientation :
2710       PRCCharacterPrimeOrientationCoherent =
2711       PRCCharacterPrimeOrientationCoherent
2712     local_prime_orientation :
2713       PRCCharacterPrimeLocalOrientation =
2714       PRCCharacterPrimeLocalOrientation
2715     no_mixed_prime_orientation :
2716       PRCCharacterNoMixedPrimeOrientation =

```

```

2717 PRCCharacterNoMixedPrimeOrientation
2718 prime_identity_trace_coherence :
2719 PRCCharacterPrimeIdentityTraceCoherent =
2720 PRCCharacterPrimeIdentityTraceCoherent
2721 prime_axis_trace_connected :
2722 PRCPrimeAxisTraceConnected =
2723 PRCPrimeAxisTraceConnected
2724 prime_axis_trace_connected_proved :
2725   ∀ p : DistinctionNat, ∀ hp : DistinctionNat.primeOrbit p,
2726   ∀ r : DistinctionNat, ∀ hr : DistinctionNat.primeOrbit r,
2727   PRCPrimeAxisTraceConnected p hp r hr
2728 orbit_trace_extends_of_toNat_le :
2729   ∀ p r : DistinctionNat,
2730   p.toNat ≤ r.toNat →
2731   Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r)
2732 orbit_trace_comparable :
2733   ∀ p r : DistinctionNat,
2734   Trace.Extends (orbitPositionTrace p) (orbitPositionTrace r) v
2735   Trace.Extends (orbitPositionTrace r) (orbitPositionTrace p)
2736 orbit_direction_toNat :
2737   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
2738   (orbitDirection p hp).toNat = (p.toNat : ℚ)
2739 orbit_direction_nonunit_not_crossEq_recip :
2740   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
2741   ¬ DistinctionNat.unit p →
2742   ¬ RatioOrbit.crossEq
2743   (orbitDirection p hp)
2744   (RatioOrbit.recip (orbitDirection p hp))
2745 orbit_direction_succ_add_one :
2746   ∀ p : DistinctionNat, ∀ hp : p ≠ DistinctionNat.zero,
2747   RatioOrbit.crossEq
2748   (orbitDirection (DistinctionNat.succ p) (orbit_succ_ne_zero p))
2749   (RatioOrbit.add (orbitDirection p hp) RatioOrbit.one)
2750 ratio_add_right_one_cancel :
2751   ∀ a b : RatioOrbit,
2752   RatioOrbit.crossEq
2753   (RatioOrbit.add a RatioOrbit.one)
2754   (RatioOrbit.add b RatioOrbit.one) →
2755   RatioOrbit.crossEq a b
2756 prime_identity_respects_trace_connected :
2757 PRCCharacterPrimeIdentityRespectsTraceConnected =
2758 PRCCharacterPrimeIdentityRespectsTraceConnected
2759 prime_identity_respects_common_trace_extension :
2760 PRCCharacterPrimeIdentityRespectsCommonTraceExtension =
2761 PRCCharacterPrimeIdentityRespectsCommonTraceExtension
2762 prime_identity_respects_comparable_trace :
2763 PRCCharacterPrimeIdentityRespectsComparableTrace =
2764 PRCCharacterPrimeIdentityRespectsComparableTrace
2765 orbit_direction_identity :
2766 PRCCharacterOrbitDirectionIdentity =
2767 PRCCharacterOrbitDirectionIdentity
2768 orbit_direction_reciprocal :
2769 PRCCharacterOrbitDirectionReciprocal =
2770 PRCCharacterOrbitDirectionReciprocal
2771 orbit_succ_not_unit :
2772   ∀ p : DistinctionNat, p ≠ DistinctionNat.zero →
2773   ¬ DistinctionNat.unit p →
2774   ¬ DistinctionNat.unit (DistinctionNat.succ p)
2775 orbit_identity_respects_successor_step :
2776 PRCCharacterOrbitIdentityRespectsSuccessorStep =
2777 PRCCharacterOrbitIdentityRespectsSuccessorStep
2778 orbit_identity_extends_successor_step :
2779 PRCCharacterOrbitIdentityExtendsSuccessorStep =
2780 PRCCharacterOrbitIdentityExtendsSuccessorStep
2781 orbit_identity_contracts_successor_step :
2782 PRCCharacterOrbitIdentityContractsSuccessorStep =
2783 PRCCharacterOrbitIdentityContractsSuccessorStep
2784 orbit_identity_successor_transport :
2785 PRCCharacterOrbitIdentitySuccessorTransport =
2786 PRCCharacterOrbitIdentitySuccessorTransport
2787 orbit_successor_additive_compat :
2788 PRCCharacterOrbitSuccessorAdditiveCompatible =
2789 PRCCharacterOrbitSuccessorAdditiveCompatible
2790 nonunit_orbit_local_orientation :
2791 PRCCharacterNonunitOrbitLocalOrientation =
2792 PRCCharacterNonunitOrbitLocalOrientation
2793 orbit_product_local_orientation :
2794 PRCCharacterOrbitProductLocalOrientationPropagates =
2795 PRCCharacterOrbitProductLocalOrientationPropagates
2796 ratio_mul_congr :
2797   ∀ a₁ a₂ b₁ b₂ : RatioOrbit,
2798   RatioOrbit.crossEq a₁ a₂ →
2799   RatioOrbit.crossEq b₁ b₂ →
2800   RatioOrbit.crossEq (RatioOrbit.mul a₁ b₁) (RatioOrbit.mul a₂ b₂)
2801 ratio_recip_congr :
2802   ∀ a b : RatioOrbit,
2803   RatioOrbit.crossEq a b →
2804   RatioOrbit.crossEq (RatioOrbit.recip a) (RatioOrbit.recip b)
2805 ratio_mul_recip_recip :
2806   ∀ a b : RatioOrbit,
2807   RatioOrbit.crossEq
2808   (RatioOrbit.mul (RatioOrbit.recip a) (RatioOrbit.recip b))
2809   (RatioOrbit.recip (RatioOrbit.mul a b))
2810 orbit_direction_mul :
2811   ∀ a b p : DistinctionNat,
2812   ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
2813   ∀ hp : p ≠ DistinctionNat.zero,
2814   a * b = p →
2815   RatioOrbit.crossEq (orbitDirection p hp)
2816   (RatioOrbit.mul (orbitDirection a ha) (orbitDirection b hb))
2817 orbit_product_display_compatible :

```

```

2818   PRCCharacterOrbitProductDisplayCompatible =
2819   PRCCharacterOrbitProductDisplayCompatible
2820   orbit_character_respects_crossEq :
2821   PRCCharacterRespectsCrossEq =
2822   PRCCharacterRespectsCrossEq
2823   normalizeRatio_canonical_target :
2824   PRCToNormalizeRatioCanonicalTarget =
2825   PRCToNormalizeRatioCanonicalTarget
2826   signed_orbit_sign_canonical :
2827   PRCSignedOrbitSignCanonical =
2828   PRCSignedOrbitSignCanonical
2829   ratio_reduced_sign_canonical :
2830   PRCRatioReducedSignCanonical =
2831   PRCRatioReducedSignCanonical
2832   signed_ofOrbit_abs_self :
2833    $\forall n : \text{DistinctionNat}, (\text{SignedOrbit.ofOrbit } n).abs = n$ 
2834   signed_neg_ofOrbit_abs_self :
2835    $\forall n : \text{DistinctionNat},$ 
2836    $(\text{SignedOrbit.negate } (\text{SignedOrbit.ofOrbit } n)).abs = n$ 
2837   signedQuotient_signCanonical :
2838    $\forall z : \text{SignedOrbit}, \forall d : \text{DistinctionNat},$ 
2839    $\forall hd : d \neq \text{DistinctionNat.zero},$ 
2840    $\text{DistinctionNat.divides } d \ z.abs \rightarrow$ 
2841    $\text{PRCSignedOrbitSignCanonical } (\text{DistinctionNat.signedQuotient } z \ d \ hd)$ 
2842   normalizeRatio_reduced_signCanonical :
2843    $\forall q : \text{RatioOrbit},$ 
2844    $\text{PRCRatioReducedSignCanonical } (\text{DistinctionNat.normalizeRatio } q)$ 
2845   signCanonical_toInt_injective :
2846    $\forall z w : \text{SignedOrbit},$ 
2847    $\text{PRCSignedOrbitSignCanonical } z \rightarrow$ 
2848    $\text{PRCSignedOrbitSignCanonical } w \rightarrow$ 
2849    $z.\text{toInt} = w.\text{toInt} \rightarrow$ 
2850    $z = w$ 
2851   reduced_den_dvd :
2852    $\forall q r : \text{RatioOrbit},$ 
2853    $\text{PRCRatioReducedSignCanonical } q \rightarrow$ 
2854    $\text{PRCRatioReducedSignCanonical } r \rightarrow$ 
2855    $\text{RatioOrbit.crossEq } q \ r \rightarrow$ 
2856    $q.\text{den.toNat} \mid r.\text{den.toNat}$ 
2857   reduced_den_eq :
2858    $\forall q r : \text{RatioOrbit},$ 
2859    $\text{PRCRatioReducedSignCanonical } q \rightarrow$ 
2860    $\text{PRCRatioReducedSignCanonical } r \rightarrow$ 
2861    $\text{RatioOrbit.crossEq } q \ r \rightarrow$ 
2862    $q.\text{den} = r.\text{den}$ 
2863   reduced_num_eq :
2864    $\forall q r : \text{RatioOrbit},$ 
2865    $\text{PRCRatioReducedSignCanonical } q \rightarrow$ 
2866    $\text{PRCRatioReducedSignCanonical } r \rightarrow$ 
2867    $\text{RatioOrbit.crossEq } q \ r \rightarrow$ 
2868    $q.\text{num} = r.\text{num}$ 
2869   reduced_signCanonical_ratio_unique_target :
2870   PRCReducedSignCanonicalRatioUniqueTarget =
2871   PRCReducedSignCanonicalRatioUniqueTarget
2872   reduced_signCanonical_ratio_unique_proved :
2873   PRCReducedSignCanonicalRatioUniqueTarget
2874   normalizeRatio_canonical_from_reduced_signCanonical_unique :
2875   PRCReducedSignCanonicalRatioUniqueTarget  $\rightarrow$ 
2876   PRCToNormalizeRatioCanonicalTarget
2877   normalizeRatio_canonical_proved :
2878   PRCToNormalizeRatioCanonicalTarget
2879   orbit_character_crossEq_from_normalizeRatio_canonical :
2880    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
2881    $\text{PRCRatioCharacter } x \rightarrow$ 
2882    $\text{PRCToNormalizeRatioCanonicalTarget} \rightarrow$ 
2883    $\text{PRCCharacterRespectsCrossEq } x$ 
2884   orbit_product_display_from_crossEq :
2885    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
2886    $\text{PRCCharacterRespectsCrossEq } x \rightarrow$ 
2887    $\text{PRCCharacterOrbitProductDisplayCompatible } x$ 
2888   orbit_product_no_mixed_orientation :
2889   PRCCharacterOrbitProductNoMixedOrientation =
2890   PRCCharacterOrbitProductNoMixedOrientation
2891   nonunit_orbit_orientation_coherent :
2892   PRCCharacterNonunitOrbitOrientationCoherent =
2893   PRCCharacterNonunitOrbitOrientationCoherent
2894   no_mixed_nonunit_orbit_orientation :
2895   PRCCharacterNoMixedNonunitOrbitOrientation =
2896   PRCCharacterNoMixedNonunitOrbitOrientation
2897   nonunit_identity_branch_transport :
2898   PRCCharacterNonunitIdentityBranchTransport =
2899   PRCCharacterNonunitIdentityBranchTransport
2900   nonunit_identity_respects_comparable_trace :
2901   PRCCharacterNonunitIdentityRespectsComparableTrace =
2902   PRCCharacterNonunitIdentityRespectsComparableTrace
2903   nonunit_local_from_coherent :
2904    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
2905    $\text{PRCCharacterNonunitOrbitOrientationCoherent } x \rightarrow$ 
2906    $\text{PRCCharacterNonunitOrbitLocalOrientation } x$ 
2907   no_mixed_nonunit_from_coherent :
2908    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
2909    $\text{PRCCharacterNonunitOrbitOrientationCoherent } x \rightarrow$ 
2910    $\text{PRCCharacterNoMixedNonunitOrbitOrientation } x$ 
2911   orbit_mul_not_unit_left :
2912    $\forall p r : \text{DistinctionNat},$ 
2913    $\neg \text{DistinctionNat.unit } p \rightarrow$ 
2914    $\neg \text{DistinctionNat.unit } (p * r)$ 
2915   no_mixed_nonunit_from_product_no_mixed :
2916    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
2917    $\text{PRCCharacterOrbitProductNoMixedOrientation } x \rightarrow$ 
2918    $\text{PRCCharacterNoMixedNonunitOrbitOrientation } x$ 

```

```

2919 orbit_product_no_mixed_from_no_mixed_nonunit :
2920   ∀ x : RatioOrbit → RatioOrbit,
2921     PRCCharacterNoMixedNonunitOrbitOrientation x →
2922     PRCCharacterOrbitProductNoMixedOrientation x
2923 orbit_product_no_mixed_iff_no_mixed_nonunit :
2924   ∀ x : RatioOrbit → RatioOrbit,
2925     PRCCharacterOrbitProductNoMixedOrientation x ↔
2926     PRCCharacterNoMixedNonunitOrbitOrientation x
2927 no_mixed_nonunit_from_identity_branch_transport :
2928   ∀ x : RatioOrbit → RatioOrbit,
2929     PRCCharacterNonunitIdentityBranchTransport x →
2930     PRCCharacterNoMixedNonunitOrbitOrientation x
2931 orbit_product_no_mixed_from_identity_branch_transport :
2932   ∀ x : RatioOrbit → RatioOrbit,
2933     PRCCharacterNonunitIdentityBranchTransport x →
2934     PRCCharacterOrbitProductNoMixedOrientation x
2935 nonunit_identity_branch_transport_from_local_no_mixed :
2936   ∀ x : RatioOrbit → RatioOrbit,
2937     PRCCharacterNonunitOrbitLocalOrientation x →
2938     PRCCharacterNoMixedNonunitOrbitOrientation x →
2939     PRCCharacterNonunitIdentityBranchTransport x
2940 nonunit_identity_branch_transport_from_coherent :
2941   ∀ x : RatioOrbit → RatioOrbit,
2942     PRCCharacterNonunitOrbitOrientationCoherent x →
2943     PRCCharacterNonunitIdentityBranchTransport x
2944 nonunit_identity_branch_transport_from_comparable_trace :
2945   ∀ x : RatioOrbit → RatioOrbit,
2946     PRCCharacterNonunitIdentityRespectsComparableTrace x →
2947     PRCCharacterNonunitIdentityBranchTransport x
2948 nonunit_identity_comparable_trace_from_branch_transport :
2949   ∀ x : RatioOrbit → RatioOrbit,
2950     PRCCharacterNonunitIdentityBranchTransport x →
2951     PRCCharacterNonunitIdentityRespectsComparableTrace x
2952 nonunit_identity_comparable_trace_iff_branch_transport :
2953   ∀ x : RatioOrbit → RatioOrbit,
2954     PRCCharacterNonunitIdentityRespectsComparableTrace x ↔
2955     PRCCharacterNonunitIdentityBranchTransport x
2956 prime_floor_successor_transport_from_nonunit_identity_comparable_trace :
2957   ∀ x : RatioOrbit → RatioOrbit,
2958     PRCCharacterNonunitIdentityRespectsComparableTrace x →
2959     PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport x
2960 nonunit_coherent_from_local_no_mixed :
2961   ∀ x : RatioOrbit → RatioOrbit,
2962     PRCCharacterNonunitOrbitLocalOrientation x →
2963     PRCCharacterNoMixedNonunitOrbitOrientation x →
2964     PRCCharacterNonunitOrbitOrientationCoherent x
2965 nonunit_coherent_from_local_identity_branch_transport :
2966   ∀ x : RatioOrbit → RatioOrbit,
2967     PRCCharacterNonunitOrbitLocalOrientation x →
2968     PRCCharacterNonunitIdentityBranchTransport x →
2969     PRCCharacterNonunitOrbitOrientationCoherent x
2970 orbit_product_no_mixed_from_nonunit_coherent :
2971   ∀ x : RatioOrbit → RatioOrbit,
2972     PRCCharacterNonunitOrbitOrientationCoherent x →
2973     PRCCharacterOrbitProductNoMixedOrientation x
2974 orbit_product_identity_identity :
2975   ∀ x : RatioOrbit → RatioOrbit,
2976     PRCRatioCharacter x →
2977     PRCCharacterOrbitProductDisplayCompatible x →
2978     ∀ a b p : DistinctionNat,
2979       ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
2980       ∀ hp : p ≠ DistinctionNat.zero,
2981         a * b = p →
2982         PRCCharacterOrbitDirectionIdentity x a ha →
2983         PRCCharacterOrbitDirectionIdentity x b hb →
2984         PRCCharacterOrbitDirectionIdentity x p hp
2985 orbit_product_reciprocal_reciprocal :
2986   ∀ x : RatioOrbit → RatioOrbit,
2987     PRCRatioCharacter x →
2988     PRCCharacterOrbitProductDisplayCompatible x →
2989     ∀ a b p : DistinctionNat,
2990       ∀ ha : a ≠ DistinctionNat.zero, ∀ hb : b ≠ DistinctionNat.zero,
2991       ∀ hp : p ≠ DistinctionNat.zero,
2992         a * b = p →
2993         PRCCharacterOrbitDirectionReciprocal x a ha →
2994         PRCCharacterOrbitDirectionReciprocal x b hb →
2995         PRCCharacterOrbitDirectionReciprocal x p hp
2996 orbit_product_local_from_display_nomix :
2997   ∀ x : RatioOrbit → RatioOrbit,
2998     PRCRatioCharacter x →
2999     PRCCharacterOrbitProductDisplayCompatible x →
3000     PRCCharacterOrbitProductNoMixedOrientation x →
3001     PRCCharacterOrbitProductLocalOrientationPropagates x
3002 nonunit_local_from_prime_product :
3003   ∀ x : RatioOrbit → RatioOrbit,
3004     PRCCharacterPrimeLocalOrientation x →
3005     PRCCharacterOrbitProductLocalOrientationPropagates x →
3006     PRCCharacterNonunitOrbitLocalOrientation x
3007 prime_floor_no_adjacent_mixed_orientation :
3008   PRCCharacterPrimeFloorNoAdjacentMixedOrientation =
3009   PRCCharacterPrimeFloorNoAdjacentMixedOrientation
3010 prime_floor_no_adjacent_from_nonunit_coherent :
3011   ∀ x : RatioOrbit → RatioOrbit,
3012     PRCCharacterNonunitOrbitOrientationCoherent x →
3013     PRCCharacterPrimeFloorNoAdjacentMixedOrientation x
3014 prime_floor_orbit_identity_extends_successor_step :
3015   PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep =
3016   PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep
3017 prime_floor_orbit_identity_contracts_successor_step :
3018   PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep =
3019   PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep

```

```

3020 prime_floor_orbit_identity_successor_transport :
3021   PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport =
3022     PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport
3023 prime_floor_extends_from_local_adjacent_nomix :
3024    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3025     PRCCharacterNonunitOrbitLocalOrientation  $x \rightarrow$ 
3026     PRCCharacterPrimeFloorNoAdjacentMixedOrientation  $x \rightarrow$ 
3027     PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep  $x$ 
3028 prime_floor_contracts_from_local_adjacent_nomix :
3029    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3030     PRCCharacterNonunitOrbitLocalOrientation  $x \rightarrow$ 
3031     PRCCharacterPrimeFloorNoAdjacentMixedOrientation  $x \rightarrow$ 
3032     PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep  $x$ 
3033 prime_floor_successor_transport_from_local_adjacent_nomix :
3034    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3035     PRCCharacterNonunitOrbitLocalOrientation  $x \rightarrow$ 
3036     PRCCharacterPrimeFloorNoAdjacentMixedOrientation  $x \rightarrow$ 
3037     PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport  $x$ 
3038 prime_floor_no_adjacent_from_successor_transport :
3039    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3040     PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport  $x \rightarrow$ 
3041     PRCCharacterPrimeFloorNoAdjacentMixedOrientation  $x$ 
3042 prime_floor_successor_transport_iff_local_adjacent_nomix :
3043    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3044     PRCCharacterNonunitOrbitLocalOrientation  $x \rightarrow$ 
3045     (PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport  $x \leftrightarrow$ 
3046      PRCCharacterPrimeFloorNoAdjacentMixedOrientation  $x$ )
3047 prime_identity_comparable_from_prime_floor_successor_transport :
3048    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3049     PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport  $x \rightarrow$ 
3050     PRCCharacterPrimeIdentityRespectsComparableTrace  $x$ 
3051 nonunit_identity_comparable_from_prime_floor_successor_transport :
3052    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3053     PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport  $x \rightarrow$ 
3054     PRCCharacterNonunitIdentityRespectsComparableTrace  $x$ 
3055 nonunit_coherent_from_local_prime_floor_successor_transport :
3056    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3057     PRCCharacterNonunitOrbitLocalOrientation  $x \rightarrow$ 
3058     PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport  $x \rightarrow$ 
3059     PRCCharacterNonunitOrbitOrientationCoherent  $x$ 
3060 orbit_identity_extends_from_additive_compat :
3061    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3062     PRCCharacterOrbitSuccessorAdditiveCompatible  $x \rightarrow$ 
3063     PRCCharacterOrbitIdentityExtendsSuccessorStep  $x$ 
3064 orbit_identity_contracts_from_additive_compat :
3065    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3066     PRCCharacterOrbitSuccessorAdditiveCompatible  $x \rightarrow$ 
3067     PRCCharacterOrbitIdentityContractsSuccessorStep  $x$ 
3068 orbit_identity_successor_transport_from_additive_compat :
3069    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3070     PRCCharacterOrbitSuccessorAdditiveCompatible  $x \rightarrow$ 
3071     PRCCharacterOrbitIdentitySuccessorTransport  $x$ 
3072 orbit_identity_respects_successor_step_from_transport :
3073    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3074     PRCCharacterOrbitIdentitySuccessorTransport  $x \rightarrow$ 
3075     PRCCharacterOrbitIdentityRespectsSuccessorStep  $x$ 
3076 orbit_identity_one_of_identity :
3077    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3078     PRCCharacterOrbitIdentityRespectsSuccessorStep  $x \rightarrow$ 
3079      $\forall p : \text{DistinctionNat}, \forall hp : p \neq \text{DistinctionNat.zero},$ 
3080     PRCCharacterOrbitDirectionIdentity  $x$   $p$   $hp \rightarrow$ 
3081     PRCCharacterOrbitDirectionIdentity  $x$ 
3082     DistinctionNat.one DistinctionNat.one_ne_zero
3083 orbit_identity_of_one :
3084    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3085     PRCCharacterOrbitIdentityRespectsSuccessorStep  $x \rightarrow$ 
3086      $\forall r : \text{DistinctionNat}, \forall hr : r \neq \text{DistinctionNat.zero},$ 
3087     PRCCharacterOrbitDirectionIdentity  $x$ 
3088     DistinctionNat.one DistinctionNat.one_ne_zero  $\rightarrow$ 
3089     PRCCharacterOrbitDirectionIdentity  $x$   $r$   $hr$ 
3090 prime_identity_comparable_from_successor_step :
3091    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3092     PRCCharacterOrbitIdentityRespectsSuccessorStep  $x \rightarrow$ 
3093     PRCCharacterPrimeIdentityRespectsComparableTrace  $x$ 
3094 prime_identity_common_trace_from_comparable_trace :
3095    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3096     PRCCharacterPrimeIdentityRespectsComparableTrace  $x \rightarrow$ 
3097     PRCCharacterPrimeIdentityRespectsCommonTraceExtension  $x$ 
3098 prime_identity_trace_connected_from_common_trace :
3099    $\forall x : \text{RatioOrbit} \rightarrow \text{RatioOrbit},$ 
3100     PRCCharacterPrimeIdentityRespectsCommonTraceExtension  $x \rightarrow$ 
3101     PRCCharacterPrimeIdentityRespectsTraceConnected  $x$ 
3102 prime_to_local_orientation_target :
3103   PRCPrimeCalibrationForcesLocalPrimeOrientationTarget =
3104     PRCPrimeCalibrationForcesLocalPrimeOrientationTarget
3105 prime_to_local_orientation_proved :
3106   PRCPrimeCalibrationForcesLocalPrimeOrientationTarget
3107 prime_no_mixed_orientation_target :
3108   PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget =
3109     PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget
3110 prime_identity_trace_coherence_target :
3111   PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget =
3112     PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget
3113 prime_identity_trace_transport_target :
3114   PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget =
3115     PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget
3116 prime_identity_common_trace_extension_target :
3117   PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget =
3118     PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget
3119 prime_identity_comparable_trace_target :
3120   PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget =

```



```

3121 PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget
3122 orbit_successor_identity_target :
3123 PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget =
3124 PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget
3125 orbit_successor_transport_target :
3126 PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget =
3127 PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget
3128 orbit_successor_additive_compat_target :
3129 PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget =
3130 PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget
3131 orbit_successor_additive_compat_target_refuted :
3132 ¬ PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget
3133 prime_floor_successor_transport_target :
3134 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget =
3135 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
3136 prime_floor_nonunit_local_orientation_target :
3137 PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget =
3138 PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget
3139 prime_floor_nonunit_product_local_orientation_target :
3140 PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget =
3141 PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget
3142 prime_floor_product_display_compatibility_target :
3143 PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget =
3144 PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget
3145 prime_floor_character_crossEq_respect_target :
3146 PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget =
3147 PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget
3148 prime_floor_character_crossEq_from_normalizeRatio_canonical :
3149 PRCNormalizeRatioCanonicalTarget →
3150 PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget
3151 prime_floor_character_crossEq_from_reduced_signCanonical_unique :
3152 PRCReducedSignCanonicalRatioUniqueTarget →
3153 PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget
3154 prime_floor_character_crossEq_respect_proved :
3155 PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget
3156 prime_floor_product_display_from_crossEq_respect :
3157 PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget →
3158 PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget
3159 prime_floor_product_display_compatibility_proved :
3160 PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget
3161 prime_floor_product_no_mixed_orientation_target :
3162 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget =
3163 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
3164 prime_floor_nonunit_orbit_orientation_coherent_target :
3165 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget =
3166 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
3167 prime_floor_no_mixed_nonunit_orbit_orientation_target :
3168 PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget =
3169 PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
3170 prime_floor_nonunit_identity_branch_transport_target :
3171 PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget =
3172 PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
3173 prime_floor_nonunit_identity_comparable_trace_target :
3174 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget =
3175 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
3176 prime_floor_nonunit_orbit_orientation_local_no_mixed_target :
3177 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget =
3178 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
3179 prime_floor_nonunit_orbit_orientation_local_product_no_mixed_target :
3180 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget =
3181 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget
3182 prime_floor_no_mixed_nonunit_from_coherent :
3183 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3184 PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
3185 prime_floor_no_mixed_nonunit_from_product_no_mixed :
3186 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
3187 PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
3188 prime_floor_product_no_mixed_from_no_mixed_nonunit :
3189 PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget →
3190 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
3191 prime_floor_product_no_mixed_iff_no_mixed_nonunit :
3192 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
3193 PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
3194 prime_floor_product_no_mixed_from_identity_branch_transport :
3195 PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget →
3196 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
3197 prime_floor_nonunit_identity_branch_transport_from_comparable_trace :
3198 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget →
3199 PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
3200 prime_floor_nonunit_identity_branch_transport_from_coherent :
3201 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3202 PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
3203 prime_floor_nonunit_local_no_mixed_from_local_product_no_mixed :
3204 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget →
3205 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
3206 prime_floor_nonunit_coherent_from_local_product_no_mixed :
3207 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget →
3208 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
3209 prime_floor_nonunit_coherent_from_product_no_mixed :
3210 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
3211 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
3212 prime_floor_product_no_mixed_iff_nonunit_coherent :
3213 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
3214 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
3215 prime_floor_nonunit_identity_branch_transport_from_product_no_mixed :
3216 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
3217 PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
3218 prime_floor_product_no_mixed_iff_identity_branch_transport :
3219 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
3220 PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
3221 prime_floor_nonunit_identity_comparable_trace_from_branch_transport :

```

```

3222 PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget →
3223 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
3224 prime_floor_nonunit_identity_comparable_trace_from_product_no_mixed :
3225 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
3226 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
3227 prime_floor_nonunit_identity_branch_transport_iff_comparable_trace :
3228 PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget →
3229 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
3230 prime_floor_product_no_mixed_iff_identity_comparable_trace :
3231 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
3232 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
3233 prime_floor_nonunit_local_no_mixed_from_coherent :
3234 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3235 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
3236 prime_floor_nonunit_coherent_from_local_no_mixed :
3237 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget →
3238 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
3239 prime_floor_nonunit_orbit_orientation_coherent_iff_local_no_mixed :
3240 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3241 PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
3242 prime_floor_nonunit_orbit_orientation_coherent_sharpened_target :
3243 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget =
3244 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget
3245 prime_floor_nonunit_orbit_orientation_coherent_from_local_successor_transport :
3246 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget →
3247 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
3248 prime_floor_product_no_mixed_from_nonunit_coherent :
3249 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3250 PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
3251 prime_floor_nonunit_local_from_nonunit_coherent :
3252 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3253 PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget
3254 prime_floor_no_adjacent_mixed_from_nonunit_coherent :
3255 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3256 PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
3257 prime_floor_no_adjacent_mixed_from_successor_transport :
3258 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget →
3259 PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
3260 prime_floor_successor_transport_from_nonunit_coherent :
3261 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3262 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
3263 prime_floor_nonunit_identity_comparable_trace_from_successor_transport :
3264 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget →
3265 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
3266 prime_floor_nonunit_orbit_orientation_sharpened_from_nonunit_coherent :
3267 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3268 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget
3269 prime_floor_nonunit_orbit_orientation_coherent_iff_sharpened :
3270 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3271 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget
3272 prime_floor_successor_transport_from_identity_comparable_trace :
3273 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget →
3274 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
3275 prime_floor_nonunit_identity_comparable_trace_iff_successor_transport :
3276 PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget →
3277 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
3278 prime_floor_successor_transport_local_adjacent_target :
3279 PRCPrimeFloorSuccessorTransportLocalAdjacentTarget =
3280 PRCPrimeFloorSuccessorTransportLocalAdjacentTarget
3281 prime_floor_successor_transport_from_local_adjacent_target :
3282 PRCPrimeFloorSuccessorTransportLocalAdjacentTarget →
3283 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
3284 prime_floor_local_adjacent_from_local_successor_transport :
3285 (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
3286 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget) →
3287 PRCPrimeFloorSuccessorTransportLocalAdjacentTarget
3288 prime_floor_local_adjacent_iff_local_successor_transport :
3289 PRCPrimeFloorSuccessorTransportLocalAdjacentTarget →
3290 (PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
3291 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget)
3292 prime_floor_local_adjacent_from_nonunit_coherent :
3293 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget →
3294 PRCPrimeFloorSuccessorTransportLocalAdjacentTarget
3295 prime_floor_nonunit_coherent_from_local_adjacent :
3296 PRCPrimeFloorSuccessorTransportLocalAdjacentTarget →
3297 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
3298 prime_floor_local_adjacent_iff_nonunit_coherent :
3299 PRCPrimeFloorSuccessorTransportLocalAdjacentTarget →
3300 PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
3301 prime_floor_product_local_orientation_sharpened_target :
3302 PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget =
3303 PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget
3304 prime_floor_product_local_orientation_from_display_nomix :
3305 PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget →
3306 PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget
3307 prime_floor_nonunit_local_orientation_from_product_local :
3308 PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget →
3309 PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget
3310 prime_floor_no_adjacent_mixed_orientation_target :
3311 PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget =
3312 PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
3313 prime_floor_successor_transport_sharpened_target :
3314 PRCPrimeFloorSuccessorTransportSharpenedTarget =
3315 PRCPrimeFloorSuccessorTransportSharpenedTarget
3316 prime_floor_successor_transport_target_from_local_adjacent_nomix :
3317 PRCPrimeFloorSuccessorTransportSharpenedTarget →
3318 PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
3319 orbit_successor_transport_target_from_additive_compat :
3320 orbitPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget →
3321 PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget
3322 prime_identity_comparable_trace_from_prime_floor_successor_transport :

```

```

3323   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget →
3324   PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget
3325   orbit_successor_identity_target_from_transport :
3326   PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget →
3327   PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget
3328   prime_identity_comparable_trace_from_successor_step :
3329   PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget →
3330   PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget
3331   prime_identity_common_trace_extension_from_comparable_trace :
3332   PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget →
3333   PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget
3334   prime_identity_trace_transport_from_common_trace :
3335   PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget →
3336   PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget
3337   prime_identity_trace_coherence_from_transport :
3338   PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget →
3339   PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget
3340   prime_no_mixed_from_trace_coherence :
3341   PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget →
3342   PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget
3343   coherent_prime_orientation_reduction :
3344   PRCPrimeCalibrationForcesLocalPrimeOrientationTarget →
3345   PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget →
3346   PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget
3347   prime_to_coherent_orientation_target :
3348   PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget =
3349   PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget
3350   coherent_prime_orientation_propagation_target :
3351   PRCCoherentPrimeOrientationPropagatesToGlobalTarget =
3352   PRCCoherentPrimeOrientationPropagatesToGlobalTarget
3353   global_orientation_reduction :
3354   PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget →
3355   PRCCoherentPrimeOrientationPropagatesToGlobalTarget →
3356   PRCPrimeCalibrationForcesGlobalOrientationTarget
3357   prime_propagation_sharpened_target :
3358   PRCPrimeCalibrationPropagationSharpenedTarget =
3359   PRCPrimeCalibrationPropagationSharpenedTarget
3360   prime_propagation_reduction :
3361   PRCPrimeCalibrationForcesGlobalOrientationTarget →
3362   PRCPrimeCalibrationPropagationTarget
3363   prime_propagation_sharpened_reduction :
3364   PRCPrimeCalibrationPropagationSharpenedTarget →
3365   PRCPrimeCalibrationPropagationTarget
3366   rigidity_sharpened_target :
3367   PRCNativeCostCharacterRigiditySharpenedTarget =
3368   PRCNativeCostCharacterRigiditySharpenedTarget
3369   rigidity_reduction :
3370   PRCTwoCalibrationForcesPrimeCalibrationTarget →
3371   PRCPrimeCalibrationPropagationTarget →
3372   PRCNativeCostCharacterRigidityTarget
3373   identity_character : PRCRatioCharacter (fun q : RatioOrbit => q)
3374   identity_rigid :
3375     ∀ q : RatioOrbit,
3376     RatioOrbit.crossEq
3377       (costFromCharacter (fun q : RatioOrbit => q) q)
3378       (onRatioOrbit q)
3379   identity_orientation :
3380   PRCCharacterGlobalCostOrientation (fun q : RatioOrbit => q)
3381   identity_prime_orientation_coherent :
3382   PRCCharacterPrimeOrientationCoherent (fun q : RatioOrbit => q)
3383   reciprocal_character :
3384   PRCRatioCharacter (fun q : RatioOrbit => RatioOrbit.recip q)
3385   reciprocal_prime_calibrated :
3386   PRCCharacterPrimeDirectionCalibrated
3387     (fun q : RatioOrbit => RatioOrbit.recip q)
3388   reciprocal_orientation :
3389   PRCCharacterGlobalCostOrientation
3390     (fun q : RatioOrbit => RatioOrbit.recip q)
3391   reciprocal_prime_orientation_coherent :
3392   PRCCharacterPrimeOrientationCoherent
3393     (fun q : RatioOrbit => RatioOrbit.recip q)
3394   sharpened_target :
3395   PRCNativeCostUniquenessSharpenedTarget =
3396   PRCNativeCostUniquenessSharpenedTarget
3397   reduction :
3398   PRCNativeCostCharacterFactorizationTarget →
3399   PRCNativeCostCharacterRigidityTarget →
3400   PRCNativeCostUniquenessTarget
3401   prime_reduction :
3402   PRCNativeCostCharacterFactorizationTarget →
3403   PRCTwoCalibrationForcesPrimeCalibrationTarget →
3404   PRCPrimeCalibrationPropagationTarget →
3405   PRCNativeCostUniquenessTarget
3406   original_target :
3407   PRCNativeCostUniquenessTarget = PRCNativeCostUniquenessTarget
3408   strength_tag : StrengthTag.deltaOnly = StrengthTag.deltaOnly
3409
3410   theorem prc_native_cost_uniqueness_blocker_certificate :
3411     PRCNativeCostUniquenessBlockerCertificate where
3412     factorization_target := rfl
3413     rigidity_target := rfl
3414     two_to_prime_target := rfl
3415     prime_propagation_target := rfl
3416     global_orientation_target := rfl
3417     coherent_prime_orientation := rfl
3418     local_prime_orientation := rfl
3419     no_mixed_prime_orientation := rfl
3420     prime_identity_trace_coherence := rfl
3421     prime_axis_trace_connected := rfl
3422     prime_axis_trace_connected_proved :=
3423     PRCPrimeAxisTraceConnected_proved

```

```

3424 orbit_trace_extends_of_toNat_le := by
3425   intro p r
3426   exact orbitPositionTrace_extends_of_toNat_le
3427 orbit_trace_comparable :=
3428   orbitPositionTrace_comparable
3429 orbit_direction_toRat := by
3430   intro p
3431   exact orbitDirection_toRat p
3432 orbit_direction_nonunit_not_crossEq_recip := by
3433   intro p
3434   exact orbitDirection_nonunit_not_crossEq_recip p
3435 orbit_direction_succ_add_one := by
3436   intro p
3437   exact orbitDirection_succ_crossEq_add_one p
3438 ratio_add_right_one_cancel := by
3439   intro a b
3440   exact RatioOrbit.add_right_one_cancel
3441 prime_identity_respects_trace_connected := rfl
3442 prime_identity_respects_common_trace_extension := rfl
3443 prime_identity_respects_comparable_trace := rfl
3444 orbit_direction_identity := rfl
3445 orbit_direction_reciprocal := rfl
3446 orbit_succ_not_unit := by
3447   intro p
3448   exact orbit_succ_not_unit_of_nonzero_not_unit p
3449 orbit_identity_respects_successor_step := rfl
3450 orbit_identity_extends_successor_step := rfl
3451 orbit_identity_contracts_successor_step := rfl
3452 orbit_identity_successor_transport := rfl
3453 orbit_successor_additive_compat := rfl
3454 nonunit_orbit_local_orientation := rfl
3455 orbit_product_local_orientation := rfl
3456 ratio_mul_congr := by
3457   intro a a' b b'
3458   exact ratioOrbit_mul_congr
3459 ratio_recip_congr := by
3460   intro a b
3461   exact ratioOrbit_recip_congr
3462 ratio_mul_recip_recip :=
3463   ratioOrbit_mul_recip_recip_crossEq_recip_mul
3464 orbit_direction_mul := by
3465   intro a b p
3466   exact orbitDirection_mul_crossEq a b p
3467 orbit_product_display_compatible := rfl
3468 orbit_character_respects_crossEq := rfl
3469 normalizeRatio_canonical_target := rfl
3470 signed_orbit_sign_canonical := rfl
3471 ratio_reduced_sign_canonical := rfl
3472 signed_ofOrbit_abs_self := signedOrbit_ofOrbit_abs_self
3473 signed_neg_ofOrbit_abs_self := signedOrbit_neg_ofOrbit_abs_self
3474 signedQuotient_signCanonical := by
3475   intro z d
3476   exact signedQuotient_signCanonical_of_divides z d
3477 normalizeRatio_reduced_signCanonical :=
3478   normalizeRatio_reduced_signCanonical
3479 signCanonical_toInt_injective := by
3480   intro z w
3481   exact PRCSignedOrbitSignCanonical.eq_of_toInt_eq
3482 reduced_den_dvd := by
3483   intro q r
3484   exact PRCReducedSignCanonical_den_dvd_of_crossEq
3485 reduced_den_eq := by
3486   intro q r
3487   exact PRCReducedSignCanonical_den_eq_of_crossEq
3488 reduced_num_eq := by
3489   intro q r
3490   exact PRCReducedSignCanonical_num_eq_of_crossEq
3491 reduced_signCanonical_ratio_unique_target := rfl
3492 reduced_signCanonical_ratio_unique_proved :=
3493   PRCReducedSignCanonicalRatioUniqueTarget_proved
3494 normalizeRatio_canonical_from_reduced_signCanonical_unique :=
3495   PRCNormalizeRatioCanonicalTarget_of_reduced_signCanonical_unique
3496 normalizeRatio_canonical_proved :=
3497   PRCNormalizeRatioCanonicalTarget_proved
3498 orbit_character_crossEq_from_normalizeRatio_canonical := by
3499   intro x
3500   exact PRCCharacterRespectsCrossEq_of_normalizeRatio_canonical
3501 orbit_product_display_from_crossEq := by
3502   intro x
3503   exact PRCCharacterOrbitProductDisplayCompatible_of_crossEq_respect
3504 orbit_product_no_mixed_orientation := rfl
3505 nonunit_orbit_orientation_coherent := rfl
3506 no_mixed_nonunit_orbit_orientation := rfl
3507 nonunit_identity_branch_transport := rfl
3508 nonunit_identity_respects_comparable_trace := rfl
3509 nonunit_local_from_coherent := by
3510   intro x
3511   exact PRCCharacterNonunitOrbitLocalOrientation_of_coherent
3512 no_mixed_nonunit_from_coherent := by
3513   intro x
3514   exact PRCCharacterNoMixedNonunitOrbitOrientation_of_coherent
3515 orbit_mul_not_unit_left := by
3516   intro p r
3517   exact orbit_mul_not_unit_of_left_not_unit
3518 no_mixed_nonunit_from_product_no_mixed := by
3519   intro x
3520   exact PRCCharacterNoMixedNonunitOrbitOrientation_of_product_no_mixed
3521 orbit_product_no_mixed_from_no_mixed_nonunit := by
3522   intro x
3523   exact PRCCharacterOrbitProductNoMixedOrientation_of_no_mixed_nonunit
3524 orbit_product_no_mixed_iff_no_mixed_nonunit := by

```

```

3525   intro x
3526   exact PRCCharacterOrbitProductNoMixedOrientation_iff_no_mixed_nonunit
3527   no_mixed_nonunit_from_identity_branch_transport := by
3528   intro x
3529   exact PRCCharacterNoMixedNonunitOrbitOrientation_of_identity_branch_transport
3530   orbit_product_no_mixed_from_identity_branch_transport := by
3531   intro x
3532   exact PRCCharacterOrbitProductNoMixedOrientation_of_identity_branch_transport
3533   nonunit_identity_branch_transport_from_local_no_mixed := by
3534   intro x
3535   exact PRCCharacterNonunitIdentityBranchTransport_of_local_no_mixed
3536   nonunit_identity_branch_transport_from_coherent := by
3537   intro x
3538   exact PRCCharacterNonunitIdentityBranchTransport_of_coherent
3539   nonunit_identity_branch_transport_from_comparable_trace := by
3540   intro x
3541   exact PRCCharacterNonunitIdentityBranchTransport_of_comparable_trace
3542   nonunit_identity_comparable_trace_from_branch_transport := by
3543   intro x
3544   exact PRCCharacterNonunitIdentityRespectsComparableTrace_of_branch_transport
3545   nonunit_identity_comparable_trace_iff_branch_transport := by
3546   intro x
3547   exact PRCCharacterNonunitIdentityRespectsComparableTrace_iff_branch_transport
3548   nonunit_coherent_from_local_no_mixed := by
3549   intro x
3550   exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_no_mixed
3551   nonunit_coherent_from_local_identity_branch_transport := by
3552   intro x
3553   exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_identity_branch_transport
3554   orbit_product_no_mixed_from_nonunit_coherent := by
3555   intro x
3556   exact PRCCharacterOrbitProductNoMixedOrientation_of_nonunit_coherent
3557   orbit_product_identity_identity := by
3558   intro x
3559   exact PRCCharacterOrbitProductIdentityIdentity
3560   orbit_product_reciprocal_reciprocal := by
3561   intro x
3562   exact PRCCharacterOrbitProductReciprocalReciprocal
3563   orbit_product_local_from_display_nomix := by
3564   intro x
3565   exact PRCCharacterOrbitProductLocalOrientationPropagates_of_display_compatible_nomix
3566   nonunit_local_from_prime_product := by
3567   intro x
3568   exact PRCCharacterNonunitOrbitLocalOrientation_of_prime_and_product_local
3569   prime_floor_no_adjacent_mixed_orientation := rfl
3570   prime_floor_no_adjacent_from_nonunit_coherent := by
3571   intro x
3572   exact PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_nonunit_coherent
3573   prime_floor_orbit_identity_extends_successor_step := rfl
3574   prime_floor_orbit_identity_contracts_successor_step := rfl
3575   prime_floor_orbit_identity_successor_transport := rfl
3576   prime_floor_extends_from_local_adjacent_nomix := by
3577   intro x
3578   exact PRCCharacterPrimeFloorOrbitIdentityExtendsSuccessorStep_of_local_adjacent_nomix
3579   prime_floor_contracts_from_local_adjacent_nomix := by
3580   intro x
3581   exact PRCCharacterPrimeFloorOrbitIdentityContractsSuccessorStep_of_local_adjacent_nomix
3582   prime_floor_successor_transport_from_local_adjacent_nomix := by
3583   intro x
3584   exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_local_adjacent_nomix
3585   prime_floor_no_adjacent_from_successor_transport := by
3586   intro x
3587   exact PRCCharacterPrimeFloorNoAdjacentMixedOrientation_of_successor_transport
3588   prime_floor_successor_transport_iff_local_adjacent_nomix := by
3589   intro x
3590   exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_iff_local_adjacent_nomix
3591   prime_floor_successor_transport_from_nonunit_identity_comparable_trace := by
3592   intro x
3593   exact PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport_of_nonunit_identity_comparable_trace
3594   prime_identity_comparable_from_prime_floor_successor_transport := by
3595   intro x
3596   exact PRCCharacterPrimeIdentityRespectsComparableTrace_of_prime_floor_successor_transport
3597   nonunit_identity_comparable_from_prime_floor_successor_transport := by
3598   intro x
3599   exact PRCCharacterNonunitIdentityRespectsComparableTrace_of_prime_floor_successor_transport
3600   nonunit_coherent_from_local_prime_floor_successor_transport := by
3601   intro x
3602   exact PRCCharacterNonunitOrbitOrientationCoherent_of_local_and_prime_floor_successor_transport
3603   orbit_identity_extends_from_additive_compat := by
3604   intro x
3605   exact PRCCharacterOrbitIdentityExtendsSuccessorStep_of_additive_compat
3606   orbit_identity_contracts_from_additive_compat := by
3607   intro x
3608   exact PRCCharacterOrbitIdentityContractsSuccessorStep_of_additive_compat
3609   orbit_identity_successor_transport_from_additive_compat := by
3610   intro x
3611   exact PRCCharacterOrbitIdentitySuccessorTransport_of_additive_compat
3612   orbit_identity_respects_successor_step_from_transport := by
3613   intro x
3614   exact PRCCharacterOrbitIdentityRespectsSuccessorStep_of_transport
3615   orbit_identity_one_of_identity := by
3616   intro x
3617   exact PRCCharacterOrbitIdentity_one_of_identity
3618   orbit_identity_of_one := by
3619   intro x
3620   exact PRCCharacterOrbitIdentity_of_one
3621   prime_identity_comparable_from_successor_step := by
3622   intro x
3623   exact PRCCharacterPrimeIdentityRespectsComparableTrace_of_successor_step
3624   prime_identity_common_trace_from_comparable_trace := by
3625   intro x

```

```

3626 exact PRCCharacterPrimeIdentityRespectsCommonTraceExtension_of_comparable_trace
3627 prime_identity_trace_connected_from_common_trace := by
3628   intro x
3629   exact PRCCharacterPrimeIdentityRespectsTraceConnected_of_common_trace_extension
3630 prime_to_local_orientation_target := rfl
3631 prime_to_local_orientation_proved :=
3632   PRCPrimeCalibrationForcesLocalPrimeOrientationTarget_proved
3633 prime_no_mixed_orientation_target := rfl
3634 prime_identity_trace_coherence_target := rfl
3635 prime_identity_trace_transport_target := rfl
3636 prime_identity_common_trace_extension_target := rfl
3637 prime_identity_comparable_trace_target := rfl
3638 orbit_successor_identity_target := rfl
3639 orbit_successor_transport_target := rfl
3640 orbit_successor_additive_compat_target := rfl
3641 orbit_successor_additive_compat_target_refuted :=
3642   PRCPrimeCalibrationForcesOrbitSuccessorAdditiveCompatibilityTarget_refuted
3643 prime_floor_successor_transport_target := rfl
3644 prime_floor_nonunit_local_orientation_target := rfl
3645 prime_floor_nonunit_product_local_orientation_target := rfl
3646 prime_floor_product_display_compatibility_target := rfl
3647 prime_floor_character_crossEq_respect_target := rfl
3648 prime_floor_character_crossEq_from_normalizeRatio_canonical :=
3649   PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_normalizeRatio_canonical
3650 prime_floor_character_crossEq_from_reduced_signCanonical_unique :=
3651   PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_of_reduced_signCanonical_unique
3652 prime_floor_character_crossEq_respect_proved :=
3653   PRCPrimeCalibrationForcesCharacterCrossEqRespectTarget_proved
3654 prime_floor_product_display_from_crossEq_respect :=
3655   PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_of_crossEq_respect
3656 prime_floor_product_display_compatibility_proved :=
3657   PRCPrimeCalibrationForcesOrbitProductDisplayCompatibilityTarget_proved
3658 prime_floor_product_no_mixed_orientation_target := rfl
3659 prime_floor_nonunit_orbit_orientation_coherent_target := rfl
3660 prime_floor_no_mixed_nonunit_orbit_orientation_target := rfl
3661 prime_floor_nonunit_identity_branch_transport_target := rfl
3662 prime_floor_nonunit_identity_comparable_trace_target := rfl
3663 prime_floor_nonunit_orbit_orientation_local_no_mixed_target := rfl
3664 prime_floor_nonunit_orbit_orientation_local_product_no_mixed_target := rfl
3665 prime_floor_no_mixed_nonunit_from_coherent :=
3666   PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_coherent
3667 prime_floor_no_mixed_nonunit_from_product_no_mixed :=
3668   PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed
3669 prime_floor_product_no_mixed_from_no_mixed_nonunit :=
3670   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_no_mixed_nonunit
3671 prime_floor_product_no_mixed_iff_no_mixed_nonunit :=
3672   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_no_mixed_nonunit
3673 prime_floor_product_no_mixed_from_identity_branch_transport :=
3674   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport
3675 prime_floor_nonunit_identity_branch_transport_from_comparable_trace :=
3676   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace
3677 prime_floor_nonunit_identity_branch_transport_from_coherent :=
3678   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_coherent
3679 prime_floor_nonunit_local_no_mixed_from_local_product_no_mixed :=
3680   PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_local_product_no_mixed
3681 prime_floor_nonunit_coherent_from_local_product_no_mixed :=
3682   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_product_no_mixed
3683 prime_floor_nonunit_coherent_from_product_no_mixed :=
3684   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed
3685 prime_floor_product_no_mixed_iff_nonunit_coherent :=
3686   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_nonunit_coherent
3687 prime_floor_nonunit_identity_branch_transport_from_product_no_mixed :=
3688   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed
3689 prime_floor_product_no_mixed_iff_identity_branch_transport :=
3690   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_branch_transport
3691 prime_floor_nonunit_identity_comparable_trace_from_branch_transport :=
3692   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_branch_transport
3693 prime_floor_nonunit_identity_comparable_trace_from_product_no_mixed :=
3694   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_product_no_mixed
3695 prime_floor_nonunit_identity_branch_transport_iff_comparable_trace :=
3696   PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_iff_comparable_trace
3697 prime_floor_product_no_mixed_iff_identity_comparable_trace :=
3698   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_comparable_trace
3699 prime_floor_nonunit_local_no_mixed_from_coherent :=
3700   PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget_of_coherent
3701 prime_floor_nonunit_coherent_from_local_no_mixed :=
3702   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_no_mixed
3703 prime_floor_nonunit_orbit_orientation_coherent_iff_local_no_mixed :=
3704   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_local_no_mixed
3705 prime_floor_nonunit_orbit_orientation_coherent_sharpened_target := rfl
3706 prime_floor_nonunit_orbit_orientation_coherent_from_local_successor_transport :=
3707   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_and_prime_floor_successor_transport
3708 prime_floor_product_no_mixed_from_nonunit_coherent :=
3709   PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_nonunit_coherent
3710 prime_floor_nonunit_local_from_nonunit_coherent :=
3711   PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_nonunit_coherent
3712 prime_floor_no_adjacent_mixed_from_nonunit_coherent :=
3713   PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_nonunit_coherent
3714 prime_floor_no_adjacent_mixed_from_successor_transport :=
3715   PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget_of_successor_transport
3716 prime_floor_successor_transport_from_nonunit_coherent :=
3717   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_nonunit_coherent
3718 prime_floor_nonunit_identity_comparable_trace_from_successor_transport :=
3719   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_prime_floor_successor_transport
3720 prime_floor_nonunit_orbit_orientation_sharpened_from_nonunit_coherent :=
3721   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget_of_nonunit_coherent
3722 prime_floor_nonunit_orbit_orientation_coherent_iff_sharpened :=
3723   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_sharpened
3724 prime_floor_successor_transport_from_identity_comparable_trace :=
3725   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_identity_comparable_trace
3726 prime_floor_nonunit_identity_comparable_trace_iff_successor_transport :=

```

```

3727   PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_iff_prime_floor_successor_transport
3728   prime_floor_successor_transport_local_adjacent_target := rfl
3729   prime_floor_successor_transport_from_local_adjacent_target :=
3730   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_target
3731   prime_floor_local_adjacent_from_local_successor_transport :=
3732   PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_local_successor_transport
3733   prime_floor_local_adjacent_iff_local_successor_transport :=
3734   PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_local_successor_transport
3735   prime_floor_local_adjacent_from_nonunit_coherent :=
3736   PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_of_nonunit_coherent
3737   prime_floor_nonunit_coherent_from_local_adjacent :=
3738   PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_local_adjacent
3739   prime_floor_local_adjacent_iff_nonunit_coherent :=
3740   PRCPrimeFloorSuccessorTransportLocalAdjacentTarget_iff_nonunit_coherent
3741   prime_floor_product_local_orientation_sharpened_target := rfl
3742   prime_floor_product_local_orientation_from_display_nomix :=
3743   PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget_of_display_compatible_nomix
3744   prime_floor_nonunit_local_orientation_from_product_local :=
3745   PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget_of_product_local_orientation
3746   prime_floor_no_adjacent_mixed_orientation_target := rfl
3747   prime_floor_successor_transport_sharpened_target := rfl
3748   prime_floor_successor_transport_target_from_local_adjacent_nomix :=
3749   PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget_of_local_adjacent_nomix
3750   orbit_successor_transport_target_from_additive_compat :=
3751   PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget_of_additive_compat
3752   prime_identity_comparable_trace_from_prime_floor_successor_transport :=
3753   PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_prime_floor_successor_transport
3754   orbit_successor_identity_target_from_transport :=
3755   PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget_of_transport
3756   prime_identity_comparable_trace_from_successor_step :=
3757   PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget_of_successor_step
3758   prime_identity_common_trace_extension_from_comparable_trace :=
3759   PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget_of_comparable_trace
3760   prime_identity_trace_transport_from_common_trace :=
3761   PRCPrimeCalibrationForcesPrimeIdentityTraceTransportTarget_of_common_trace_extension
3762   prime_identity_trace_coherence_from_transport :=
3763   PRCPrimeCalibrationForcesPrimeIdentityTraceCoherenceTarget_of_trace_transport
3764   prime_no_mixed_from_trace_coherence :=
3765   PRCPrimeCalibrationForcesNoMixedPrimeOrientationTarget_of_trace_coherence
3766   coherent_prime_orientation_reduction :=
3767   PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget_of_local_and_nomixed
3768   prime_to_coherent_orientation_target := rfl
3769   coherent_prime_orientation_propagation_target := rfl
3770   global_orientation_reduction :=
3771   PRCPrimeCalibrationForcesGlobalOrientationTarget_of_prime_orientation_targets
3772   prime_propagation_sharpened_target := rfl
3773   prime_propagation_reduction :=
3774   PRCPrimeCalibrationPropagationTarget_of_global_orientation
3775   prime_propagation_sharpened_reduction :=
3776   PRCPrimeCalibrationPropagationTarget_of_sharpened_orientation
3777   rigidity_sharpened_target := rfl
3778   rigidity_reduction := PRCNativeCostCharacterRigidityTarget_of_prime_targets
3779   identity_character := identity_ratio_character
3780   identity_rigid := identity_character_rigid
3781   identity_orientation := identity_character_global_orientation
3782   identity_prime_orientation_coherent :=
3783   identity_character_prime_orientation_coherent
3784   reciprocal_character := reciprocal_ratio_character
3785   reciprocal_prime_calibrated := reciprocal_character_prime_calibrated
3786   reciprocal_orientation := reciprocal_character_global_orientation
3787   reciprocal_prime_orientation_coherent :=
3788   reciprocal_character_prime_orientation_coherent
3789   sharpened_target := rfl
3790   reduction := PRCNativeCostUniquenessTarget_of_character_targets
3791   prime_reduction := PRCNativeCostUniquenessTarget_of_prime_character_targets
3792   original_target := rfl
3793   strength_tag := rfl
3794
3795 end PRCJCost
3796 end PrimitiveRecognitionCalculus
3797 end Foundation
3798 end IndisputableMonolith

```

32. Kernel.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/Kernel.lean · Lines: 253

```

0001 /-
0002   PrimitiveRecognitionCalculus/Kernel.lean
0003
0004   Round-trip source:
0005     PRC_Kernel_Spec_20260526.html
0006
0007   Aggregate import for the first PRC kernel pass:
0008
0009      $\delta \rightarrow \text{trace} \rightarrow \text{SameT/DiffT} \rightarrow \text{substitution} \rightarrow \text{quotient} \rightarrow \mathbb{N} \rightarrow \mathbb{Z} \rightarrow \mathbb{Q}$ 
0010
0011   The  $\mathbb{Z}$  and  $\mathbb{Q}$  stages are PRC-owned quotient surfaces with internal
0012   equivalence relations (balanced length for 'PRCInt'; cross-multiplication
0013   for 'PRCRat'). The maps into Lean's ' $\mathbb{Z}$ ' and ' $\mathbb{Q}$ ' are conservative verifier
0014   displays, proved well-defined from the internal characterizations.
0015 -/
0016
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Strength
0018 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Basic
0019 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.SameDiff
0020 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.TraceLogic
0021 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.FormalSystem
0022 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Inevitability
0023 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Quotient
0024 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Orbit
0025 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitArithmetic
0026 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerRational
0027 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.IntegerOrder
0028 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitDivisibility
0029 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.OrbitEuclidean
0030 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCost
0031 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RationalField
0032 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RecognizerBridge
0033 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.TraceClosure
0034 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCauchy
0035 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealNullSetoid
0036 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceTriangle
0037 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceVerifierTriangle
0038 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCJCostDistanceIncrementTriangle
0039 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteOrderedField
0040 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealMulBoundedContinuity
0041 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealBoundednessModulus
0042 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealProductContinuity
0043 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealOrderCongruence
0044 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteness
0045 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompleteOrderedFieldPromoted
0046 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.RealCompletion
0047
0048 namespace IndisputableMonolith
0049 namespace Foundation
0050 namespace PrimitiveRecognitionCalculus
0051
0052 /-- K7/A2. First-pass PRC kernel certificate:
0053 the analytic specification has concrete Lean objects for each stage in the
0054 first theorem chain. This is a bundling certificate, not yet the final
0055 inevitability theorem. -/
0056 structure KernelFirstPassCertificate : Prop where
0057   strength_tags_exist : Nonempty StrengthTag
0058   trace_syntax_exists : Nonempty Trace
0059   judgment_surface_exists : Nonempty TraceJudgment
0060   /-- Stable finite-trace predicates carry the first PRC logic surface. -/
0061   trace_logic : TraceLogicCertificate
0062   /-- Expressive formal systems admit a PRC trace embedding. -/
0063   formal_system : FormalSystemCertificate
0064   /-- Every admissible foundation carries a PRC trace core. -/
0065   inevitability : PRCInevitabilityCertificate
0066   quotient_surface_exists :
0067      $\forall (J : \text{TraceJudgment}) (T : \text{Trace}), \text{Nonempty } (\text{EndpointClass } J \ T)$ 
0068   orbit_nat_equivalence : Nonempty (DistinctionNat = Nat)
0069   /-- Orbit addition is verifier-faithful. -/
0070   orbit_add_faithful :
0071      $\forall a \ b : \text{DistinctionNat}, (a + b).\text{toNat} = a.\text{toNat} + b.\text{toNat}$ 
0072   /-- Orbit multiplication is verifier-faithful. -/
0073   orbit_mul_faithful :
0074      $\forall a \ b : \text{DistinctionNat}, (a * b).\text{toNat} = a.\text{toNat} * b.\text{toNat}$ 
0075   prc_integer_surface_exists : Nonempty PRCInt
0076   prc_rational_surface_exists : Nonempty PRCRat
0077   /-- The internal balanced-length relation characterizes signed-orbit
0078   equivalence in PRC integers (K4.9). -/
0079   prc_int_balanced_iff_display :
0080      $\forall a \ b : \text{SignedOrbit},$ 
0081      $\text{SignedOrbit.balanced } a \ b \leftrightarrow a.\text{toInt} = b.\text{toInt}$ 
0082   /-- The internal cross-multiplication relation characterizes ratio-orbit
0083   equivalence in PRC rationals (K4.10). -/
0084   prc_rat_cross_iff_display :
0085      $\forall a \ b : \text{RatioOrbit},$ 
0086      $\text{RatioOrbit.crossseq } a \ b \leftrightarrow a.\text{toRat} = b.\text{toRat}$ 
0087   /-- The PRC integer display is injective on the quotient. -/
0088   prc_int_display_injective : Function.Injective PRCInt.toInt
0089   /-- The PRC rational display is injective on the quotient. -/
0090   prc_rat_display_injective : Function.Injective PRCRat.toRat

```



```

0091 /-- The PRC integer surface is isomorphic to verifier `Z`. -/
0092 prc_int_equiv_int : Nonempty (PRCInt = Z)
0093 /-- Internal signed-orbit order and absolute value are closed. -/
0094 integer_order : IntegerOrderCertificate
0095 /-- Native orbit divisibility, units, factorization, and prime-orbit predicates are closed. -/
0096 orbit_divisibility : DistinctionNat.OrbitDivisibilityCertificate
0097 /-- Native Euclidean quotient/remainder, GCD, and coprime predicates are closed. -/
0098 orbit_euclidean : DistinctionNat.OrbitEuclideanCertificate
0099 /-- PRC rational J-cost, canonical RCL, and bridge to existing real uniqueness are closed. -/
0100 prc_jcost : PRCJCost.PRCJCostCertificate
0101 /-- PRC rational field-style laws, division, positivity, and quotient-level J-cost are packaged. -/
0102 rational_field : RationalFieldCertificate
0103 /-- Positive PRC ratios carry recognition cost and bridge to the Law-of-Logic J-cost chain. -/
0104 recognizer_bridge : PRCRecognizerBridgeCertificate
0105 /-- Addition on PRC integers is commutative. -/
0106 prc_int_add_comm : ∀ a b : PRCInt, a + b = b + a
0107 /-- Addition on PRC integers is associative. -/
0108 prc_int_add_assoc : ∀ a b c : PRCInt, a + b + c = a + (b + c)
0109 /-- Multiplication on PRC integers is commutative. -/
0110 prc_int_mul_comm : ∀ a b : PRCInt, a * b = b * a
0111 /-- Multiplication on PRC integers is associative. -/
0112 prc_int_mul_assoc : ∀ a b c : PRCInt, a * b * c = a * (b * c)
0113 /-- Multiplication distributes over addition. -/
0114 prc_int_left_distrib :
0115   ∀ a b c : PRCInt, a * (b + c) = a * b + a * c
0116 /-- Negation is the additive inverse on PRC integers. -/
0117 prc_int_add_negate : ∀ a : PRCInt, a + (-a) = 0
0118 /-- The PRC rational display preserves addition. -/
0119 prc_rat_display_add :
0120   ∀ a b : PRCRat, (a + b).toRat = a.toRat + b.toRat
0121 /-- The PRC rational display preserves multiplication. -/
0122 prc_rat_display_mul :
0123   ∀ a b : PRCRat, (a * b).toRat = a.toRat * b.toRat
0124 /-- The PRC rational display preserves reciprocal. -/
0125 prc_rat_display_inv :
0126   ∀ a : PRCRat, (a⁻¹).toRat = (a.toRat)⁻¹
0127 /-- Ratio reciprocal uses internal signed-orbit absolute value for the denominator. -/
0128 ratio_recip_internal_den :
0129   ∀ (a : RatioOrbit)
0130     (h : ¬ SignedOrbit.balanced a.num SignedOrbit.zero),
0131     (RatioOrbit.recipNonzero a h).den = a.num.abs
0132 /-- Addition on PRC rationals is commutative. -/
0133 prc_rat_add_comm : ∀ a b : PRCRat, a + b = b + a
0134 /-- Multiplication on PRC rationals is commutative. -/
0135 prc_rat_mul_comm : ∀ a b : PRCRat, a * b = b * a
0136 /-- Multiplication distributes over addition on PRC rationals. -/
0137 prc_rat_left_distrib :
0138   ∀ a b c : PRCRat, a * (b + c) = a * b + a * c
0139 /-- Nonzero PRC rationals multiply by their reciprocal to one. -/
0140 prc_rat_mul_recip_cancel :
0141   ∀ a : PRCRat, a.toRat ≠ 0 → a * a⁻¹ = 1
0142 /-- The completed trace boundary is inhabited and trace-closure tagged. -/
0143 trace_closure_boundary : TraceClosureCertificate
0144 /-- Internal Cauchy ledgers and the first PRC real quotient carrier are trace-closure tagged. -/
0145 real_cauchy : PRCRealCauchyCertificate
0146 /-- The final null-distance setoid is reduced to the exact J-cost triangle-modulus target. -/
0147 real_null_setoid : PRCRealNullSetoidConditionalCertificate
0148 /-- The J-cost distance triangle target is reduced to an exact rational display inequality. -/
0149 jcost_distance_triangle : PRCJCostDistanceTriangleConditionalCertificate
0150 /-- The verifier-rational triangle target is reduced to an increment-only modulus. -/
0151 jcost_distance_verifier_triangle : PRCJCostDistanceVerifierTriangleConditionalCertificate
0152 /-- The increment-only J-cost modulus closes the null-distance setoid chain. -/
0153 jcost_distance_increment_triangle : PRCJCostDistanceIncrementTriangleCertificate
0154 /-- The first complete ordered field pass closes add/neg and names mul/order/completeness targets. -/
0155 real_complete_ordered_field : PRCRealCompleteOrderedFieldConditionalCertificate
0156 /-- Multiplication on the null quotient is reduced to boundedness and bounded product continuity. -/
0157 real_mul_bounded_continuity : PRCRealMulBoundedContinuityConditionalCertificate
0158 /-- Every J-cost Cauchy ledger is eventually PRC-bounded. -/
0159 real_boundedness_modulus : PRCRealBoundednessModulusCertificate
0160 /-- Bounded product-continuity closes multiplication on the null quotient. -/
0161 real_product_continuity : PRCRealProductContinuityCertificate
0162 /-- Eventual non-strict order descends to the null-distance quotient. -/
0163 real_order_congruence : PRCRealOrderCongruenceCertificate
0164 /-- Internal completeness is sharpened to the exact Cauchy-of-Cauchy diagonal target. -/
0165 real_completeness : PRCRealCompletenessSharpenedCertificate
0166 /-- The complete ordered-field certificate surface now carries proved mul, order, and completeness targets. -/
0167 real_complete_ordered_field_promoted :
0168   PRCRealCompleteOrderedFieldPromotedCertificate
0169 /-- The real-completion boundary is inhabited and classical-extension tagged. -/
0170 real_completion_boundary : RealCompletionBoundaryCertificate
0171 signed_orbit_display : Nonempty (SignedOrbit → Z)
0172 ratio_orbit_display : Nonempty (RatioOrbit → Q)
0173
0174 /-- K7/A2. The first-pass kernel certificate is inhabited. -/
0175 theorem kernel_first_pass_certificate :
0176   KernelFirstPassCertificate where
0177     strength_tags_exist := (StrengthTag.deltaOnly)
0178     trace_syntax_exists := (Trace.empty)
0179     judgment_surface_exists := (verifierEqualityJudgment)
0180     trace_logic := trace_logic_certificate
0181     formal_system := formal_system_certificate
0182     inevitability := prc_inevitability_certificate
0183     quotient_surface_exists := by
0184       intro J T
0185       exact (endpointClassOf J T Endpoint.left)
0186     orbit_nat_equivalence := (DistinctionNat.equivNat)
0187     orbit_add_faithful := DistinctionNat.toNat_add
0188     orbit_mul_faithful := DistinctionNat.toNat_mul
0189     prc_integer_surface_exists := (PRCInt.zero)
0190     prc_rational_surface_exists := by
0191       let one : DistinctionNat := DistinctionNat.succ DistinctionNat.zero

```

```

0192   have hone : one ≠ DistinctionNat.zero := by
0193     intro h
0194     exact DistinctionNat.zero_ne_succ DistinctionNat.zero h.symm
0195     exact (PRCRat.mk (SignedOrbit.zero, one, hone))
0196   prc_int_balanced_iff_display := SignedOrbit.balanced_iff_toInt_eq
0197   prc_rat_cross_iff_display := RatioOrbit.crossEq_iff_toRat_eq
0198   prc_int_display_injective := PRCInt.toInt_injective
0199   prc_rat_display_injective := PRCRat.toRat_injective
0200   prc_int_equiv_int := (PRCInt.equivInt)
0201   integer_order := integer_order_certificate
0202   orbit_divisibility := DistinctionNat.orbit_divisibility_certificate
0203   orbit_euclidean := DistinctionNat.orbit_euclidean_certificate
0204   prc_jcost := PRCJCost.prc_jcost_certificate
0205   rational_field := rational_field_certificate
0206   recognizer_bridge := prc_recognizer_bridge_certificate
0207   prc_int_add_comm := PRCInt.add_comm
0208   prc_int_add_assoc := PRCInt.add_assoc
0209   prc_int_mul_comm := PRCInt.mul_comm
0210   prc_int_mul_assoc := PRCInt.mul_assoc
0211   prc_int_left_distrib := PRCInt.left_distrib
0212   prc_int_add_negate := PRCInt.add_negate
0213   prc_rat_display_add := PRCRat.toRat_add'
0214   prc_rat_display_mul := PRCRat.toRat_mul'
0215   prc_rat_display_inv := PRCRat.toRat_inv'
0216   ratio_recip_internal_den := by
0217     intro a h
0218     rfl
0219   prc_rat_add_comm := PRCRat.add_comm
0220   prc_rat_mul_comm := PRCRat.mul_comm
0221   prc_rat_left_distrib := PRCRat.left_distrib
0222   prc_rat_mul_recip_cancel := by
0223     intro a h
0224     simp using PRCRat.mul_recip_cancel (a := a) h
0225   trace_closure_boundary := trace_closure_certificate
0226   real_cauchy := real_cauchy_certificate
0227   real_null_setoid := real_null_setoid_conditional_certificate
0228   jcost_distance_triangle := prc_jcost_distance_triangle_conditional_certificate
0229   jcost_distance_verifier_triangle :=
0230     prc_jcost_distance_verifier_triangle_conditional_certificate
0231   jcost_distance_increment_triangle :=
0232     prc_jcost_distance_increment_triangle_certificate
0233   real_complete_ordered_field :=
0234     prc_real_complete_ordered_field_conditional_certificate
0235   real_mul_bounded_continuity :=
0236     prc_real_mul_bounded_continuity_conditional_certificate
0237   real_boundedness_modulus :=
0238     prc_real_boundedness_modulus_certificate
0239   real_product_continuity :=
0240     prc_real_product_continuity_certificate
0241   real_order_congruence :=
0242     prc_real_order_congruence_certificate
0243   real_completeness :=
0244     prc_real_completeness_sharpened_certificate
0245   real_complete_ordered_field_promoted :=
0246     prc_real_complete_ordered_field_promoted_certificate
0247   real_completion_boundary := real_completion_boundary_certificate
0248   signed_orbit_display := (SignedOrbit.toInt)
0249   ratio_orbit_display := (RatioOrbit.toRat)
0250
0251 end PrimitiveRecognitionCalculus
0252 end Foundation
0253 end IndisputableMonolith

```

33. UniversalFoundation.lean

Path: IndisputableMonolith/Foundation/PrimitiveRecognitionCalculus/UniversalFoundation.lean · Lines: 336

```

0001 /-
0002   PrimitiveRecognitionCalculus/UniversalFoundation.lean
0003
0004   Round-trip source:
0005   6/PRC_Universal_Foundation_Execution_Plan_20260526.html
0006
0007   Spec anchor:
0008   Build Order step 15: compose arithmetic, structural surfaces, cost, logic,
0009   real completion, recognizer, and admissible inevitability.
0010
0011   This file deliberately does not declare the final theorem
0012   'prc_universal_foundation'. It gives the current top-level conditional
0013   certificate and names the exact remaining targets that prevent final closure.
0014 -/
0015
0016 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.Kernel
0017 import IndisputableMonolith.Foundation.PrimitiveRecognitionCalculus.PRCNativeCostUniqueness
0018
0019 namespace IndisputableMonolith
0020 namespace Foundation
0021 namespace PrimitiveRecognitionCalculus
0022
0023 /-- Exact open targets remaining before the final unqualified universal
0024 foundation theorem can be declared. -/
0025 structure PRCUniversalFoundationOpenTargets : Prop where
0026   native_cost_uniqueness :
0027     PRCJCost.PRCNativeCostUniquenessTarget =
0028     PRCJCost.PRCNativeCostUniquenessTarget
0029   native_cost_character_factorization :
0030     PRCJCost.PRCNativeCostCharacterFactorizationTarget =
0031     PRCJCost.PRCNativeCostCharacterFactorizationTarget
0032   native_cost_character_rigidity :
0033     PRCJCost.PRCNativeCostCharacterRigidityTarget =
0034     PRCJCost.PRCNativeCostCharacterRigidityTarget
0035   native_cost_sharpened :
0036     PRCJCost.PRCNativeCostUniquenessSharpenedTarget =
0037     PRCJCost.PRCNativeCostUniquenessSharpenedTarget
0038   two_to_prime_calibration :
0039     PRCJCost.PRCTwoCalibrationForcesPrimeCalibrationTarget =
0040     PRCJCost.PRCTwoCalibrationForcesPrimeCalibrationTarget
0041   prime_calibration_propagation :
0042     PRCJCost.PRCPrimeCalibrationPropagationTarget =
0043     PRCJCost.PRCPrimeCalibrationPropagationTarget
0044   prime_global_orientation :
0045     PRCJCost.PRCPrimeCalibrationForcesGlobalOrientationTarget =
0046     PRCJCost.PRCPrimeCalibrationForcesGlobalOrientationTarget
0047   coherent_prime_orientation :
0048     PRCJCost.PRCCharacterPrimeOrientationCoherent =
0049     PRCJCost.PRCCharacterPrimeOrientationCoherent
0050   local_prime_orientation :
0051     PRCJCost.PRCCharacterPrimeLocalOrientation =
0052     PRCJCost.PRCCharacterPrimeLocalOrientation
0053   no_mixed_prime_orientation :
0054     PRCJCost.PRCCharacterNoMixedPrimeOrientation =
0055     PRCJCost.PRCCharacterNoMixedPrimeOrientation
0056   prime_identity_trace_coherence :
0057     PRCJCost.PRCCharacterPrimeIdentityTraceCoherent =
0058     PRCJCost.PRCCharacterPrimeIdentityTraceCoherent
0059   prime_axis_trace_connected :
0060     PRCJCost.PRCPrimeAxisTraceConnected =
0061     PRCJCost.PRCPrimeAxisTraceConnected
0062   prime_identity_respects_trace_connected :
0063     PRCJCost.PRCCharacterPrimeIdentityRespectsTraceConnected =
0064     PRCJCost.PRCCharacterPrimeIdentityRespectsTraceConnected
0065   prime_identity_respects_common_trace_extension :
0066     PRCJCost.PRCCharacterPrimeIdentityRespectsCommonTraceExtension =
0067     PRCJCost.PRCCharacterPrimeIdentityRespectsCommonTraceExtension
0068   prime_identity_respects_comparable_trace :
0069     PRCJCost.PRCCharacterPrimeIdentityRespectsComparableTrace =
0070     PRCJCost.PRCCharacterPrimeIdentityRespectsComparableTrace
0071   orbit_identity_respects_successor_step :
0072     PRCJCost.PRCCharacterOrbitIdentityRespectsSuccessorStep =
0073     PRCJCost.PRCCharacterOrbitIdentityRespectsSuccessorStep
0074   orbit_identity_successor_transport :
0075     PRCJCost.PRCCharacterOrbitIdentitySuccessorTransport =
0076     PRCJCost.PRCCharacterOrbitIdentitySuccessorTransport
0077   orbit_successor_additive_compat :
0078     PRCJCost.PRCCharacterOrbitSuccessorAdditiveCompatible =
0079     PRCJCost.PRCCharacterOrbitSuccessorAdditiveCompatible
0080   prime_floor_orbit_identity_successor_transport :
0081     PRCJCost.PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport =
0082     PRCJCost.PRCCharacterPrimeFloorOrbitIdentitySuccessorTransport
0083   nonunit_orbit_local_orientation :
0084     PRCJCost.PRCCharacterNonunitOrbitLocalOrientation =
0085     PRCJCost.PRCCharacterNonunitOrbitLocalOrientation
0086   orbit_product_local_orientation :
0087     PRCJCost.PRCCharacterOrbitProductLocalOrientationPropagates =
0088     PRCJCost.PRCCharacterOrbitProductLocalOrientationPropagates
0089   orbit_product_display_compatible :
0090     PRCJCost.PRCCharacterOrbitProductDisplayCompatible =

```

```

0091 PRCJCost.PRCCharacterOrbitProductDisplayCompatible
0092 orbit_product_no_mixed_orientation :
0093 PRCJCost.PRCCharacterOrbitProductNoMixedOrientation =
0094 PRCJCost.PRCCharacterOrbitProductNoMixedOrientation
0095 prime_floor_no_adjacent_mixed_orientation :
0096 PRCJCost.PRCCharacterPrimeFloorNoAdjacentMixedOrientation =
0097 PRCJCost.PRCCharacterPrimeFloorNoAdjacentMixedOrientation
0098 prime_identity_common_trace_extension_target :
0099 PRCJCost.PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget =
0100 PRCJCost.PRCPrimeCalibrationForcesPrimeIdentityCommonTraceExtensionTarget
0101 prime_identity_comparable_trace_target :
0102 PRCJCost.PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget =
0103 PRCJCost.PRCPrimeCalibrationForcesPrimeIdentityComparableTraceTarget
0104 orbit_successor_identity_target :
0105 PRCJCost.PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget =
0106 PRCJCost.PRCPrimeCalibrationForcesOrbitSuccessorIdentityTarget
0107 orbit_successor_transport_target :
0108 PRCJCost.PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget =
0109 PRCJCost.PRCPrimeCalibrationForcesOrbitSuccessorTransportTarget
0110 prime_floor_successor_transport_target :
0111 PRCJCost.PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget =
0112 PRCJCost.PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
0113 prime_floor_nonunit_local_orientation_target :
0114 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget =
0115 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget
0116 prime_floor_nonunit_product_local_orientation_target :
0117 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget =
0118 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationTarget
0119 prime_floor_nonunit_orbit_orientation_coherent_target :
0120 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget =
0121 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
0122 prime_floor_no_mixed_nonunit_orbit_orientation_target :
0123 PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget =
0124 PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
0125 prime_floor_nonunit_identity_branch_transport_target :
0126 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget =
0127 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
0128 prime_floor_nonunit_identity_comparable_trace_target :
0129 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget =
0130 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
0131 prime_floor_nonunit_orbit_orientation_local_no_mixed_target :
0132 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget =
0133 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
0134 prime_floor_nonunit_orbit_orientation_local_product_no_mixed_target :
0135 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget =
0136 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalProductNoMixedTarget
0137 prime_floor_no_mixed_nonunit_from_product_no_mixed :
0138 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
0139 PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
0140 prime_floor_product_no_mixed_from_no_mixed_nonunit :
0141 PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget →
0142 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
0143 prime_floor_product_no_mixed_iff_no_mixed_nonunit :
0144 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
0145 PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget
0146 prime_floor_product_no_mixed_from_identity_branch_transport :
0147 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget →
0148 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget
0149 prime_floor_nonunit_identity_branch_transport_from_comparable_trace :
0150 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget →
0151 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
0152 prime_floor_nonunit_coherent_from_product_no_mixed :
0153 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
0154 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
0155 prime_floor_product_no_mixed_iff_nonunit_coherent :
0156 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
0157 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
0158 prime_floor_nonunit_identity_branch_transport_from_product_no_mixed :
0159 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
0160 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
0161 prime_floor_product_no_mixed_iff_identity_branch_transport :
0162 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
0163 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget
0164 prime_floor_nonunit_identity_comparable_trace_from_product_no_mixed :
0165 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget →
0166 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
0167 prime_floor_nonunit_identity_branch_transport_iff_comparable_trace :
0168 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget ↔
0169 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
0170 prime_floor_product_no_mixed_iff_identity_comparable_trace :
0171 PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget ↔
0172 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget
0173 prime_floor_nonunit_identity_comparable_trace_iff_successor_transport :
0174 PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget ↔
0175 PRCJCost.PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget
0176 prime_floor_successor_transport_local_adjacent_target :
0177 PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget =
0178 PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget
0179 prime_floor_successor_transport_local_adjacent_iff_local_successor_transport :
0180 PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
0181 (PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitLocalOrientationTarget ∧
0182 PRCJCost.PRCPrimeCalibrationForcesPrimeFloorSuccessorTransportTarget)
0183 prime_floor_successor_transport_local_adjacent_iff_nonunit_coherent :
0184 PRCJCost.PRCPrimeFloorSuccessorTransportLocalAdjacentTarget ↔
0185 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget
0186 prime_floor_nonunit_orbit_orientation_coherent_iff_local_no_mixed :
0187 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ↔
0188 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationLocalNoMixedTarget
0189 prime_floor_nonunit_orbit_orientation_coherent_sharpened_target :
0190 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget =
0191 PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget

```

```

0192 prime_floor_nonunit_orbit_orientation_coherent_iff_sharpened :
0193   PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget ←
0194     PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentSharpenedTarget
0195 prime_floor_product_local_orientation_sharpened_target :
0196   PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget =
0197     PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitProductLocalOrientationSharpenedTarget
0198 prime_floor_no_adjacent_mixed_orientation_target :
0199   PRCJCost.PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget =
0200     PRCJCost.PRCPrimeCalibrationForcesPrimeFloorNoAdjacentMixedOrientationTarget
0201 prime_floor_successor_transport_sharpened :
0202   PRCJCost.PRCPrimeFloorSuccessorTransportSharpenedTarget =
0203     PRCJCost.PRCPrimeFloorSuccessorTransportSharpenedTarget
0204 prime_to_coherent_orientation :
0205   PRCJCost.PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget =
0206     PRCJCost.PRCPrimeCalibrationForcesCoherentPrimeOrientationTarget
0207 coherent_prime_orientation_propagation :
0208   PRCJCost.PRCCoherentPrimeOrientationPropagatesToGlobalTarget =
0209     PRCJCost.PRCCoherentPrimeOrientationPropagatesToGlobalTarget
0210 prime_propagation_sharpened :
0211   PRCJCost.PRCPrimeCalibrationPropagationSharpenedTarget =
0212     PRCJCost.PRCPrimeCalibrationPropagationSharpenedTarget
0213 native_cost_rigidity_sharpened :
0214   PRCJCost.PRCNativeCostCharacterRigiditySharpenedTarget =
0215     PRCJCost.PRCNativeCostCharacterRigiditySharpenedTarget
0216 external_foundation_parsing_schema :
0217   ∀ (ExternalFoundation : Type)
0218     (FaithfulParse : ExternalFoundation → FormalSystem → Prop),
0219     ExternalFoundationParsingTarget ExternalFoundation FaithfulParse =
0220       ExternalFoundationParsingTarget ExternalFoundation FaithfulParse
0221
0222 /-- Current top-level conditional certificate: all built PRC surfaces compose,
0223 and the two remaining nonlocal obligations are exposed by name. -/
0224 structure PRCUniversalFoundationConditionalCertificate : Prop where
0225   kernel : KernelFirstPassCertificate
0226   real_complete_ordered_field :
0227     PRCRealCompleteOrderedFieldPromotedCertificate
0228   trace_logic : TraceLogicCertificate
0229   formal_system : FormalSystemCertificate
0230   inevitability : PRCInevitabilityCertificate
0231   recognizer_bridge : PRCRecognizerBridgeCertificate
0232   native_cost_blocker : PRCJCost.PRCNativeCostUniquenessBlockerCertificate
0233   open_targets : PRCUniversalFoundationOpenTargets
0234   no_project_local_axioms_audit :
0235     StrengthTag.classicalExtension = StrengthTag.classicalExtension
0236
0237 theorem prc_universal_foundation_conditional_certificate :
0238   PRCUniversalFoundationConditionalCertificate where
0239     kernel := kernel_first_pass_certificate
0240     real_complete_ordered_field :=
0241       prc_real_complete_ordered_field_promoted_certificate
0242     trace_logic := trace_logic_certificate
0243     formal_system := formal_system_certificate
0244     inevitability := prc_inevitability_certificate
0245     recognizer_bridge := prc_recognizer_bridge_certificate
0246     native_cost_blocker := PRCJCost.prc_native_cost_uniqueness_blocker_certificate
0247     open_targets := {
0248       native_cost_uniqueness := rfl
0249       native_cost_character_factorization := rfl
0250       native_cost_character_rigidity := rfl
0251       native_cost_sharpened := rfl
0252       two_to_prime_calibration := rfl
0253       prime_calibration_propagation := rfl
0254       prime_global_orientation := rfl
0255       coherent_prime_orientation := rfl
0256       local_prime_orientation := rfl
0257       no_mixed_prime_orientation := rfl
0258       prime_identity_trace_coherence := rfl
0259       prime_axis_trace_connected := rfl
0260       prime_identity_respects_trace_connected := rfl
0261       prime_identity_respects_common_trace_extension := rfl
0262       prime_identity_respects_comparable_trace := rfl
0263       orbit_identity_respects_successor_step := rfl
0264       orbit_identity_successor_transport := rfl
0265       orbit_successor_additive_compat := rfl
0266       prime_floor_orbit_identity_successor_transport := rfl
0267       nonunit_orbit_local_orientation := rfl
0268       orbit_product_local_orientation := rfl
0269       orbit_product_display_compatible := rfl
0270       orbit_product_no_mixed_orientation := rfl
0271       prime_floor_no_adjacent_mixed_orientation := rfl
0272       prime_identity_common_trace_extension_target := rfl
0273       prime_identity_comparable_trace_target := rfl
0274       orbit_successor_identity_target := rfl
0275       orbit_successor_transport_target := rfl
0276       prime_floor_successor_transport_target := rfl
0277       prime_floor_nonunit_local_orientation_target := rfl
0278       prime_floor_nonunit_product_local_orientation_target := rfl
0279       prime_floor_nonunit_orbit_orientation_coherent_target := rfl
0280       prime_floor_no_mixed_nonunit_orbit_orientation_target := rfl
0281       prime_floor_nonunit_identity_branch_transport_target := rfl
0282       prime_floor_nonunit_identity_comparable_trace_target := rfl
0283       prime_floor_nonunit_orbit_orientation_local_no_mixed_target := rfl
0284       prime_floor_nonunit_orbit_orientation_local_product_no_mixed_target := rfl
0285       prime_floor_no_mixed_nonunit_from_product_no_mixed :=
0286         PRCJCost.PRCPrimeCalibrationForcesNoMixedNonunitOrbitOrientationTarget_of_product_no_mixed
0287       prime_floor_product_no_mixed_from_no_mixed_nonunit :=
0288         PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_no_mixed_nonunit
0289       prime_floor_product_no_mixed_iff_no_mixed_nonunit :=
0290         PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_no_mixed_nonunit
0291       prime_floor_product_no_mixed_from_identity_branch_transport :=
0292         PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_of_identity_branch_transport

```

```

0293 prime_floor_nonunit_identity_branch_transport_from_comparable_trace :=
0294   PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_comparable_trace
0295 prime_floor_nonunit_coherent_from_product_no_mixed :=
0296   PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_of_product_no_mixed
0297 prime_floor_product_no_mixed_iff_nonunit_coherent :=
0298   PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_nonunit_coherent
0299 prime_floor_nonunit_identity_branch_transport_from_product_no_mixed :=
0300   PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_of_product_no_mixed
0301 prime_floor_product_no_mixed_iff_identity_branch_transport :=
0302   PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_branch_transport
0303 prime_floor_nonunit_identity_comparable_trace_from_product_no_mixed :=
0304   PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_of_product_no_mixed
0305 prime_floor_nonunit_identity_branch_transport_iff_comparable_trace :=
0306   PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityBranchTransportTarget_iff_comparable_trace
0307 prime_floor_product_no_mixed_iff_identity_comparable_trace :=
0308   PRCJCost.PRCPrimeCalibrationForcesOrbitProductNoMixedOrientationTarget_iff_identity_comparable_trace
0309 prime_floor_nonunit_identity_comparable_trace_iff_successor_transport :=
0310   PRCJCost.PRCPrimeCalibrationForcesNonunitIdentityComparableTraceTarget_iff_prime_floor_successor_transport
0311 prime_floor_successor_transport_local_adjacent_target := rfl
0312 prime_floor_successor_transport_local_adjacent_iff_local_successor_transport :=
0313   PRCJCost.PRCPrimeFloorsSuccessorTransportLocalAdjacentTarget_iff_local_successor_transport
0314 prime_floor_successor_transport_local_adjacent_iff_nonunit_coherent :=
0315   PRCJCost.PRCPrimeFloorsSuccessorTransportLocalAdjacentTarget_iff_nonunit_coherent
0316 prime_floor_nonunit_orbit_orientation_coherent_iff_local_no_mixed :=
0317   PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_local_no_mixed
0318 prime_floor_nonunit_orbit_orientation_coherent_sharpened_target := rfl
0319 prime_floor_nonunit_orbit_orientation_coherent_iff_sharpened :=
0320   PRCJCost.PRCPrimeCalibrationForcesNonunitOrbitOrientationCoherentTarget_iff_sharpened
0321 prime_floor_product_local_orientation_sharpened_target := rfl
0322 prime_floor_no_adjacent_mixed_orientation_target := rfl
0323 prime_floor_successor_transport_sharpened := rfl
0324 prime_to_coherent_orientation := rfl
0325 coherent_prime_orientation_propagation := rfl
0326 prime_propagation_sharpened := rfl
0327 native_cost_rigidity_sharpened := rfl
0328 external_foundation_parsing_schema := by
0329   intro ExternalFoundation FaithfulParse
0330   rfl
0331 }
0332 no_project_local_axioms_audit := rfl
0333
0334 end PrimitiveRecognitionCalculus
0335 end Foundation
0336 end IndisputableMonolith

```